

ACSL by Example

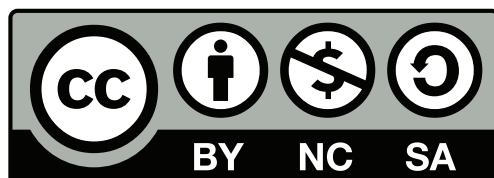
Towards a Formally Verified Standard Library

Version 33.0.1
for Frama-C 33.0 (Arsenic)
September 2026

Jens Gerlach

Former Authors

Malte Brodmann
Jochen Burghardt
Andreas Carben
Robert Clausecker
Denis Efremov
Liangliang Gu
Kerstin Hartig
Timon Lapawczyk
Hans Werner Pohl
Tim Sikatzki
Juan Soto
Kim Völlinger



Except where otherwise noted, this work is licensed under the Creative Commons
Attribution–NonCommercial–ShareAlike 4.0 International License:
<https://creativecommons.org/licenses/by-nc-sa/4.0>

Acknowledgements and Publication Information

Many members of the Frama-C community provided valuable input and comments during the course of the development of this document. In particular, we wish to thank our project partners Patrick Baudin, Allan Blanchard, Loïc Correnson, Zaynah Dargaye, Florent Kirchner, Virgile Prevosto, and Armand Puccetti from CEA LIST¹ and Pascal Cuoq from TrustInSoft².

We also express our gratitude to Claude Marché (LRI/INRIA)³ and Yannick Moy (AdaCore)⁴ for their helpful comments and detailed suggestions for improvement. Finally, we thank Aaron Rocha for valuable improvement suggestions and error reports, and Adam Berger (addaon) for the pull request from which the auxiliary function `heap_sift_down`, the bottom-up implementation of `make_heap` and the shorter `pop_heap` arose.

The research presented in this report was supported by the following projects:

- **VESEDIA** — European Union Horizon 2020 research and innovation programme, Grant Agreement No 731453 (2017–2019), <https://vessedia.eu>.
- **STANCE** — European Union Seventh Framework Programme (FP7/2007–2013), Grant Agreement No 317753 (2012–2016), <http://www.stance-project.eu>.
- **DeviceSoft** — supported by the Programme Inter Carnot Fraunhofer from BMBF (Grant 01SF0804) and ANR (2009–2012).

Availability. This document is hosted at

<https://github.com/fraunhoferfokus/acsl-by-example>

The repository also contains the source code of all algorithms discussed in this report, their formal contracts, and the associated predicate definitions and lemmas. All examples are developed and proved using the Frama-C/WP [1] plugin.

Suggestions or error reports may be submitted via the GitHub issue tracker:

<https://github.com/fraunhoferfokus/acsl-by-example/issues>

Alternatively, correspondence may be addressed to:

jens.gerlach@fokus.fraunhofer.de

¹<http://www-list.cea.fr/en>

²<http://trust-in-soft.com>

³https://www.lri.fr/index_en.php?lang=EN

⁴<http://www.adacore.com>

Contents

1	Introduction	9
1.1	Frama-C	9
1.2	Structure of this document	10
1.3	Recent changes	11
	List of Open Issues	13
I	Deductive Verification Basics	15
	Introduction	17
2	Modeling conventions	19
2.1	Types, arrays, and ranges	19
3	The Hoare calculus	23
3.1	Introduction	23
3.2	The assignment rule	25
3.3	The sequence rule	27
3.4	The implication rule	27
3.5	The choice rule	28
3.6	The loop rule	29
3.7	Derived rules	31
II	Nonmutating Algorithms for Selection and Search	33
	Introduction	35
4	Nonmutating algorithms	37
4.1	The find algorithm	38
4.2	The find2 algorithm—reuse of specification elements	41
4.3	The find3 algorithm—using a logic function	44
4.4	The find_last algorithm	49
4.5	The find_if_not algorithm	52
4.6	The find_first_of algorithm	56
4.7	The adjacent_find algorithm	59
4.8	The equal and mismatch algorithms	62
4.9	The all_of, any_of, and none_of algorithms	66
4.10	The search algorithm	68
4.11	The search_n algorithm	71
4.12	The find_end algorithm	74
4.13	The count algorithm	77

5	Minimum and maximum algorithms	81
5.1	A note on relational operators	82
5.2	Predicates for bounds and extrema of arrays	83
5.3	The auxiliary function <code>make_pair</code>	84
5.4	The <code>clamp</code> algorithm	85
5.5	The <code>max_element</code> algorithm	87
5.6	The <code>max_element2</code> algorithm (predicate-based specification)	89
5.7	The <code>max_seq</code> algorithm	91
5.8	The <code>min_element</code> algorithm	92
5.9	The <code>minmax_element</code> algorithm	94
6	Binary search algorithms	97
6.1	The predicate <code>Increasing</code>	97
6.2	Global vs. local monotonicity	98
6.3	The <code>lower_bound</code> algorithm	101
6.4	The <code>upper_bound</code> algorithm	103
6.5	The <code>equal_range</code> algorithm	105
6.6	The <code>binary_search</code> algorithm	110
III Mutating and Numeric Algorithms		113
Introduction		115
7	Mutating algorithms	117
7.1	The predicate <code>Unchanged</code>	118
7.2	Reasoning about local array updates	121
7.3	The <code>fill</code> algorithm	122
7.4	The <code>swap</code> algorithm	124
7.5	The <code>swap_ranges</code> algorithm	125
7.6	The <code>copy</code> algorithm	127
7.7	The <code>copy_backward</code> algorithm	129
7.8	The <code>reverse_copy</code> algorithm	131
7.9	The <code>reverse</code> algorithm	134
7.10	The <code>rotate_copy</code> algorithm	136
7.11	The <code>rotate</code> algorithm	139
7.12	The <code>replace_copy</code> algorithm	143
7.13	The <code>replace</code> algorithm	145
7.14	The <code>remove_copy</code> algorithm (basic contract)	147
7.15	The <code>remove_copy2</code> algorithm (number of copied elements)	150
7.16	The <code>remove_copy3</code> algorithm (final contract)	153
7.17	The <code>remove</code> algorithm	160
7.18	The <code>shuffle</code> algorithm	162
7.19	Verifying a random number generator	167
8	Numeric algorithms	169
8.1	The <code>iota</code> algorithm	170
8.2	The <code>accumulate</code> algorithm	172
8.3	The <code>inner_product</code> algorithm	176
8.4	The <code>partial_sum</code> algorithm	179
8.5	The <code>adjacent_difference</code> algorithm	182

8.6	Inverting <code>partial_sum</code> and <code>adjacent_difference</code>	185
IV	Heaps and Sorting Algorithms	189
	Introduction	191
9	Heap Algorithms	193
9.1	Basic heap concepts	194
9.2	Representation of heap concepts in ACSL	197
9.3	The auxiliary functions <code>heap_parent</code> and <code>heap_child</code>	200
9.4	The <code>is_heap_until</code> algorithm	203
9.5	The <code>is_heap</code> algorithm	205
9.6	Reorderings and temporary changes in multiplicity	206
9.7	The <code>push_heap</code> algorithm	211
9.8	The auxiliary function <code>heap_sift_down</code>	217
9.9	The <code>pop_heap</code> algorithm	221
9.10	The <code>make_heap</code> algorithm	224
9.11	The <code>sort_heap</code> algorithm	226
10	Sorting Algorithms	228
10.1	Common contract of sorting algorithms	230
10.2	The <code>is_sorted_until</code> algorithm	230
10.3	The <code>is_sorted</code> algorithm	232
10.4	The <code>partial_sort</code> algorithm	234
10.5	The <code>bubble_sort</code> algorithm	240
10.6	The <code>selection_sort</code> algorithm	242
10.7	The <code>insertion_sort</code> algorithm	244
10.8	The <code>heap_sort</code> algorithm	248
10.9	The <code>merge</code> algorithm	249
10.10	The <code>inplace_merge</code> algorithm	258
10.11	The <code>merge_sort</code> algorithm	261
V	Verification of data structures	265
	Introduction	267
11	The Stack data type	269
11.1	Methodology overview	270
11.2	Axioms and invariants of finite-capacity stacks	272
11.3	The structure <code>Stack</code> and its associated functions	274
11.4	Logical interface and invariants of <code>Stack</code>	275
11.5	Equality of stacks	277
11.6	Specification and implementation of stack functions	279
11.7	Witnesses for the stack axioms	286
11.8	Witnesses for well-definedness	292
VI	Appendices	297
	Introduction	299

A	Results of Formal Verification with Frama-C	301
A.1	Verification settings	301
A.2	Consistency of the specifications	302
A.3	Verification results	302
	Bibliography	311
	Index of ACSL definitions	313
	Index of examples	321

1. Introduction

This report presents a collection of examples illustrating the formal specification, implementation, and deductive verification of C programs using the ANSI/ISO-C Specification Language (ACSL [2]) and the Frama-C/WP plug-in [1] of Frama-C [3] (Framework for Modular Analysis of C programs).

Our examples are drawn from the C++ Standard Library, whose initial version is still known as the *Standard Template Library* (STL). The C++ Standard Library comprises a broad collection of *generic* algorithms that operate not only on C arrays but also on more elaborate container data structures. For the purposes of this document, we select representative algorithms and translate their implementations from C++ function templates into C functions operating on arrays of type `int`.

We intend to continue extending and refining this report by describing additional STL algorithms and data structures. In this way, the document is meant to evolve gradually from an ACSL tutorial into a report on a formally specified and deductively verified Standard Library for ANSI/ISO-C. Moreover, as ACSL is extended towards a specification language for C++, our approach may naturally be carried over to a deductively verified C++ Standard Library.

We encourage the reader to examine critically whether the formal specifications presented here faithfully capture the intent of the corresponding informal descriptions of the STL algorithms. We welcome feedback⁵ and hope that this document contributes to a broader adoption of deductive verification techniques.

1.1. Frama-C

The Framework for Modular Analysis of C, Frama-C [3], is a suite of software tools dedicated to the analysis of C source code. Its development is conducted and coordinated primarily at CEA LIST [4], a laboratory for applied research on software-intensive technologies, and at INRIA Saclay [5], the French National Institute for Research in Computer Science and Control, in collaboration with LRI [6], the Laboratory for Computer Science at Université Paris-Sud.

ACSL (ANSI/ISO-C Specification Language) [2] is a formal specification language for expressing behavioral properties of C programs. It allows programs to be annotated with precise contracts, including preconditions and postconditions, as well as with type invariants, global invariants, and logical specifications such as predicates, lemmas, axioms, and logic functions. In addition, ACSL supports statement-level annotations, including assertions and loop annotations, which play a central role in modular verification.

Within Frama-C, the Frama-C/WP plug-in [1] supports the deductive verification of C programs annotated with ACSL. It is based on Hoare-style weakest precondition reasoning and generates verification conditions that are discharged by external automatic theorem provers or, where necessary, by interactive proof assistants.

The Verification Group at Fraunhofer FOKUS [7] recognizes the considerable potential of deductive verification based on ACSL. At the same time, we acknowledge that the effective use of the Frama-C/WP plug-in poses non-trivial challenges, particularly for users who are new to formal methods. In order to lower this

⁵We suggest GitHub's issue tracker: <https://github.com/fraunhoferfokus/acsl-by-example/issues>

barrier to entry, this document focuses on concrete, worked examples that demonstrate how to annotate existing C programs and how to guide the deductive verification process in practice.

For a thorough introduction to ACSL and its underlying concepts, we strongly recommend that readers consult the official Frama-C introductory tutorial [8] before proceeding. The principles employed throughout this report are also documented in the ACSL reference manual [9].

1.2. Structure of this document

The functions presented in this document were selected from the C++ Standard Library. The original C++ implementation was stripped of its generic implementation and mapped to C arrays of type `value_type`.

The document is structured into several parts, each of which groups chapters with a common conceptual focus.

- Part **I**, *Deductive Verification Basics*, introduces the methodological foundations on which the remainder of the document is built. It first fixes basic modeling conventions for types, arrays, and memory ranges in C and ACSL, and then introduces the Hoare calculus as the central framework for reasoning about the functional correctness of imperative programs. Together, these chapters establish the conceptual and technical prerequisites required for all subsequent verification examples.
- Part **II**, *Nonmutating Algorithms for Selection and Search*, groups algorithms that operate on sequences without modifying their contents. The chapters in this part study traversal-based inspection algorithms, the determination of extremal elements, and binary search under suitable structural assumptions. Together, they introduce fundamental patterns of search and selection that rely solely on observation and comparison, and that serve as a conceptual bridge between the verification basics established earlier and the mutating and reordering algorithms studied in later parts.
- Part **III**, *Mutating and Numeric Algorithms*, groups algorithms that modify program state or compute numeric results during traversal of a sequence. The chapters in this part study in-place transformations of data and the accumulation of numeric values derived from sequence elements. Together, they introduce algorithmic patterns in which memory updates and numeric computation play a central role, and they extend the nonmutating inspection and selection schemes studied earlier to settings that involve state change and aggregation.
- Part **IV**, *Heaps and Sorting Algorithms*, groups algorithms whose primary purpose is the reorganization of sequence elements according to an ordering relation. The chapters in this part study heap-based algorithms and classical sorting algorithms, focusing on the use of structured ordering constraints and in-place permutation of data. Together, they address algorithmic patterns that combine mutation with global ordering properties, and they require reasoning about both structural invariants and the preservation of multisets of values.
- Part **V**, *Verification of Data Structures*, shifts the focus from individual algorithms to abstract data types and their associated operations. The chapters in this part study the specification and verification of data structures with internal state and representation invariants, using a stack as a first and representative example. Together, they introduce verification techniques that go beyond functional correctness of individual operations and address well-definedness with respect to user-defined equality and abstraction, thereby laying the groundwork for reasoning about the correctness of complete data structure interfaces.
- Part **VI**, *Appendices*, collects supplementary material that supports the main exposition but is not required for a linear reading of the document. The chapters in this part provide additional technical details, background information, and reference material that complement the core verification

examples. Together, they document the verification setup and tool configuration as well as the results obtained with them, thereby preserving context and traceability without interrupting the conceptual flow of the main text.

1.3. Recent changes

The changes of earlier versions are recorded in the file `CHANGES.md` of the repository that hosts this document:

<https://github.com/fraunhoferfokus/acsl-by-example/blob/master/CHANGES.md>

New in Version 33.0.1 (September 2026, Arsenic, 33.0)

This release is intended for Frama-C [3, p. v33.0] issued in July 2026. We are also using for this release the Why3 platform[10, p. v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
ALT-ERGO	automatic	2.6.3	[11]
CVC5	automatic	1.3.4	[12]
Z3	automatic	5.1.0	[13]
Coq	interactive	8.20.1	[14]

Table 1.1.: Information on automatic and interactive theorem provers

Most of the changes listed below were prepared with the help of AI coding agents, chiefly Claude Code but also Codex. The agents proposed code, annotations and prose; what to keep, and how to say it, was decided by us. Every contract, proof and description was reviewed before it entered the report, and every proof result reported here comes from an actual run of Frama-C/WP. The responsibility for this release is ours.

New examples

- `is_sorted_until` returns the length of the longest increasing prefix of an array and is thus to `is_sorted` what `is_heap_until` is to `is_heap`. `is_sorted` no longer scans the array itself but delegates to it; its contract is unchanged, and its proof obligations drop from 29 to 9.
- `merge_sort` splits an array into two halves, sorts each of them recursively, and joins the results with a single call to `merge`. It receives its working space as a third parameter, where `std::stable_sort` hides it [15, §28.7.1.2]. It is our first *recursive* example and thus the first whose contract carries a *decreases* clause. The new Lemmas `MultisetUnion_Reorder`, `Increasing_Unshift` and `MultisetReorder_Shift` carry what the recursive calls establish over to the postcondition of the whole array.
- `inplace_merge` merges two ordered ranges that lie next to each other in one array [15, §28.7.5] and is presented in §10.10. Its implementation reuses the Lemmas `Rotate_Shift`, `RotateOne_StrictLowerBound` and `RotateOne_MultisetReorder` as they stand, so no new lemma was needed. The verification of `inplace_merge` is based on previous work of Denis Efremov.

Implementations

- the heap algorithms now share a single descent. The new auxiliary function `heap_sift_down`, §9.8, repairs a heap whose defect is confined to a single node, and the new predicates `HeapForest` and `HeapAncestor` state its precondition. `make_heap` applies it from the last internal node down to the root and so builds a heap in $O(n)$ operations instead of the $O(n \log n)$ of repeated insertion with `push_heap`, while `pop_heap` repairs its prefix by a single call instead of a copy of the same descent and its proof shrinks from 189 proof obligations to 104. Both contracts are unchanged. These implementations go back to a pull request by Adam Berger (addaon), which we merged and modified.
- the assertions that follow the swap in `bubble_sort`'s inner loop now stand in front of the one that needs them, in `ArraySwap`'s own order, and the range that was not stated at all raises the proof obligations of the example from 109 to 110. The epilogues of `push_heap` and `pop_heap` were given the same treatment in Version 33.0.0.

Contracts

- the postcondition last of `is_heap_until` no longer quantifies over all longer prefixes of the array. It denies the heap property of the shortest prefix that exceeds the returned index, from which Lemma `Heap_Shrink` carries the failure to every longer one; a loop invariant that this made redundant was removed. The new Lemma `Increasing_Shrink` serves `is_sorted_until` in the same way.
- the postcondition of `merge` now states what the algorithm produces, and not merely that the result is ordered. The new predicate `MultisetUnion` relates three arrays in one program state and, together with `increasing`, determines the output uniquely. This settles the first item of the open issue on the completeness of `merge`'s postcondition; the two remaining items concern stability and stay open.

ACSL definitions

- the predicate `Partition` now takes the segment it refers to as an explicit pair of outer indices, with the position of the split between them, in the same way as `Increasing`, `Unchanged` and `Rotate`. The former two-index form remains available as the abbreviation of the case where the segment starts at 0, so that the contract of `partial_sort` is unchanged.
- the predicates `SomeEqual` and `NoneEqual`, and likewise `AllEqual` and `SomeNotEqual`, are now related by a single equivalence each: the new Lemmas `NoneEqual_SomeEqual` and `SomeNotEqual_AllEqual` replace the pairs that stated the halves of one $\Leftrightarrow$, in the form that `LessOrEqual_Less` already uses for the relational operators. Every example that includes one of the two definitions has one goal fewer.
- `MultisetRetainRest` now carries two lemmas of its own, which state that the predicate survives a stretch in which the array does not change, once before and once after the temporary change. The corresponding lemmas of `MultisetParity` now reduce to the two, and none of them is near the prover timeout any more. `push_heap` and `heap_sift_down`, the two examples that include the definition, have two proof obligations more.
- the labels of ACSL annotations now follow one vocabulary: `bounds` is reserved for the predicates of that name, `bound` denotes a plain range, `index` a relation between indices, and the invariants of `search`, `search_n` and `find_end` say `no_match` where their contracts do. In `selection_sort` two loop invariants shared the same label, and a loop invariant of `find_last` that a neighbouring one already implied was removed.

Verification

- the specifications are now searched for contradictions. A smoke test of Frama-C/WP asks whether `\false` follows from a precondition, a behaviour assumption or a loop invariant; no contradiction was found. §A.2 describes what such a search can and cannot show.
- `rewrite_array_unchanged` and `rewrite_array_nothing` now take part in the verification run, without being examples in their own right. The two files of §7.1 make a point about assigns rather than compute anything, and the contract of `rewrite_array_nothing` is deliberately incorrect; each is now checked against the outcome it is meant to have, so that a counterexample that quietly began to prove would not pass unnoticed.
- the list of Frama-C options in §A.1 now matches the command line that produced the results of this release. It had omitted `-wp-split`, which decides how a proof obligation is decomposed and therefore how many verification conditions each example has; `-no-warn-unaligned-pointer` and `-rte-no-pointer-call` were added as well.
- the proof obligations that no automatic prover can discharge are now named. The result tables count them per algorithm; the table in §A.3.1 lists the 52 ACSL lemmas they belong to, with the listing that shows each and the number of examples that carry it, and says why every one of them is a lemma. A comparable table was part of the report between versions 14.1.0 and 17.1.0.

Descriptions

- the agreement of the prose with the listings is now checked mechanically, and the descriptions that had drifted are again phrased in the terms of the code, among them the loop invariants of `remove_copy3` and `partial_sum`, the generator state of `shuffle` and the parameter order of `search_n`.
- §7.16 now says why `IndexOfNotEqual_Core` states three properties of the partitioning sequence in a single lemma: they are established by one induction on the rank k whose hypothesis has to supply all of them at once. The Coq script that carries the induction names its facts and records where each conjunct is used.
- the terms *array* and *range* are no longer used interchangeably: §2.1.2 calls the region described by a pointer and a length an array and reserves *range* for a contiguous part of one.
- the chapter on sorting says how its algorithms stand to one another: the overview is grouped, the section on the common contract records which five algorithms carry it and how the other five depart from it, `merge_sort` says why it keeps its auxiliary array rather than merging in place, and `insertion_sort` and `inplace_merge` point at each other.
- the report prints only the release notes of the version it is; the complete record of all releases is the file `CHANGES.md` of the repository, which is generated from these notes.

List of Open Issues

10.9.1 Completeness of the postcondition of <code>merge</code>	250
--	-----

Part I.

Deductive Verification Basics

Introduction

This part introduces the basic principles and assumptions underlying the deductive verification of C programs in this document. Its purpose is to establish a common framework for modeling programs and for reasoning about their functional correctness.

We first fix a set of modeling conventions for types, arrays, and memory ranges, which are used implicitly throughout all specifications and proofs. Building on these conventions, we then introduce the Hoare calculus as the central reasoning framework employed in the remainder of this work.

2. Modeling conventions

This chapter introduces a number of basic modeling conventions that are used consistently throughout the remainder of this document. These conventions concern the representation of types, arrays, and memory ranges in C programs, as well as their specification in ACSL.

The purpose of this chapter is not to develop new verification techniques or reasoning principles. Rather, it serves to fix terminology, notation, and assumptions that underlie all subsequent specifications and proofs. Making these conventions explicit at an early stage helps avoid ambiguities and allows later chapters to focus on the essential aspects of deductive verification.

In particular, we clarify how abstract type names are used to separate conceptual roles such as values and indices, how arrays and ranges are modeled at function boundaries, and how validity of memory accesses is expressed in ACSL. These notions are prerequisites for the correct formulation of preconditions, postconditions, and invariants, and they will be used implicitly in all following chapters.

The next chapter builds on these conventions and introduces the Hoare calculus as the central framework for reasoning about the functional correctness of C programs.

2.1. Types, arrays, and ranges

Before introducing the Hoare calculus, we fix a number of basic modeling conventions that are used throughout this document. These concern the representation of types, arrays, and memory ranges in C and their specification in ACSL. Although largely technical in nature, these conventions are essential for understanding the specifications and proofs presented later.

2.1.1. Types

In order to keep algorithms and specifications as general as possible, we employ abstract type names on almost all occasions. At present, the following type definitions are used:

```
typedef int value_type;  
typedef unsigned int size_type;  
typedef int bool;
```

Readers familiar with the types associated with C++ Standard Library containers will recognize this approach. The type `value_type` denotes the type of elements stored in an array, whereas `size_type` is used for array indices.

This convention allows, for example, an algorithm operating on an `int` array to be adapted to a different element type by changing a single line of code, namely the definition of `value_type`. In addition, it improves readability by making the intended role of a variable apparent from its type: index variables and data values are clearly distinguished.

The same consideration motivates the explicit use of the type `bool`. Values of this type are represented by the identifiers `false` and `true`, which are defined to be 0 and 1, respectively. While any non-zero value may

denote **true** in both C and ACSL, all algorithms in this document produce the value 1 to represent truth. Due to these definitions, the ACSL truth-value constants `\false` and `\true` may be used interchangeably with **false** and **true** in ACSL clauses, but not in C code.

2.1.2. Arrays and ranges

According to the C Standard, an array is a “contiguously allocated nonempty set of objects” [16, §6.2.5.20]. If n is a constant integer expression with a value greater than zero, then the declaration

```
int a[n];
```

defines an array of type **int**. In particular, for each index i satisfying $0 \leq i < n$, the pointer $a+i$ may be safely dereferenced.

In this document, we frequently consider functions of the following form:

```
void example(value_type* a, size_type n);
```

Here, the pointer a denotes the address of the first element of a memory region, and n denotes the length of that region. Strictly speaking, such a pair describes a *range* rather than an array, since functions may be called with $n == 0$. Empty arrays, however, are not permitted by the C definition. We nevertheless call the region described by such a pair an *array*, even when it is empty, and reserve the term *range* for a contiguous part of an array.

2.1.3. Notations

Throughout this document, ranges are described by a half-open notation that we introduce now.

- For integers m and n with $0 \leq m \leq n$, we denote by

$$[m \dots n) \tag{2.1}$$

the right-open integer interval $\{i \in \mathbb{Z} \mid m \leq i < n\}$.

- For an array a and integers m and n with $0 \leq m \leq n$, we denote by

$$a[m \dots n) \tag{2.2}$$

the following *array slice*

$$(a[m], a[m+1], \dots, a[n-1]).$$

In particular, the array described by the pair a and n above is the array slice $a[0 \dots n)$, and an empty range is one with $m = n$.

Note that the preceding sentence uses two typefaces for what may appear to be the same symbol, and the distinction is deliberate.

Typefaces. A symbol in *typewriter type*, such as `a` or `n`, refers to the **program text**: a parameter as it occurs in a signature, an identifier from an annotation, or the name of a clause.

A symbol in *italic type*, such as a or n , refers to the **mathematical object** that this program text denotes: the array itself, its length, or an index used in reasoning about it.

Both denote the same elements. A sentence that moves from the one to the other is moving from the program to its meaning. Where an annotation is quoted literally, typewriter type is used throughout.

For a label K , the notation $a^K[m \dots n]$ denotes the same slice, with each element evaluated in the program state K , that is,

$$a^K[i] = \text{at}(a[i], K). \quad (2.3)$$

Here, $\text{at}(x, K)$ denotes the value of the expression x in the program state K , corresponding to the ACSL construct `\at`. Labels and their semantics are discussed in §4.8.

2.1.4. Specification of valid ranges in ACSL

The following ACSL contract expresses the precondition that the function `example` may safely access all elements of the range $a[0 \dots n]$:

```
/*@
  requires 0 <= n;
  requires \valid(a + (0.. n-1));
*/
void example(value_type* a, size_type n);
```

In this context, each index i satisfying $0 \leq i < n$ is called a *valid index* of a .

The built-in ACSL predicates `\valid(a + (0.. n))` and `\valid_read(a + (0.. n))` refer to all addresses $a+i$ with $0 \leq i \leq n$. By contrast, the C array declaration `int a[n]` refers only to the elements $a+i$ for which $0 \leq i < n$ holds. Consequently, when specifying arrays of length n in ACSL, users must employ the range notation $a+(0.. n-1)$.

Note that ACSL provides no half-open range notation, so the same elements are denoted by $a+(0.. n-1)$ in an ACSL annotation and by $a[0 \dots n)$ in the mathematical descriptions of this document. We keep the closed form wherever we quote an annotation literally.

3. The Hoare calculus

The reasoning principles introduced in this chapter build on the modeling conventions established in the preceding chapter. In particular, the treatment of arrays, ranges, valid indices, and abstract types provides the semantic background against which assertions, preconditions, postconditions, and invariants are interpreted.

With these conventions fixed, we now introduce the Hoare calculus as the central framework for reasoning about the functional correctness of C programs. The calculus allows program fragments to be related systematically to logical assertions about program states and forms the basis for all subsequent verification examples in this document.

3.1. Introduction

In 1969, C. A. R. Hoare introduced a calculus for the formal reasoning about properties of imperative programs [17]. This calculus, now commonly referred to as the *Hoare calculus*, provides a systematic framework for relating program fragments to logical assertions about program states.

The central notion of the Hoare calculus may be illustrated as follows.

<pre>//@ assert P; Q; //@ assert R;</pre>

Here, P and R denote logical expressions describing properties of program states, while Q denotes a fragment of source code. Informally, this annotation expresses the following meaning:

If P holds before the execution of Q, then R holds after the execution of Q.

Usually, P and R are referred to as the *precondition* and *postcondition* of Q, respectively. The syntax of logical expressions in ACSL is described in full detail in [9, §2.2]. For the purposes of this tutorial, however, it suffices to consider the constructs listed in Table 3.1. Note that these constructs closely resemble the logical and relational operators provided by C.

The following examples illustrate the intended meaning of Hoare-style annotations by means of three simple source-code fragments.

<pre>//@ assert x % 2 == 1; ++x; //@ assert x % 2 == 0;</pre>	If x has an odd value before execution of the code ++x then x has an even value thereafter.
---	---

ACSL syntax	Name	Reading
<code>!P</code>	negation	P is not true
<code>P && Q</code>	conjunction	P is true and Q is true
<code>P Q</code>	disjunction	P is true or Q is true
<code>P ==> Q</code>	implication	if P is true, then Q is true
<code>P <==> Q</code>	equivalence	if, and only if, P is true, then Q is true
<code>x < y == z</code>	relation chain	x is less than y and y is equal to z
<code>\forall int x; P(x)</code>	universal quantifier	P(x) is true for every int value of x
<code>\exists int x; P(x)</code>	existential quantifier	P(x) is true for some int value of x

Table 3.1.: Some ACSL formula syntax

<pre>//@ assert 0 <= x <= y; ++x; //@ assert 0 <= x <= y + 1;</pre>	If the value of x is in the range $\{0, \dots, y\}$ before execution of the same code, then x's value is in the range $\{0, \dots, y + 1\}$ after execution.
<pre>//@ assert \true; while (--x != 0) sum += a[x]; //@ assert x == 0;</pre>	Under any circumstances, the value of x is zero after execution of the loop code.

Any experienced C programmer will readily confirm that these properties are valid.⁶ The examples were chosen not only for their simplicity, but also to illustrate the following general aspects of pre- and postconditions.

- For a given code fragment, there is no uniquely determined precondition or postcondition. Rather, the choice of suitable formulas depends on the particular property to be verified, which in turn arises from the application context. The first two examples illustrate this point by sharing the same code fragment while employing different pre- and postconditions.
- A postcondition need not be the strongest property that could be derived. In the second example, it is therefore not an error to state only $0 \leq x$, even though the stronger property $1 \leq x$ would also hold.
- Pre- and postconditions are not required to mention all variables occurring in the corresponding code fragment. In particular, neither `sum` nor `a[]` appears in the formulas associated with the loop example.
- The logical constant `\true` of ACSL may be used to express the absence of a restrictive precondition, as demonstrated before the `while` loop.
- Termination properties cannot, in general, be expressed by pre- and postconditions alone. The loop example therefore states only that *if* the loop terminates, then $x == 0$ will hold afterward. Indeed, if x has a negative value on entry, the loop will not terminate. Nevertheless, whenever the loop does terminate, the stated postcondition is satisfied, which is precisely the claim made by the example.

⁶For the time being, we deliberately ignore potential issues related to arithmetic overflow.

In practice, termination is usually treated separately from functional correctness. Termination proofs may, however, rely on properties that are stated and verified using the Hoare calculus.

Hoare introduced a collection of inference rules for reasoning about imperative programs. In Listings 3.2–3.14 we collect the rules and accompanying examples that will be introduced and discussed step by step in the following sections.

3.2. The assignment rule

We begin with the assignment rule, which is often regarded as the least intuitive rule of the Hoare calculus. Nevertheless, it is one of its most fundamental components. The rule is shown in Listing 3.2. Here,

$$P \{x \mapsto e\}$$

denotes the predicate obtained by substituting every occurrence of the variable x in the predicate P by the expression e .

```
//@ assert P {x |--> e};
x = e;
//@ assert P;
```

Listing 3.2: The assignment rule

As a concrete example, let P be the predicate

$$x > 0 \ \&\& \ a[2*x] == 0$$

Then the predicate $P \{x \mapsto y + 1\}$ is given by

$$y+1 > 0 \ \&\& \ a[2*(y+1)] == 0$$

Applying the assignment rule with this substitution yields Listing 3.3 as a concrete instance of the rule. Note that parentheses are required in the index expression in order to obtain the correct term $2*(y+1)$ rather than the incorrect $2*y+1$.

```
//@ assert y+1 > 0 && a[2*(y+1)] == 0;
x = y+1;
//@ assert x > 0 && a[2*x] == 0;
```

Listing 3.3: An assignment rule example instance

It is important to observe that different predicates P may yield the same substituted predicate $P \{x \mapsto e\}$. For example, under the substitution $P \{x \mapsto y + 1\}$, each of the following predicates

$$\begin{aligned} x > 0 \ \&\& \ a[2*x] &== 0 \\ x > 0 \ \&\& \ a[2*(y+1)] &== 0 \\ y+1 > 0 \ \&\& \ a[2*x] &== 0 \\ y+1 > 0 \ \&\& \ a[2*(y+1)] &== 0 \end{aligned}$$

is transformed into

$$y+1 > 0 \ \&\& \ a[2*(y+1)] == 0$$

As a consequence, for a given assignment statement, the same precondition may give rise to several different postconditions. All four predicates listed above, for instance, are valid postconditions for the

assignment shown in Listing 3.3. Conversely, for a fixed statement and a given postcondition, there is exactly one corresponding precondition determined by the assignment rule.

When first encountering Hoare’s assignment rule, many readers are tempted to consider a seemingly simpler and more intuitive alternative, shown in Listing 3.4.

```
//@ assert P;
x = e;
//@ assert P && x == e;
```

Listing 3.4: Simpler, but *faulty* assignment rule

Listings 3.5–3.7 illustrate several concrete instances of this alternative rule.

```
//@ assert y > 0;
x = y+1;
//@ assert y > 0 && x == y+1;
```

Listing 3.5: An example instance of the faulty rule from Listing 3.4

The instance shown in Listing 3.5 happens to be valid. However, the instances in Listings 3.6 and 3.7 give rise to postconditions that are clearly nonsensical.

```
//@ assert true;
x = x+1;
//@ assert x == x+1;
```

Listing 3.6: An example instance of the faulty rule from Listing 3.4

In Listing 3.6, the problem arises because the variable x occurring on the left-hand side of the assignment also appears in the right-hand side expression e . In contrast, the assignment in Listing 3.7 destroys the property stated in the precondition altogether.

```
//@ assert x < 0;
x = 5;
//@ assert x < 0 && x == 5;
```

Listing 3.7: An example instance of the faulty rule from Listing 3.4

Finally, observe that the valid instance shown in Listing 3.5 can also be derived using the correct assignment rule from Listing 3.2. Indeed, substituting x by $y+1$ in the postcondition yields the precondition $y > 0 \ \&\& \ y+1 == y+1$, which is logically equivalent to the simpler predicate $y > 0$.

3.3. The sequence rule

The sequence rule, shown in Listing 3.8, captures the effect of sequential composition. It allows two code fragments Q and S to be combined into a single fragment $Q ; S$ and relates the corresponding pre- and postconditions.

<code>//@ assert P;</code>		<code>//@ assert R;</code>		<code>//@ assert P;</code>
<code>Q;</code>	and	<code>S;</code>	$\rightsquigarrow$	<code>Q ; S;</code>
<code>//@ assert R;</code>		<code>//@ assert T;</code>		<code>//@ assert T;</code>

Listing 3.8: The sequence rule

The key requirement of the sequence rule is that the postcondition of Q must coincide with the precondition of S . This intermediate assertion represents the program state after the execution of Q and before the execution of S . In this way, the rule reflects the intuitive reading of sequential execution—“first execute Q , then execute S ”—in a precise logical form.

An important consequence of the sequence rule is that it allows programs to be annotated with assertions at intermediate program points. This style of annotation, in which assertions are interspersed with statements, is characteristic of Hoare-style reasoning and is directly supported by tools such as Frama-C.

As a simple example, consider the sequential composition of two assignments. After the first assignment, the value of x is known explicitly, which can then be used as a precondition for the second statement.

<code>//@ assert true;</code>
<code>x = y + 1;</code>
<code>//@ assert x == y + 1;</code>
<code>z = 2 * x;</code>
<code>//@ assert z == 2 * (y + 1);</code>

Listing 3.9: An example instance of the sequence rule

Here, the intermediate assertion $x == y + 1$ serves as the postcondition of the first assignment and simultaneously as the precondition of the second. By applying the sequence rule, the two annotated fragments can be combined into a single correctness argument for the sequentially composed code.

3.4. The implication rule

The implication rule, shown in Listing 3.10, provides flexibility in the choice of pre- and postconditions. It allows a precondition to be strengthened and a postcondition to be weakened without invalidating the correctness of the annotated code fragment.

<code>//@ assert P;</code>		<code>//@ assert P';</code>
<code>Q;</code>	$\rightsquigarrow$	<code>Q;</code>
<code>//@ assert R;</code>		<code>//@ assert R';</code>

Listing 3.10: The implication rule

More precisely, suppose that P' implies P and that R implies R' . In this situation, any execution that satisfies the original contract also satisfies the modified one. Consequently, the contract on the left-hand side of Listing 3.10 may safely be replaced by the contract on the right-hand side.

The implication rule plays a crucial role in practice. It allows assertions to be adapted to the surrounding context and to be matched with the requirements of other rules, such as the sequence rule. In particular, it

is frequently used to align intermediate assertions so that sequentially composed code fragments can be combined.

As a simple example, consider the following two annotations of the same code fragment.

<pre>//@ assert x > 0; x = x + 1; //@ assert x > 1;</pre>	$\rightsquigarrow$	<pre>//@ assert x > 0; x = x + 1; //@ assert x >= 0;</pre>
---	--------------------	--

Listing 3.11: An example instance of the implication rule

Here, the postcondition on the right-hand side is weaker than the one on the left-hand side. Since $x > 1 \implies x \geq 0$ holds, the implication rule allows the original contract to be replaced by the weaker one without affecting correctness.

Such a weakening is often required in practice in order to adapt a postcondition to the precondition of a subsequent code fragment.

3.5. The choice rule

The choice rule, shown in Listing 3.12, is used to reason about conditional statements of the form

```
if (C) {
    X;
}
else {
    Y;
}
```

The essential idea of the choice rule is that both the *then*-branch and the *else*-branch must establish the same postcondition, denoted by S . In each branch, the precondition is strengthened by the information provided by the condition C : in the then-branch, one may assume that C holds, whereas in the else-branch one may assume that C is false.

<pre>//@ assert P && C; X; //@ assert S;</pre>	and	<pre>//@ assert P && !C; Y; //@ assert S;</pre>	$\rightsquigarrow$	<pre>//@ assert P; if (C) X; else Y; //@ assert S;</pre>
--	-----	---	--------------------	--

Listing 3.12: The choice rule

If the two branches naturally yield different postconditions, say S_1 and S_2 , the implication rule may be used to weaken them to a common postcondition. In particular, both S_1 and S_2 imply the disjunction $S_1 \mid\mid S_2$, which can therefore serve as a unified postcondition for the conditional statement.

If the else-branch is omitted, it can be treated as an empty sequence. In this case, its postcondition coincides with its precondition, namely $P \ \&\& \ !C$.

Listing 3.13 illustrates the application of the choice rule to a simple conditional statement. The annotations are written so that each assertion refers directly to the program state immediately before or after the corresponding statement.

```

//@ assert 0 <= i < n;           // given precondition
if (i < n-1) {
  //@ assert 0 <= i < n - 1;       // using that i < n-1 holds in this branch
  //@ assert 1 <= i+1 < n;         // by the implication rule
  i = i+1;
  //@ assert 1 <= i < n;           // by the assignment rule
  //@ assert 0 <= i < n;           // weakened by the implication rule
} else {
  //@ assert 0 <= i == n-1 < n;    // using that !(i < n-1) holds in else part
  //@ assert 0 == 0 && 0 < n;      // weakened by the implication rule
  i = 0;
  //@ assert i == 0 && 0 < n;       // by the assignment rule
  //@ assert 0 <= i < n;           // weakened by the implication rule
}
//@ assert 0 <= i < n;           // by the choice rule from both branches

```

Listing 3.13: An example application of the choice rule

In this example, the variable `i` is intended to serve as an index into a ring buffer `int a[n]`. The code fragment advances the index `i` in a wrap-around fashion. The annotations show that `i` remains a valid index into the array `a[]` after execution of the conditional statement, provided that it was valid initially.

The example also illustrates the interplay between the choice rule, the assignment rule, and the implication rule. The latter is used both to establish suitable preconditions for the assignment statements and to align the postconditions of the two branches, as required by the choice rule.

3.6. The loop rule

The loop rule, shown in Listing 3.14, is used to verify `while` loops. Its central requirement is the identification of a suitable formula `P` that is preserved by every execution of the loop body. Such a formula is called a *loop invariant*.

<pre> //@ assert P && B; S; //@ assert P; </pre>	$\rightsquigarrow$	<pre> //@ assert P; while (B) { S; } //@ assert !B && P; </pre>
--	--------------------	--

Listing 3.14: The loop rule

Intuitively, the invariant `P` describes a property that holds before the first iteration of the loop and is re-established at the end of each execution of the loop body. Finding a suitable invariant often requires *insight* into the algorithm being verified and *intuition* and is therefore a major challenge in deductive verification. For this reason, the automatic discovery of loop invariants remains difficult for automated theorem provers.

As already noted, the loop rule does not guarantee termination. It only ensures that, if the loop terminates, the stated postcondition holds. For this reason, the properties established using the Hoare calculus are usually referred to as *partial correctness* properties. By contrast, properties that additionally guarantee termination are called *total correctness* properties.

As an example application, consider the abstract ring buffer shown in Listing 3.15. The goal of the verification is to show that the index variable i always remains within the valid range $[0 \dots n)$ during the execution of the loop and after its termination. The proof relies on the reasoning developed earlier in Listing 3.13 as a sub-derivation.

```

//@ assert 0 < n;                                // given precondition

int i = 0;
//@ assert 0 <= i < n;                            // by the assignment rule

while (!done) {
  //@ assert 0 <= i < n && !done;                // may be assumed by the loop rule

  a[i] = getchar();
  //@ assert 0 <= i < n && !done;                // required property of getchar
  //@ assert 0 <= i < n;                          // weakened by the implication rule

  i = (i < n-1) ? i+1 : 0;
  //@ assert 0 <= i < n;                          // follows by the choice rule

  process(a, i, &done);
  //@ assert 0 <= i < n;                          // required property of process
}
//@ assert 0 <= i < n;                            // by the loop rule

```

Listing 3.15: An abstract ring buffer loop

In order to reuse the proof from Listing 3.13, the conjunct `!done` had to be dropped, since it was not considered in that proof. In general, one may *not* infer

<pre> //@ assert P && S; Q; //@ assert R && S; </pre>	from	<pre> //@ assert P; Q; //@ assert R; </pre>
---	------	---

since the code fragment Q may destroy the property S . This is already evident if Q is taken from Listing 3.13 and S is, for example, $i \neq 0$.

Suppose, for the sake of argument, that the function `process` were implemented in such a way that it refuses to set `done` to `true` unless it is `false` on entry. In that case, it would indeed be necessary to ensure that `!done` still holds after executing the code from Listing 3.13. This would require redoing the proof while threading the additional conjunct `!done` through all intermediate assertions.

Similar considerations apply when propagating properties such as $0 \leq i < n \ \&\& \ !done$ or $0 \leq i < n$ across the statements `a[i] = getchar()` and `process(a, i, &done)`. To justify such reasoning, it must be specified that neither `getchar` nor `process` is allowed to modify the values of i or n . In ACSL, this is expressed using the `assigns` clause, which is introduced in §7.4 on Page 124.

In the present example, the loop invariant $0 \leq i < n$ happens to hold between all statements of the loop body. This need not be the case in general. The loop rule only requires the invariant to hold before the loop and at the end of each execution of the loop body. For instance, the function `process` could temporarily modify the values of i ⁷ or even n , as long as it re-establishes the invariant before returning.

⁷This would require changing the call to `process(a, &i, &done)` and adapting the implementation of `process` accordingly. In that case, the `assigns` clause discussed above could no longer be relied upon.

Finally, note that the loop invariant $0 \leq i < n$ is established by the proof in Listing 3.13 not only during the loop, but also after its termination. Consequently, a statement such as `a[i] = '\0'` placed after the loop would be guaranteed not to cause a bounds violation.

Even if a property like $0 \leq i < n$ were needed only immediately before a particular statement inside the loop body, it would still have to be established as a loop invariant. Within the Hoare calculus, the loop rule is the only mechanism by which properties can be obtained inside a loop body. Apart from this formal consideration, it is also intuitively clear that the property $0 \leq i < n$ would not hold in later iterations unless it were re-established at the end of each iteration, which is precisely what the loop rule enforces.

3.7. Derived rules

The rules presented so far do not directly cover all statement forms allowed in C. Nevertheless, statements that are not handled explicitly can be rewritten into semantically equivalent forms that fall within the scope of the basic Hoare rules.

As a first example, consider a **switch** statement. If the controlling expression *E* has no side effects, then a statement of the form

```
switch (E) {
    case E1: Q1; break; ...
    case En: Qn; break;
    default: Q0; break;
}
```

is semantically equivalent to a sequence of nested conditional statements,

```
if (E == E1) {
    Q1;
} else ... if (E == En) {
    Qn;
} else {
    Q0;
}
```

While the **if-else** form may be less efficient in terms of execution time on actual hardware, this distinction is irrelevant for verification purposes. Deductive verification is concerned with semantic correctness, not with performance characteristics.

Similarly, a loop of the form

```
for (P; Q; R) {
    S;
}
```

can be rewritten as

```
P;
while (Q) {
    S;
    R;
}
```

and verified using the sequence rule and the loop rule.

In this way, Hoare-style rules can be derived for many statement forms that were not introduced explicitly, simply by applying the basic rules to suitably transformed code. For this reason, we do not present such derived rules separately.

Procedures require a slightly different treatment. Non-recursive procedures can, in principle, be inlined, thereby reducing their verification to the application of the basic Hoare rules. Recursive procedures, however, cannot be handled in this way. Instead, their verification requires the formulation of suitable *procedure invariants*, which play a role analogous to loop invariants. The decreases clause of ACSL serves the corresponding purpose for termination; `merge_sort` in §10.11 is the recursive example of this document and the only one that carries such a clause.

Finally, note that `goto` statements cannot be rewritten into the restricted statement forms considered above. As a consequence, programs containing arbitrary `goto` statements cannot be handled directly by the Hoare calculus. For calculi capable of reasoning about arbitrary control flow graphs, including unrestricted jumps, we refer to Floyd's work [18], which predates Hoare's calculus and strongly influenced its development. Later extensions and variants of Hoare-style reasoning have reintroduced support for such constructs.

In this tutorial, however, we restrict our attention to structured control flow and therefore do not consider examples involving `goto`.

Part II.

Nonmutating Algorithms for Selection and Search

Introduction

This part collects algorithms that operate on sequences without modifying their contents. In contrast to mutating, numeric, or sorting algorithms, the procedures studied here restrict themselves to inspection, comparison, and selection. Their purpose is not to transform a sequence, but to extract information from it.

Chapter 4 introduces the basic pattern of nonmutating traversal algorithms. These algorithms examine the elements of a sequence sequentially, typically maintaining auxiliary state such as counters, accumulators, or predicates, while leaving the underlying data unchanged. They form the conceptual foundation for all subsequent algorithms in this part.

Building on this pattern, Chapter 5 studies algorithms for determining extremal elements. Minimum and maximum selection illustrates how simple traversal schemes can be refined to solve well-defined selection problems efficiently and reliably.

Finally, Chapter 6 is devoted to binary search. Unlike the preceding algorithms, binary search relies on additional structural assumptions about the input, most notably ordering. It therefore serves as a first example of how stronger invariants enable asymptotically more efficient search procedures, while still adhering to the nonmutating paradigm.

Together, these chapters establish a class of algorithms whose defining characteristic is search and selection without modification, and they provide a point of reference for later parts of the document that introduce mutation, reordering, and structural verification.

4. Nonmutating algorithms

In this chapter, we study *non-mutating* algorithms of the C++ Standard Library [15, §28.5], that is, algorithms that do not modify their arguments nor any other memory locations. This property is uniformly expressed by the following *assigns clause*:

```
assigns \nothing;
```

This assigns clause is therefore used consistently in the specification of all algorithms considered in this chapter.

Although the functional behavior of non-mutating algorithms is often simpler than that of mutating ones, their specifications nevertheless illustrate a variety of important specification techniques. The examples are arranged in increasing order of complexity, and all algorithms operate on one-dimensional arrays.

- We begin with linear search algorithms. `find` in §4.1 returns the smallest index at which a given value occurs in a range. Two alternative specifications are then introduced: `find2` in §4.2, which relies on a user-defined predicate to factor out common specification elements, and `find3` in §4.3, which uses a user-defined logic function to express linear search directly at the logic level. We also show that these different specifications are equivalent.
- `find_last` in §4.4 returns the *last* occurrence of a value in a range and is thus the counterpart of `find`.
- `find_if_not` in §4.5 is a simple variation of `find` that searches for the first position at which a given value does not occur.
- `find_first_of` in §4.6 generalizes linear search by searching for the first occurrence of an arbitrary element drawn from a given set.
- `adjacent_find` in §4.7 detects equal neighboring elements in an array.
- Range comparison algorithms are treated next. `equal` and `mismatch` in §4.8 compare two ranges element-by-element and, in the case of `merge`, identify the first position at which they differ.
- `all_of`, `any_of`, and `none_of` in §4.9 do not return a position at all, but only report whether a property holds for all, for some, or for no element of a range. Their contracts consist of a single postcondition each, since the relevant properties are already available as ACSL predicates.
- `search` and `search_n` in §4.10 and §4.11, respectively, search for a subsequence that matches a given sequence element-by-element and return the position of its first occurrence.
- `find_end` in §4.12 searches for a matching subsequence as well, but returns the *largest* index at which such a match starts, and can thus be seen as the counterpart of `search` that identifies the last occurrence rather than the first.
- The chapter concludes with `count` in §4.13, which returns the number of occurrences of a given value in a range. This example introduces an explicit logic function for counting elements and demonstrates how an implementation can be shown to comply with such a declarative specification.

4.1. The find algorithm

The `find` algorithm in the C++ Standard Library [15, §28.5.5] implements *sequential search* for general sequences. We have modified the generic implementation, which relies heavily on C++ templates, to that of a range of type `value_type`. The signature now reads:

```
size_type find(const value_type* a, size_type n, value_type v);
```

The function `find` returns the least *valid* index `i` of `a` where the condition `a[i] == v` holds. If no such index exists then `find` returns the length `n` of the array.

As an example, we consider in Figure 4.1 an array. The arrows indicate which indices will be returned by `find` for a given value. Note that the index 9 points *one past end* of the array. Values that are not contained in the array are colored in gray.

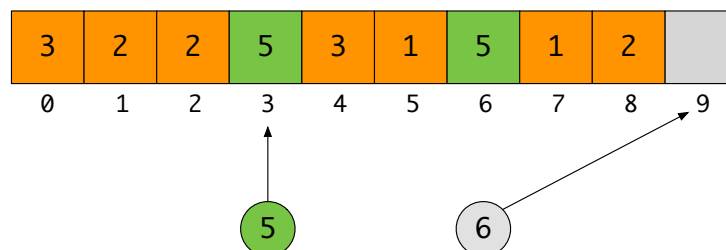


Figure 4.1.: Some simple examples for `find`

4.1.1. Formal specification of `find`

Listing 4.2 presents our first formal specification of `find`. It illustrates the basic structure of an ACSL function contract and introduces the most common specification clauses that will be used throughout this document.

- The *preconditions* of the function are expressed by the `requires`-clauses. They state that the length `n` of the array is non-negative and that the pointer `a` denotes a contiguous block of `n` objects of type `value_type`. This pattern of specifying valid memory ranges occurs frequently and will be reused in many subsequent specifications (see §2).
- The `assigns`-clause specifies the memory locations that may be modified by the function. Since `find` is a non-mutating algorithm, it does not modify any memory location visible to its caller; this is expressed by the clause `assigns \nothing` (see Page 37).
- The clause `terminates \true` states that the function is guaranteed to terminate for all inputs satisfying the preconditions. In particular, the implementation contains neither infinite loops nor unbounded recursion.
- The clause `exits \false` specifies that the function never terminates by an abnormal exit. That is, it cannot call `exit()`, trigger a runtime abort, or otherwise leave its execution context without returning normally to its caller.

```

/*@
  requires    \valid_read(a + (0..n-1));

  terminates  \true;
  exits       \false;
  assigns     \nothing;

  ensures     0 <= \result <= n;

  behavior some:
    assumes   \exists integer i; 0 <= i < n && a[i] == v;
    assigns   \nothing;
    ensures   0 <= \result < n;
    ensures   a[\result] == v;
    ensures   \forall integer i; 0 <= i < \result ==> a[i] != v;

  behavior none:
    assumes   \forall integer i; 0 <= i < n ==> a[i] != v;
    assigns   \nothing;
    ensures   \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find(const value_type* a, size_type n, value_type v);

```

Listing 4.2: Formal specification of find

At this point, the contract only guarantees that `find` returns a non-negative index that is less than or equal to the array length `n`. More precise statements about the return value can be made once additional assumptions about the contents of the array are introduced. For this reason, the specification of `find` is subdivided into two *behaviors*, named *some* and *none*.

- The behavior *some* applies when the sought-after value is contained in the array. This situation is characterized by the *assumes*-clause of the behavior. Under this assumption, the contract ensures that `find` returns a valid index `i` such that `a[i] == v`. Moreover, the final postcondition of this behavior states that the returned index is the *smallest* index with this property, thereby capturing the fact that `find` returns the first occurrence of the value in the array.
- The behavior *none* covers the complementary case in which the sought-after value does not occur in the array, as expressed by its *assumes*-clause. In this case, the contract requires `find` to return the value `n`, which indicates that the search was unsuccessful.

The formulas used in the *assumes*-clauses of the behaviors *some* and *none* are logical negations of each other. Consequently, the two behaviors are *disjoint* and *complete*: for every input satisfying the preconditions, exactly one of the two behaviors applies.

4.1.2. Implementation of find

The implementation of find [4.3] follows a simple linear search scheme.

```
size_type find(const value_type* a, size_type n, value_type v)
{
    /*@
        loop invariant 0 <= i <= n;
        loop invariant \forall integer k; 0 <= k < i ==> a[k] != v;
        loop assigns i;
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
        if (a[i] == v) {
            return i;
        }
    }

    return n;
}
```

Listing 4.3: Implementation of find

The array *a* is traversed from left to right until either an occurrence of the sought-after value is found or the end of the array is reached. If the value is found, the function returns its index; otherwise it indicates failure in accordance with the contract of find [4.2]. The algorithm performs no modification of the input array and executes at most *n* iterations.

The noteworthy elements of the implementation are the *loop annotations*. The first loop *invariant* establishes that all accesses to *a* are performed with valid indices. The second loop *invariant* supports the proof of the postconditions of the behavior some in the contract of find [4.2] by expressing that, at each iteration step, the sought-after value has not been encountered in the already inspected prefix of the array. Finally, the loop *variant* *n-i* is required to generate the verification conditions that establish termination of the loop.

4.2. The find2 algorithm—reuse of specification elements

In this section we specify `find` in a slightly different way. The approach is motivated by the observation that a considerable number of closely related ACSL formulas occur both in the contract of `find` [4.2] and in the implementation `find` [4.3]. Typical examples are the following formulas:

```
\exists integer i; 0 <= i < n      &&  a[i] == v;
\forall integer i; 0 <= i < \result ==> a[i] != v;
\forall integer i; 0 <= i < n      ==> a[i] != v;
\forall integer k; 0 <= k < i      ==> a[k] != v;
```

Observe that the first formula is the logical negation of the third one. More generally, these formulas express two complementary situations: either some element of the array equals the searched value, or no such element exists in a given range.

4.2.1. The predicates `SomeEqual` and `NoneEqual`

To make these common patterns explicit and reusable, we introduce in the following listing two predicates, `SomeEqual` [4.4] and `NoneEqual` [4.4]. Intuitively, `SomeEqual` expresses that there exists a valid index `i` such that the array element `a[i]` equals the value `v`, whereas `NoneEqual` states that no such index exists. These predicates abstract recurring existential and universal quantification into named logical concepts that can be used uniformly in both specifications and proofs.

```
/*@
  predicate SomeEqual{A}(value_type* a, integer m, integer n, value_type v) =
    \exists integer i; m <= i < n && a[i] == v;

  predicate SomeEqual{A}(value_type* a, integer n, value_type v) =
    SomeEqual(a, 0, n, v);

  predicate NoneEqual(value_type* a, integer m, integer n, value_type v) =
    \forall integer i; m <= i < n ==> a[i] != v;

  predicate NoneEqual(value_type* a, integer n, value_type v) =
    NoneEqual(a, 0, n, v);

  lemma NoneEqual_SomeEqual:
    \forall value_type *a, v, integer m, n;
      NoneEqual(a, m, n, v) <==> !SomeEqual(a, m, n, v);
*/
```

Listing 4.4: `SomeNone` (ACSL)

Note that both predicates are provided in overloaded variants. One variant applies to ranges, while the other applies to complete arrays. This avoids the need for separate predicate names while still covering both use cases.

Finally, note that the predicate `SomeEqual` is equipped with an explicit label, namely `A`. This label indicates that the evaluation of the predicate depends on a specific memory state, namely the contents of `a[0 .. n)` at label `A`. Formally, the defining formula corresponds to

```
\exists integer i; 0 <= i < n && \at(a[i],A) == v;
```

In accordance with ACSL conventions, the explicit use of `\at` can be omitted whenever the label `A` is the only label in scope. For this reason, the abbreviated form `a[i]` is used in the definition of `SomeEqual` and `NoneEqual`, and the label is omitted entirely in the overloaded versions for complete arrays.

4.2.2. Formal specification of `find2`

With the predicates `SomeEqual` [4.4] and `NoneEqual` [4.4] we encapsulate all occurrences of existential and universal quantification in both the specification and the implementation of `find2` [4.5].

```

/*@
requires    valid:    \valid_read(a + (0..n-1));

terminates      \true;
exits          \false;
assigns        \nothing;

ensures    result:    0 <= \result <= n;

behavior some:
  assumes      SomeEqual(a, n, v);
  assigns      \nothing;
  ensures bound:    0 <= \result < n;
  ensures result:  a[\result] == v;
  ensures first:   NoneEqual(a, \result, v);

behavior none:
  assumes      NoneEqual(a, n, v);
  assigns      \nothing;
  ensures result: \result == n;

complete behaviors;
disjoint behaviors;
*/
size_type find2(const value_type* a, size_type n, value_type v);

```

Listing 4.5: Formal specification of `find2`

As a result, the revised contract is more concise and more readable than that of `find` [4.2]. In particular, the assumptions of the two behaviors `some` and `none` are manifestly mutually exclusive, since one is the literal negation of the other. This mutual exclusivity, which was previously implicit in the structure of quantified formulas, is now expressed explicitly through predicate usage.

Moreover, the requirement that `find` returns the smallest index at which the searched value occurs can be expressed naturally using the `NoneEqual` [4.4] predicate. This is reflected in the final postcondition of behavior `some`, which states that no occurrence of the value precedes the returned index.

Finally, the specification of `find2` makes systematic use of user-defined names for `requires` and `ensures` clauses (sometimes also called *labels*, the distinction from program-state identifiers being clear from the context). We strongly recommend this practice, especially for more complex annotations. Among other benefits, Frama-C can be instructed to verify only clauses with a given name, which significantly improves proof control and diagnosability.

4.2.3. Implementation of `find2`

The predicate `NoneEqual` is also used in the loop annotations of the implementation `find2` [4.6].

```
size_type find2(const value_type* a, size_type n, value_type v)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant none:      NoneEqual(a, i, v);
    loop assigns i;
    loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
        if (a[i] == v) {
            return i;
        }
    }

    return n;
}
```

Listing 4.6: Implementation of find2

As in the function contract, this allows us to avoid explicit universal quantification in the annotations and to express the relevant properties in a uniform and declarative way. In particular, `NoneEqual` is used to state that all elements examined so far differ from the searched value. This closely mirrors the informal loop invariant of the algorithm and aligns the structure of the implementation annotations with that of the formal specification.

As in the specification, we systematically use labels to name individual annotations. This improves readability and enables selective verification of specific invariants or assertions.

4.3. The find3 algorithm—using a logic function

In this section we present a third specification of linear search. While this approach requires additional preparatory work, it results in a significantly more concise and transparent function contract. At the end of this section we will discuss in what sense the contracts of find2 and find3 are equivalent.

4.3.1. The logic function Find

We begin with a *recursive* definition of the ACSL logic function Find. Due to the number of lemmas associated with Find, its definition is distributed over the Listings 4.7 and 4.8. The function is provided in two *overloaded* variants: the first operates on *ranges*, whereas the second is intended for *complete arrays*.

An automatic prover will unfold the definition of Find a step at a time, but it does not invent the induction that a statement about a whole range needs. Anything to be known about Find beyond a single step therefore has to be written down as a *lemma*, proved once, and used thereafter as a named fact. That is why the listings below carry so many, and why the lemmas accompany the definition rather than find3: they are properties of Find, and every algorithm specified through it draws on them. Most are discharged by the automatic provers; those that are not are proved interactively with Coq and are listed in §A.3.1.

```
/*@
logic integer
Find(value_type* a, integer m, integer n, value_type v) =
  (n <= m) ?
  0 : ((0 <= Find(a, m, n-1, v) < n-m-1) ?
    Find(a, m, n-1, v) : ((a[n-1] == v) ? n-m-1 : n-m));

logic integer
Find(value_type* a, integer n, value_type v) = Find(a, 0, n, v);

lemma Find_Empty:
  \forallall value_type *a, v, integer m, n;
    n <= m ==> Find(a, m, n, v) == 0;

lemma Find_Hit:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
    Find(a, m, n, v) < n-m ==>
    Find(a, m, n+1, v) == Find(a, m, n, v);

lemma Find_MissHit:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
    a[n] == v ==>
    Find(a, m, n, v) == n-m ==>
    Find(a, m, n+1, v) == n-m;

lemma Find_MissMiss:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
    a[n] != v ==>
    Find(a, m, n, v) == n-m ==>
    Find(a, m, n+1, v) == (n+1)-m;
```

Listing 4.7: The logic function Find (1)

The first group of lemmas captures elementary properties of `Find` that are directly related to extending the search range from $a[0 \dots n]$ to $a[0 \dots n + 1]$. Lemmas of the first group characterize how the value of `Find` changes when the search interval is extended by one element.

The second group collects the lemmas that make `Find` usable in proofs. `Find_Lower` and `Find_Upper` confine its value to the interval $[0, n - m]$, while `Find_WeaklyIncreasing` and `Find_Increasing` state that extending the search range can never move the reported position backwards. `Find_Limit` bounds the result by any index at which v is known to occur.

```

lemma Find_Lower:
  \forallall value_type *a, v, integer m, n;
    0 <= Find(a, m, n, v);

lemma Find_Upper:
  \forallall value_type *a, v, integer m, n;
    m <= n ==> Find(a, m, n, v) <= n-m;

lemma Find_WeaklyIncreasing:
  \forallall value_type *a, v, integer m, n;
    m <= n ==> Find(a, m, n, v) <= Find(a, m, n+1, v);

lemma Find_Increasing:
  \forallall value_type *a, v, integer k, m, n;
    m <= k <= n ==>
      Find(a, m, k, v) <= Find(a, m, n, v);

lemma Find_Limit:
  \forallall value_type *a, v, integer k, m, n;
    m <= k < n ==>
      a[k] == v ==>
        Find(a, m, n, v) <= k-m;

lemma Find_NoneEqual:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
      NoneEqual(a, m, n, v) ==>
        Find(a, m, n, v) == n-m;

lemma Find_SomeEqual:
  \forallall value_type *a, v, integer k, m, n;
    m <= k < n ==>
      a[k] == v ==>
        NoneEqual(a, m, k, v) ==>
          Find(a, m, n, v) == k-m;

lemma Find_ResultNoneEqual:
  \forallall value_type *a, v, integer m, n;
    m <= n ==> NoneEqual(a, m, m + Find(a, m, n, v), v);

lemma Find_ResultEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= Find(a, m, n, v) < n-m ==>
      a[m + Find(a, m, n, v)] == v;
*/

```

Listing 4.8: The logic function `Find` (2)

The remaining lemmas connect `Find` with the predicate `NoneEqual` [4.4] and thereby with the predicate-based formulation of `find2` [4.5]: `Find_NoneEqual` covers the case that v is absent from the range, and `Find_SomeEqual` the case that its first occurrence is at a known index k . Finally, `Find_ResultNoneEqual`

and `Find_ResultEqual` characterize the position that `Find` itself returns — no element before it equals `v`, and, whenever that position lies inside the range, the element at it does.

4.3.2. Formal specification of `find3`

Using the logic function `Find` we can now give a third formal specification of linear search. In contrast to `find2` [4.5], all quantification over the input array is encapsulated in the logic function itself. As a result, the contract of `find3` [4.9] is significantly more compact and its logical structure is easier to recognize at a glance. This conciseness is achieved at the cost of a more elaborate logical infrastructure: the burden of reasoning about existence, minimality, and absence of matching elements is shifted from the function contract to the definition of the ACSL logic function `Find` [4.7] and its associated lemmas.

```

/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures      result:  0 <= \result <= n;
  ensures      result:  \result == Find(a, n, v);
*/
size_type find3(const value_type* a, size_type n, value_type v);

```

Listing 4.9: Formal specification of `find3`

4.3.3. Implementation of `find3`

The following listing shows the implementation of `find3` [4.10]. Operationally, the code is similar to the previous implementations of linear search. However, the verification now relies on relating the operational behavior of the loop to the abstract specification given by `Find` [4.7]. To this end, we introduce the assertion `found`, which makes explicit the connection between the current loop index and the value returned by `Find`. This assertion serves as a bridge between the concrete control flow of the implementation and the functional characterization used in the contract.

```

size_type find3(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:    0 <= i <= n;
    loop invariant not_found: Find(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      //@ assert found: Find(a, n, v) == i;
      return i;
    }
  }

  return n;
}

```

Listing 4.10: Implementation of `find3`

4.3.4. The equivalence of find2 and find3

A natural question at this point is in what precise sense the contracts of find2 [4.5] and find3 [4.9] can be regarded as equivalent. In this section we make this notion of equivalence explicit and show how it can be established by verification.

We call two function contracts *equivalent* if each of them is sufficiently strong to verify an implementation that is specified using the other. In other words, either contract can be used as a drop-in replacement for the other in a verified context.

To demonstrate this mutual sufficiency, we introduce two auxiliary examples, find4 and find5. Both functions are trivial wrappers whose implementations consist of a single call to an already verified linear search routine, while their contracts are taken from the alternative specification.

The implementation of find4 [4.11] consists solely of a call to find3:

```
size_type find4(const value_type* a, size_type n, value_type v)
{
    return find3(a, n, v);
}
```

Listing 4.11: Implementation of find4

The contract of find4 [4.12], however, is identical to that of find2 [4.5]:

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            \nothing;

  ensures     result:  0 <= \result <= n;

  behavior some:
    assumes      SomeEqual(a, n, v);
    assigns      \nothing;
    ensures bound:  0 <= \result < n;
    ensures result: a[\result] == v;
    ensures first:  NoneEqual(a, \result, v);

  behavior none:
    assumes      NoneEqual(a, n, v);
    assigns      \nothing;
    ensures result: \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find4(const value_type* a, size_type n, value_type v);
```

Listing 4.12: Formal specification of find4

Analogously, the implementation of find5 [4.13] is a direct call to find2:

```

size_type find5(const value_type* a, size_type n, value_type v)
{
    return find2(a, n, v);
}

```

Listing 4.13: Implementation of find5

The contract of find5 [4.14], on the other hand, coincides with that of find3 [4.9]. Thus, find5 reverses the roles of implementation and specification compared to find4.

```

/*@
    requires    valid:    \valid_read(a + (0..n-1));

    terminates          \true;
    exits              \false;
    assigns            \nothing;

    ensures    result:  0 <= \result <= n;
    ensures    result:  \result == Find(a, n, v);
*/
size_type find5(const value_type* a, size_type n, value_type v);

```

Listing 4.14: Formal specification of find5

The successful verification of both wrapper functions (cf. Table A.2) shows that each contract is sufficient to justify an implementation written against the other. This establishes the equivalence of the contracts of find2 [4.5] and find3 [4.9].

The notion of equivalence used here is inherently *contextual*. It is defined relative to the underlying specification language, the available logical infrastructure, and the verification framework. Two contracts are considered equivalent not in an abstract semantic sense, but insofar as each can be used to verify implementations specified with the other within the same formal setting.

4.4. The `find_last` algorithm

The `find_last` algorithm is the counterpart of `find` (see §4.1): instead of the first, it returns the *last* occurrence of a given value in a sequence. It is not contained in [15]; the C++ Standard Library provides it as one of the range algorithms introduced with the 2023 revision of the C++ standard. As before, we adapt the generic interface to arrays of type `value_type`:

```
size_type find_last(const value_type* a, size_type n, value_type v);
```

The function `find_last` returns the greatest *valid* index `i` of `a` for which `a[i] == v` holds. If no such index exists then `find_last` returns the length `n` of the array.

As an example, we consider in Figure 4.15 the same array as in Figure 4.1. The arrows indicate which indices will be returned by `find_last` for a given value. The value 5 occurs twice, at the indices 3 and 6. Whereas `find` returns the smaller of these two indices, `find_last` returns the greater one. As before, the index 9 points *one past end* of the array, and values that are not contained in the array are colored in gray.

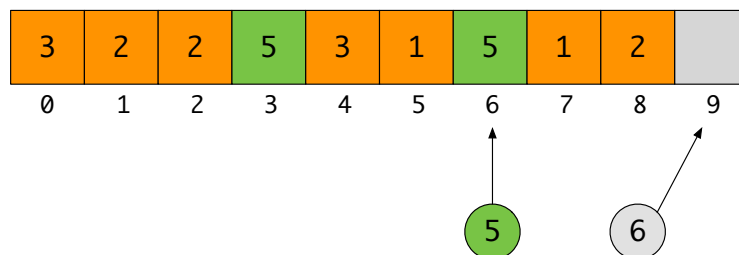


Figure 4.15.: Some simple examples for `find_last`

4.4.1. Formal specification of `find_last`

No new logical notions are required for the specification of `find_last`. Its contract has the same structure as that of `find2` [4.5] and uses the same predicates `SomeEqual` [4.4] and `NoneEqual` [4.4].

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures      result:  0 <= \result <= n;

  behavior some:
    assumes      SomeEqual(a, n, v);
    assigns      \nothing;
    ensures bound:  0 <= \result < n;
    ensures result: a[\result] == v;
    ensures last:  NoneEqual(a, \result+1, n, v);

  behavior none:
    assumes      NoneEqual(a, n, v);
    assigns      \nothing;
    ensures result: \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find_last(const value_type* a, size_type n, value_type v);
```

Listing 4.16: Formal specification of `find_last`

The two behaviors are again distinguished by `SomeEqual` and `NoneEqual` and are therefore complete and disjoint. The only difference to `find2` lies in the behavior `some`: the postcondition `first` of `find2` states that no occurrence of the value *precedes* the returned index, whereas the postcondition `last` of `find_last` states that no occurrence *follows* it. Both are expressed by `NoneEqual`, applied to the prefix $a[0 \dots \text{\result}]$ respectively to the suffix $a[\text{\result} + 1 \dots n]$.

4.4.2. Implementation of find_last

Since the last occurrence of a value can only be known once the whole array has been examined, the implementation of find_last [4.17] does not stop at the first match.

```
size_type find_last(const value_type* a, size_type n, value_type v)
{
    size_type r = n;

    /*@
    loop invariant bound: 0 <= i <= n;
    loop invariant index: r == n || 0 <= r < i;
    loop invariant none:  r == n ==> NoneEqual(a, i, v);
    loop invariant hit:   r < n ==> a[r] == v;
    loop invariant last:  r < n ==> NoneEqual(a, r+1, i, v);
    loop assigns i, r;
    loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] == v) {
            r = i;
        }
    }

    return r;
}
```

Listing 4.17: Implementation of find_last

The array is traversed from left to right, and the variable `r` always holds the index of the most recent occurrence found so far, or `n` if no occurrence has been encountered yet. The loop invariants reflect this case distinction: the invariant `none` covers the situation `r == n`, whereas the invariants `hit` and `last` state, for `r < n`, that `a[r]` equals `v` and that no later occurrence exists in the already inspected prefix `a[0 .. i)`. Upon termination of the loop, the latter two invariants are exactly the postconditions `result` and `last` of the behavior `some`.

4.5. The `find_if_not` algorithm

Many algorithms in the C++ standard library are parameterized not only by the type of the input sequence but also by so-called *function objects*. A representative example is the `find_if_not` algorithm, which takes a *predicate function object* P and returns the first position i in the input sequence for which $P(i)$ does *not* hold.

Although function objects could in principle be emulated in C by means of *function pointers*, we deliberately do not pursue this approach here. At present, function pointers are only partially supported by Frama-C, and there is currently no support for parameterized ACSL predicates. For these reasons, our implementation of `find_if_not` is restricted to the special case of locating the first position in an array at which a given value does *not* occur. The resulting function signature is therefore

```
size_type find_if_not(const value_type* a, size_type n, value_type v);
```

From an algorithmic point of view, this is not a particularly exciting addition to our collection of verified algorithms. Its value lies instead in providing a clean setting for introducing the predicates `AllEqual` and `SomeNotEqual` [4.18], and, more importantly, the logic function `FindNotEqual` [4.19]. These definitions will later play a central role in the specification of the algorithm `remove_copy`, and in particular of its variant `remove_copy3` [7.53].

```
/*@  
  predicate AllEqual(value_type* a, integer m, integer n, value_type v) =  
    \forall integer i; m <= i < n ==> a[i] == v;  
  
  predicate AllEqual(value_type* a, integer m, integer n) =  
    AllEqual(a, m, n, a[m]);  
  
  predicate AllEqual(value_type* a, integer n, value_type v) =  
    AllEqual(a, 0, n, v);  
  
  predicate SomeNotEqual{A}(value_type* a, integer m, integer n, value_type v) =  
    \exists integer i; m <= i < n && a[i] != v;  
  
  predicate SomeNotEqual{A}(value_type* a, integer n, value_type v) =  
    SomeNotEqual(a, 0, n, v);  
  
  lemma SomeNotEqual_AllEqual:  
    \forall value_type *a, v, integer m, n;  
      SomeNotEqual(a, m, n, v) <==> !AllEqual(a, m, n, v);  
*/
```

Listing 4.18: `AllSomeNot` (ACSL)

The predicate `AllEqual` expresses that every element of the range $a[m .. n)$ is equal to the value v . The predicate `SomeNotEqual` states that some element of that range differs from v , and the lemma `SomeNotEqual_AllEqual` records that this is exactly the negation of `AllEqual`. Together, these predicates naturally complement the previously introduced predicates `SomeEqual` [4.4] and `NoneEqual`.

In addition, two overloaded variants of `AllEqual` are provided. The first variant uses the value $a[m]$ itself as the reference value v . The second variant is a convenient abbreviation for the common case where the lower bound m is equal to 0.

4.5.1. The logic function FindNotEqual

The definition of the overloaded logic function FindNotEqual is given in Listings 4.19. This function is closely related to the logic function Find [4.7]. While Find [4.7] returns the first position in a sequence at which a given value occurs, FindNotEqual returns the first position at which the array element *differs* from a given reference value.

More precisely, FindNotEqual characterizes the smallest index i in the range under consideration such that $a[i] \neq v$ holds. If no such position exists, the function yields the upper bound of the considered range. This behavior mirrors that of Find [4.7] and allows the two functions to be used in a largely symmetric manner in specifications.

```
/*@
logic integer
FindNotEqual(value_type* a, integer m, integer n, value_type v) =
  (n <= m) ?
    0 : ((0 <= FindNotEqual(a, m, n-1, v) < n-m-1) ?
      FindNotEqual(a, m, n-1, v) : ((a[n-1] != v) ? n-m-1 : n-m));

logic integer
FindNotEqual(value_type* a, integer n, value_type v) =
  FindNotEqual(a, 0, n, v);

lemma FindNotEqual_Empty:
  \forall value_type *a, v, integer m, n;
    n <= m ==> FindNotEqual(a, m, n, v) == 0;

lemma FindNotEqual_Hit:
  \forall value_type *a, v, integer m, n;
    m <= n ==> FindNotEqual(a, m, n, v) < n-m ==>
      FindNotEqual(a, m, n+1, v) == FindNotEqual(a, m, n, v);

lemma FindNotEqual_MissHit:
  \forall value_type *a, v, integer m, n;
    m <= n ==> a[n] != v ==>
      FindNotEqual(a, m, n, v) == n-m ==>
      FindNotEqual(a, m, n+1, v) == n-m;

lemma FindNotEqual_MissMiss:
  \forall value_type *a, v, integer m, n;
    m <= n ==> a[n] == v ==>
      FindNotEqual(a, m, n, v) == n-m ==>
      FindNotEqual(a, m, n+1, v) == (n+1)-m;

lemma FindNotEqual_Lower:
  \forall value_type *a, v, integer m, n;
    0 <= FindNotEqual(a, m, n, v);

lemma FindNotEqual_Upper:
  \forall value_type *a, v, integer m, n;
    m <= n ==> FindNotEqual(a, m, n, v) <= n-m;
```

Listing 4.19: The logic function FindNotEqual (1)

In addition to the core definition, several lemmas establish basic properties of `FindNotEqual`. Among them is the lemma `FindNotEqual_Unchanged`, which expresses the stability of the function under modifications of ranges that are known to be unchanged. This lemma relies on the predicate `Unchanged` [7.1], which will be introduced formally in a later chapter. The forward reference is intentional, as the lemma already reflects a recurring verification pattern that will be used extensively in subsequent algorithm specifications.

```

lemma FindNotEqual_Unchanged{K,L}:
  \forallall value_type *a, v, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
      FindNotEqual{K}(a, m, n, v) == FindNotEqual{L}(a, m, n, v);

lemma FindNotEqual_WeaklyIncreasing:
  \forallall value_type *a, v, integer m, n;
    m <= n ==> FindNotEqual(a, m, n, v) <= FindNotEqual(a, m, n+1, v);

lemma FindNotEqual_Increasing:
  \forallall value_type *a, v, integer k, m, n;
    m <= k <= n ==> FindNotEqual(a, m, k, v) <= FindNotEqual(a, m, n, v);

lemma FindNotEqual_Limit:
  \forallall value_type *a, v, integer k, m, n;
    m <= k < n ==>
      a[k] != v ==> FindNotEqual(a, m, n, v) <= k-m;

lemma FindNotEqual_AllEqual:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
      AllEqual(a, m, n, v) ==> FindNotEqual(a, m, n, v) == n-m;

lemma FindNotEqual_SomeNotEqual:
  \forallall value_type *a, v, integer k, m, n;
    m <= k < n ==> a[k] != v ==>
      AllEqual(a, m, k, v) ==> FindNotEqual(a, m, n, v) == k-m;

lemma FindNotEqual_ResultAllEqual:
  \forallall value_type *a, v, integer m, n;
    m <= n ==> AllEqual(a, m, m + FindNotEqual(a, m, n, v), v);

lemma FindNotEqual_ResultNotEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= FindNotEqual(a, m, n, v) < n-m ==>
      a[m + FindNotEqual(a, m, n, v)] != v;
  */

```

Listing 4.20: The logic function `FindNotEqual` (2)

4.5.2. Formal specification of `find_if_not`

The contract of `find_if_not` [4.21] is, unsurprisingly, very similar to that of `find3` [4.9]. Both specifications follow the same overall structure and rely on a logic function that characterizes the first position in the array at which a distinguishing property holds.

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            \nothing;

  ensures      result:  0 <= \result <= n;
  ensures      result:  \result == FindNotEqual(a, n, v);
*/
size_type find_if_not(const value_type* a, size_type n, value_type v);
```

Listing 4.21: Formal specification of `find_if_not`

The only substantive difference is that the logic function `Find` [4.7] is replaced by `FindNotEqual` [4.19]. As a consequence, the postcondition no longer characterizes the first occurrence of a given value, but instead the first position at which the array element differs from the reference value. All other aspects of the contract — in particular the treatment of the case where no such position exists — remain unchanged.

4.5.3. Implementation of `find_if_not`

The implementation of `find_if_not` [4.22] closely mirrors that of `find3` [4.10]. The control structure, loop invariants, and termination argument are identical. As in the specification, the only essential change is the systematic replacement of `Find` by `FindNotEqual`.

```
size_type find_if_not(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:    0 <= i <= n;
    loop invariant not_found: FindNotEqual(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] != v) {
      //@ assert found: FindNotEqual(a, n, v) == i;
      return i;
    }
  }

  return n;
}
```

Listing 4.22: Implementation of `find_if_not`

In the loop body, this change manifests itself in the condition that is checked at each iteration: instead of testing whether `a[i]` equals the given value, we now test that it *differs* from `v`. The proof obligations generated by this implementation are discharged in the same way as for `find3` [4.10], using the corresponding lemmas for `FindNotEqual`.

4.6. The find_first_of algorithm

The algorithm `find_first_of` [15, §28.5.7] can be viewed as a natural generalization of `find` (see §4.1 and §4.2). Whereas `find` performs a sequential search for a single, fixed value, `find_first_of` searches for the first occurrence of *any* value drawn from a given set.

```
size_type find_first_of(const value_type* a, size_type m,
                       const value_type* b, size_type n);
```

Operationally, `find_first_of` performs a linear scan of the array $a[0 \dots m)$ and returns the least index i such that the element $a[i]$ is equal to at least one element of the array $b[0 \dots n)$. If no such index exists, the function indicates failure in the same way as `find`. The algorithm thus combines an outer sequential search over a with an implicit membership test against the array b .

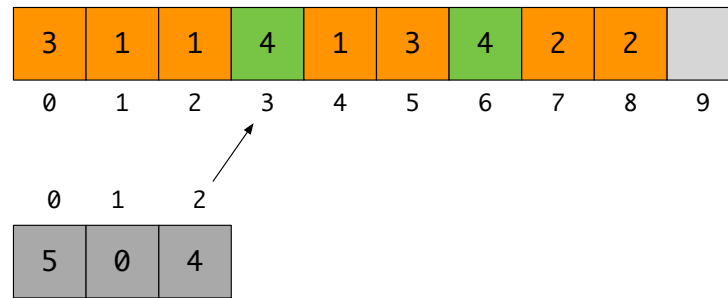


Figure 4.23.: A simple example of `find_first_of`

Figure 4.23 illustrates this behavior. Among the elements of the left-hand array, the arrow marks the smallest index at which a value occurs that is also present in the three-element array on the right.

4.6.1. The predicate `HasValueOf`

In order to express this membership test at the logic level, we introduce the predicate `HasValueOf`. Intuitively, `HasValueOf` expresses that the two arrays a and b share at least one common value at valid indices. More precisely, it asserts the existence of indices i and j within bounds such that $a[i] == b[j]$ holds.

```
/*@
  predicate HasValueOf{A}(value_type* a, integer m, value_type* b, integer n) =
    \existss integer i; 0 <= i < m && SomeEqual{A}(b, n, a[i]);
*/
```

Listing 4.24: `HasValueOf` (ACSL)

As in our treatment of `find2` (§4.2), we deliberately encapsulate this existential quantification in a named predicate. This keeps the function contract of `find_first_of` free of explicit quantifiers and clarifies the logical structure: the algorithm succeeds exactly when `HasValueOf` holds.

Rather than introducing a new primitive notion of equality, `HasValueOf` is defined in terms of the established predicate `SomeEqual` [4.4]. This highlights the close conceptual relationship between `find_first_of` and earlier variants of `find`, while avoiding unnecessary duplication at the specification level.

4.6.2. Formal specification of `find_first_of`

The following listing presents the formal specification of `find_first_of`. As in the specification of `find2` [4.5], the function contract is formulated in terms of named predicates rather than explicit quantification. In particular, it relies on the predicates `HasValueOf` [4.24] and `SomeEqual` [4.4] to characterize the existence of matching elements between the two input arrays.

```
/*@
requires    valid:  \valid_read(a + (0..m-1));
requires    valid:  \valid_read(b + (0..n-1));

terminates      \true;
exits          \false;
assigns        \nothing;

ensures    result:  0 <= \result <= m;

behavior found:
  assumes      HasValueOf(a, m, b, n);
  assigns      \nothing;
  ensures bound:  0 <= \result < m;
  ensures result: SomeEqual(b, n, a[\result]);
  ensures first: !HasValueOf(a, \result, b, n);

behavior not_found:
  assumes      !HasValueOf(a, m, b, n);
  assigns      \nothing;
  ensures result: \result == m;

complete behaviors;
disjoint behaviors;
*/
size_type find_first_of(const value_type* a, size_type m,
                        const value_type* b, size_type n);
```

Listing 4.25: Formal specification of `find_first_of`

This choice makes the logical structure of the contract immediately apparent. The successful behavior corresponds to the case where `HasValueOf` holds, that is, where at least one element of *a* occurs in *b*. Conversely, the failure behavior captures the absence of any such common value. By reusing `SomeEqual` [4.4], the specification emphasizes the close relationship between `find_first_of` and the earlier variants of `find`, while avoiding a proliferation of ad hoc predicates.

4.6.3. Implementation of `find_first_of`

The implementation of `find_first_of` [4.26] is deliberately structured as a thin wrapper around the previously verified function `find2` [4.5]. Rather than performing a nested search explicitly, the algorithm delegates the inner membership test to `find2`, thereby making reuse an explicit design choice.

```
size_type find_first_of (const value_type* a, size_type m,
                        const value_type* b, size_type n)
{
    /*@
      loop invariant bound:      0 <= i <= m;
      loop invariant not_found: !HasValueOf(a, i, b, n);
      loop assigns i;
      loop variant m-i;
    */
    for (size_type i = 0u; i < m; i++) {
        if (find2(b, n, a[i]) < n) {
            return i;
        }
    }

    return m;
}
```

Listing 4.26: Implementation of `find_first_of`

This approach has two immediate benefits. First, it leads to a concise and readable implementation whose control flow closely mirrors the informal description of the algorithm. Second, and more importantly from a verification perspective, it significantly reduces the annotation burden: no additional loop invariants are required beyond those already established for `find2`. As a result, the correctness of `find_first_of` follows largely from the existing specification and proof of `find2`, illustrating how verified components can be composed to obtain new, verified functionality with minimal additional effort.

From an algorithmic point of view, this implementation realizes `find_first_of` as a linear scan over the array a . At each position, the current element $a[i]$ is tested for membership in the array b by invoking `find2`. The function thus combines an outer sequential traversal with a logically specified membership test, closely reflecting the structure captured by the predicate `HasValueOf` [4.24] at the specification level.

4.7. The adjacent_find algorithm

The algorithm `adjacent_find`, as specified in the C++ Standard Library [15, §28.5.8], searches an array for the first occurrence of two equal adjacent elements.

```
size_type adjacent_find(const value_type* a, size_type n);
```

More precisely, `adjacent_find` returns the smallest index i such that both i and $i+1$ are valid indices of the array and

$$a[i] == a[i+1]$$

holds. If no such index exists, the algorithm returns n .

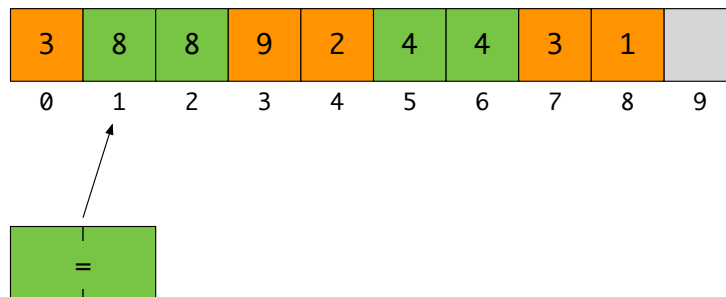


Figure 4.27.: A simple example of `adjacent_find`

Figure 4.27 illustrates this behavior. The arrow marks the smallest index at which two adjacent elements of the array are equal.

4.7.1. The predicate `HasEqualNeighbors`

As in the case of the other search algorithms considered so far, we first introduce a predicate that captures the essential logical property underlying the algorithm. The predicate `HasEqualNeighbors` [4.28] formalizes the existence of two adjacent valid indices at which the array holds equal values.

```
/*@  
  predicate HasEqualNeighbors{L}(value_type* a, integer n) =  
    \exists integer i; 0 <= i < n-1 && a[i] == a[i+1];  
*/
```

Listing 4.28: The predicate `HasEqualNeighbors`

By encapsulating this property in a named predicate, we avoid explicit quantification in the function contract and make the specification more readable and structured.

4.7.2. Formal specification of `adjacent_find`

Our specification of `adjacent_find` [4.29] relies on the predicate `HasEqualNeighbors` [4.28]. This predicate captures the essential property underlying the algorithm, namely the existence of two adjacent valid indices at which the array contains equal values.

```
/*@
  requires    valid:      \valid_read(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            \nothing;

  ensures    result:    0 <= \result <= n;

  behavior some:
    assumes      HasEqualNeighbors(a, n);
    assigns      \nothing;
    ensures result: 0 <= \result < n-1;
    ensures adjacent: a[\result] == a[\result+1];
    ensures first:  !HasEqualNeighbors(a, \result);

  behavior none:
    assumes      !HasEqualNeighbors(a, n);
    assigns      \nothing;
    ensures result: \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type adjacent_find(const value_type* a, size_type n);
```

Listing 4.29: Formal specification of `adjacent_find`

As in the specifications of the other search algorithms, the function contract is structured into distinct behaviors. The successful behavior corresponds exactly to the case where `HasEqualNeighbors` holds, while the failure behavior captures its negation. By formulating the contract in terms of this predicate, the specification avoids explicit quantification and makes the intended meaning of the postcondition immediately apparent.

4.7.3. Implementation of `adjacent_find`

The implementation of `adjacent_find` [4.30] follows closely the informal description of the algorithm. As a preliminary step, it checks whether the array contains at least two elements. If this is not the case, no pair of adjacent elements can exist, and the function can immediately return failure.

```
size_type adjacent_find(const value_type* a, size_type n)
{
    if (1u < n) {
        /*@
        loop invariant bound:  0 <= i < n;
        loop invariant none:   !HasEqualNeighbors(a, i+1);
        loop assigns i;
        loop variant n-i;
        */
        for (size_type i = 0u; i + 1u < n; ++i) {
            if (a[i] == a[i + 1u]) {
                return i;
            }
        }
    }

    return n;
}
```

Listing 4.30: Implementation of `adjacent_find`

For the general case, the algorithm performs a linear scan of the array, comparing each element with its immediate successor. The loop invariant makes explicit use of the predicate `HasEqualNeighbors` [4.28] to relate the progress of the scan to the postcondition of the successful behavior. In particular, it ensures that, at each iteration, either a matching pair has already been encountered or no such pair exists in the portion of the array examined so far.

This close correspondence between the loop invariant and the function postcondition allows the correctness of the implementation to be established in a direct and transparent manner.

4.8. The equal and mismatch algorithms

The algorithms `equal` [15, §28.5.11] and `mismatch` [15, §28.5.10] of the C++ Standard Library compare two sequences element by element. For our purposes, we restrict attention to arrays of a fixed value type `value_type` and adapt the generic interfaces accordingly. The resulting function signatures are

```
bool    equal(const value_type* a, size_type n, const value_type* b);

size_type mismatch(const value_type* a, size_type n, const value_type* b);
```

The function `equal` returns `true` if and only if $a[i] == b[i]$ holds for every index $0 \leq i < n$. If this condition fails for at least one index, the function returns `false`.

The algorithm `mismatch` is slightly more informative. Rather than merely reporting whether the two arrays are equal, it returns the smallest index at which the arrays a and b differ. If no such index exists—that is, if the arrays agree at all positions $0 \leq i < n$ —then `mismatch` returns the common length n of the two arrays.

4.8.1. The Equal predicate

The fact that two array slices $a[0 \dots n)$ and $b[0 \dots n)$ coincide element by element is a basic property that arises naturally in many specifications. Rather than restating this condition repeatedly, it is preferable to capture it once in a dedicated predicate. To this end, we define several *overloaded* versions of the predicate `Equal` [4.31], shown in the following listing.

```
/*@
  predicate Equal{K,L}(value_type* a, integer m, integer n, value_type* b) =
    \forall integer i; m <= i < n ==> \at(a[i],K) == \at(b[i],L);

  predicate Equal{K,L}(value_type* a, integer n, value_type* b) =
    Equal{K,L}(a, 0, n, b);

  predicate Equal{K,L}(value_type* a, integer m, integer n,
                        value_type* b, integer p) =
    \forall integer k; 0 <= k < n-m ==> \at(a[m+k],K) == \at(b[p+k],L);

  predicate Equal{K,L}(value_type* a, integer m, integer n, integer p) =
    Equal{K,L}(a, m, n, a, p);
*/
```

Listing 4.31: Equal (ACSL)

The parameters K and L in the definition of `Equal` are so-called *labels*. They denote the program states in which the arrays a and b are to be evaluated. The Frama-C tool chain provides several standard labels, such as `Old` and `Post`, which refer to the pre-state and post-state of a function, respectively. User-defined labels may be introduced to name intermediate program points. For a detailed discussion of labels and their semantics, we refer to the ACSL specification [9, §2.6.9].

Mathematical description

We use the array slice notation introduced in §2.1.2, in particular the slice $a[m \dots n]$ of Equation (2.2) and its evaluation $a^K[m \dots n]$ in a program state K .

The following equations provide a concise mathematical description of the overloaded variants of the predicate `Equal` defined in Listing 4.31.

Equal $\{K, L\}(a, m, n, b)$ expresses equality of two array slices over the same index range:

$$a^K[m \dots n] = b^L[m \dots n], \quad \text{Base variant} \quad (4.1a)$$

that is,

$$a^K[i] = b^L[i] \quad \forall i \in [m \dots n].$$

Equal $\{K, L\}(a, n, b)$ is a shorthand for (4.1a) with $m = 0$:

$$a^K[0 \dots n] = b^L[0 \dots n], \quad \text{Prefix variant} \quad (4.1b)$$

Equal $\{K, L\}(a, m, n, b, p)$ generalizes (4.1a) by starting the right-hand slice at index p :

$$a^K[m \dots n] = b^L[p \dots p + (n - m)], \quad \text{Shifted-slice variant} \quad (4.1c)$$

Equal $\{K, L\}(a, m, n, p)$ instantiates (4.1c) for the case $a = b$:

$$a^K[m \dots n] = a^L[p \dots p + (n - m)], \quad \text{Same-array variant} \quad (4.1d)$$

4.8.2. Formal specification of `equal` and `mismatch`

Using the predicate `Equal` [4.31], we can give a concise and precise specification of `equal` [4.32].

```

/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures   result: \result <==> Equal{Here,Here}(a, n, b);
*/
bool equal(const value_type* a, size_type n, const value_type* b);

```

Listing 4.32: Formal specification of `equal`

In the `ensures` clause, `Equal` is instantiated with the predefined label `Here`, which refers to the post-state of the function. This states that, upon termination, the two input arrays are equal element by element. Note that the postcondition is formulated as a logical equivalence. An equality would not be well-formed in ACSL, since `Equal` is a predicate, not a logic function, and therefore cannot appear as the argument of the equality operator.

The formal specification of `mismatch` [4.33] is necessarily more elaborate. Unlike `equal`, which merely reports whether the two arrays coincide, `mismatch` returns additional information in the form of an index.

```

/*@
requires    valid:    \valid_read(a + (0..n-1));
requires    valid:    \valid_read(b + (0..n-1));

terminates          \true;
exits               \false;
assigns             \nothing;

ensures    result:    0 <= \result <= n;

behavior all_equal:
  assumes          Equal{Here,Here}(a, n, b);
  assigns          \nothing;
  ensures result:    \result == n;

behavior some_not_equal:
  assumes          !Equal{Here,Here}(a, n, b);
  assigns          \nothing;
  ensures bound:    0 <= \result < n;
  ensures result:    a[\result] != b[\result];
  ensures first:    Equal{Here,Here}(a, \result, b);

complete behaviors;
disjoint behaviors;
*/
size_type mismatch(const value_type* a, size_type n, const value_type* b);

```

Listing 4.33: Formal specification of `mismatch`

Conceptually, the specification of `mismatch` closely parallels that of `find2` [4.5]. While `find2` returns the smallest index i such that $a[i] == v$ holds, `mismatch` returns the smallest index i such that $a[i] != b[i]$. In both cases, the returned index identifies the first position at which a distinguishing condition becomes true.

As in the specification of `find2`, the behaviors of `mismatch` are both complete and mutually exclusive. This is apparent from the `assumes` clauses of the behaviors `all_equal` and `some_not_equal`, which are exact negations of one another. The predicate `Equal` [4.31] plays a central role here, both in expressing the case where no mismatch exists and in delimiting the prefix of equal elements in the case where a mismatch is found.

4.8.3. Implementation of equal and mismatch

The implementation of `equal` [4.34] is deliberately minimal: it consists solely of a call to `mismatch` and a comparison of the returned index with the array length `n`. This directly reflects the relationship between the two algorithms established at the level of their specifications.

```
bool equal(const value_type* a, size_type n, const value_type* b)
{
    return mismatch(a, n, b) == n;
}
```

Listing 4.34: Implementation of `equal`

The implementation of `mismatch` [4.35] follows the expected linear scan of the two arrays, but is enriched with loop annotations to support deductive verification.

```
size_type mismatch(const value_type* a, size_type n, const value_type* b)
{
    /*@
        loop invariant bound:  0 <= i <= n;
        loop invariant equal:  Equal{Here,Here}(a, i, b);
        loop assigns i;
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
        if (a[i] != b[i]) {
            return i;
        }
    }

    return n;
}
```

Listing 4.35: Implementation of `mismatch`

As in the specification, we employ the predicate `Equal` [4.31] in the loop invariant to express that all indices `k` strictly smaller than the current index `i` satisfy `a[k] == b[k]`. This invariant captures the essential progress property of the algorithm: at each iteration, the examined prefix of the arrays is known to be equal.

The presence of this invariant is crucial for proving that the value returned by `mismatch` is indeed the smallest index at which the two arrays differ, or `n` if no such index exists.

4.9. The `all_of`, `any_of`, and `none_of` algorithms

The algorithms `all_of`, `any_of`, and `none_of` of the C++ Standard Library [15, §28.5.1–28.5.3] do not return a position but merely report *whether* a property holds for all, for some, or for no element of a sequence.

As in the case of `find_if_not` (see §4.5), we do not model the predicate function object of the generic versions but restrict ourselves to the comparison with a fixed value v . The three function signatures then read:

```
bool all_of (const value_type* a, size_type n, value_type v);  
bool any_of (const value_type* a, size_type n, value_type v);  
bool none_of (const value_type* a, size_type n, value_type v);
```

4.9.1. Formal specification of `all_of`, `any_of`, and `none_of`

The specifications of these algorithms are particularly simple, because the properties in question have already been introduced as ACSL predicates. Each contract consists of a single postcondition stating that the returned truth value is equivalent to the corresponding predicate.

The algorithm `all_of` reports whether *all* elements of the array are equal to v , which is precisely the property expressed by `AllEqual` [4.18].

```
/*@  
  requires   valid:   \valid_read(a + (0..n-1));  
  
  terminates \true;  
  exits      \false;  
  assigns    \nothing;  
  
  ensures    result:  \result <==> AllEqual(a, n, v);  
*/  
bool all_of(const value_type* a, size_type n, value_type v);
```

Listing 4.36: Formal specification of `all_of`

The algorithm `any_of` reports whether *at least one* element is equal to v , as expressed by `SomeEqual` [4.4].

```
/*@  
  requires   valid:   \valid_read(a + (0..n-1));  
  
  terminates \true;  
  exits      \false;  
  assigns    \nothing;  
  
  ensures    result:  \result <==> SomeEqual(a, n, v);  
*/  
bool any_of(const value_type* a, size_type n, value_type v);
```

Listing 4.37: Formal specification of `any_of`

The algorithm `none_of` reports whether *no* element is equal to v , as expressed by `NoneEqual` [4.4].

```

/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates      \true;
  exits          \false;
  assigns        \nothing;

  ensures      result: \result <==> NoneEqual(a, n, v);
*/
bool none_of(const value_type* a, size_type n, value_type v);

```

Listing 4.38: Formal specification of `none_of`

Note that the contracts of `none_of` [4.38] and `any_of` [4.37] are negations of each other, which is a direct consequence of the lemmas relating `SomeEqual` and `NoneEqual` (see §4.2).

4.9.2. Implementation of `all_of`, `any_of`, and `none_of`

None of the three algorithms performs a linear scan of its own. Each of them delegates the search to `find` (see §4.1) or to `find_if_not` (see §4.5) and merely inspects the index that is returned, so that no loop annotations are needed at all.

Since `find_if_not` returns the index of the first element that differs from `v`, and the array length `n` if there is no such element, all elements are equal to `v` if and only if that index equals `n`.

```

bool all_of(const value_type* a, size_type n, value_type v)
{
    return find_if_not(a, n, v) == n;
}

```

Listing 4.39: Implementation of `all_of`

Since `find` returns the index of the first occurrence of `v`, and the array length `n` if there is none, the value occurs in the array if and only if that index is smaller than `n`.

```

bool any_of(const value_type* a, size_type n, value_type v)
{
    return find(a, n, v) < n;
}

```

Listing 4.40: Implementation of `any_of`

By the same argument, `v` is absent from the array exactly when `find` returns `n`.

```

bool none_of(const value_type* a, size_type n, value_type v)
{
    return find(a, n, v) == n;
}

```

Listing 4.41: Implementation of `none_of`

The reductions of `any_of` and `none_of` are verified against the contract of `find` [4.2] alone: the distinction between its behaviors `some` and `none` carries over directly to the two postconditions. The contract of `find_if_not` [4.21], in contrast, describes the result by the logic function `FindNotEqual` [4.19]. The verification of `all_of` [4.39] therefore relies on the lemmas that relate this function to `AllEqual`, in particular `FindNotEqual_AllEqual` and `FindNotEqual_ResultAllEqual`.

4.10. The search algorithm

The search algorithm of the C++ Standard Library [15, §28.5.13] searches for the first occurrence of a sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we adapt the generic formulation to arrays of type `value_type`. The resulting function signature is as follows:

```
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p);
```

The function `search` returns the smallest index s such that the subrange $a[s \dots s + p)$ is element-wise equal to the range $b[0 \dots p)$, that is, $a[s+k] == b[k]$ holds for every index k with $0 \leq k < p$ (see Figure 4.42). If no such index exists, the function returns the end of the searched range, represented here by the length n of the array a .

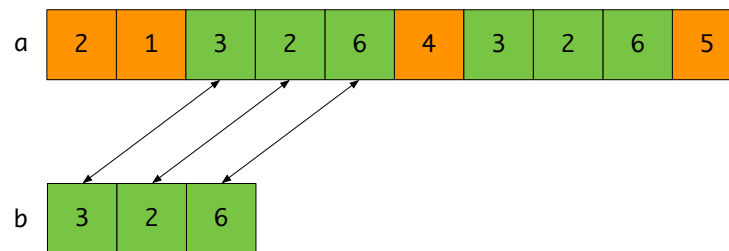


Figure 4.42.: A simple example of search

4.10.1. The predicate `HasSubRange`

The formal specification of `search` begins with the introduction of the predicate `HasSubRange`. This predicate expresses that the array a contains b as a *contiguous* subrange. Using the predicate `Equal` [4.31], it formalizes the existence of an index s such that the ranges $a[s \dots s + p)$ and $b[0 \dots p)$ are equal element by element.

```
/*@
  predicate HasSubRange{L}(value_type* a, integer m, integer n, value_type* b,
    integer p) =
    \exists integer k; (m <= k <= n-p) && Equal{L,L}(a+k, p, b);

  predicate HasSubRange{L}(value_type* a, integer n, value_type* b, integer p) =
    HasSubRange{L}(a, 0, n, b, p);

  lemma HasSubRange_Sizes:
    \forall value_type *a, *b, integer m, n, p;
      HasSubRange(a, m, n, b, p) ==> p <= n-m;
*/
```

Listing 4.43: `HasSubRange` (ACSL)

For such a subrange of length p to exist, the array a must be long enough to accommodate it. This necessary size constraint is captured by the lemma `HasSubRange_Sizes`.

4.10.2. Formal specification of search

The following listing shows the formal specification of search [4.44].

```
/*@
requires    valid:    \valid_read(a + (0..n-1));
requires    valid:    \valid_read(b + (0..p-1));

terminates          \true;
exits               \false;
assigns             \nothing;

ensures    result:    0 <= \result <= n;

behavior has_match:
  assumes      HasSubRange(a, n, b, p);
  assigns      \nothing;
  ensures bound:  0 <= \result <= n-p;
  ensures result: Equal{Here,Here}(a+\result, p, b);
  ensures first: !HasSubRange(a, \result+p-1, b, p);

behavior no_match:
  assumes      !HasSubRange(a, n, b, p);
  assigns      \nothing;
  ensures result: \result == n;

complete behaviors;
disjoint behaviors;
*/
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p);
```

Listing 4.44: Formal specification of search

At a conceptual level, the specification of search closely resembles that of find [4.2]. As before, we distinguish two behaviors in order to capture the essential cases.

- The behavior `has_match` applies when the array a contains the sequence b as a contiguous subrange. This situation is characterized in the `assumes` clause by the predicate `HasSubRange` [4.43].

The `ensures` clause `bound` of behavior `has_match` states that the returned index lies in the range $[0 .. n - p + 1)$. Clause `result` expresses that `search` returns an index at which a copy of b occurs in a . Finally, clause `first` enforces minimality: the returned index is the least one with this property, i.e. the sequence b does not occur in the prefix $a[0 .. \text{result} + p - 1)$.

- The behavior `no_match` covers the complementary case in which the array a does not contain b as a subrange. In this situation, `search` is required to return the length n of the array a . If either of the ranges a or b is empty, the return value is 0 , in accordance with the definition of the algorithm.

The `assumes` clause of behavior `has_match` is the logical negation of the `assumes` clause of behavior `no_match`. Consequently, the two behaviors are explicitly specified as *complete* and *disjoint*.

4.10.3. Implementation of search

The implementation of search [4.45] is simple and easy to follow: it checks every possible starting position in a until it finds a match for b or reaches the end of a . Because it may compare up to p elements at each of the $n-p+1$ starting positions, its worst-case number of comparisons grows proportionally to $n \cdot p$.

More advanced string search algorithms, such as the classic method of Knuth, Morris, and Pratt [19], avoid this quadratic behavior and guarantee at most $n + p$ comparisons.⁸

```
size_type search(const value_type* a, size_type n,
                const value_type* b, size_type p)
{
    if (p <= n) {
        /*@
        loop invariant bound:      i <= n-p+1;
        loop invariant no_match:  !HasSubRange(a, p+i-1, b, p);
        loop assigns i;
        loop variant n-i;
        */
        for (size_type i = 0u; i <= n - p; ++i) {
            if (equal(a + i, p, b)) {
                /*@ assert has_match: HasSubRange(a, n, b, p);
                return i;
            }
        }
    }

    /*@ assert no_match: !HasSubRange(a, n, b, p);
    return n;
}
```

Listing 4.45: Implementation of search

Depending on the version of Frama-C/WP, the loop invariant `no_match` may be essential for establishing the postconditions of the behavior `has_match`. It expresses that no occurrence of the sequence b has been found in a up to the current iteration.

Neither the case $p == 0$ nor the case $n == 0$ requires special treatment, not even for efficiency reasons. In the former case, the call `equal(a+i, p, b)` succeeds immediately, while in the latter case the condition $p > n$ applies.

⁸ The efficiency of search has also been discussed by the C++ standardization committee; see <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3905.html>.

4.11. The search_n algorithm

The `search_n` algorithm of the C++ Standard Library [15, §28.5.13] locates the first position at which a given value occurs a prescribed number of consecutive times in a sequence. For our purposes, we adapt the generic interface to operate on an array of elements of type `value_type`. The resulting function signature is

```
size_type
search_n(const value_type* a, size_type n,
         value_type v, size_type p);
```

This interface closely resembles that of `search` (Section 4.10). The only difference is that `search_n` takes a single value `v` instead of a pattern array.

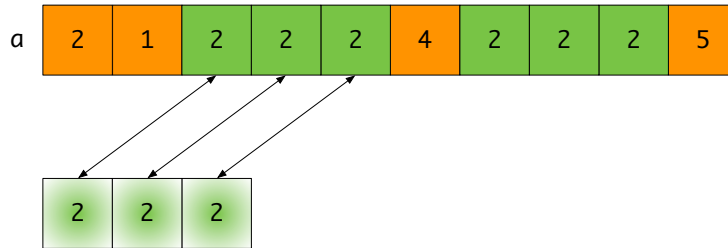


Figure 4.46.: A simple example of `search_n`

The function `search_n` returns the smallest index `s` such that `a[s+k] == v` holds for all indices `k` with $0 \leq k < p$ (see Figure 4.46). If no such index exists, the function returns the array length `n`.

4.11.1. The predicate `HasConstantSubRange`

The specification of `search_n` is based on the predicate `HasConstantSubRange` [4.47]. This predicate expresses that the array `a` of length `n` contains a contiguous subsequence consisting of `p` occurrences of the value `v`. Its definition reuses the predicate `AllEqual` [4.18] to characterize constant ranges.

```
/*@
  predicate HasConstantSubRange{L}(value_type* a, integer m, integer n, value_type
    v, integer p) =
    \exists integer k; m <= k <= n-p && AllEqual(a, k, k+p, v);

  predicate HasConstantSubRange{L}(value_type* a, integer n, value_type v, integer
    p) =
    HasConstantSubRange(a, 0, n, v, p);

  lemma HasConstantSubRange_Sizes:
    \forall value_type *a, v, integer n, p;
      HasConstantSubRange(a, n, v, p) ==> p <= n;
*/
```

Listing 4.47: `HasConstantSubRange` (ACSL)

As with `HasSubRange` [4.43], the existence of such a subsequence requires that the array be sufficiently long. This size constraint is made explicit by the lemma `HasConstantSubRange_Sizes` [4.47].

4.11.2. Formal specification of search_n

As for search [4.44], the specification of search_n [4.48] closely resembles that of find2 [4.5].

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns             \nothing;

  ensures     result:   0 <= \result <= n;

  behavior has_match:
    assumes      HasConstantSubRange(a, n, v, p);
    assigns      \nothing;
    ensures result: 0 <= \result <= n-p;
    ensures match: AllEqual(a, \result, \result+p, v);
    ensures first: !HasConstantSubRange(a, \result+p-1, v, p);

  behavior no_match:
    assumes      !HasConstantSubRange(a, n, v, p);
    assigns      \nothing;
    ensures result: \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type search_n(const value_type* a, size_type n, value_type v, size_type p);
```

Listing 4.48: Formal specification of search_n

We again distinguish two behaviors capturing the essential cases.

- The behavior `has_match` applies if the array `a` contains a p -fold repetition of the value `v`. This condition is expressed in the `assumes` clause using the predicate `HasConstantSubRange` [4.47]. The `ensures` clauses state that the return value lies in $[0 .. n - p + 1)$, that p or more consecutive occurrences of `v` start at this position, and that this position is minimal.
- The behavior `no_match` covers the complementary case in which no such subsequence exists. Here, `search_n` returns the length n of the array `a`.

4.11.3. Implementation of search_n

Although the specification of `search_n` [4.48] closely mirrors that of `search` [4.44], their implementations differ substantially. While `search` [4.45] uses a simple but non-optimal $\mathcal{O}(n \cdot p)$ algorithm, the implementation of `search_n` [4.49] runs in linear time $\mathcal{O}(n)$.

The implementation tracks, in the variable `start`, the beginning of the current maximal block of consecutive values `v`. The loop invariant `no_match` expresses that no p -fold repetition has been encountered so far; once such a repetition is found, the loop terminates and `start` is returned.

The boundary cases $n < p$ and $p == 0$ are handled explicitly in separate branches, which simplifies the verification by automatic provers.

```

size_type search_n(const value_type* a, size_type n, value_type v, size_type p)
{
    if (0u < p) {
        if (p <= n) {
            size_type start = 0u;

            /*@
            loop invariant match:    AllEqual(a, start, i, v);
            loop invariant start:    0 < start ==> a[start-1] != v;
            loop invariant bound:    start <= i + 1 <= start + p;
            loop invariant no_match: !HasConstantSubRange(a, i, v, p);
            loop assigns i, start;
            loop variant n - i;
            */
            for (size_type i = 0u; i < n; ++i) {
                if (a[i] != v) {
                    start = i + 1u;
                    /*@ assert no_match: !HasConstantSubRange(a, i+1, v, p);
                    */
                }
                else {
                    /*@ assert match: a[i] == v;
                    /*@ assert match: AllEqual(a, start, i+1, v);
                    if (p == i + 1u - start) {
                        /*@ assert bound: start + p == i + 1;
                        /*@ assert match: AllEqual(a, start, start+p, v);
                        /*@ assert match: \exists integer k;
                                0 <= k <= n-p && AllEqual(a, k, k+p, v);
                        */
                        /*@ assert has_match: HasConstantSubRange(a, n, v, p);
                        return start;
                    }
                    else {
                        /*@ assert bound: i + 1 < start + p;
                        continue;
                    }
                }

                /*@ assert no_match: !HasConstantSubRange(a, i+1, v, p);
                */
            }

            /*@ assert no_match: !HasConstantSubRange(a, n, v, p);
            return n;
            */
        }
        else {
            /*@ assert bound: n < p;
            /*@ assert no_match: !HasConstantSubRange(a, n, v, p);
            return n;
            */
        }
    }
    else {
        /*@ assert bound: p == 0;
        /*@ assert match: AllEqual(a, 0, 0, v);
        /*@ assert has_match: HasConstantSubRange(a, n, v, 0);
        return 0u;
        */
    }
}

```

Listing 4.49: Implementation of search_n

4.12. The `find_end` algorithm

The algorithm `find_end`, as specified in the C++ Standard Library [15, §28.5.6], searches for the *last* occurrence of a given sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we specialize the generic formulation to arrays of type `value_type`. The resulting function signature is as follows:

```
size_type  
find_end(const value_type* a, size_type n,  
         const value_type* b, size_type p);
```

The function `find_end` returns the greatest index s such that $a[s+k] == b[k]$ holds for all indices k with $0 \leq k < p$. In other words, s denotes the starting position of the last occurrence of the subarray $b[0 \dots p)$ within $a[0 \dots n)$ (see Figure 4.50). If no such index exists, the function returns the value n .

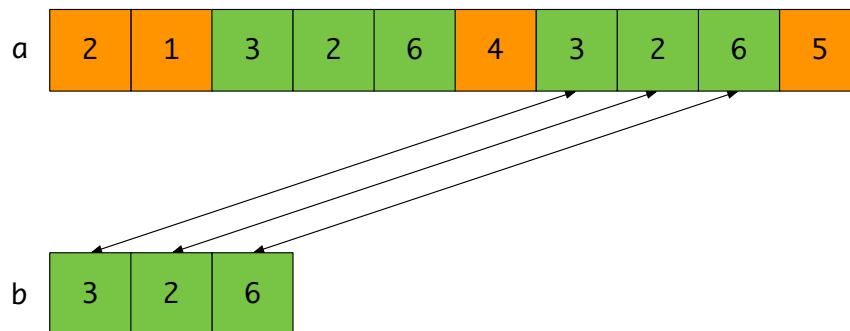


Figure 4.50.: A simple example of `find_end`

A special case arises when $p == 0$. In this situation, the empty sequence is considered to occur at the end of the array, and the function therefore returns n .

4.12.1. Formal specification of find_end

Listing 4.51 shows the formal specification of find_end [4.51]. From a conceptual point of view, the contract of find_end is very similar to that of find2 [4.5]. As before, we use behaviors to capture the essential cases of the algorithm. These behaviors are both *complete* and *disjoint*.

```
/*@
requires    valid:    \valid_read(a + (0..n-1));
requires    valid:    \valid_read(b + (0..p-1));

terminates          \true;
exits               \false;
assigns             \nothing;

ensures    result:    0 <= \result <= n;

behavior has_match:
  assumes      HasSubRange(a, n, b, p);
  assigns      \nothing;
  ensures bound:    0 <= \result <= n-p;
  ensures result:  Equal{Here,Here}(a + \result, p, b);
  ensures last:    !HasSubRange(a, \result + 1, n, b, p);

behavior no_match:
  assumes      !HasSubRange(a, n, b, p);
  assigns      \nothing;
  ensures result: \result == n;

complete behaviors;
disjoint behaviors;
*/
size_type find_end(const value_type* a, size_type n,
                  const value_type* b, size_type p);
```

Listing 4.51: Formal specification of find_end

The behavior has_match applies when the array a contains a contiguous subsequence that is identical to b . This condition is expressed in the corresponding assumes clause using the predicate HasSubRange [4.43]. The ensures clause bound states that the return value lies in the range $[0 \dots n - p + 1)$. The result clause of behavior has_match expresses that the returned index indeed denotes a position in a where the sequence b occurs. Finally, the clause last formalizes the defining property of find_end: it ensures that there is no occurrence of b in a starting at any index strictly greater than \result.

The behavior no_match covers the complementary case in which no subsequence of a is equal to b . In this situation, find_end must return the value n , that is, the length of the array a .

4.12.2. Implementation of find_end

The implementation of `find_end` [4.52] closely follows that of `search` [4.45], with the essential difference that the last matching position is recorded rather than the first.

```
size_type find_end(const value_type* a, size_type n,
                  const value_type* b, size_type p)
{
    size_type r = n;

    if ((0u < p) && (p <= n)) {
        /*@
        loop invariant bound:      r <= n - p || r == n;
        loop invariant no_match:  r == n ==> !HasSubRange(a, p+i-1, b, p);
        loop invariant found:     r < n ==> Equal{Here,Here}(a+r, p, b);
        loop invariant last:      r < n ==> !HasSubRange(a, r+1, i+p-1, b, p);
        loop assigns i, r;
        loop variant n - i;
        */
        for (size_type i = 0u; i <= n - p; ++i) {
            if (equal(a + i, p, b)) {
                r = i;
            }
        }
    }

    return r;
}
```

Listing 4.52: Implementation of `find_end`

The variable `r` stores the prospective return value according to the current state of the search. It is initially set to `n`, indicating that no occurrence of `b` has been found so far. Whenever an occurrence is detected, `r` is updated to the corresponding starting index.

The invariant `bound` states that `r` either retains the value `n` or lies in the range $[0 \dots n - p + 1]$. In the former case, the invariant `no_match` expresses that no occurrence of `b` has been identified. In the latter case, the invariant `found` states that the subsequence $b[0 \dots p)$ occurs in `a` starting at index `r`. Finally, the invariant `last` ensures that no occurrence of `b` has been found at any index strictly greater than `r`.

4.13. The count algorithm

The count algorithm of the C++ Standard Library [15, §28.5.9] determines how often a given value occurs in a sequence. For the purposes of this chapter, we restrict attention to arrays of elements of type `value_type` and adapt the generic interface accordingly. The resulting function signature is:

```
size_type count(const value_type* a, size_type n, value_type v);
```

Given an array `a` of length `n`, the function returns the number of indices `i` with $0 \leq i < n$ for which `a[i] == v` holds.

4.13.1. The logic function Count

In order to specify `count`, we first need a formal notion of counting occurrences of a value in a range. The ACSL specification language does not provide such a notion as a primitive.⁹ We therefore introduce a definition of the *logic function* `Count` [4.53], which captures the standard intuitive behavior of counting a given value in a range. The expression `Count(a, m, n, v)` denotes the number of occurrences of `v` in the range `a[m .. n)`.

```
/*@
logic integer
Count(value_type* a, integer m, integer n, value_type v) =
  n <= m ? 0 : Count(a, m, n-1, v) + (a[n-1] == v ? 1 : 0);

logic integer
Count(value_type* a, integer n, value_type v) = Count(a, 0, n, v);

lemma Count_Empty:
  \forallall value_type *a, v, integer m, n;
    n <= m ==> Count(a, m, n, v) == 0;

lemma Count_Hit:
  \forallall value_type *a, v, integer n, m;
    m < n ==>
      a[n-1] == v ==> Count(a, m, n, v) == Count(a, m, n-1, v) + 1;

lemma Count_Miss:
  \forallall value_type *a, v, integer n, m;
    m < n ==>
      a[n-1] != v ==> Count(a, m, n, v) == Count(a, m, n-1, v);

lemma Count_One:
  \forallall value_type *a, v, integer m, n;
    m <= n ==>
      Count(a, m, n+1, v) == Count(a, m, n, v) + Count(a, n, n+1, v);

lemma Count_Single{K,L}:
  \forallall value_type *a, *b, v, integer m, n;
    \at(a[m],K) == \at(b[n],L) ==>
      Count{K}(a, m, m+1, v) == Count{L}(b, n, n+1, v);
```

Listing 4.53: The logic function Count (1)

⁹Strictly speaking, the ACSL documentation mentions the operator `numof` as part of a collection of so-called *higher-order logic constructions* [9, §2.6.7]. These *extended quantifiers*, however, have originally been described as experimental features and are therefore not used here.

- The bounds m and n , as well as the return value of `Count`, are of type `integer`.
- The logic function `Count` is defined recursively. The definition distinguishes the empty range from the non-empty case and, in the latter, inspects the value of the current array element. The recursion decreases the length of the range. An overloaded variant of `Count` is provided that takes only the array length, corresponding to the common case $m = 0$.
- Lemma `Count_Empty` [4.53] characterizes the value of `Count` on empty ranges.
- Lemmas `Count_Hit` [4.53] and `Count_Miss` express how counting over a range of length $n-m$ reduces to counting over a range of length $n-m-1$, depending on whether the last element matches the value v .
- Lemmas `Count_One` [4.53] and `Count_Single` build on `Count_Hit` and `Count_Miss`. They are convenient derived results that simplify several Coq proofs by shifting the induction scheme from $n-1 \rightarrow n$ to $n \rightarrow n+1$.

Listing 4.54 shows another group of `Count` lemmas

```

lemma Count_Equal{K,L}:
  \forallall value_type *a, v, integer m, n, p;
    0 <= m <= n ==>
    Equal{K,L}(a, m, n, p) ==>
    Count{K}(a, m, n, v) == Count{L}(a, p, p + (n-m), v);

lemma Count_Unchanged{K,L}:
  \forallall value_type *a, v, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
    Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

lemma Count_Union:
  \forallall value_type *a, v, integer k, m, n;
    0 <= k <= m <= n ==>
    Count(a, k, n, v) == Count(a, k, m, v) + Count(a, m, n, v);

lemma Count_Cut:
  \forallall value_type *a, v, integer k, m, n;
    0 <= k <= m < n ==> Count(a, k, n, v) ==
    Count(a, k, m, v) + Count(a, m, m+1, v) + Count(a, m+1, n, v);

lemma Count_Single_Bounds:
  \forallall value_type *a, v, integer n;
    0 <= Count(a, n, n+1, v) <= 1;

```

Listing 4.54: The logic function `Count` (2)

- The logic function `Count` depends only on the values in the memory locations $a[m \dots n]$. Lemma `Count_Unchanged` makes this explicit by stating that `Count` yields the same result in two program states K and L , provided that the values of $a[m \dots n]$ are unchanged. The predicate `Unchanged` is used to express this premise.
- Lemma `Count_Equal` generalizes `Count_Unchanged` to the comparison of two arrays.
- Lemmas `Count_Union` and `Count_Cut` support reasoning about partitions of ranges.

The final group of initial Count lemmas are shown in Listing 4.55.

- Lemmas Count_Single_Bounds and Count_Bounds establish lower and upper bounds for the value of Count. Lemma Count_Increasing states that Count is monotonic with respect to range extension.
- Finally, lemmas Count_Single_Shift and Count_Shift express that Count is invariant under shifts of the array index.

```

lemma Count_Bounds:
  \forallall value_type *a, v, integer m, n;
    0 <= m <= n ==> 0 <= Count(a, m, n, v) <= n-m;

lemma Count_Increasing:
  \forallall value_type *a, v, integer m, n, p;
    m <= n <= p ==> Count(a, m, n, v) <= Count(a, m, p, v);

lemma Count_Single_Shift:
  \forallall value_type *a, v, integer n;
    0 <= n ==> Count(a+n, 0, 1, v) == Count(a, n, n+1, v);

lemma Count_Shift:
  \forallall value_type *a, v, integer m, n;
    0 <= m ==> 0 <= n ==> Count(a+m, 0, n, v) == Count(a, m, m+n, v);
*/

```

Listing 4.55: The logic function Count (3)

4.13.2. Relating Count and Find

So far, the logic function Count has been treated independently of the predicates used in the specifications of searching algorithms. In those specifications, existential properties are expressed using the predicate SomeEqual [4.4], introduced earlier in the discussion of the various find variants.

Lemma Count_SomeEqual [4.56], shown below, establishes a direct connection between these two notions. It states that, for a non-empty range $[m, n)$, a strictly positive value of Count implies that SomeEqual holds on the same range. In other words, if the count of occurrences of a value v in $a[m \dots n)$ is positive, then v occurs at least once in that range.

```

/*@
lemma Count_SomeEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= m < n ==>
    0 < Count(a, m, n, v) ==>
    SomeEqual(a, m, n, v);
*/

```

Listing 4.56: CountFind (ACSL)

This lemma makes explicit a basic consistency property between counting and existence. It is stated only in this direction; the converse implication, although expected from the definition of Count, is not required here and is therefore not included.

4.13.3. Formal specification of count

The contract of `count` [4.57] is formulated using the logic function `Count` [4.53], which expresses the number of occurrences of a given value in a range. In addition, the specification explicitly states that the result returned by `count` is non-negative and does not exceed the length of the array.

```
/*@
  requires    valid:  \valid_read(a + (0..n-1));

  terminates      \true;
  exits          \false;
  assigns        \nothing;

  ensures      bound:  0 <= \result <= n;
  ensures      count:  \result == Count(a, n, v);
*/
size_type count(const value_type* a, size_type n, value_type v);
```

Listing 4.57: Formal specification of count

4.13.4. Implementation of count

The following listing shows a straightforward implementation of `count` [4.58]. One of the loop invariants refers to the logic function `Count`, thereby establishing a direct connection between the operational behavior of the loop and the abstract counting property used in the specification.

```
size_type count(const value_type* a, size_type n, value_type v)
{
    size_type counted = 0u;

    /*@
      loop invariant bound: 0 <= i <= n;
      loop invariant bound: 0 <= counted <= i;
      loop invariant count: counted == Count(a, i, v);
      loop assigns i, counted;
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] == v) {
            counted++;
        }
    }

    return counted;
}
```

Listing 4.58: Implementation of count

5. Minimum and maximum algorithms

In this chapter, we study the formal specification of algorithms of the C++ Standard Library [15, §28.7.8] that compute minimum or maximum values of their arguments. Like the algorithms discussed in Chapter 4, these algorithms do not modify any memory locations and are therefore specified uniformly using nonmutating contracts.

The algorithms considered here are characterized by their essential reliance on comparisons between values. Their specifications are formulated in terms of binary relational operators such as `<`, and the mathematical properties of these operators play a central role in the correctness arguments.

The chapter is organized to reflect this structure. We begin by introducing auxiliary notions that support precise, reusable, and modular specifications of comparison-based algorithms. We then turn to the specification and verification of the minimum and maximum algorithms provided by the library.

- We first establish the necessary foundations. §5.1 discusses basic properties that relational operators must satisfy in order to be suitable for use in minimum and maximum algorithms. §5.2 then introduces logical predicates that describe order-related properties of arrays with elements of type `value_type`, which are used throughout the remainder of the chapter.
- §5.3 introduces the type `size_type_pair` together with the function `make_pair`, providing a uniform way of representing pairs of indices in specifications. These definitions are used in particular for the specification of `minmax_element` [5.17].
- We then study individual algorithms. `clamp` (§5.4) serves as a simple introductory example that restricts a value to lie within given boundary values.
- `max_element` (§5.5) returns an index to a maximum element of an array, choosing the smallest index among all valid candidates. An alternative specification based on user-defined predicates, `max_element2`, is presented in §5.6.
- `max_seq` (§5.7) is closely related to `max_element` and illustrates modular verification. Unlike `max_element`, it returns the maximum value itself rather than an index.
- `min_element` (§5.8) computes an index to a smallest element of an array.
- The chapter concludes with `minmax_element` (§5.9), which computes both the smallest and the largest element of an array simultaneously. Its specification and implementation rely on the auxiliary function `make_pair` introduced earlier.

We begin with a discussion of relational operators in §5.1, which provides the foundation for all subsequent specifications.

5.1. A note on relational operators

Note that in order to compare values, algorithms in the C++ Standard Library [15, §28.7.8] usually rely solely on the *less than* operator `<` or on special function objects. To be precise, the operator `<` must induce a *strict partial order*, which means that the following rules must hold.¹⁰

$$\text{Irreflexivity:} \quad \forall x : \neg(x < x) \quad (5.1a)$$

$$\text{Asymmetry:} \quad \forall x, y : x < y \implies \neg(y < x) \quad (5.1b)$$

$$\text{Transitivity:} \quad \forall x, y, z : x < y \implies y < z \implies x < z \quad (5.1c)$$

We adopt the usual convention that the implication symbol $\implies$ associates to the right. Thus an expression of the form $A \implies B \implies C$ is understood as $A \implies (B \implies C)$.

If one wishes to check that the operator `<` of our `value_type` (see §2). satisfies these properties, the corresponding lemmas of [5.1] can be formulated and verified with Frama-C.

```

/*@
lemma Less_Irreflexivity:
  \forallall value_type a; !(a < a);

lemma Less_Antisymmetry:
  \forallall value_type a, b; (a < b) ==> !(b < a);

lemma Less_Transitivity:
  \forallall value_type a, b, c; (a < b) ==> (b < c) ==> (a < c);

lemma Greater_Less:
  \forallall value_type a, b; (a > b) <==> (b < a);

lemma LessOrEqual_Less:
  \forallall value_type a, b; (a <= b) <==> !(b < a);

lemma GreaterOrEqual_Less:
  \forallall value_type a, b; (a >= b) <==> !(a < b);
*/

```

Listing 5.1: LessThanComparable (ACSL)

It is possible to specify and implement the algorithms of this chapter by using only operator `<`. For example, `a <= b` can be written as `a < b || a == b`, or, for our ordering on `value_type`, as `!(b < a)`. Listing [5.1] therefore also contains lemmas on representing the operators `>`, `<=`, and `>=` in terms of operator `<`.

Remark on StrictWeakOrdering. The C++ Standard formally requires comparison predicates used by most algorithms to satisfy the concept of a *strict weak ordering*. This notion slightly relaxes that of a strict partial order by allowing distinct elements to be considered *equivalent* with respect to the ordering, that is, neither $x < y$ nor $y < x$ holds. In this chapter, we restrict attention to orderings that are strict partial orders. This simplification is sufficient for the algorithms considered here and avoids the additional technical overhead associated with equivalence classes induced by weak orderings.

¹⁰See http://en.wikipedia.org/wiki/Partially_ordered_set

5.2. Predicates for bounds and extrema of arrays

We begin by introducing predicates that express how all elements of an range compare to a given value. The predicates `LowerBound` [5.2] and `UpperBound`, together with their strict variants `StrictLowerBound` and `StrictUpperBound`, are shown in the following listing.

```
/*@
  predicate LowerBound{L}(value_type* a, integer m, integer n, value_type v) =
    \forall integer i; m <= i < n ==> v <= a[i];

  predicate LowerBound{L}(value_type* a, integer n, value_type v) =
    LowerBound{L}(a, 0, n, v);

  predicate StrictLowerBound{L}(value_type* a, integer m, integer n, value_type v)
    =
    \forall integer i; m <= i < n ==> v < a[i];

  predicate StrictLowerBound{L}(value_type* a, integer n, value_type v) =
    StrictLowerBound{L}(a, 0, n, v);

  predicate UpperBound{L}(value_type* a, integer m, integer n, value_type v) =
    \forall integer i; m <= i < n ==> a[i] <= v;

  predicate UpperBound{L}(value_type* a, integer n, value_type v) =
    UpperBound{L}(a, 0, n, v);

  predicate StrictUpperBound{L}(value_type* a, integer m, integer n, value_type v)
    =
    \forall integer i; m <= i < n ==> a[i] < v;

  predicate StrictUpperBound{L}(value_type* a, integer n, value_type v) =
    StrictUpperBound{L}(a, 0, n, v);
*/
```

Listing 5.2: ArrayBounds (ACSL)

These predicates provide a concise and uniform way to state global ordering properties of ranges. They will be used extensively throughout this chapter and again in Chapter 6.

Using these bounds predicates as building blocks, we now define predicates that characterize extrema of ranges. An element is a minimum (respectively maximum) if it constitutes an lower (respectively upper) bound for all elements in the range. This idea is captured by the predicates `MaxElement` [5.3] and `MinElement` [5.3] shown below.

```
/*@
  predicate MaxElement{L}(value_type* a, integer n, integer max) =
    0 <= max < n && UpperBound(a, n, a[max]);

  predicate MinElement{L}(value_type* a, integer n, integer min) =
    0 <= min < n && LowerBound(a, n, a[min]);
*/
```

Listing 5.3: ArrayExtrema (ACSL)

We will discuss these extrema predicates in more detail in §5.6 and §5.8.

5.3. The auxiliary function `make_pair`

In several specifications we need to reason about *pairs of indices* rather than individual positions. For this purpose, and purely as a technical convenience, we introduce a small auxiliary data type. This construction is independent of any particular algorithm and, in particular, is *not* conceptually related to algorithms computing minimum or maximum elements.

The following listing introduces the type `size_type_pair`, which simply bundles two values of type `size_type` into a single object. It serves as a lightweight vehicle for passing index pairs through specifications and implementations.

```
struct size_type_pair {  
    size_type first;  
    size_type second;  
};  
  
typedef struct size_type_pair size_type_pair;
```

Listing 5.4: The auxiliary type `size_type_pair`

To construct values of type `size_type_pair`, we provide the auxiliary function `make_pair`, which combines two indices `first` and `second` into a single object. The function serves purely as a technical convenience. The formal specification of `make_pair` [5.5] is shown in the following listing.

```
/*@  
    terminates      \true;  
    exits           \false;  
    assigns         \nothing;  
  
    ensures   result: \result.first == first;  
    ensures   result: \result.second == second;  
*/  
size_type_pair make_pair(size_type first, size_type second);
```

Listing 5.5: Formal specification of `make_pair`

The next listing gives a straightforward implementation that directly reflects the specification.

```
size_type_pair make_pair(size_type first, size_type second)  
{  
    size_type_pair pair = {first, second};  
    return pair;  
}
```

Listing 5.6: Implementation of `make_pair`

5.4. The clamp algorithm

The clamp algorithm of the C++ Standard Library [15, §28.7.9] restricts a value to a given closed interval. If the value of v is within the interval $[lower, upper]$, then `clamp` returns v ; otherwise returns the nearest boundary. The standard also states that if `upper` is less than `lower`, then the behaviour of `clamp` is undefined. The signature of our version of `clamp` is as follows:

```
value_type clamp(value_type v, value_type lower, value_type upper);
```

More formally, the specification of `clamp` can be stated as follows:

- If $lower \leq v \leq upper$, the function returns v .
- If $v < lower$, the function returns `lower`.
- If $v > upper$, the function returns `upper`.

5.4.1. Formal specification of clamp

The following listing shows the formal specification of `clamp` [5.7]. As a precondition, we require that the lower bound is strictly smaller than the upper bound, that is, $lower < upper$. This condition excludes degenerate or ill-formed intervals and reflects the intended use of `clamp` as an operation on a proper closed interval.

```
/*@
  requires    bound:  lower < upper;

  terminates    \true;
  exits        \false;
  assigns      \nothing;

  ensures    bound:  lower <= \result <= upper;

  behavior lower_bound:
    assumes    v < lower;
    ensures    result: \result == lower;

  behavior between:
    assumes    lower <= v <= upper;
    ensures    result: \result == v;

  behavior upper_bound:
    assumes    upper < v;
    ensures    result: \result == upper;

  complete behaviors;
  disjoint behaviors;
*/
value_type clamp(value_type v, value_type lower, value_type upper);
```

Listing 5.7: Formal specification of `clamp`

The function is specified as terminating normally, without exceptional exits, and as having no side effects. In particular, the `assigns` clause expresses that `clamp` is a pure function whose result depends solely on its input parameters.

The global postcondition states that the returned value always lies within the interval defined by the bounds. Formally, the result satisfies the inequality $\text{lower} \leq \text{result} \leq \text{upper}$, independently of the relative position of the input value v . The specification then refines this general guarantee by distinguishing three exclusive behaviors that cover all possible cases.

5.4.2. Implementation of `clamp`

The implementation of `clamp` [5.8] is straightforward. Due to its simple control flow, the correctness of the implementation can be verified without introducing any additional annotations.

```
value_type clamp(value_type v, value_type lower, value_type upper)
{
    return (v < lower) ? lower : (upper < v) ? upper : v;
}
```

Listing 5.8: Implementation of `clamp`

5.5. The max_element algorithm

The max_element algorithm of the C++ Standard Library [15, §28.7.8] determines the position of a maximal element in a finite sequence. The signature of our version of max_element is

```
size_type max_element(const value_type* a, size_type n);
```

The algorithm distinguishes two conceptually simple cases. If the array $a[0 \dots n)$ is empty ($n == 0$), no maximum exists and the function returns the sentinel value n . Otherwise ($n > 0$), the algorithm scans the array from left to right and determines the first occurrence of a maximum value.

5.5.1. Formal specification of max_element

The following listing shows the formal ACSL specification of max_element.

```
/*@
  requires    valid:  \valid_read(a + (0..n-1));

  terminates  \true;
  exits      \false;
  assigns     \nothing;

  ensures     result:  0 <= \result <= n;

  behavior empty:
    assumes   n == 0;
    assigns   \nothing;
    ensures   result: \result == 0;

  behavior not_empty:
    assumes   0 < n;
    assigns   \nothing;
    ensures   result: 0 <= \result < n;
    ensures   upper:  \forall integer i; 0 <= i < n      ==> a[i] <= a[\result];
    ensures   first:  \forall integer i; 0 <= i < \result ==> a[i] < a[\result];

  complete behaviors;
  disjoint behaviors;
*/
size_type max_element(const value_type* a, size_type n);
```

Listing 5.9: Formal specification of max_element

The specification distinguishes two behaviors, empty and not_empty, corresponding to the cases $n == 0$ and $n > 0$, respectively.

In behavior not_empty, the ensures clauses max and first state that the returned index refers to a maximal element and denotes its first occurrence.

5.5.2. Implementation of `max_element`

The following listing shows a straightforward implementation of `max_element` [5.10] together with its loop annotations.

```
size_type max_element(const value_type* a, size_type n)
{
    if (0u < n) {
        size_type max = 0u;

        /*@
        loop invariant bound:  0 <= i <= n;
        loop invariant max:    0 <= max < n;
        loop invariant upper:  \forall integer k; 0 <= k < i ==> a[k] <= a[max];
        loop invariant first:  \forall integer k; 0 <= k < max ==> a[k] < a[max];
        loop assigns max, i;
        loop variant n-i;
        */
        for (size_type i = 1u; i < n; i++) {
            if (a[max] < a[i]) {
                max = i;
            }
        }

        return max;
    }

    return n;
}
```

Listing 5.10: Implementation of `max_element`

The implementation reflects the same two cases as the specification. The case $n == 0$ is handled explicitly before the loop. For $n > 0$, the loop traverses the array and maintains the index of a candidate maximum.

The loop invariant `max` anticipates the postcondition `max` of behavior `not_empty` by stating that the current candidate index refers to a maximal element of the prefix inspected so far. The invariant `upper` ensures that the candidate index remains within bounds. Finally, the invariant `first` records that the current candidate index refers to the first occurrence of the maximal value seen so far, which allows the postcondition `first` to be proved at loop termination.

5.6. The max_element2 algorithm (predicate-based specification)

In this section we present an alternative specification of the `max_element` algorithm. Instead of expressing maximality directly through quantified conditions, this variant relies on previously introduced predicates.

In particular, we make use of the predicate `UpperBound` [5.2], which expresses that a given value is greater than or equal to all elements of a given range. Its strict counterpart, `StrictUpperBound` [5.2], is used to capture the notion of a *first* maximum.

In addition, we employ the predicate `MaxElement` [5.3]. This predicate characterizes an index `max` whose associated element is an upper bound of the range $a[0 \dots n)$ and, at the same time, belongs to that range. Together, these predicates allow a more declarative formulation of the specification.

5.6.1. Formal specification of max_element2

The following listing shows the formal specification of `max_element2` [5.11].

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures    result:  0 <= \result <= n;

  behavior empty:
    assumes      n == 0;
    assigns      \nothing;
    ensures result: \result == 0;

  behavior not_empty:
    assumes      0 < n;
    assigns      \nothing;
    ensures result: 0 <= \result < n;
    ensures max:  MaxElement(a, n, \result);
    ensures first: StrictUpperBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type max_element2(const value_type* a, size_type n);
```

Listing 5.11: Formal specification of `max_element2`

The predicate `MaxElement` [5.3] is used to express that the returned index refers to a maximum element of the array. The predicate `StrictUpperBound` [5.2] is employed to specify that this index denotes the first occurrence of such a maximum.

5.6.2. Implementation of max_element2

The implementation of max_element2 [5.12] is structurally identical to that of max_element [5.10]. The difference lies solely in the loop annotations, which now make use of the predicate-based characterizations introduced above.

```
size_type max_element2(const value_type* a, size_type n)
{
    if (0u < n) {
        size_type max = 0u;

        /*@
        loop invariant bound:    0 <= i <= n;
        loop invariant max:      0 <= max < n;
        loop invariant upper:    UpperBound(a, i, a[max]);
        loop invariant first:    StrictUpperBound(a, max, a[max]);
        loop assigns max, i;
        loop variant n-i;
        */
        for (size_type i = 0u; i < n; i++) {
            if (a[max] < a[i]) {
                max = i;
            }
        }

        return max;
    }

    return n;
}
```

Listing 5.12: Implementation of max_element2

5.7. The max_seq algorithm

In this section we consider the function `max_seq` [8, Ch. 3], which is closely related to `max_element`. The essential difference is that `max_seq` returns the maximum value itself, rather than its position. Consequently, `max_seq` requires a *non-empty* input array.

The function `max_seq` can easily be implemented using `max_element2` [5.12]. Moreover, relying solely on the formal specification of `max_element2` [5.11], we can deductively verify the correctness of this implementation. This provides a simple example of *modular verification* in the following sense:

Any implementation of `max_element2` that is separately proven to satisfy the contract `max_element2` [5.11] guarantees the correctness of `max_seq`. Once the contracts have been defined, the function `max_element2` may be implemented independently of `max_seq`, without affecting its verification.

5.7.1. Formal specification of max_seq

The following listing shows the formal specification of `max_seq` [5.13].

```
/*@
  requires    0 < n;
  requires    \valid_read(p + (0..n-1));

  terminates  \true;
  exits       \false;
  assigns     \nothing;

  ensures     \forall integer i; 0 <= i <= n-1 ==> \result >= p[i];
  ensures     \exists integer e; 0 <= e <= n-1 && \result == p[e];
*/
value_type max_seq(const value_type* p, size_type n);
```

Listing 5.13: Formal specification of `max_seq`

The first `requires`-clause expresses that `max_seq` expects a *non-empty* array. The postconditions formalize that the returned value is indeed a maximum of the array.

5.7.2. Implementation of max_seq

The implementation of `max_seq` [5.14] consists of a single call to `max_element2` [5.12]. Since `max_seq` requires a non-empty array, this call returns an index of a maximum element. The fact that `max_element2` returns the smallest such index is irrelevant in this context.

```
value_type max_seq(const value_type* p, size_type n)
{
  return p[max_element2(p, n)];
}
```

Listing 5.14: Implementation of `max_seq`

5.8. The min_element algorithm

The `min_element` algorithm of the C++ Standard Library [15, §28.7.8] determines the position of a minimal element in a finite sequence. The signature of our version of `min_element` is

```
size_type min_element(const value_type* a, size_type n);
```

The function operates on the array $a[0 \dots n)$. If the array is non-empty, it returns the unique index i such that $a[i]$ is minimal among all its elements and i denotes the first position with this property. If the array is empty ($n == 0$), the function returns n .

Conceptually, `min_element` is the natural dual of the `max_element` algorithm. Accordingly, its specification and implementation closely follow the predicate-based variant of `max_element` discussed in §5.6.

5.8.1. Formal specification of min_element

The following listing shows the formal specification of `min_element` [5.15].

```
/*@
requires    valid:  \valid_read(a + (0..n-1));

terminates      \true;
exits           \false;
assigns         \nothing;

ensures    result:  0 <= \result <= n;

behavior empty:
  assumes      n == 0;
  assigns      \nothing;
  ensures result: \result == 0;

behavior not_empty:
  assumes      0 < n;
  assigns      \nothing;
  ensures result: 0 <= \result < n;
  ensures min:  MinElement(a, n, \result);
  ensures first: StrictLowerBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type min_element(const value_type* a, size_type n);
```

Listing 5.15: Formal specification of `min_element`

As in the predicate-based specification of `max_element`, the contract is formulated using previously introduced predicates. The predicate `LowerBound` [5.2] expresses that a given value is less than or equal to all elements of a range, while `StrictLowerBound` [5.2] is used to characterize the first occurrence of a minimum. The predicate `MinElement` [5.3] combines these properties with the requirement that the value occurs in the range.

5.8.2. Implementation of min_element

The implementation of `min_element` [5.16] closely follows that of `max_element` [5.10]. Its loop annotations make systematic use of the predicates `LowerBound` [5.2] and `StrictLowerBound` [5.2] to establish the postconditions of the specification.

```
size_type min_element(const value_type* a, size_type n)
{
    if (0u < n) {
        size_type min = 0u;

        /*@
        loop invariant bound:  0 <= i    <= n;
        loop invariant min:    0 <= min <  n;
        loop invariant lower:  LowerBound(a, i, a[min]);
        loop invariant first:  StrictLowerBound(a, min, a[min]);
        loop assigns min, i;
        loop variant n-i;
        */
        for (size_type i = 0u; i < n; i++) {
            if (a[i] < a[min]) {
                min = i;
            }
        }

        return min;
    }

    return n;
}
```

Listing 5.16: Implementation of `min_element`

5.9. The minmax_element algorithm

The `minmax_element` algorithm of the C++ Standard Library [15, §28.7.8] determines both a minimum and a maximum element in a finite sequence. The signature of our version of `minmax_element` is

```
size_type_pair minmax_element(const value_type* a, size_type n);
```

The function returns a pair of indices (see §5.3). The first component denotes the first position at which a minimum value occurs in the range $a[0 \dots n)$, while the second component denotes the last position at which a maximum value occurs.

The properties of the index identifying the minimum are identical to those of `min_element` [5.15]. By contrast, the properties of the index identifying the maximum differ slightly from those specified for `max_element2` [5.11]. While `max_element` returns the first occurrence of a maximum value, `minmax_element` returns the last occurrence of such a value.

5.9.1. Formal specification of minmax_element

The following listing shows the formal ACSL specification of `minmax_element` [5.17].

```
/*@
  requires    valid:  \valid_read(a + (0..n-1));

  terminates      \true;
  exits          \false;
  assigns        \nothing;

  ensures      result:  0 <= \result.first  <= n;
  ensures      result:  0 <= \result.second <= n;

  behavior empty:
    assumes      0 == n;
    assigns      \nothing;
    ensures      result: \result.first == 0;
    ensures      result: \result.second == 0;

  behavior not_empty:
    assumes      0 < n;
    assigns      \nothing;
    ensures      result: 0 <= \result.first < n;
    ensures      result: 0 <= \result.second < n;

    ensures      min:    MinElement(a, n, \result.first);
    ensures      first:  StrictLowerBound(a, \result.first, a[\result.first]);
    ensures      max:    MaxElement(a, n, \result.second);
    ensures      last:   StrictUpperBound(a, \result.second+1, n, a[\result.second]);
*/
size_type_pair minmax_element(const value_type* a, size_type n);
```

Listing 5.17: Formal specification of `minmax_element`

The specification combines the ideas used for `min_element` and `max_element`. The predicates `MinElement` [5.3] and `MaxElement` [5.3] are used to characterize minimality and maximality, respectively. The predicate `StrictLowerBound` [5.2] specifies that the minimum index refers to the first occurrence of a minimum value, while `StrictUpperBound` [5.2] is adapted to express that the returned maximum index denotes the last occurrence of a maximum value.

Accordingly, the postcondition `last` differs from the corresponding postcondition of `max_element`: it states that no element at or after the returned index is greater than or equal to the maximum value.

5.9.2. Implementation of `minmax_element`

The implementation of `minmax_element` [5.18] uses the auxiliary function `make_pair` [5.5] to construct the result. Apart from this, it closely follows the structure of the implementations of `min_element` and `max_element`.

```

size_type_pair minmax_element(const value_type* a, size_type n)
{
    if (0u < n) {
        size_type min = 0u;
        size_type max = 0u;

        /*@
        loop invariant bound: 0 <= i < n;
        loop invariant min:  0 <= min < n;
        loop invariant max:  0 <= max < n;
        loop invariant lower: LowerBound(a, i, a[min]);
        loop invariant upper: UpperBound(a, i, a[max]);
        loop invariant first: StrictLowerBound(a, min, a[min]);
        loop invariant last:  StrictUpperBound(a, max+1, i, a[max]);
        loop assigns min, max, i;

        loop variant n-i;
        */
        for (size_type i = 0u; i < n; i++) {
            if (a[i] >= a[max]) {
                max = i;
            }

            if (a[i] < a[min]) {
                min = i;
            }
        }

        return make_pair(min, max);
    }

    return make_pair(n, n);
}

```

Listing 5.18: Implementation of `minmax_element`

We focus on the loop invariant `last`, which is the only invariant that differs from the earlier implementations. It reflects the modified use of the predicate `StrictUpperBound` [5.2], whose range is adjusted to capture the fact that the algorithm returns the last occurrence of a maximum value.

6. Binary search algorithms

In this chapter, we study the formal specification of the binary search algorithms of the C++ Standard Library [15, §28.7.3]. These algorithms determine whether a given value is contained in a sorted array, and, if so, identify positions at which it may occur.

As in the case of the maximum and minimum algorithms discussed in Chapter 5, the binary search algorithms rely essentially on comparisons between values. Their specifications are formulated primarily in terms of the less-than operator `<` and the relational operators derived from it, while the equality operator `==` plays only a supporting role.

A distinctive aspect of binary search is that its correctness depends crucially on structural assumptions about the input array. In particular, the array is required to be ordered, and this monotonicity assumption is made explicit in the form of logical predicates introduced in §6.1 and §6.2. These predicates provide the foundation for the specifications and proofs of the binary search algorithms considered in this chapter.

The chapter then turns to the specification and verification of the individual binary search algorithms provided by the library.

- `lower_bound` in §6.3 computes the first position at which a given value could be inserted without violating the ordering of the array.
- `upper_bound` in §6.4 computes the first position at which a value strictly greater than the given one occurs.
- `equal_range`, discussed in §6.5, is presented in two implementation variants that illustrate alternative ways of combining `lower_bound` and `upper_bound`.
- `binary_search` in §6.6 decides whether a given value is contained in the array.

6.1. The predicate `Increasing`

We begin by introducing a logic predicate that expresses an ordering property of ranges. This predicate abstracts a recurring universal quantification pattern over array indices and allows us to state ordering assumptions in a concise and uniform manner. It serves as a foundational precondition for the formal specification of all binary search algorithms discussed in this chapter.

The following listing introduces the overloaded predicate `Increasing` [6.1] together with the lemma `Increasing_Shrink` [6.1].

```

/*@
predicate Increasing{L}(value_type* a, integer m, integer n) =
  \forall integer i, j; m <= i < j < n ==> a[i] <= a[j];

predicate Increasing{L}(value_type* a, integer n) = Increasing{L}(a, 0, n);

lemma Increasing_Shrink{L}:
  \forall value_type *a, integer m, n;
    0 <= m <= n ==> Increasing(a, n) ==> Increasing(a, m);
*/

```

Listing 6.1: Increasing (ACSL)

This predicate characterizes ranges or entire arrays whose elements are *increasing*, in the sense that they are ordered with respect to the non-strict order $\leq$. By convention, order properties based on the strict relation $<$ are referred to as *strictly increasing*.

More precisely, the predicate `Increasing` requires that, for all indices within the specified range, each element is smaller than or equal to any element at a greater index. This captures the notion of an increasing range at the logic level. In this chapter, the predicate `Increasing` is used primarily as a *precondition* that formalizes the ordering assumptions under which binary search algorithms operate. In later chapters, for example in the specification of `sort_heap` [9.40], the same predicate appears as an essential *postcondition*, expressing that an algorithm has produced an increasing array (see in particular Chapter 10).

The lemma `Increasing_Shrink` captures a simple but frequently used reasoning principle: the increasing property survives a restriction of the index range, so that every prefix of an increasing array is itself increasing. It is not needed for the binary search algorithms of this chapter, but it is what allows a contract to deny the increasing property of a single prefix and leave the conclusion for all longer ones to the caller, as the specification of `is_sorted_until` [10.2] does.

6.2. Global vs. local monotonicity

In the verification of algorithms that construct or transform arrays, it is often inconvenient to maintain global ordering properties of the form expressed by the predicate `Increasing` as loop invariants. Instead, such algorithms typically establish ordering properties through local comparisons between neighbouring elements.

For this reason, we introduce the predicate `WeaklyIncreasing` [6.2], which provides a weaker, neighbour-based formulation of monotonicity. This predicate is particularly well suited for use in assertions and loop invariants, where it allows us to express intermediate ordering properties that arise naturally during algorithm execution.

```

/*@
predicate WeaklyIncreasing{L}(value_type* a, integer m, integer n) =
  \forall integer i; m <= i < n-1 ==> a[i] <= a[i+1];

predicate WeaklyIncreasing{L}(value_type* a, integer n) = WeaklyIncreasing{L}(a,
  0, n);
*/

```

Listing 6.2: WeaklyIncreasing (ACSL)

Users inexperienced in formal verification often underestimate the difference between `Increasing` [6.1] and `WeaklyIncreasing` [6.2]. Although both predicates characterize non-decreasing order of ranges, they

differ in the scope of the comparisons they express: `WeaklyIncreasing` only compares direct neighbours, whereas `Increasing` relates all pairs of elements at increasing indices.

As a consequence, `Increasing` immediately implies `WeaklyIncreasing`, while the converse implication typically requires an induction over the length of the range. Such reasoning relies on the transitivity of the operator `<=` and therefore exceeds the capabilities of automated provers. For this reason, both directions of the relationship between `Increasing` and `WeaklyIncreasing` are made explicit by dedicated auxiliary lemmas (see, in particular, `Increasing_WeaklyIncreasing` [6.3] and `WeaklyIncreasing_Increasing`).

In addition to relating the predicates `Increasing` and `WeaklyIncreasing`, the lemma collection also contains auxiliary results that support the reuse of `Increasing` under common transformations. The lemma `Increasing_Shift` expresses that the increasing property is preserved under shifts of the index range, which is essential when reasoning about subranges addressed via pointer arithmetic. Its converse, `Increasing_Unshift`, restates in the coordinates of the whole array a property established for a shifted subrange, and is needed wherever the postcondition of a call on such a subrange has to be combined with what is known about the rest of the array. The lemma `Increasing_Equal` captures the fact that equality of array elements at corresponding positions preserves the increasing property, and is used to transfer ordering assumptions between logically equivalent ranges. Both lemmas also play an essential role in the verification of sorting algorithms, where ordering properties must be maintained and propagated through successive transformations of the array.

```

/*@
  lemma Increasing_Shift{L}:
    \forallall value_type *a, integer l, r;
      0 <= l <= r ==>
      Increasing{L}(a, l, r) ==>
      Increasing{L}(a+l, r-l);

  lemma Increasing_Unshift{L}:
    \forallall value_type *a, integer p, q, r;
      0 <= p ==> 0 <= q <= r ==>
      Increasing{L}(a+p, q, r) ==>
      Increasing{L}(a, p+q, p+r);

  lemma Increasing_Equal{K,L}:
    \forallall value_type* a, integer m, n, p;
      Increasing{K}(a, m, n) ==>
      Equal{K,L}(a, m, n, m+p) ==>
      Increasing{L}(a, m+p, n+p);

  lemma WeaklyIncreasing_Increasing{L}:
    \forallall value_type* a, integer m, n;
      0 <= m <= n ==>
      WeaklyIncreasing(a, m, n) ==>
      Increasing(a, m, n);

  lemma Increasing_WeaklyIncreasing{L}:
    \forallall value_type* a, integer m, n;
      0 <= m <= n ==>
      Increasing(a, m, n) ==>
      WeaklyIncreasing(a, m, n);
*/

```

Listing 6.3: IncreasingLemmas (ACSL)

The auxiliary lemmas provided above make the necessary reasoning steps explicit at the logic level. They are reused throughout the verification of binary search algorithms and, as discussed earlier, also play a central role in the verification of sorting algorithms.

In practice, we exploit the relationship between the predicates `Increasing` and `WeaklyIncreasing` in a systematic manner:

- The predicate `Increasing` [6.1] is used in preconditions and postconditions of function contracts, where global ordering assumptions on ranges are required.
- The predicate `WeaklyIncreasing` [6.2] is employed in assertions and loop invariants whenever we need to express that an algorithm (typically a sorting algorithm) maintains or produces a weakly increasing array through local comparisons.
- When it is necessary to conclude that a *weakly increasing* array is in fact *increasing*, we rely on the lemma `WeaklyIncreasing_Increasing` [6.3].

6.3. The lower_bound algorithm

The `lower_bound` algorithm is one of the four binary search algorithms of the C++ Standard Library [15, §28.7.3.1]. For our purposes we have modified the generic implementation to that of an array of type `value_type`. The signature now reads:

```
size_type
lower_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms `lower_bound` requires that its input array is in increasing order. The index `lb`, that `lower_bound` returns satisfies the inequality

$$0 \leq lb \leq n \quad (6.1)$$

and has the following properties for any valid index `k` of the array.

$$0 \leq k < lb \implies a[k] < v \quad (6.2)$$

$$lb \leq k < n \implies v \leq a[k] \quad (6.3)$$

Conditions (6.2) and (6.3) imply that `v` can only occur in the range $a[lb \dots n)$. In this sense `lower_bound` returns a *lower bound* for the potential indices.

As an example, we consider in Figure 6.4 an increasingly ordered array. The arrows indicate which indices are returned by `lower_bound` for a given value. Note that the index 9 points *one past the end* of the array. Values that are not contained in the array are colored in gray.

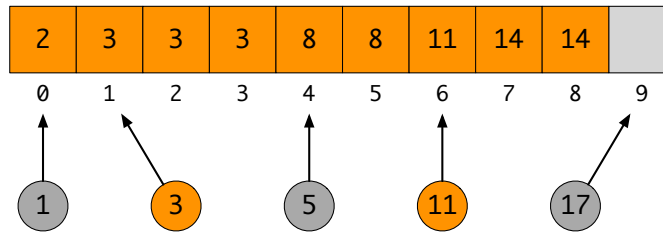


Figure 6.4.: Some examples for `lower_bound`

Figure 6.4 also clarifies that care must be taken when interpreting the return value of `lower_bound`. An important difference to the algorithms in Chapter 4 is that a return value of `lower_bound` that is less than `n` does not necessarily imply $a[lb] == v$. We can only be sure that $v \leq a[lb]$ holds.

6.3.1. Formal specification of lower_bound

The specification of `lower_bound` [6.5] is shown in the following listing. The precondition `increasing` formalizes the requirement that the array values are in increasing order. The postconditions reflect the conditions listed above and are expressed using the predicates `LowerBound` [5.2] and `StrictUpperBound` [5.2].

- Condition (6.1) is reflected by the postcondition `result`.
- Condition (6.2) is reflected by the postcondition `left`.
- Condition (6.3) is reflected by the postcondition `right`.

```

/*@
requires    valid:      \valid_read(a + (0..n-1));
requires    increasing: Increasing(a, n);

terminates          \true;
exits               \false;
assigns             \nothing;

ensures    result:      0 <= \result <= n;
ensures    left:        StrictUpperBound(a, 0, \result, v);
ensures    right:       LowerBound(a, \result, n, v);
*/
size_type lower_bound(const value_type* a, size_type n, value_type v);

```

Listing 6.5: Formal specification of `lower_bound`

6.3.2. Implementation of `lower_bound`

Our implementation of `lower_bound` [6.6] follows the standard binary search scheme and maintains a shrinking search interval `[left, right)` that is known to contain the result.

```

size_type lower_bound(const value_type* a, size_type n, value_type v)
{
    size_type left  = 0u;
    size_type right = n;

    /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   StrictUpperBound(a, 0, left, v);
    loop invariant right:  LowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
    */
    while (left < right) {
        const size_type middle = left + (right - left) / 2u;

        if (a[middle] < v) {
            left = middle + 1u;
        }
        else {
            right = middle;
        }
    }

    return left;
}

```

Listing 6.6: Implementation of `lower_bound`

Each iteration step narrows down the range that contains the sought-after result. The loop invariants express that in each iteration step all indices less than the temporary left bound `left` contain values that are less than `v` and all indices not less than the temporary right bound `right` contain values that are greater or equal than `v`. The expression to compute `middle` is slightly more complex than the naïve $(\text{left} + \text{right}) / 2$, but it avoids potential overflows.

6.4. The upper_bound algorithm

The `upper_bound` algorithm of the C++ Standard Library [15, §28.7.3.2] is a variant of binary search and is closely related to `lower_bound` [6.5]. The signature reads:

```
size_type
upper_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms, `upper_bound` requires that its input array is in increasing order. The index `ub` returned by `upper_bound` satisfies the inequality

$$0 \leq ub \leq n \quad (6.4)$$

and is involved in the following implications for any valid index `k` of the array under consideration

$$0 \leq k < ub \implies a[k] \leq v \quad (6.5)$$

$$ub \leq k < n \implies v < a[k] \quad (6.6)$$

Conditions (6.5) and (6.6) imply that `v` can only occur in the range $a[0 \dots ub)$. In this sense, `upper_bound` returns an *upper bound* for the potential indices at which `v` can occur. It also follows that the searched-for value `v` can *never* be located at the index `ub`.

Figure 6.7 is a variant of Figure 6.4 for the `upper_bound` algorithm, using the same example array. The arrows indicate which indices are returned by `upper_bound` for a given value. Compared to Figure 6.4, only the arrows corresponding to values that *are present* in the array change their target index.

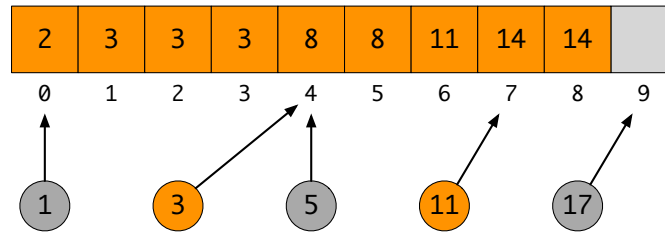


Figure 6.7.: Some examples for `upper_bound`

6.4.1. Formal specification of `upper_bound`

The following listing shows the specification of `upper_bound` [6.8], which is closely related to the specification of `lower_bound` [6.5]. The precondition `increasing` formalizes the requirement that the array values are in increasing order.

The postconditions reflect the conditions listed above and are expressed using the predicates `UpperBound` [5.2] and `StrictLowerBound` [5.2], namely,

- Condition (6.4) is reflected by the postcondition `result`.
- Condition (6.5) is reflected by the postcondition `left`.
- Condition (6.6) is reflected by the postcondition `right`.

```

/*@
requires    valid:      \valid_read(a + (0..n-1));
requires    increasing: Increasing(a, n);

terminates           \true;
exits                \false;
assigns              \nothing;

ensures    result:      0 <= \result <= n;
ensures    left:        UpperBound(a, 0, \result, v);
ensures    right:       StrictLowerBound(a, \result, n, v);
*/
size_type upper_bound(const value_type* a, size_type n, value_type v);

```

Listing 6.8: Formal specification of upper_bound

6.4.2. Implementation of upper_bound

Our implementation of upper_bound [6.9] follows the standard binary search scheme and maintains a shrinking search interval [left, right) that is known to contain the sought-after result.

```

size_type upper_bound(const value_type* a, size_type n, value_type v)
{
    size_type left  = 0u;
    size_type right = n;

    /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   UpperBound(a, 0, left, v);
    loop invariant right:  StrictLowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
    */
    while (left < right) {
        const size_type middle = left + (right - left) / 2u;

        if (a[middle] <= v) {
            left = middle + 1u;
        }
        else {
            right = middle;
        }
    }

    return right;
}

```

Listing 6.9: Implementation of upper_bound

The loop invariants express that, throughout the loop, all indices less than the temporary left bound `left` contain values not greater than `v`, and all indices not less than the temporary right bound `right` contain values greater than `v`.

6.5. The `equal_range` algorithm

The `equal_range` algorithm is one of the four binary search algorithms provided by the C++ Standard Library [15, §28.7.3.3]. As with the other binary search algorithms, `equal_range` requires that its input array is sorted in increasing order.

The specification of this algorithm in the C++ standard is particularly simple, since it explicitly states that `equal_range` combines the results of `lower_bound` [6.5] and `upper_bound` [6.8]. In C++, `equal_range` therefore returns a pair of iterators that delimit the subrange of elements equal to a given value.

As usual for our purposes, we adapt `equal_range` to operate on an array of type `value_type`. Instead of returning a pair of iterators, our version returns a pair of indices. The return type of `equal_range` is the struct `size_type_pair` shown in Listing 5.4, and its signature reads as follows:

```
size_type_pair  
equal_range(const value_type* a, size_type n, value_type v);
```

In accordance with the preceding discussion, Figure 6.10 combines Figures 6.4 and 6.7 in order to visualize the behavior of `equal_range` for selected test cases. The two types of arrows, $\rightarrow$ and $\dashrightarrow$, represent the respective fields `first` and `second` of the return value. For values that are not contained in the array, the two arrows point to the same index.

More generally, if `equal_range` returns the pair (lb, ub) , then the difference $ub - lb$ is equal to the number of occurrences of the argument v in the array.

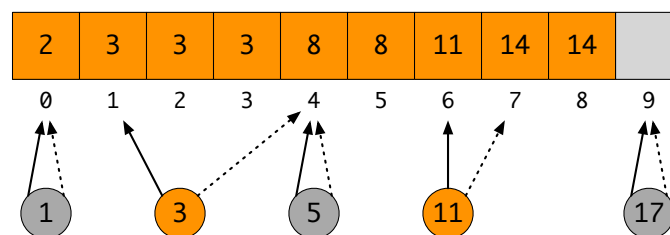


Figure 6.10.: Some examples for `equal_range`

We present two different implementations of `equal_range` and verify both of them against the *same formal specification*. Although the implementations differ substantially in structure, they are intended to be observationally equivalent.

The first implementation, `equal_range` [6.12], is deliberately straightforward: it invokes `lower_bound` and `upper_bound` separately and returns the pair consisting of their respective results. The second implementation, `equal_range2` [6.13], is more elaborate and follows an efficiency-oriented strategy by attempting to minimize duplicate computations.

6.5.1. Formal specification of `equal_range`

Let (lb, ub) be the return value of `equal_range`. Then the conditions (6.1)–(6.6) can be combined into the inequality

$$0 \leq lb \leq ub \leq n. \quad (6.7)$$

Moreover, for any valid index k of the array under consideration, the following three implications hold:

$$0 \leq k < lb \implies a[k] < v, \quad (6.8)$$

$$lb \leq k < ub \implies a[k] = v, \quad (6.9)$$

$$ub \leq k < n \implies a[k] > v. \quad (6.10)$$

Here are some justifications for these conditions.

- Conditions (6.8) and (6.10) are exactly Conditions (6.2) and (6.6), respectively.
- The inequality (6.7) follows from (6.1) and (6.4) together with the following observation. If ub were smaller than lb , then by (6.8) we would have $a[ub] < v$. On the other hand, (6.10) implies $v < a[ub]$. This contradiction shows that $lb \leq ub$ must hold.
- Condition (6.9) follows from the combination of (6.3) and (6.5), together with the fact that $\leq$ is a total order on the integers.

The following listing shows the shared ACSL specification of `equal_range` [6.11] and `equal_range2`. The precondition `increasing` expresses that the array values are required to be in increasing order.

```

/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   increasing: Increasing(a, n);

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures      result:      0 <= \result.first <= \result.second <= n;
  ensures      left:       StrictUpperBound(a, 0, \result.first, v);
  ensures      middle:    AllEqual(a, \result.first, \result.second, v);
  ensures      right:    StrictLowerBound(a, \result.second, n, v);
*/
size_type_pair equal_range(const value_type* a, size_type n, value_type v);

```

Listing 6.11: Formal specification of `equal_range`

The postconditions reflect the conditions listed above and can be expressed using the already introduced predicates `AllEqual` [4.18], `StrictUpperBound` [5.2], and `StrictLowerBound`. In particular, we observe:

- Condition (6.7) becomes postcondition `result`.
- Condition (6.8) becomes postcondition `left`.
- Condition (6.9) becomes postcondition `middle`.
- Condition (6.10) becomes postcondition `right`.

6.5.2. Implementation of `equal_range`

Our first implementation of `equal_range` [6.12] is shown in the following listing. It serves as a baseline implementation and closely mirrors the structure suggested by the specification: it invokes `lower_bound` [6.5] and `upper_bound` [6.8] independently and returns the resulting pair.

```
size_type_pair equal_range(const value_type* a, size_type n, value_type v)
{
    size_type first = lower_bound(a, n, v);
    size_type second = upper_bound(a, n, v);
    //@ assert aux: second < n ==> v < a[second];
    return make_pair(first, second);
}
```

Listing 6.12: Implementation of `equal_range`

In an early version of this document, the relation `first <= second` was established using the interactive theorem prover Coq. After revisiting this proof, we reformulated the argument by introducing the auxiliary assertion `aux`, which exploits a property already guaranteed by the postcondition of `upper_bound` [6.8]. As a result, both the assertion `aux` and the postcondition `first <= second` can now be verified fully automatically.

6.5.3. Implementation of `equal_range2`

The first implementation of `equal_range` [6.12] performs more work than necessary, since it computes the lower and upper bounds independently. The second implementation, `equal_range2` [6.13], follows a more typical and efficient C++ approach.

The algorithm first performs a binary-search-style range reduction that narrows the search interval as much as possible with respect to the argument `v`. Only once this reduced range has been determined are `upper_bound` and `lower_bound` applied to the corresponding subranges. In this way, duplicate comparisons are avoided while preserving the same observable behavior and the same formal specification.

Due to the higher code complexity of the second implementation, additional assertions had to be inserted in order to ensure that Frama-C/WP is able to verify the correctness of the code. In contrast to the first implementation, these assertions do not concern the algorithmic behavior itself, but rather properties of pointer arithmetic and shifting base pointers. All of these assertions fall into three closely related groups, which are briefly discussed after the listing of `equal_range2`.

```

size_type_pair equal_range2(const value_type* a, size_type n, value_type v)
{
    size_type first = 0u;
    size_type middle = 0u;
    size_type last = n;

    /*@
    loop invariant bound: 0 <= first <= last <= n;
    loop invariant left:  StrictUpperBound(a, 0, first, v);
    loop invariant right: StrictLowerBound(a, last, n, v);
    loop assigns first, last, middle;
    loop variant last - first;
    */
    while (last > first) {
        middle = first + (last - first) / 2u;

        if (a[middle] < v) {
            first = middle + 1u;
        }
        else if (v < a[middle]) {
            last = middle;
        }
        else {
            break;
        }
    }

    if (first < last) {
        /*@ assert increasing: Increasing(a, first, middle);
        size_type left = first + lower_bound(a + first, middle - first, v);
        /*@ assert middle: LowerBound(a, left, middle, v);
        /*@ assert left:  StrictUpperBound(a, first, left, v);
        ++middle;
        /*@ assert increasing: Increasing(a, middle, last);
        size_type right = middle + upper_bound(a + middle, last - middle, v);
        /*@ assert middle: UpperBound(a, middle, right, v);
        /*@ assert right:  StrictLowerBound(a, right, last, v);
        /*@ assert middle: AllEqual(a, left, right, v);
        return make_pair(left, right);
    }
    else {
        return make_pair(first, first);
    }
}

```

Listing 6.13: Implementation of `equal_range2`

The increasing properties. Both `lower_bound` and `upper_bound` require that they operate on increasingly ordered arrays. This requirement also applies to `equal_range`. However, in the second implementation a more specific formulation is needed for the subarrays on which these functions are invoked. In particular, using the predicate `Increasing` to write

```
Increasing(a, middle, last)
```

expresses the required ordering property for an intermediate subarray. Making this property explicit allows the automatic provers to discharge the preconditions of the call to `lower_bound`. A corresponding assertion is used before the call to `upper_bound`.

To enable the automatic verification of the corresponding proof obligations, we introduce a small collection of auxiliary lemmas, summarized under the name [6.14]. These lemmas capture general facts about array bounds under pointer shifts and can be reused independently of the present algorithm.

```

/*@
lemma LowerBound_Shift{L}:
  \forall value_type *a, v, integer b, c, d;
    LowerBound{L}(a+b, c, d, v) ==> LowerBound{L}(a, c+b, d+b, v);

lemma StrictLowerBound_Shift{L}:
  \forall value_type *a, v, integer b, c, d;
    StrictLowerBound{L}(a+b, c, d, v) ==> StrictLowerBound{L}(a, c+b, d+b, v);

lemma UpperBound_Shift{L}:
  \forall value_type *a, v, integer b, c;
    UpperBound{L}(a+b, 0, c-b, v) ==> UpperBound{L}(a, b, c, v);

lemma StrictUpperBound_Shift{L}:
  \forall value_type *a, v, integer b, c;
    StrictUpperBound{L}(a+b, 0, c-b, v) ==> StrictUpperBound{L}(a, b, c, v);
*/

```

Listing 6.14: ArrayBoundsShift (ACSL)

The strict and constant properties. Part of the postconditions of `equal_range` [6.11] is that the argument `v` is both a strict upper bound and a strict lower bound for the corresponding ranges. However, the calls to `lower_bound` and `upper_bound` only establish the following properties:

```

StrictUpperBound(a + first, 0, left - first, v)
StrictLowerBound(a + middle, right - middle, last - middle, v)

```

Taken in isolation, these properties are not sufficient to derive the required postconditions automatically. In each case, a single intermediate assertion is introduced to bridge this gap and to guide the prover towards the desired result.

Closely related to the strict properties are the constant properties, which capture bounds that are invariant with respect to the current subarray. They guide the prover towards the following assertions:

```

LowerBound(a, left, n, v)
UpperBound(a, 0, right, v)

```

Together, the strict and constant properties allow the postcondition `middle` to be derived automatically.

6.6. The `binary_search` algorithm

The `binary_search` algorithm is one of the four binary search algorithms of the C++ Standard Library [15, §28.7.3.4]. For the purposes of this chapter, we adapt the generic, iterator-based interface of the standard algorithm to operate on a contiguous array of elements of type `value_type`. Its signature therefore reads:

```
bool binary_search(const value_type* a, size_type n, value_type v);
```

The function `binary_search` requires that the input array $a[0 \dots n)$ is sorted in increasing order. It returns **true** if and only if there exists an index i such that $a[i] == v$. More precisely, the C++ Standard Library specifies this condition in terms of the ordering relation, requiring that both $a[i] \leq v$ and $v \leq a[i]$ hold. For our definition of `value_type` (see §2), this is equivalent to the equality $a[i] == v$.

The following figure illustrates the behavior of `binary_search` on several example arrays.

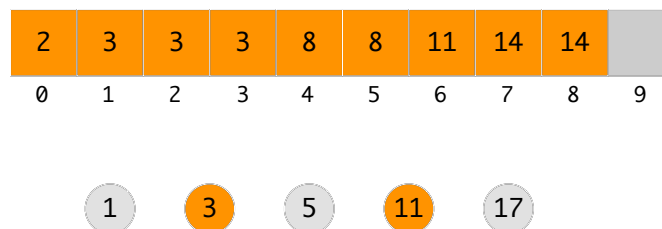


Figure 6.15.: Some examples for `binary_search`

In contrast to the other binary search algorithms discussed in this chapter, `binary_search` does not return a specific array position or range, but only a boolean result. For this reason, no arrows are required in Figure 6.15. Instead, the colors orange and grey of the sought-after values indicate whether the algorithm returns **true** or **false**, respectively.

6.6.1. Formal specification of `binary_search`

The ACSL specification of `binary_search` is shown in the following listing.

```
/*@
requires   valid:      \valid_read(a + (0..n-1));
requires   increasing: Increasing(a, n);

terminates           \true;
exits                \false;
assigns              \nothing;

ensures   result:     \result <==> SomeEqual(a, n, v);
*/
bool binary_search(const value_type* a, size_type n, value_type v);
```

Listing 6.16: Formal specification of `binary_search`

Conceptually, the result of `binary_search` must express the existence of an index at which the searched value occurs in the input array. At the level of ACSL formulas, this condition can be written as

```
\exists integer i; 0 <= i < n && a[i] == v;
```

While this formulation is logically correct, repeating such existential expressions verbatim across multiple specifications is undesirable. As discussed earlier in this report, careful *specification and proof engineering* aims at factoring out recurring logical patterns into named predicates. This improves readability, promotes uniformity across specifications, and reduces the effort required to maintain and adapt proofs.

For this reason, we reuse the predicate `SomeEqual` [4.4], which captures the above existential condition once and for all. Using this predicate leads to a more concise and robust specification of `binary_search` and serves as a further illustration of the benefits of systematic abstraction at the specification level.

It is instructive to compare the specification of `binary_search` [6.16] with that of `find2` [4.5]. At the level of functional behavior, both algorithms serve the same purpose: they determine whether a given value is contained in an array. Consequently, their ACSL specifications are closely related.

By contrast, the essential difference between the two algorithms lies in their complexity guarantees. The C++ Standard Library requires `find` to have *linear* complexity, whereas `binary_search` must have *logarithmic* complexity. Such asymptotic complexity requirements, however, are beyond the *current expressive power* of ACSL and therefore cannot be captured directly in the specification.

6.6.2. Implementation of `binary_search`

Our implementation of `binary_search` [6.17] is deliberately simple and closely follows the structure suggested by the specification. It relies on a single call to `lower_bound` [6.5] to identify a candidate position at which the searched value may occur.

```
bool binary_search(const value_type* a, size_type n, value_type v)
{
    const size_type i = lower_bound(a, n, v);
    return (i < n) && (a[i] <= v);
}
```

Listing 6.17: Implementation of `binary_search`

Recall that `lower_bound` returns the smallest index `i` such that `v <= a[i]` holds, or `n` if no such index exists. Consequently, if `lower_bound` returns an index `0 <= i < n` and, in addition, `a[i] <= v` holds, then both ordering relations `a[i] <= v` and `v <= a[i]` are satisfied, which implies that `a[i] == v`.

The implementation therefore checks whether the index returned by `lower_bound` lies within the array bounds and whether the corresponding array element is equal to the searched value. Based on this test, it returns either `true` or `false`, in accordance with the specification.

Part III.

Mutating and Numeric Algorithms

Introduction

This part is devoted to algorithms that modify program state or compute numeric results derived from the elements of a sequence. In contrast to the nonmutating algorithms studied earlier, the procedures considered here actively update memory locations or accumulate values during traversal.

Chapter 7 introduces mutating algorithms. These algorithms transform a sequence in place, for example by overwriting elements or rearranging their order. From a verification perspective, mutation requires more detailed reasoning about memory updates, frame conditions, and the preservation of unaffected data.

Chapter 8 focuses on numeric algorithms. These algorithms compute aggregated numeric values, such as sums or products, from the elements of one sequence or several sequences. Although they may or may not modify the input data, their defining characteristic is the systematic accumulation of intermediate results, which gives rise to characteristic loop invariants and correctness arguments.

Together, these chapters introduce algorithmic patterns in which state changes and numeric computation play a central role. They prepare the ground for the subsequent parts of the document, where mutation is combined with reordering and more complex structural properties of data.

7. Mutating algorithms

Let us now turn our attention to another class of algorithms, viz. *mutating* algorithms of the C++ Standard Library [15, §28.6], i.e., algorithms that change one or more arrays. In Frama-C, you can explicitly specify that, e.g., entries in an array `a` may be modified by a function `f`, by including the following *assigns clause* into the `f`'s specification:

```
assigns a[0..length-1];
```

The expression `length-1` refers to the value of `length` when `f` is entered, see [9, §2.3.2].

The sections of this chapter are organized to progressively develop techniques for specifying and verifying mutating algorithms.

- We begin with general specification mechanisms for array mutation. §7.1 introduces the predicate `Unchanged` for expressing preservation properties, while §7.2 adds the logic function `At` and the predicate `ArrayUpdate` to reason about localized updates at the level of individual array elements.
- We then study basic mutating operations. `fill` (§7.3) initializes an array, and `swap` (§7.4) exchanges two values.
- More complex algorithms are obtained by composition. `swap_ranges` (§7.5) exchanges two arrays element by element and serves as a first example of modular verification through reuse of the verified `swap` operation.
- Copying algorithms are treated next. `copy` (§7.6) and `copy_backward` (§7.7) copy arrays under different separation assumptions.
- Reordering algorithms follow. `reverse_copy` and `reverse` (§7.8, §7.9) reverse arrays, while `rotate_copy` and `rotate` (§7.10, §7.11) perform rotations.
- Replacement and filtering algorithms are then considered. `replace_copy` and `replace` (§7.12, §7.13) substitute values, whereas `remove_copy` and `remove` (§7.14–§7.17) filter values. The multiple specifications of `remove_copy` highlight the iterative nature of contract development.
- The chapter concludes with `shuffle` (§7.18), which randomly reorders an array and relies on the random number generator `random_number` (§7.19).

7.1. The predicate `Unchanged`

Many of the algorithms in this section iterate sequentially over one or several sequences. For the verification of such algorithms, it is often important to express that a range of an array, or even the complete array, has remained *unchanged*. Since this property cannot always be expressed using an `assigns` clause alone, we introduce in the following listing the overloaded predicate `Unchanged` [7.1]. The expression `Unchanged{K, L}(a, m, n)` holds if the range $a[m \dots n]$ in state K is element-wise equal to the corresponding range in state L .

```

/*@
  predicate Unchanged{K, L}(value_type* a, integer m, integer n) =
    \forallall integer i; m <= i < n ==> \at(a[i], K) == \at(a[i], L);

  predicate Unchanged{K, L}(value_type* a, integer n) = Unchanged{K, L}(a, 0, n);
*/

```

Listing 7.1: `Unchanged` (ACSL)

Using the slice notation of Equation (2.2), the predicate `Unchanged{K, L}(a, m, n)` expresses that the array slice $a[m \dots n]$ has the same values in program states K and L , that is,

$$a^K[m \dots n] = a^L[m \dots n]. \quad (7.1)$$

Although the predicate `Unchanged` is closely related to `Equal`, it serves a different purpose. The predicate `Equal` is used to express equality between two ranges, which may belong to different arrays or to the same array. By contrast, `Unchanged` always refers to the same range, but compares its values across two different program states. It therefore captures preservation across program states and can be combined naturally with operational `assigns` clauses in loop invariants and postconditions. The following example illustrates why this distinction is essential.

Operational vs. semantic preservation

Consider the following function, which explicitly writes back each element of an array:

```

void rewrite_array(value_type *a, size_type n)
{
  for (int i = 0; i < n; i++) {
    a[i] = a[i];
  }
}

```

From a semantic point of view, the array is not changed: every element in the post-state is equal to the corresponding element in the pre-state. It is therefore tempting to specify the function as follows:

```

/*@
  requires    \valid(a + (0..n-1));

  terminates  \true;
  exits      \false;
  assigns     \nothing;

  ensures     Unchanged{Pre,Post}(a,0,n);
*/
void rewrite_array(value_type* a, size_type n);

```

However, this specification is incorrect. Although the values stored in the array remain the same, the function performs write operations to memory. More generally, an `assigns` clause does not specify precisely which memory locations are modified (only which are permitted to be modified). In particular, the clause `assigns \nothing;` forbids *any* write, even if the written value coincides with the previous one.

A correct specification must therefore allow the writes explicitly:

```

/*@
  requires    \valid(a + (0..n-1));

  terminates  \true;
  exits      \false;
  assigns     a[0..n-1];

  ensures     Unchanged{Pre,Post}(a,0,n);
*/
void rewrite_array(value_type* a, size_type n);

```

The next listing shows the ACSL-annotated implementation of the correct specification. In particular:

- The two invariants `array_prefix` and `array_suffix` split the array into a prefix $a[0 \dots i)$ and a suffix $a[i \dots n)$. The invariant `array_prefix` asserts that all elements already visited by the loop have the same value as in the pre-state. The invariant `array_suffix` asserts the same for the elements not yet visited.
- The loop `assigns` clause specifies which locations the loop body is allowed to write to. Since the statement `a[i] = a[i];` performs a write, we must permit writes to (parts of) the array. At the same time, the invariants ensure that these writes do not change the values, yielding the postcondition `Unchanged{Pre,Post}(a,0,n)` after loop termination.

```

void rewrite_array(value_type* a, size_type n)
{
  /*@
    loop invariant 0u <= i <= n;
    loop invariant array_prefix: Unchanged{Pre,Here}(a, 0, i);
    loop invariant array_suffix: Unchanged{Pre,Here}(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant n - i;
  */
  for (size_type i = 0u; i < n; i++) {
    a[i] = a[i];
  }
}

```

In summary, this pair of specifications highlights an essential distinction. The `assigns` clause is an operational statement about memory writes, whereas `Unchanged [7.1]` is a semantic statement about equality of values between program states.

Lemmas for Unchanged

The following listing collects a small number of auxiliary lemmas about the predicate `Unchanged [7.1]` that are used repeatedly in the verification of subsequent algorithms.

```

/*@
lemma Unchanged_Shrink{K,L}:
  \forallall value_type *a, integer m, n, p, q;
    m <= p <= q <= n      ==>
    Unchanged{K,L}(a, m, n) ==>
    Unchanged{K,L}(a, p, q);

lemma Unchanged_Extend{K,L}:
  \forallall value_type *a, integer n;
    Unchanged{K,L}(a, n) ==>
    \at(a[n],K) == \at(a[n],L) ==>
    Unchanged{K,L}(a, n+1);

lemma Unchanged_Shift{K,L}:
  \forallall value_type *a, integer p, q, r;
    Unchanged{K,L}(a+p, q, r) ==> Unchanged{K,L}(a, p+q, p+r);

lemma Unchanged_Symmetric{K,L}:
  \forallall value_type *a, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
    Unchanged{L,K}(a, m, n);

lemma Unchanged_Transitive{K,L,M}:
  \forallall value_type *a, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
    Unchanged{L,M}(a, m, n) ==>
    Unchanged{K,M}(a, m, n);
*/

```

Listing 7.2: UnchangedLemmas (ACSL)

Using the array slice notation used in Equation (7.1) to characterize the predicate `Unchanged`, the above lemmas express the following.

- Lemma `Unchanged_Shrink [7.2]` states that equality of array slices is preserved under restriction to subranges. If the slice $a^K[m \dots n]$ coincides with $a^L[m \dots n]$, then any sub-slice $a^K[p \dots q]$ coincides with $a^L[p \dots q]$ as well, provided $0 \leq m \leq p \leq q \leq n$.
- Lemma `Unchanged_Extend` expresses that equality of adjacent slices is preserved under concatenation. If $a^K[0 \dots n] = a^L[0 \dots n]$ and $a^K[n] = a^L[n]$, then $a^K[0 \dots n+1] = a^L[0 \dots n+1]$. This property allows equality of prefixes to be established inductively.
- Lemma `Unchanged_Shift` describes how slice equality behaves under pointer arithmetic. It relates equality of the slice $(a+p)^K[q \dots r]$ to equality of the slice $(a+p)^L[q \dots r]$, or equivalently, equality of $a^K[p+q \dots p+r]$ to equality of $a^L[p+q \dots p+r]$.
- Lemmas `Unchanged_Symmetric` and `Unchanged_Transitive` establish that slice equality is symmetric and transitive with respect to program states.

7.2. Reasoning about local array updates

When updating a single element of an array, verification typically requires expressing two complementary properties. One distinguished element may change its value. All remaining elements must be shown to remain unchanged. Such properties are sometimes difficult to express directly using `assigns` clauses alone. This is particularly the case when reasoning about intermediate program points or when the relevant program state cannot be named explicitly.

To simplify comparisons of array elements across program states, we introduce the auxiliary logic function `At` [7.3]. The notation $\text{At}\{L\}(a, i)$ denotes the value of the array element $a[i]$ in the program state labeled L . Compared to the built-in notation $\text{at}(a[i], L)$, this formulation is slightly more concise. It also reads more fluently in specifications, reflecting the intended meaning “at L , consider a at index i ”.

```
/*@  
  logic value_type At{L}(value_type* x, integer i) = \at(x[i], L);  
*/
```

Listing 7.3: `At` (ACSL)

In order to describe updates of individual array elements more concisely, we also introduce the predicate `ArrayUpdate` [7.4]. This predicate relies on `Unchanged` [7.1] and uses the logic function `At`. It characterizes the effect of updating one array element while preserving all others. The accompanying lemmas `ArrayUpdate_Shrink` and `ArrayUpdate_UpperBound` are intended to support common proof patterns involving localized array updates.

```
/*@  
  predicate ArrayUpdate{K,L}(value_type* a, integer n, integer i, value_type v) =  
    0 <= i < n &&  
    Unchanged{K,L}(a, 0, i) &&  
    Unchanged{K,L}(a, i+1, n) &&  
    At{K}(a, i) != v &&  
    At{L}(a, i) == v;  
  
  lemma ArrayUpdate_Shrink{K,L}:  
    \forallall value_type *a, v, integer n, i;  
    0 <= i < n-1 ==>  
    ArrayUpdate{K,L}(a, n, i, v) ==>  
    ArrayUpdate{K,L}(a, n-1, i, v);  
  
  lemma ArrayUpdate_UpperBound{K,L}:  
    \forallall value_type *a, v, w, integer n, i;  
    ArrayUpdate{K,L}(a, n, i, v) ==>  
    v <= w ==>  
    UpperBound{K}(a, n, w) ==>  
    UpperBound{L}(a, n, w);  
*/
```

Listing 7.4: `ArrayUpdate` (ACSL)

7.3. The fill algorithm

The fill algorithm of the C++ Standard Library [15, §28.6.6] initializes a sequence by assigning a fixed value to each of its elements. The signature of the modified variant considered here is

```
void fill(value_type* a, size_type n, value_type v);
```

7.3.1. Formal specification of fill

The following listing presents the formal specification of fill [7.5]. The postcondition of fill can be expressed concisely by means of the overloaded predicate AllEqual [4.18]. Note that is postcondition could equivalently be written using an explicit universal quantification, stating

```
ensures constant: \forall i; 0 <= i < n ==> a[i] == v;
```

While this formulation is perfectly adequate, the use of the predicate AllEqual [4.18] avoids repeating this quantifier structure verbatim. More importantly, it allows the same logical concept to be reused uniformly in postconditions, loop invariants, and auxiliary lemmas, thereby keeping the specification concise and conceptually consistent.

```
/*@
  requires    valid:    \valid(a + (0..n-1));

  terminates      \true;
  exits          \false;
  assigns        a[0..n-1];

  ensures      constant: AllEqual(a, n, v);
*/
void fill(value_type* a, size_type n, value_type v);
```

Listing 7.5: Formal specification of fill

The assigns clauses specify that fill may modify only the elements of the range $a[0 \dots n)$. In general, if more than one *assigns clause* appears in a function contract, the function is permitted to modify any of the referenced memory locations. If, however, no *assigns clause* is present at all, the function is allowed to modify arbitrary memory locations, see [9, §2.3.2]. To explicitly forbid any modification outside the intended scope, an `assigns \nothing;` clause must be used, as already practised in the example specifications of Chapter 4.

7.3.2. Implementation of fill

The implementation of fill [7.6] is annotated with a loop invariant named constant. This invariant expresses that, at each iteration, the array prefix up to the current loop index *i* is entirely filled with the value *v*. Here again, the predicate AllEqual [4.18] is used to formulate this property.

```
void fill(value_type* a, size_type n, value_type v)
{
    /*@
      loop invariant bound:    0 <= i <= n;
      loop invariant constant: AllEqual(a, i, v);
      loop assigns i, a[0..n-1];
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        a[i] = v;
    }
}
```

Listing 7.6: Implementation of fill

7.4. The swap algorithm

The swap algorithm [15, § 28.6.3] in the C++ Standard Library exchanges the contents of two variables. Closely related is the algorithm `iter_swap` [15, § 28.6.3], which exchanges the contents referenced by two iterators. Since C and hence ACSL do not support C++ references, we restrict ourselves to a pointer-based variant and refer to it simply as `swap`.

7.4.1. Formal specification of swap

The contract of `swap` [7.7] is shown in the following listing. The preconditions require that both pointer arguments are valid and dereferenceable. No further assumptions are needed.

```
/*@
  requires   valid:    \valid(p);
  requires   valid:    \valid(q);

  terminates          \true;
  exits               \false;
  assigns             *p, *q;

  ensures   exchange: *p == \old(*q);
  ensures   exchange: *q == \old(*p);
*/
void swap(value_type* p, value_type* q);
```

Listing 7.7: Formal specification of swap

The postcondition specifies that the values stored at the two memory locations have been exchanged. The expression `\old(*p)` denotes the value of `*p` in the pre-state of the function call. Unless stated otherwise, postconditions are interpreted with respect to the state after normal termination of the function.

7.4.2. Implementation of swap

The following listing shows a straightforward implementation of `swap` [7.8]. It follows the conventional three-step pattern using a temporary variable.

```
void swap(value_type* p, value_type* q)
{
  value_type save = *p;
  *p = *q;
  *q = save;
}
```

Listing 7.8: Implementation of swap

No auxiliary annotations are required. The implementation satisfies the contract directly and can be verified by Frama-C/WP without additional assertions or loop invariants.

7.5. The swap_ranges algorithm

The swap_ranges algorithm in the C++ Standard Library [15, §28.6.3] exchanges the contents of two arrays element-wise. After translating C++ reference types and iterators to C, our version of the signature reads:

```
void swap_ranges(value_type* a, size_type n, value_type* b);
```

We do not return a value. In the original interface, the return value merely identifies the end position, which in our setting is determined by *n* anyway.

7.5.1. Formal specification of swap_ranges

The following listing shows a specification for swap_ranges [7.9].

```
/*@
requires   valid:  \valid(a + (0..n-1));
requires   valid:  \valid(b + (0..n-1));
requires   sep:    \separated(a+(0..n-1), b+(0..n-1));

terminates      \true;
exits           \false;
assigns         a[0..n-1];
assigns         b[0..n-1];

ensures   equal:  Equal{Old,Here}(a, n, b);
ensures   equal:  Equal{Old,Here}(b, n, a);
*/
void swap_ranges(value_type* a, size_type n, value_type* b);
```

Listing 7.9: Formal specification of swap_ranges

The algorithm is well-defined only if the two arrays do not overlap. This requirement is expressed by the precondition labeled *sep*, which states that the memory regions $a[0 \dots n)$ and $b[0 \dots n)$ are separated.

The assigns clauses specify that swap_ranges may modify exactly the two arrays $a[0 \dots n)$ and $b[0 \dots n)$, and no other memory locations.

The postconditions labeled *equal* specify the mutual exchange of the two arrays. They are formulated using the predicate *Equal* [4.31] together with the labels *Old* and *Here*. Since these formulas occur in a postcondition, *Here* refers to the post-state of swap_ranges, whereas *Old* refers to its pre-state. Together, the two equal postconditions express that each element of *a* is replaced by the corresponding pre-state element of *b*, and vice versa.

7.5.2. Implementation of swap_ranges

The implementation of swap_ranges [7.10] together with the loop annotations is shown in the following listing.

```
void swap_ranges(value_type* a, size_type n, value_type* b)
{
    /*@
        loop invariant bound:      0 <= i <= n;
        loop invariant equal:      Equal{Pre,Here}(a, i, b);
        loop invariant equal:      Equal{Pre,Here}(b, i, a);

        loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
        loop invariant unchanged:  Unchanged{Pre,Here}(b, i, n);

        loop assigns i, a[0..n-1], b[0..n-1];
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        swap(a + i, b + i);
    }
}
```

Listing 7.10: Implementation of swap_ranges

The loop repeatedly calls swap [7.7] to exchange the current pair of elements. To establish the postconditions labeled equal, the loop invariants must capture both the progress of the swap and the preservation of the untouched suffixes.

- The loop invariants labeled equal state that, at each iteration, the prefixes $a[0 \dots i)$ and $b[0 \dots i)$ have already been exchanged. They use the predicate Equal [4.31] with the labels Pre and Here, where Pre denotes the loop-entry state and Here the current state.
- In addition, the loop invariants labeled unchanged state that the suffixes $a[i \dots n)$ and $b[i \dots n)$ still coincide with their values at loop entry. This property is expressed using the predicate Unchanged.

These unchanged invariants are essential because the loop assigns clause allows modification of the full arrays $a[0 \dots n)$ and $b[0 \dots n)$. They recover the finer-grained information needed to conclude the postconditions labeled equal.

7.6. The copy algorithm

The copy algorithm in the C++ Standard Library [15, §28.6.1] implements the duplication of a finite sequence. In its generic form, the algorithm operates on iterator pairs and supports a wide range of value types. For the purposes of this presentation, we restrict attention to contiguous arrays of elements of type `value_type`. This specialization allows us to focus on the essential behaviour of the algorithm without distraction from iterator abstractions. After this simplification, the signature of copy reads:

```
void copy(const value_type* a, size_type n, value_type* b);
```

Informally, the function copies each element of the source array $a[0 \dots n)$ to the corresponding position in the destination array $b[0 \dots n)$. No other memory locations are affected. The overall effect of the algorithm is illustrated in Figure 7.11.

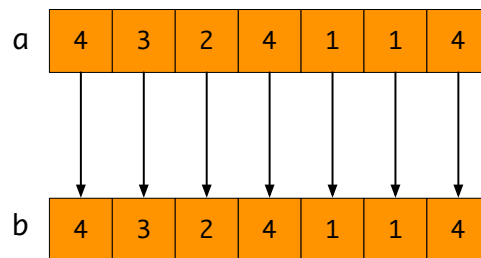


Figure 7.11.: Effects of copy

7.6.1. Formal specification of copy

The formal specification of copy is shown in the following listing. As usual, the preconditions require that the source array $a[0 \dots n)$ is valid for reading and that the destination array $b[0 \dots n)$ is valid for writing. The distinctive precondition is `sep`, which captures the non-overlapping property discussed below and rules out only those memory configurations that would invalidate the left-to-right copying semantics.

```
/*@
requires   valid:  \valid_read(a + (0..n-1));
requires   valid:  \valid(b + (0..n-1));
requires   sep:    \separated(a + (0..n-1), b);

terminates      \true;
exits           \false;
assigns         b[0..n-1];

ensures   equal:  Equal{Old,Here}(a, n, b);
*/
void copy(const value_type* a, size_type n, value_type* b);
```

Listing 7.12: Formal specification of copy

At first glance, Figure 7.11 may suggest that the source and destination arrays $a[0 \dots n)$ and $b[0 \dots n)$ must be completely disjoint. This requirement would be natural for a specification that abstracts away from the operational behaviour of the algorithm. However, the informal description of copy is more precise: elements are copied sequentially in order of increasing indices. As a consequence, a strictly weaker separation condition is sufficient. More precisely, it is only required that the destination pointer b does not point into the source array $a[0 \dots n)$. In this situation, copying from left to right cannot overwrite values that are still needed. An example of such a permitted overlap is illustrated in Figure 7.13.

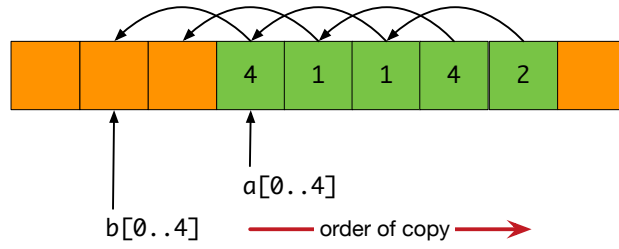


Figure 7.13.: Possible overlap of copy arrays

The postcondition expresses that, upon termination, the destination array contains an element-wise copy of the source array as it was in the pre-state. This is concisely stated using the predicate `Equal` [4.31]. The assigns-clause restricts modifications to the destination array $b[0 \dots n)$ only. In particular, no guarantee is made that the source array $a[0 \dots n)$ remains unchanged. This asymmetry is intentional and reflects the weak separation requirement discussed above: in the situation illustrated in Figure 7.13, writing to the destination array may overwrite parts of the source array without violating the intended left-to-right copying semantics.

7.6.2. Implementation of copy

The following listing shows an implementation of the copy function together with its ACSL annotations.

```

void copy(const value_type* a, size_type n, value_type* b)
{
    /*@
      loop invariant bound:      0 <= i <= n;
      loop invariant equal:      Equal{Pre,Here}(a, i, b);
      loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
      loop assigns    i, b[0..n-1];
      loop variant    n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        b[i] = a[i];
    }
}

```

Listing 7.14: Implementation of copy

In order for the postcondition `equal` to hold, it must be guaranteed that, for every index i , the value $a[i]$ is read before it can be modified by any write operation performed by the algorithm. This requirement follows directly from the left-to-right copying semantics and is essential in the presence of the weak separation condition discussed in the previous subsection.

Within the loop, this property is expressed by means of the predicate `Unchanged`, which asserts that the relevant part of the source array has not been altered at the current iteration. The invariant therefore connects the operational order of the copy with the logical requirement needed to establish the postcondition.¹¹

The function-level `assigns` clause restricts side effects to the destination array $b[0 \dots n)$ and to the loop variable i . As in the specification, no attempt is made to preserve the source array $a[0 \dots n)$. If the arrays overlap as in Figure 7.13, writes to the destination array may overwrite elements of the source array without violating either the annotations or the intended behaviour of the algorithm.

¹¹Alternatively, the same effect could be achieved by strengthening the `loop assigns` clause to `i, b[0..i-1]`. At present, however, Frama-C does not support `loop assigns` clauses that refer to the loop variable itself.

7.7. The `copy_backward` algorithm

The `copy_backward` algorithm in the C++ Standard Library [15, §28.6.1] implements a duplication algorithm that is complementary to `copy`. As before, we restrict the generic iterator-based interface to contiguous arrays of elements of type `value_type`. The resulting signature reads:

```
void copy_backward(const value_type* a, size_type n, value_type* b);
```

The primary motivation for `copy_backward` is to support copying in memory configurations that violate the separation requirement of `copy`. In particular, `copy_backward` can be applied when the beginning of the destination array $b[0 \dots n)$ lies inside the source array $a[0 \dots n)$. In such situations, the precondition `sep` of `copy` [7.12] is violated, and left-to-right copying would overwrite values that are still required.

To address this issue, the informal specification of `copy_backward` prescribes copying in reverse order, starting from the end of the source array. For this strategy to be safe, it is required that the pointer $b+n$ does not point into the source array. Figure 7.15 illustrates a typical overlap pattern in which `copy_backward`, but not `copy`, can be applied.

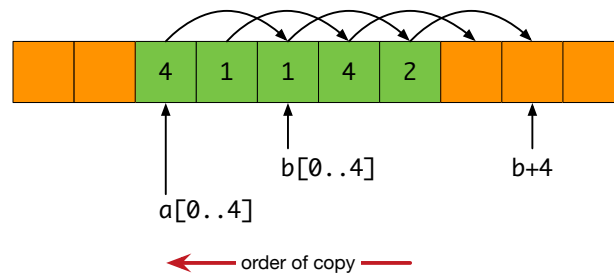


Figure 7.15.: Possible overlap of `copy_backward` arrays

In the original C++ interface, the destination iterator passed to `copy_backward` refers to one past the end of the destination array. This convention is motivated by the use of bidirectional iterators, which do not support random access. Since our C formulation operates directly on pointers, we instead let b denote the start of the destination array. This avoids the need for additional pointer arithmetic in both the specification and the implementation.

7.7.1. Formal specification of `copy_backward`

The formal specification of `copy_backward` is shown in the following listing. As in the case of `copy`, the source array $a[0 \dots n)$ must be valid for reading and the destination array $b[0 \dots n)$ must be valid for writing. The precondition `sep` captures the admissible overlap between the two arrays and reflects the reverse copying order discussed above.

```

/*@
requires    valid:  \valid_read(a + (0..n-1));
requires    valid:  \valid(b + (0..n-1));
requires    sep:    \separated(a + (0..n-1), b + n);

terminates      \true;
exits           \false;
assigns         b[0..n-1];

ensures    equal:  Equal{Old,Here}(a, n, b);
*/
void copy_backward(const value_type* a, size_type n, value_type* b);

```

Listing 7.16: Formal specification of `copy_backward`

The postcondition states that, upon termination, the destination array contains an element-wise copy of the source array as it was in the pre-state. This property is concisely expressed using the predicate `Equal` [4.31]. As with `copy`, no guarantee is made that the source array $a[0 \dots n)$ remains unchanged, since parts of it may be overwritten in the presence of overlapping arrays.

7.7.2. Implementation of `copy_backward`

The following listing shows an implementation of the `copy_backward` function together with its ACSL annotations.

```

void copy_backward(const value_type* a, size_type n, value_type* b)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant equal:      Equal{Pre,Here}(a, i, n, b);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, i);
    loop assigns i, b[0..n-1];
    loop variant i;
    */
    for (size_type i = n; i > 0u; --i) {
        b[i - 1u] = a[i - 1u];
    }
}

```

Listing 7.17: Implementation of `copy_backward`

The loop invariants are closely related to those used for `copy`. They constrain the range of the loop variable using `bound` and express which part of the destination array has already been filled using the predicate `equal`. In addition, an invariant based on `Unchanged` ensures that, at each iteration, those elements of the source array that still need to be read have not yet been modified. This invariant is essential for establishing the postcondition in the presence of overlapping arrays and reflects the right-to-left traversal order of the algorithm.

7.8. The reverse_copy algorithm

The `reverse_copy` algorithm of the C++ Standard Library [15, §28.6.10] copies the elements of a source array into a destination array such that the order of elements is reversed. The source and destination arrays are required to be non-overlapping. As in the previous sections, we restrict the generic interface to contiguous arrays of elements of type `value_type`. The resulting signature reads:

```
void reverse_copy(const value_type* a, size_type n, value_type* b);
```

Figure 7.18 illustrates the effect of applying `reverse_copy` to an array $a[0 \dots n)$, producing a reversed copy in the array $b[0 \dots n)$.

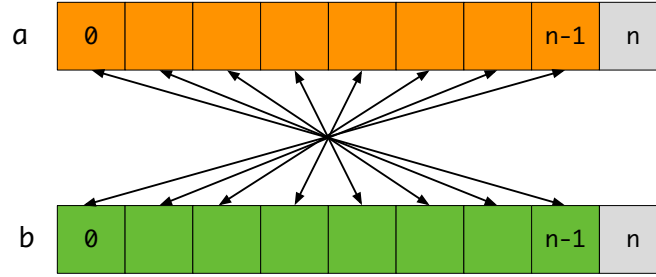


Figure 7.18.: Effects of `reverse_copy`

7.8.1. The predicate Reverse

In order to concisely formalize the reversal relation employed by the `reverse_copy` algorithm, we introduce in the following listing several overloaded versions of the predicate `Reverse` [7.20]. These variants provide convenient default parameters and allow reversal properties to be expressed succinctly in subsequent ACSL annotations.

The first predicate in this family constitutes the core definition of `Reverse`. It relates two array slices $a[m \dots n)$ and $b[q - 1 - (n - m) \dots q)$ and captures their element-wise reversal property—see Figure 7.19. In this formulation, the parameter q denotes the one-past-end index of the destination slice. Thus, both source and destination arrays are described in the same half-open form, which is convenient for reasoning in ACSL. The first index $q - 1 - (n - m)$ of the second slice is determined implicitly by the integer parameters m , n , and q .

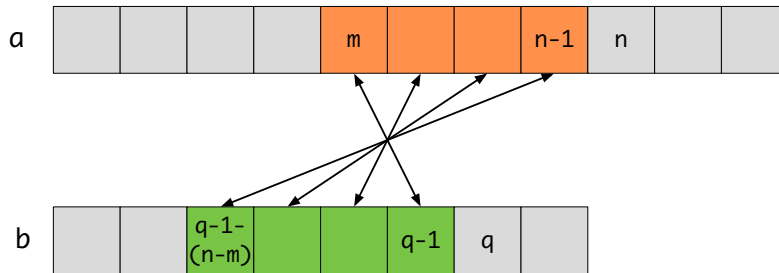


Figure 7.19.: Sketch of predicate `Reverse`

The subsequent overloaded versions of `Reverse` [7.20] are all defined in terms of this first, foundational predicate. They introduce no new semantics, but merely provide convenient default values for certain parameters.

```

/*@
  predicate Reverse{K,L}(value_type* a, integer m, integer n, value_type* b,
    integer q) =
    \forall integer i; m <= i < n ==> \at(a[i],K) == \at(b[q-1 - (i-m)],L);

  predicate Reverse{K,L}(value_type* a, integer m, integer n, value_type* b) =
    Reverse{K,L}(a, m, n, b, n);

  predicate Reverse{K,L}(value_type* a, integer n, value_type* b) =
    Reverse{K,L}(a, 0, n, b, n);

  predicate Reverse{K,L}(value_type* a, integer m, integer n, integer q) =
    Reverse{K,L}(a, m, n, a, q);

  predicate Reverse{K,L}(value_type* a, integer m, integer n) =
    Reverse{K,L}(a, m, n, n, n);

  predicate Reverse{K,L}(value_type* a, integer n) = Reverse{K,L}(a, 0, n, n);
*/

```

Listing 7.20: Reverse (ACSL)

The first three predicates cover the general case of two explicit array slices, possibly with different starting indices. The remaining three predicates specialize this relation to the frequent case in which a slice of a single array is reversed in place.

We also add `Reverse_Shift` [7.21] in the following listing. This lemma captures the semantic invariance of the `Reverse` [7.20] predicate under pointer shifts and allows us to transfer reversal properties between shifted and unshifted views of the same memory region.

```

/*@
  lemma Reverse_Shift{K,L}:
    \forall value_type* a, integer n, p;
    0 <= p <= n ==>
    (Reverse{K,L}(a + p, 0, n - p) ==> Reverse{K,L}(a, p, n));
*/

```

Listing 7.21: ReverseLemmas (ACSL)

7.8.2. Formal specification of `reverse_copy`

The formal specification of `reverse_copy` is given in the following listing. Its central postcondition `reverse` is expressed using a suitable variant of the predicate `Reverse` [7.20], which directly captures the intended element-wise reversal relation between the source and destination arrays.

The preconditions `valid` ensure that the source array is readable and that the destination array is writable for the specified number of elements. In addition, the precondition `sep` expresses that the two arrays do not overlap. This separation condition is essential, since `reverse_copy` performs the copy operation without any precautions against aliasing between source and destination.

```

/*@
requires   valid:      \valid_read(a + (0..n-1));
requires   valid:      \valid(b + (0..n-1));
requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

terminates          \true;
exits               \false;
assigns             b[0..(n-1)];

ensures   reverse:    Reverse{Old,Here}(a, n, b);
ensures   unchanged:  Unchanged{Old,Here}(a, n);
*/
void reverse_copy(const value_type* a, size_type n, value_type* b);

```

Listing 7.22: Formal specification of reverse_copy

Besides the reversal property, the specification includes a postcondition `unchanged` stating that the source array is not altered by the algorithm. Together with the `assigns` clause, this precisely characterizes the observable effects of `reverse_copy`: the destination array is written according to the reversal relation, while all other memory locations are preserved.

7.8.3. Implementation of reverse_copy

The implementation of `reverse_copy` [7.23] is shown in the following listing. The algorithm performs a single traversal of the *destination* array and, at each index `i`, assigns the value `a[n-1-i]` from the source array.

```

void reverse_copy(const value_type* a, size_type n, value_type* b)
{
    /*@
        loop invariant bound:    0 <= i <= n;
        loop invariant reverse: Reverse{Here,Pre}(b, 0, i, a, n);
        loop assigns i, b[0..n-1];
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        b[i] = a[n - 1u - i];
    }
}

```

Listing 7.23: Implementation of reverse_copy

To establish the postcondition `reverse`, the loop maintains an invariant of the same name, which employs the first (core) version of `Reverse` [7.20]. At loop counter value `i`, this invariant states that the already written prefix `b[0 .. i]` equals the reverse of the corresponding suffix `a[(n - i) .. n)` of the source array. Upon loop termination, the invariant therefore yields the desired reversal property for the full arrays.

The remaining loop annotations are standard. They ensure that index bounds are respected, restrict the set of memory locations that may be modified, and guarantee termination of the loop.

7.9. The reverse algorithm

The reverse algorithm of the C++ Standard Library [15, § 28.6.10] reverses the order of elements *in place* within a given sequence. In contrast to `reverse_copy`, the reversal is performed directly on the input array and therefore does not require a separate destination array.

As before, we restrict the generic interface to a contiguous array of elements of type `value_type`. The resulting function signature is

```
void reverse(value_type* a, size_type n);
```

Figure 7.24 illustrates the effect of applying reverse *in place* to an array slice $a[0 \dots n)$.

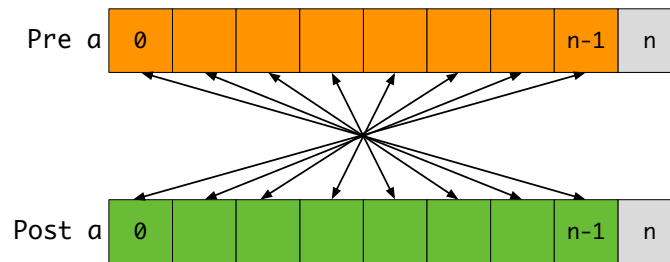


Figure 7.24.: Effect of reverse

7.9.1. Formal specification of reverse

The formal specification of reverse [7.25] is shown in the following listing.

```
/*@  
  requires    valid:    \valid(a + (0..n-1));  
  
  terminates      \true;  
  exits           \false;  
  assigns         a[0..n-1];  
  
  ensures    reverse: Reverse{Old,Here}(a, n);  
*/  
void reverse(value_type* a, size_type n);
```

Listing 7.25: Formal specification of reverse

Since reverse operates *in place*, its postcondition relates the final contents of the array to its own pre-state. This relation is expressed using an appropriate variant of the predicate Reverse [7.20], instantiated such that both ranges refer to the same array but are evaluated in different program states.

The precondition `valid` ensures that the array is valid for both reading and writing over the specified range. The `assigns` clause restricts the effects of the function to the elements of the array itself.

7.9.2. Implementation of reverse

The implementation of reverse [7.26] is shown in the following listing. Since the algorithm operates *in place*, it reverses the sequence by pairwise exchanging elements. To this end, the loop iterates over the first half of the array and invokes swap [7.7] to exchange $a[i]$ with its symmetric counterpart $a[n - 1 - i]$. Iterating beyond this point would merely undo exchanges that have already been performed.

```
void reverse(value_type* a, size_type n)
{
    const size_type half = n / 2u;

    //@ assert half:    half <= n - half;
    //@ assert half:  2*half <= n <= 2*half + 1;
    /*
        loop invariant bound:    0 <= i <= half <= n-i;
        loop invariant left:     Reverse{Pre,Here}(a, 0, i, n);
        loop invariant middle:   Unchanged{Pre,Here}(a, i, n-i);
        loop invariant right:    Reverse{Pre,Here}(a, n-i, n, i);
        loop assigns i, a[0..n-1];
        loop variant half - i;
    */
    for (size_type i = 0u; i < half; ++i) {
        swap(&a[i], &a[n - 1u - i]);
    }
}
```

Listing 7.26: Implementation of reverse

The central loop invariants *left* and *right* employ an appropriate variant of the predicate *Reverse* [7.20] to relate the already processed prefix and suffix of the array to the pre-state. At iteration counter *i*, they express that the elements in $a[0 \dots i]$ and $a[(n - i) \dots n]$ appear in reversed order with respect to their initial values. In addition, the predicate *Unchanged* [7.1] is used to state that the remaining middle range $a[i \dots (n - i)]$ has not been modified yet.

Taken together, these invariants capture the characteristic structure of the in-place reversal: the algorithm progresses symmetrically from both ends of the array, and termination occurs once the two halves meet.

7.10. The rotate_copy algorithm

The `rotate_copy` algorithm of the C++ Standard Library [15, §28.6.11] copies a sequence of length n into a separate sequence while performing a rotation by p positions. We adapt the generic iterator-based formulation to contiguous arrays of type `value_type`, yielding the signature:

```
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b);
```

The parameter p , with $0 \leq p \leq n$, denotes the rotation point. The source array $a[0 \dots n)$ is split into a prefix $a[0 \dots p)$ and a suffix $a[p \dots n)$, which are copied to $b[0 \dots n)$ in order: the suffix becomes the prefix of b , and the prefix becomes its suffix. Figure 7.27 illustrates this mapping.

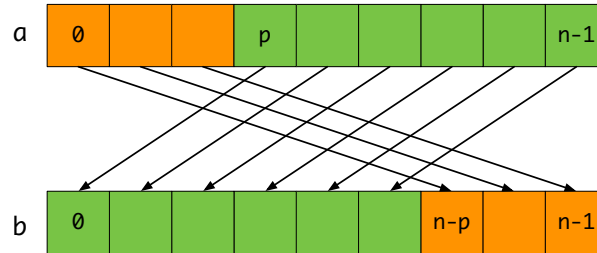


Figure 7.27.: Effects of `rotate_copy`

7.10.1. The predicate Rotate

The overloaded versions of the predicate `Rotate` in the following listing are defined in terms of appropriate overloaded versions of the predicate `Equal` [4.31]. They express that one array slice is obtained from another by a cyclic shift over a fixed index range.

```
/*@
predicate Rotate{K,L}(value_type* a, integer p, integer n, value_type* b) =
    Equal{K,L}(a, 0, p, b, n-p) &&
    Equal{K,L}(a, p, n, b, 0);

predicate Rotate{K,L}(value_type* a, integer m, integer p, integer n) =
    Equal{K,L}(a, m, p, m+n-p) &&
    Equal{K,L}(a, p, n, m);

predicate Rotate{K,L}(value_type* a, integer p, integer n) =
    Rotate{K,L}(a, 0, p, n);
*/
```

Listing 7.28: `Rotate` (ACSL)

Using the slice notation of Equation (2.2), the predicate $\text{Rotate}\{K, L\}(a, p, n, b)$ expresses that the array slice $a[0 \dots n)$ in state K is rotated by p positions to form the slice $b[0 \dots n)$ in state L . That is, the prefix of a of length p is mapped to the corresponding suffix of b , while the suffix of a starting at index p is mapped to the prefix of b . Formally, this predicate holds if and only if

$$\text{Rotate}\{K, L\}(a, p, n, b) \iff \begin{cases} a^K[0 \dots p) = b^L[(n-p) \dots n), \\ a^K[p \dots n) = b^L[0 \dots (n-p)). \end{cases} \quad (7.2)$$

The second version of `Rotate` will be explained in §7.11.

7.10.2. Formal specification of rotate_copy

The contract of rotate_copy is shown in the following listing. It formalizes the fact that the algorithm performs a pure copy operation with a well-defined reordering of elements.

```
/*@
  requires bound:      0 <= p <= n;
  requires valid:      \valid_read(a + (0..n-1));
  requires valid:      \valid(b + (0..n-1));
  requires sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns              b[0..(n-1)];

  ensures rotate:      Rotate{Old,Here}(a, p, n, b);
  ensures unchanged:  Unchanged{Old,Here}(a, n);
*/
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b);
```

Listing 7.29: Formal specification of rotate_copy

As for the other _copy algorithms discussed in this chapter, the preconditions require that the source and destination pointers are valid for arrays of length n and that the two arrays do not overlap. The assigns clause restricts all modifications to the destination array $b[0 \dots n)$, thereby formalizing that the source array $a[0 \dots n)$ is accessed in a read-only manner.

The essential effect of the algorithm is expressed by the predicate Rotate [7.28]. This predicate provides a logical characterization of rotation from one array into another and captures the complete postcondition of rotate_copy at the level of ACSL. It states that the array b represents a rotation of the array a by p positions over a range of length n , independently of any particular implementation strategy.

By using Rotate [7.28], the contract directly reflects the abstract operation performed by rotate_copy. The element-wise correspondence between source and destination arrays is encapsulated in a single predicate, which keeps the specification concise and emphasizes the conceptual intent of the algorithm rather than its low-level realization.

7.10.3. Implementation of rotate_copy

The following listing shows a straightforward implementation of the rotate_copy function. The algorithm is realized by decomposing the rotation into two applications of copy, in accordance with the prefix-suffix decomposition introduced earlier.

First, the suffix $a[p \dots n)$ is copied to the prefix $b[0 \dots (n - p))$ of the destination array. Second, the prefix $a[0 \dots p)$ is copied to the remaining suffix $b[(n - p) \dots n)$. Together, these two copy operations establish exactly the rotation of the two arrays.

```
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b)
{
    copy(a, p, b + (n - p));
    copy(a + p, n - p, b);
}
```

Listing 7.30: Implementation of rotate_copy

The precondition that the source and destination arrays do not overlap ensures that both calls to copy satisfy their separation requirements. Moreover, since copy accesses its source array in a read-only manner and writes exclusively to its destination array, the implementation preserves the intended access discipline of rotate_copy.

From a verification perspective, the effect of the two calls to copy composes naturally into the predicate Rotate [7.28]. Rather than reasoning about individual subranges, the implementation can be shown to establish the abstract rotation property expressed by Rotate [7.28], which directly matches the postcondition of rotate_copy.

7.11. The rotate algorithm

The algorithm `rotate` is an *in-place* variant of `rotate_copy` [7.29]. As before, the elements of a sequence of length n are rotated by p positions, but in contrast to `rotate_copy`, no auxiliary destination array is used. Instead, the rearrangement is performed directly within the array $a[0 \dots n)$. As in the previous section, we adapt the generic iterator-based formulation from the C++ Standard Library [15, §28.6.11] to contiguous arrays of objects of type `value_type`. The resulting signature reads:

```
size_type rotate(value_type* a, size_type p, size_type n);
```

The parameter p again denotes the rotation point and satisfies $0 \leq p \leq n$. Conceptually, the array is decomposed into a prefix $a[0 \dots p)$ and a suffix $a[p \dots n)$, which exchange their relative positions. Unlike `rotate_copy`, however, this exchange must be realized without overwriting elements that are still needed. Figure 7.31 illustrates the effect of `rotate`. The figure closely resembles that of `rotate_copy` (see Figure 7.27), but now both the prefix and the suffix are rearranged *in place* within the same array.

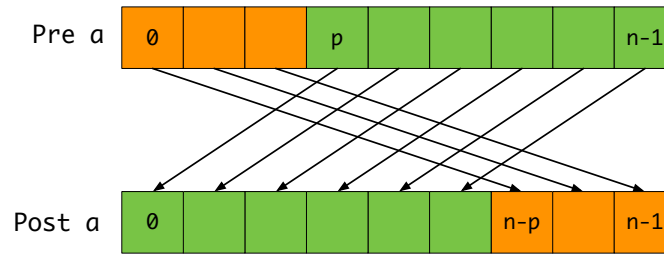


Figure 7.31.: Effects of `rotate`

7.11.1. Mathematical description of in-place rotation

In contrast to `rotate_copy`, where rotation is expressed between two distinct arrays, the specification of `rotate` relies on a second and a third version of the predicate `Rotate` [7.28], both of which refer to a single array across two program states.

1. Following the mathematical description (7.2), the second version of `Rotate` expresses a general in-place rotation. It states that the slice $a[m \dots n)$ in state K is rotated by $p - m$ positions to form the same slice $a[m \dots n)$ in state L . That is, the prefix $a[m \dots p)$ is mapped to the corresponding suffix of $a[m \dots n)$, while the suffix $a[p \dots n)$ is mapped to the corresponding prefix of $a[m \dots n)$.

Formally, this predicate holds if and only if

$$\text{Rotate}\{K, L\}(a, m, p, n) \iff \begin{cases} a^K[m \dots p) = a^L[(m + n - p) \dots n), \\ a^K[p \dots n) = a^L[m \dots (m + n - p)). \end{cases} \quad (7.3)$$

This general form will be used in the verification of `insertion_sort` [10.20].

2. The third version of the predicate `Rotate` [7.28] is the special case of the second version obtained by fixing $m = 0$. This version serves as the basis for the formal specification of `rotate`. By instantiating (7.3) with $m = 0$, we see that it expresses that the slice $a[0 \dots n)$ in state K is rotated in place by p positions to form the same slice $a[0 \dots n)$ in state L . Formally, this predicate holds if and only if

$$\text{Rotate}\{K, L\}(a, p, n) \iff \begin{cases} a^K[0 \dots p) = a^L[(n-p) \dots n), \\ a^K[p \dots n) = a^L[0 \dots (n-p)). \end{cases} \quad (7.4)$$

7.11.2. Formal specification of `rotate`

The specification of `rotate` is shown in the following listing. It specifies an in-place rotation of the array $a[0 \dots n)$ by p positions.

```

/*@
  requires   valid:   \valid(a + (0..n-1));
  requires   bound:   p <= n;

  terminates
  exits
  assigns    a[0..n-1];

  ensures    result:   \result == n-p;
  ensures    rotate:   Rotate{Old,Here}(a, p, n);
*/
size_type rotate(value_type* a, size_type p, size_type n);

```

Listing 7.32: Formal specification of `rotate`

The postconditions relate the final contents of a to its initial contents using the third version of the predicate `Rotate` [7.28], as characterized in Equation (7.4). They assert that the suffix $a[p \dots n)$ in the initial state becomes the prefix of the array in the final state, while the initial prefix $a[0 \dots p)$ becomes the final suffix.

As is typical for in-place algorithms, the preconditions require that the pointer a is valid for a range of length n and that the rotation point p satisfies $0 \leq p \leq n$. The `assigns` clause permits modification of the entire range $a[0 \dots n)$, while excluding any writes outside this range. This reflects that all elements may be moved during the rotation, but that no auxiliary memory is used.

Finally, the return value of `rotate` is specified to be $n - p$. It denotes the index of the first element of the original suffix $a[p \dots n)$ in the rotated array, in accordance with the standard interface.

7.11.3. Implementation of rotate

The following listing shows an implementation of the rotate function together with its ACSL annotations. Although there are several possible ways to implement rotate, we have chosen a well-established variant derived from an implementation of `std::rotate` for *bidirectional iterators* [15, §27.2.6]. This algorithm is concise and efficient, but its control structure is somewhat subtle and therefore benefits from a careful formal annotation.

```

size_type rotate(value_type* a, size_type p, size_type n)
{
    // if one subrange is empty, then nothings needs to be done
    if ((0u < p) && (p < n)) {
        reverse(a, p);
        reverse(a + p, n - p);
        //@ assert reverse: Reverse{Pre,Here}(a, p, n);
        reverse(a, n);
    }

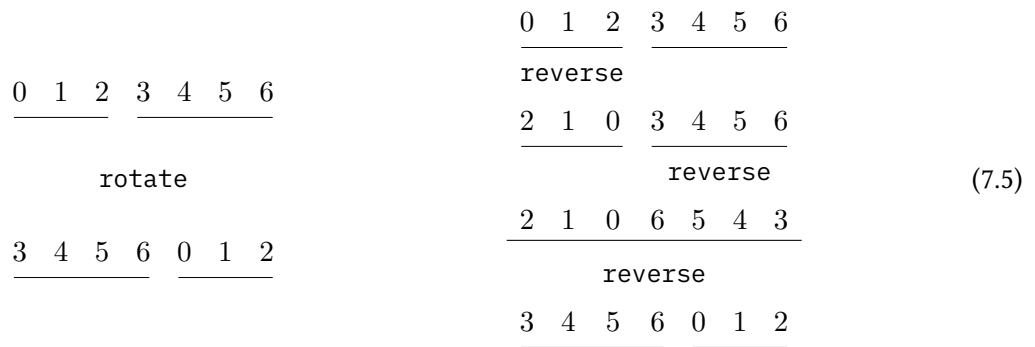
    return n - p;
}

```

Listing 7.33: Implementation of rotate

The implementation reduces the rotation to a sequence of calls to the algorithm reverse [7.25]. This classical decomposition avoids auxiliary storage and expresses the entire algorithm solely in terms of a single, already verified primitive.

The diagram in (7.5) illustrates how the implementation of rotate [7.33] operates on an array of seven elements, namely $a = \{0, 1, 2, 3, 4, 5, 6\}$. On the left-hand side, it shows the effect of calling `rotate(a, 3, 7)`. The right-hand side depicts the successive applications of reverse to the corresponding array slices. This example makes concrete how the implementation achieves the rotation by composing three reversals.



With this example in mind, we now analyze the implementation of rotate [7.33] in more detail. The following itemized discussion recalls the effect of each of the three calls to reverse [7.25] as specified by their contracts. Although each contract describes only a local transformation of the array, their combined effect is nontrivial. The automatic provers perform the necessary reasoning to combine these effects and establish that the overall behavior of the implementation satisfies the rotation property.

- First, the prefix $a[0 \dots p]$ is reversed. The contract of `reverse` [7.25] ensures that, after this call, the following assertion holds implicitly:

```
//@ assert reverse: Reverse{Pre,Here}(a, 0, p);
```

Although it is sometimes beneficial, for the sake of automated provers, to state such consequences explicitly, no additional assertion is required in this case.

- Next, the suffix $a[p \dots n]$ is reversed by calling `reverse` with a shifted address. As a consequence, the contract of `reverse` ensures only the following implicit assertion:

```
//@ assert reverse: Reverse{Pre,Here}(a+p, 0, n-p);
```

Using the lemma `Reverse_Shift` [7.21], this postcondition can be rewritten as the following assertion for the suffix $a[p \dots n]$. We make this fact explicit by enriching the implementation with:

```
//@ assert reverse: Reverse{Pre,Here}(a, p, n);
```

- the complete range $a[0 \dots n]$ is reversed, thereby establishing the desired in-place rotation.

From a verification perspective, this implementation is particularly remarkable. Its correctness relies entirely on the contract of algorithm `reverse` [7.25]. The `assigns` clauses ensure that each call affects only the intended subrange, while the postconditions precisely characterize the local effect of each reversal. Although the combination of these effects is nontrivial, Frama-C/WP and the involved provers are able to perform the necessary reasoning automatically and establish that the overall behavior of the implementation satisfies the rotation property specified by `Rotate` [7.28], without requiring substantial auxiliary annotations.

7.12. The replace_copy algorithm

The `replace_copy` algorithm of the C++ Standard Library [15, §28.6.5] copies a sequence while substituting selected elements. Here, the iterator-based interface has been adapted to process `value_type` arrays. The resulting signature reads:

```
size_type replace_copy(const value_type* a, size_type n,
                      value_type* b, value_type v, value_type w);
```

The `replace_copy` algorithm copies the elements of the input array $a[0 \dots n)$ to the output array $b[0 \dots n)$, substituting every occurrence of v by w . The return value is the length of the output array. Since this length equals n , which is already passed as an argument, the return value does not convey additional information.

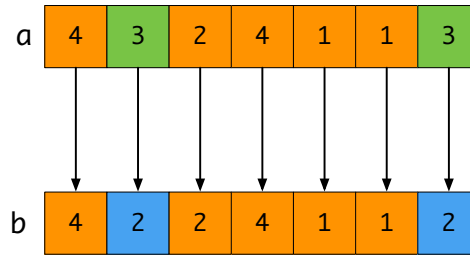


Figure 7.34.: Effects of `replace_copy`

Figure 7.34 illustrates the behavior of `replace_copy` by means of an example. Here, all occurrences of the value 3 in $a[0 \dots n)$ are replaced by the value 2 in $b[0 \dots n)$.

7.12.1. The predicate `Replace`

We begin by defining in the following listing the predicate `Replace` [7.35], which formalizes the intended element-wise relationship between an input array $a[0 \dots n)$ and an output array $b[0 \dots n)$.

```
/*@
predicate Replace{K,L}(value_type* a, integer n,
                      value_type* b, value_type v, value_type w) =
  \forall integer i; 0 <= i < n ==>
    \let ai = \at(a[i],K);
    \let bi = \at(b[i],L);
    (ai == v ==> bi == w) && (ai != v ==> bi == ai) ;

predicate Replace{K,L}(value_type* a, integer n, value_type v, value_type w) =
  Replace{K,L}(a, n, a, v, w);
*/
```

Listing 7.35: `Replace` (ACSL)

The predicate expresses that each element of the output array equals either the corresponding input element or a designated replacement value, depending on whether the input element matches the value to be replaced. In order to keep the definition concise and readable, we introduce *local bindings* using the ACSL construct `\let`. In particular, the bindings `\let ai = ...` and `\let bi = ...` denote the elements $a[i]$ and $b[i]$ at the current index, as evaluated in the appropriate program state (see [9, §2.2]).

The listing also contains a second, overloaded variant of the predicate `Replace` [7.35]. This variant relates an array to itself across two program states and will be used in the specification of the corresponding in-place algorithm `replace` [7.38].

7.12.2. Formal specification of `replace_copy`

The formal specification of `replace_copy` [7.36] is shown in the following listing.

```
/*@
  requires  valid:      \valid_read(a + (0..n-1));
  requires  valid:      \valid(b + (0..n-1));
  requires  sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates
  exits
  assigns   b[0..n-1];

  ensures   result:     \result == n;
  ensures   replace:    Replace{Old,Here}(a, n, b, v, w);
  ensures   unchanged:  Unchanged{Old,Here}(a, n);
*/
size_type replace_copy(const value_type* a, size_type n, value_type* b,
                      value_type v, value_type w);
```

Listing 7.36: Formal specification of `replace_copy`

The contract closely mirrors the informal description of the algorithm. The preconditions require that the input array $a[0 \dots n)$ is readable and that the output array $b[0 \dots n)$ is writable. In addition, the two arrays are required to be non-overlapping, as `replace_copy` performs a duplication rather than an in-place update. The assigns clause restricts side effects to the output array $b[0 \dots n)$. The postcondition is expressed concisely using the predicate `Replace` [7.35].

7.12.3. Implementation of `replace_copy`

The implementation of `replace_copy` [7.37], together with its loop annotations, is shown in the following listing.

```
size_type replace_copy(const value_type* a, size_type n, value_type* b,
                      value_type v, value_type w)
{
  /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant replace:    Replace{Pre,Here}(a, i, b, v, w);
    loop assigns i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = (a[i] == v ? w : a[i]);
  }

  return n;
}
```

Listing 7.37: Implementation of `replace_copy`

The algorithm consists of a single loop that traverses the input array $a[0 \dots n)$ from left to right and constructs the output array $b[0 \dots n)$ element by element. At each iteration, the current input element is compared with the value v . If the comparison succeeds, the value w is written to the corresponding position in the output array; otherwise, the input element is copied unchanged.

The loop invariants mirror the contract of `replace_copy` [7.36]. The invariant bound constrains the loop index to the valid range. The central invariant `replace` uses the predicate `Replace` [7.35] to express that the prefix of the output array that has already been produced satisfies the same element-wise replacement relation as required by the postcondition. The loop `assigns` clause records that all memory updates are confined to the output array. The loop variant `n-i` establishes termination by monotonically decreasing towards zero.

7.13. The `replace` algorithm

The `replace` algorithm of the C++ Standard Library [15, §28.6.5] performs an in-place substitution of selected values in a sequence. Here, the iterator-based interface of the standard algorithm is adapted to arrays of type `value_type`. The resulting signature reads:

```
void replace(value_type* a, size_type n, value_type v, value_type w);
```

The `replace` algorithm traverses the range $a[0 \dots n)$ and replaces every element that is equal to `v` by the value `w`. All other elements remain unchanged.

7.13.1. Formal specification of `replace`

The formal specification of `replace` [7.38] is shown in the following listing.

```
/*@
  requires    valid:    \valid(a + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         a[0..n-1];

  ensures    replace:  Replace{Old,Here}(a, n, v, w);
*/
void replace(value_type* a, size_type n, value_type v, value_type w);
```

Listing 7.38: Formal specification of `replace`

Since `replace` performs an in-place update, its contract differs from that of `replace_copy` in two important respects. First, the `assigns` clause restricts side effects to the range $a[0 \dots n)$, reflecting that all modifications occur within the input array itself. Second, the postcondition relates the final state of the array to its initial state, rather than to a separate output array.

This relationship is expressed using the second, overloaded variant of the predicate `Replace` [7.35]. Unlike the version used for `replace_copy`, this variant relates a single array across two program states. It specifies that, for each index i in the range $[0 \dots n)$, the value stored in $a[i]$ after execution equals `w` if the corresponding value in the pre-state equals `v`, and otherwise remains unchanged. In this way, the predicate concisely captures the element-wise effect of the in-place replacement.

7.13.2. Implementation of `replace`

The implementation of `replace` [7.39], together with its loop annotations, is shown in the following listing.

```

void replace(value_type* a, size_type n, value_type v, value_type w)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant replace:    Replace{Pre,Here}(a, i, v, w);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] == v) {
            a[i] = w;
        }
    }
}

```

Listing 7.39: Implementation of replace

The algorithm traverses the array $a[0 \dots n)$ once from left to right and updates the elements in place. At each iteration, the current element $a[i]$ is compared with the value v . If the comparison succeeds, the element is overwritten with the value w ; otherwise, it is left unchanged.

The invariant `unchanged` expresses that, upon entry to iteration i , the suffix $a[i \dots n)$ still coincides with its initial state. This invariant is characteristic for in-place algorithms that process an array sequentially and complements the use of the state-based predicate `Replace` [7.35] in the postcondition. The `loop assigns` clause makes explicit that all side effects are confined to the array a and the loop index.

7.14. The `remove_copy` algorithm (basic contract)

The `remove_copy` algorithm of the C++ Standard Library [15, §28.6.8] constructs a new sequence by copying all elements of an input array except those equal to a given value. In contrast to many of the algorithms discussed so far, the presentation of `remove_copy` will be distributed over several sections. In the present section we therefore restrict attention to the basic contract and to the externally observable behavior of the algorithm, postponing a more refined specification and the implementation details to later sections.

As before, the generic iterator-based formulation has been specialized to contiguous arrays of elements of type `value_type`. The resulting function signature reads as follows:

```
size_type  
remove_copy(const value_type* a, size_type n, value_type* b, value_type v);
```

Intuitively, `remove_copy` scans the input array $a[0 \dots n)$, selects those elements that are not equal to v , and copies them, in order, into the output array starting at b . The function returns the number of elements that have been copied.

The essential requirements and properties of `remove_copy` are summarized in Table 7.40. These requirements will later be reflected as precisely as currently possible in the formal specification.

Requirements	Description
<i>Remove Copy Size</i>	The output array must be large enough to hold all elements of the input array except those equal to v .
<i>Remove Copy Separated</i>	The input array and the output array do not overlap.
<i>Remove Copy Elements</i>	All elements of $a[0 \dots n)$ that are not equal to v are copied to the range $b[0 \dots \text{return}-1]$.
<i>Remove Copy Stability</i>	The algorithm is stable, that is, the relative order of the copied elements in b is the same as in a .
<i>Remove Copy Return</i>	The return value is the length of the resulting output array.
<i>Remove Copy Complexity</i>	The algorithm performs exactly n comparisons.

Table 7.40.: Properties of `remove_copy`

Figure 7.41 illustrates the effect of `remove_copy` for a concrete input, showing how all elements different from 4 are copied from the input array to the output array while preserving their relative order.

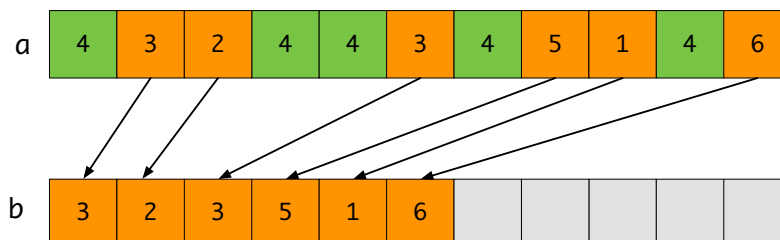


Figure 7.41.: Effects of `remove_copy`

7.14.1. Formal specification of `remove_copy`

The following listing presents a first attempt at a formal specification of `remove_copy`. At this stage, the contract captures only basic properties of the resulting output array and deliberately avoids committing to a more detailed structural relationship between input and output.

In the postcondition `discard` we use the predicate `NoneEqual` [4.4] to express that the value `v` does not occur in the output array `b[0 .. \result)`. This ensures that all copied elements differ from the value being removed, but it does not yet constrain which elements have actually been copied.

```
/*@
  requires  valid:      \valid_read(a + (0..n-1));
  requires  valid:      \valid(b + (0..n-1));
  requires  sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates
  exits
  assigns   b[0..n-1];

  ensures   bound:      0 <= \result <= n;
  ensures   discard:    NoneEqual(b, \result, v);
  ensures   unchanged:  Unchanged{Old,Here}(a, n);
  ensures   unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy(const value_type *a, size_type n, value_type *b,
                      value_type v);
```

Listing 7.42: Formal specification of `remove_copy`

One limitation of this specification is that the postcondition `bound` only provides a very coarse upper bound on the number of copied elements. While it guarantees that the result does not exceed the length of the input, it does not characterize the result size more precisely. This issue will be addressed in the refined contract of `remove_copy2` [7.46].

A more fundamental shortcoming, however, is that the present specification does not describe the relationship between the elements of the input array `a[0 .. n)` and those of the output array `b[0 .. \result)`. In particular, no statement is made about which input elements are copied or about the order in which they appear in the output. As a consequence, the stability of `remove_copy` is not yet expressible at this level. Both the precise correspondence between input and output elements and the formalization of stability will be tackled in the contract of `remove_copy3` [7.53].

7.14.2. Implementation of `remove_copy`

An implementation of `remove_copy` together with its ACSL annotations is shown in the following listing. The implementation follows the standard structure of a single linear scan over the input array $a[0 \dots n)$. An index variable i traverses the input array, while a second index k denotes the length of the output prefix that has already been constructed in the array b . Whenever the current input element $a[i]$ differs from the value v , it is copied to position $b[k]$, and the index k is incremented. Elements equal to v are simply skipped, leaving the output array unchanged.

```
size_type remove_copy(const value_type *a, size_type n, value_type *b,
                     value_type v)
{
    size_type k = 0u;

    /*@
    loop invariant bound:      0 <= k <= i <= n;
    loop invariant discard:    NoneEqual(b, k, v);
    loop invariant unchanged:  Unchanged{Pre,Here}(b, k, n);
    loop assigns      k, i, b[0..n-1];
    loop variant      n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] != v) {
            b[k++] = a[i];
        }
    }

    return k;
}
```

Listing 7.43: Implementation of `remove_copy`

The loop invariants reflect this separation of concerns. The invariant `bound` relates the output index k to the progress of the input index i and ensures that the output array is never written out of bounds. In addition, we must explicitly state that none of the elements written so far equals the value being removed.

This property is captured by the additional loop invariant `discard`, which asserts that the value v does not occur in the partially constructed output array $b[0 \dots k)$ at any loop iteration. Without this invariant, the verifier would have no means of propagating the postcondition that excludes v from the output array.

At this level of specification, the loop invariants intentionally do not express any correspondence between elements of the input array and those of the output array beyond the exclusion of v . In particular, neither the origin nor the relative order of the copied elements is specified. This mirrors the limitations already observed in the contract and prepares the ground for later refinements, where stability and a precise input-output relationship will be formalized.

7.15. The `remove_copy2` algorithm (number of copied elements)

In this section we refine the contract of `remove_copy` [7.42] by specifying the number of elements copied to the output array. Unlike the basic contract of Section 7.14, the present specification characterizes the return value precisely.

By definition, the number of copied elements equals the number of elements in the input array $a[0 \dots n)$ that are different from the value v . This quantity can be expressed conveniently by means of a logic function that counts the elements of an array not equal to v . One straightforward possibility is to express this number using the already available logic function `Count` [4.53]:

```
logic integer  
CountNotEqual(value_type* a, integer n, value_type v) =  
  n - Count(a, n, v);
```

While such a definition is sufficient from a mathematical point of view, we introduce a dedicated logic function `CountNotEqual` [7.44] together with basic properties and supporting lemmas. This allows contracts and invariants to be stated directly in terms of the quantity of interest, rather than via derived expressions. For this reason, `CountNotEqual` is introduced as an independent logic function. The relationship between `CountNotEqual` and the previously introduced logic function `Count` [4.53] is captured explicitly by the lemma `CountNotEqual_Count`, shown in Listing 7.45.

```
/*@  
logic integer  
CountNotEqual(value_type* a, integer m, integer n, value_type v) =  
  n <= m ? 0 : CountNotEqual(a, m, n-1, v) + (a[n-1] == v ? 0 : 1);  
  
logic integer  
CountNotEqual(value_type* a, integer n, value_type v) =  
  CountNotEqual(a, 0, n, v);  
  
lemma CountNotEqual_Empty:  
  \forall value_type *a, v, integer m, n;  
    n <= m ==> CountNotEqual(a, m, n, v) == 0;  
  
lemma CountNotEqual_Hit:  
  \forall value_type *a, v, integer m, n;  
    m <= n ==>  
    a[n] != v ==>  
    CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v) + 1;  
  
lemma CountNotEqual_Miss:  
  \forall value_type *a, v, integer m, n;  
    m <= n ==>  
    a[n] == v ==>  
    CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v);  
  
lemma CountNotEqual_Lower:  
  \forall value_type *a, v, integer m, n;  
    m <= n ==> 0 <= CountNotEqual(a, m, n, v);  
  
lemma CountNotEqual_Upper:  
  \forall value_type *a, v, integer m, n;  
    m <= n ==> CountNotEqual(a, m, n, v) <= n-m;
```

Listing 7.44: The logic function `CountNotEqual` (1)

```

lemma CountNotEqual_WeaklyIncreasing:
  \forall value_type *a, v, integer m, n;
    m <= n ==> CountNotEqual(a, m, n, v) <= CountNotEqual(a, m, n+1, v);

lemma CountNotEqual_Increasing:
  \forall value_type *a, v, integer k, m, n;
    m <= k <= n ==> CountNotEqual(a, m, k, v) <= CountNotEqual(a, m, n, v);

lemma CountNotEqual_Unchanged{K,L}:
  \forall value_type *a, v, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
      CountNotEqual{K}(a, m, n, v) == CountNotEqual{L}(a, m, n, v);

lemma CountNotEqual_Count:
  \forall value_type *a, v, integer m, n;
    m <= n ==> CountNotEqual(a, m, n, v) == n - m - Count(a, m, n, v);

lemma CountNotEqual_Union:
  \forall value_type *a, v, integer k, m, n;
    0 <= k <= m <= n ==>
      CountNotEqual(a, k, n, v) ==
      CountNotEqual(a, k, m, v) + CountNotEqual(a, m, n, v);
*/

```

Listing 7.45: The logic function CountNotEqual (2)

7.15.1. Formal specification of remove_copy2

The formal specification of `remove_copy2` extends the basic contract of `remove_copy` [7.42] by precisely characterizing the number of elements copied to the output array.

```

/*@
requires   valid:    \valid_read(a + (0..n-1));
requires   valid:    \valid(b + (0..n-1));
requires   sep:      \separated(a + (0..n-1), b + (0..n-1));

terminates      \true;
exits           \false;
assigns         b[0..n-1];

ensures   size:      \result == CountNotEqual(a, n, v);
ensures   bound:     0 <= \result <= n;
ensures   discard:   NoneEqual(b, \result, v);
ensures   unchanged: Unchanged{Old,Here}(a, n);
ensures   unchanged: Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy2(const value_type* a, size_type n, value_type* b,
                       value_type v);

```

Listing 7.46: Formal specification of remove_copy2

To this end, we introduce an additional postcondition `size`, which states that the return value of `remove_copy2` equals the number of elements in the input array $a[0 \dots n)$ that are different from v . This property is expressed using the logic function `CountNotEqual` [7.44].

7.15.2. Implementation of `remove_copy2`

The following listing shows the implementation of the extended algorithm `remove_copy2` together with its ACSL annotations. Compared to the basic implementation of `remove_copy`, the algorithmic structure remains unchanged. The only conceptual extension concerns the precise tracking of the number of elements copied to the output array. This is reflected by the additional loop invariant `size`, which states that the current value of the output index equals the number of elements in the already processed prefix of the input array that are different from `v`. The invariant directly mirrors the postcondition of `remove_copy2` [7.46] and makes the result size available throughout the loop.

```
size_type remove_copy2(const value_type* a, size_type n, value_type* b,
                      value_type v)
{
    size_type k = 0u;

    /*@
    loop invariant size:      k == CountNotEqual(a, i, v);
    loop invariant bound:    0 <= k <= i <= n;
    loop invariant discard:  NoneEqual(b, k, v);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns    k, i, b[0..n-1];
    loop variant    n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] != v) {
            b[k++] = a[i];
            //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, n);
            //@ assert size:      k == CountNotEqual(a, 0, i+1, v);
        }
    }

    return k;
}
```

Listing 7.47: Implementation of `remove_copy2`

To allow the verifier to re-establish this invariant after each iteration, the implementation is augmented with the assertions `size` and `unchanged`. These assertions make explicit the effect of each loop body execution on the output index and on the portions of the arrays that are not modified.

Despite this refinement, the implementation still does not specify how individual elements of the input array relate to those of the output array. In particular, no correspondence between input and output positions is yet expressed. This remaining gap, and with it the formalization of stability, will be addressed in the contract of `remove_copy3` [7.53].

7.16. The `remove_copy3` algorithm (final contract)

In this section we complete the refinement of the contracts introduced in `remove_copy` [7.42] and `remove_copy2` [7.46]. The final step consists in introducing a logic function that formally describes the relationship between the elements of the input array $a[0 \dots n)$ and those of the output array $b[0 \dots \text{\texttt{result}})$. Recall from the previous section that the return value `\result` equals `CountNotEqual(a, n, v)`.

7.16.1. A closer look at the properties of `remove_copy`

Figure 7.48 shows a refined version of Figure 7.41. For clarity, indices of elements that are not copied into the target array have been omitted. In addition, a dashed arrow has been added to indicate the *one-past-the-end* position of both the input and the output array.

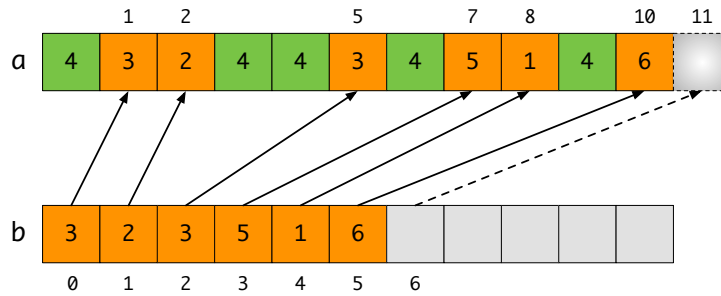


Figure 7.48.: Partitioning the input of `remove_copy`

The arrows connecting indices of the array b to indices of the array a induce a sequence p of indices. In the example shown, this sequence consists of seven indices, where the *one-past-the-end* index is underlined:

$$p = (1, 2, 5, 7, 8, 10, \underline{11}).$$

More generally, we refer to p as the *partitioning sequence* associated with an application of `remove_copy` to the array $a[0 \dots n)$. Let m denote the length of this sequence. Then p is a **strictly increasing** sequence satisfying

$$0 \leq p_0 < \dots < p_m = n. \quad (7.6)$$

The open index intervals (p_i, p_{i+1}) marks **consecutive ranges** of the value v in the source array:

$$a[k] = v \quad \forall k : p_i < k < p_{i+1}. \quad (7.7)$$

In addition, the half-open interval $[0, p_0)$ also forms a **consecutive range** of the value v :

$$a[k] = v \quad \forall k : 0 \leq k < p_0. \quad (7.8)$$

The elements of the input array selected by the partitioning sequence are all different from v :

$$a[p_i] \neq v \quad \forall i : 0 \leq i < m. \quad (7.9)$$

Finally, the defining correspondence between input and output elements is given by

$$a[p_i] = b[i] \quad \forall i : 0 \leq i < m. \quad (7.10)$$

Together with (7.9), this immediately implies that the output array does not contain the value v :

$$b[i] \neq v \quad \forall i : 0 \leq i < m. \quad (7.11)$$

7.16.2. Additional lemmas for CountNotEqual

The formalization of the properties in Section 7.16.1 relies on the logic function `CountNotEqual` [7.44]. In order to reason about the correspondence between input and output indices, we additionally make use of the logic function `FindNotEqual` [4.19].

The accompanying lemmas collected under [7.49] establish further relationships between `CountNotEqual` and `FindNotEqual`. These results are of a technical nature: they do not introduce new conceptual ideas, but provide the auxiliary facts required to carry out the verification of the final contract.

```

/*@
lemma CountNotEqual_AllEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= m <= n      ==>
    AllEqual(a, m, n, v) ==> CountNotEqual(a, m, n, v) == 0;

lemma CountNotEqual_SomeNotEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= m < n      ==>
    0 < CountNotEqual(a, m, n, v) ==> SomeNotEqual(a, m, n, v);

lemma CountNotEqual_FindNotEqual:
  \forallall value_type *a, v, integer m, n;
    0 <= m < n      ==>
    0 < CountNotEqual(a, m, n, v) ==> FindNotEqual(a, m, n, v) < n-m;

lemma CountNotEqual_Zero:
  \forallall value_type *a, v, integer m, n;
    0 <= m < n      ==>
    CountNotEqual(a, m, m + FindNotEqual(a, m, n, v), v) == 0;

lemma CountNotEqual_Decrement:
  \forallall value_type *a, v, integer m, n;
    0 <= m < n      ==>
    CountNotEqual(a, m + FindNotEqual(a, m, n, v), n, v) ==
    CountNotEqual(a, 0, n, v) - CountNotEqual(a, 0, m, v);
*/

```

Listing 7.49: CountFindNotEqual (ACSL)

7.16.3. Formalizing the properties of the partitions

The logic function `IndexOfNotEqual`, whose definition is given in Listings 7.50 and 7.51, formalizes the partitioning sequence p introduced in Section 7.16.1. Intuitively, `IndexOfNotEqual` (a , n , v , k) returns the array index corresponding to the k -th element of the range $a[0 \dots n)$ that is not equal to v .

```
/*@
logic integer
IndexOfNotEqual(value_type* a, integer n, value_type v, integer k) =
  \let count = CountNotEqual(a, n, v);
  \let start = IndexOfNotEqual(a, n, v, k-1) + 1;
  // Negative rank: invalid request
  k < 0 ? -1 :
    // Now we know 0 <= k
    (n <= 0 ? 0 :
      // Now we know 0 < n
      k < count ? start + FindNotEqual(a, start, n, v) : n
    );

lemma IndexOfNotEqual_Left:
  \forall value_type *a, v, integer n, k;
  k < 0 ==> IndexOfNotEqual(a, n, v, k) == -1;

lemma IndexOfNotEqual_Empty:
  \forall value_type *a, v, integer n, k;
  n <= 0 <= k ==>
  IndexOfNotEqual(a, n, v, k) == 0;

lemma IndexOfNotEqual_Right:
  \forall value_type *a, v, integer n, k;
  0 <= n ==>
  CountNotEqual(a, n, v) <= k ==> IndexOfNotEqual(a, n, v, k) == n;

lemma IndexOfNotEqual_Next:
  \forall value_type *a, v, integer n, k;
  \let x = IndexOfNotEqual(a, n, v, k-1) + 1;
  0 <= n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  IndexOfNotEqual(a, n, v, k) == x + FindNotEqual(a, x, n, v);

lemma IndexOfNotEqual_Lower:
  \forall value_type *a, v, integer n, k;
  0 < n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  0 <= IndexOfNotEqual(a, n, v, k);
```

Listing 7.50: The logic function `IndexOfNotEqual` (1)

The behavior of `IndexOfNotEqual` can be summarized as follows.

- Negative values of k correspond to invalid requests and yield -1 .
- Out-of-range requests, that is, values of k greater than or equal to the number of elements of $a[0 \dots n)$ that are different from v , yield n , the first index past the end of the array.
- For intermediate values, the function is defined inductively: the index for rank k is obtained from the index for rank $k - 1$ by advancing to the next array position whose value differs from v .

We recall that logic functions (and predicates) in ACSL must be total, that is, they must be defined for all possible argument values. The explicit treatment of invalid and out-of-range arguments in the definition of `IndexOfNotEqual` serves precisely this purpose.

```

lemma IndexOfNotEqual_Core:
  \forallall value_type *a, v, integer n, k;
  \let R = IndexOfNotEqual(a, n, v, k);
  0 < n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  (R < n &&
   a[R] != v &&
   CountNotEqual(a, R, n, v) == CountNotEqual(a, 0, n, v) - k);

lemma IndexOfNotEqual_Upper:
  \forallall value_type *a, v, integer n, k;
  0 < n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  IndexOfNotEqual(a, n, v, k) < n;

lemma IndexOfNotEqual_NotEqual:
  \forallall value_type *a, v, integer n, k;
  0 < n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  a[IndexOfNotEqual(a, n, v, k)] != v;

lemma IndexOfNotEqual_Count:
  \forallall value_type *a, v, integer n, k;
  0 < n ==>
  0 <= k < CountNotEqual(a, n, v) ==>
  CountNotEqual(a, IndexOfNotEqual(a, n, v, k), n, v) ==
  CountNotEqual(a, 0, n, v) - k;

lemma IndexOfNotEqual_StrictlyWeakIncreasing:
  \forallall value_type *a, v, integer n, k;
  0 < n ==>
  0 < k < CountNotEqual(a, n, v) ==>
  IndexOfNotEqual(a, n, v, k-1) < IndexOfNotEqual(a, n, v, k);

lemma IndexOfNotEqual_Segment:
  \forallall value_type *a, v, integer n, k;
  0 < n ==>
  0 <= k ==>
  k + 1 < CountNotEqual(a, n, v) ==>
  AllEqual(a, IndexOfNotEqual(a, n, v, k) + 1,
           IndexOfNotEqual(a, n, v, k+1), v);

lemma IndexOfNotEqual_Unchanged{K,L}:
  \forallall value_type *a, v, integer n, k;
  Unchanged{K,L}(a, n) ==>
  IndexOfNotEqual{K}(a, n, v, k) == IndexOfNotEqual{L}(a, n, v, k);
*/

```

Listing 7.51: The logic function `IndexOfNotEqual` (2)

The lemmas associated with `IndexOfNotEqual` correspond directly to the properties identified in Section 7.16.1.

- Property (7.6) is captured by the lemmas `IndexOfNotEqual_Empty`, `IndexOfNotEqual_Left`, `IndexOfNotEqual_Right`, and `IndexOfNotEqual_StrictlyWeakIncreasing`, which together establish boundary behavior and monotonicity of the partitioning sequence.

- Properties (7.7) and (7.8) are formalized by the lemma `IndexOfNotEqual_Segment`, which characterizes the ranges of consecutive occurrences of the value v in the input array.
- Property (7.9) is expressed by the lemma `IndexOfNotEqual_NotEqual`.
- Property (7.10) is formulated using the predicate `Remove` [7.52], which relates the partitioning sequence to the construction of the target array.

We single out the lemma `IndexOfNotEqual_Core`, which subsumes the statements of the following lemmas:

- `IndexOfNotEqual_Upper`,
- `IndexOfNotEqual_NotEqual`, and
- `IndexOfNotEqual_Count`.

None of the three introduces additional information. `IndexOfNotEqual_Upper` restates Property (7.6) of Section 7.16.1, and `IndexOfNotEqual_NotEqual` restates Property (7.9); both are retained because they provide a convenient and focused reference point. `IndexOfNotEqual_Count` has no counterpart there: it counts rather than locates, and states that exactly k of the elements different from v lie below p_k .

The presence of the combined lemma `IndexOfNotEqual_Core` may nevertheless raise the question why such a formulation is needed at all. `IndexOfNotEqual` is defined by recursion on the rank k , so the three statements are established by one induction on k , and its hypothesis has to supply all three of them at once. Each is consumed at a different point of the step from k to $k + 1$: that $a[p_k]$ differs from v is what makes the single cell p_k contribute one to the count, the counting identity is what shows that a further element different from v remains beyond p_k , and $p_k < n$ is what permits the count over $[p_k .. n)$ to be split at $p_k + 1$. An induction that carried only one of the three would thus have too weak a hypothesis, and `IndexOfNotEqual_Core` is the strengthened statement that carries them jointly.

7.16.4. The predicate `Remove`

The predicate `Remove` [7.52] is introduced to improve the readability and modularity of the specification of `remove_copy3` [7.53]. Rather than restating the correspondence between input and output elements directly in the contract, this relationship is encapsulated in a dedicated predicate.

```

/*@
  predicate Remove{K,L}(value_type* a, integer n, integer i, value_type* b,
    value_type v) =
    \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v) ==>
      \let j = IndexOfNotEqual{K}(a, n, v, k);
      \at(b[k],L) == \at(a[j],K);

  predicate Remove{K,L}(value_type* a, integer n, value_type* b, value_type v) =
    Remove{K,L}(a, n, n, b, v);

  predicate Remove{K,L}(value_type* a, integer n, integer i, value_type v) =
    \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v) ==>
      \let j = IndexOfNotEqual{K}(a, n, v, k);
      \at(a[k],L) == \at(a[j],K);

  predicate Remove{K,L}(value_type* a, integer n, value_type v) =
    Remove{K,L}(a, n, n, v);
*/

```

Listing 7.52: `Remove` (ACSL)

Conceptually, `Remove` formalizes Property (7.10) from Section 7.16.1, namely the fact that the i -th element of the output array originates from the input element at index p_i of the partitioning sequence. By isolating this relationship in a separate predicate, the specification becomes both shorter and easier to maintain.

As in earlier chapters, the predicate `Remove` is provided in multiple, overloaded variants. In addition to the version used here for `remove_copy3`, an alternative formulation is included that will be employed in the specification of the *in-place* variant `remove` [7.56].

7.16.5. Formal specification of `remove_copy3`

The following listing presents the formal specification of `remove_copy3` [7.53], which completes the refinement of the contracts introduced in `remove_copy` [7.42] and `remove_copy2` [7.46].

```

/*@
requires   valid:      \valid_read(a + (0..n-1));
requires   valid:      \valid(b + (0..n-1));
requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

terminates           \true;
exits                \false;
assigns              b[0..n-1];

ensures   size:       \result == CountNotEqual{Old}(a, n, v);
ensures   bound:      0 <= \result <= n;
ensures   remove:     Remove{Old,Here}(a, n, b, v);
ensures   discard:    NoneEqual(b, \result, v);
ensures   unchanged:  Unchanged{Old,Here}(a, n);
ensures   unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy3(const value_type* a, size_type n, value_type* b,
                       value_type v);

```

Listing 7.53: Formal specification of `remove_copy3`

Compared to the specification of `remove_copy2` [7.46], the essential extension is the addition of the postcondition `remove`. This postcondition makes use of the predicate `Remove` to formalize the precise correspondence between elements of the input array $a[0 \dots n)$ and those of the output array $b[0 \dots \text{result})$. In particular, it captures both the selection of elements that differ from v and the preservation of their relative order. All other postconditions are inherited unchanged. In particular, the postcondition `unchanged` again states that the source array $a[0 \dots n)$ is not modified by the execution of the algorithm.

7.16.6. Implementation of `remove_copy3`

We now highlight the aspects of the implementation of `remove_copy3` [7.54] that differ from the earlier versions of the algorithm. The algorithmic structure itself is unchanged; the refinements concern exclusively the loop annotations required to establish the final contract.

The central addition is the pair of loop invariants `interval`. Together they relate the progress of the input index i to the current output index k by confining i to the closed interval bounded by two consecutive members of the partitioning sequence:

$$p_{k-1} \leq i \leq p_k.$$

Here p_k stands for `IndexOfNotEqual(a, n, v, k)`. The degenerate case $k = 0$ is covered by the lemma `IndexOfNotEqual_Left`, which yields $p_{-1} = -1$. The upper bound expresses that the algorithm has not

yet skipped past the input position corresponding to the k -th element copied to the output array, while the lower bound records that the preceding such position has already been passed.

To enable the prover to re-establish these invariants after each loop iteration, the implementation includes an explicit assertion interval. As in earlier implementations, such assertions serve as intermediate proof obligations and guide the automated reasoning.

```

size_type remove_copy3(const value_type* a, size_type n, value_type* b,
                      value_type v)
{
    size_type k = 0u;

    /*@
    loop invariant size:      k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:    0 <= k <= i <= n;
    loop invariant remove:   Remove{Pre,Here}(a, n, i, b, v);
    loop invariant discard:  NoneEqual(b, k, v);
    loop invariant interval: IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval: i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged: Unchanged{Pre,Here}(a, n);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns    k, i, b[0..n-1];
    loop variant    n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        if (a[i] != v) {
            b[k++] = a[i];
            /*@ assert size:      k == CountNotEqual{Pre}(a, i+1, v);
            /*@ assert update:    b[k-1] == At{Pre}(a, i);
            /*@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
            /*@ assert remove:    Remove{Pre,Here}(a, n, i, b, v);
            /*@ assert remove:    Remove{Pre,Here}(a, n, i+1, b, v);
            /*@ assert unchanged: Unchanged{Pre,Here}(a, n);
            /*@ assert unchanged: Unchanged{Pre,Here}(b, k, n);
        }
        else {
            /*@ assert unchanged: Unchanged{Pre,Here}(a, n);
        }

        /*@ assert remove:      Remove{Pre,Here}(a, n, i, b, v);
        /*@ assert unchanged:   Unchanged{Pre,Here}(a, n);
    }

    return k;
}

```

Listing 7.54: Implementation of `remove_copy3`

In addition, the logic function `At` [7.3] is used in the annotations to refer to values of array elements at specific program points. This allows the correspondence between input and output elements, as expressed by the predicate `Remove` [7.52], to be stated concisely.

Finally, the implementation contains an explicit `else` branch whose body is empty apart from the assertion `unchanged`. Although this branch has no operational effect, its presence significantly reduces verification times by making the control-flow structure explicit and thereby simplifying the generation of proof obligations.

7.17. The remove algorithm

In addition to `remove_copy`, the C++ Standard Library provides the algorithm `remove` [15, p. 28.6.8], which performs the same logical operation as an *in-place* transformation. Rather than copying the selected elements to a separate output array, `remove` compacts the input array itself by overwriting those elements that are equal to the given value. Accordingly, the signature of `remove` is similar to that of `remove_copy`, but does not require a separate output array:

```
size_type remove(value_type* a, size_type n, value_type v);
```

The algorithm returns the length of the resulting prefix of the array that contains all elements different from `v`. Elements beyond this prefix are left in an unspecified state.

Figure 7.55 illustrates the effect of `remove` for a concrete example, showing how all occurrences of the value 4 are removed from the array by shifting the remaining elements towards the beginning of the array.

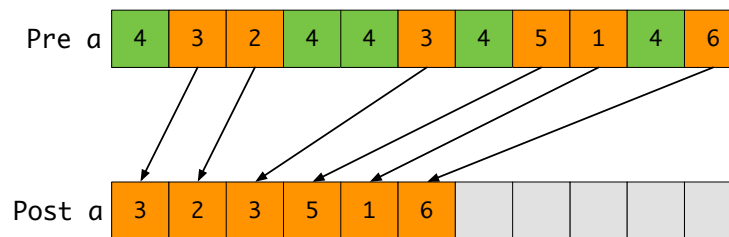


Figure 7.55.: Effects of `remove`

7.17.1. Formal specification of `remove`

The specification of `remove` [7.56] is closely related to that of `remove_copy3` [7.53]. Both algorithms select exactly those elements of the input array that are different from `v`, preserve their relative order, and return the number of selected elements.

```
/*@
requires   valid:      \valid(a + (0..n-1));

terminates      \true;
exits           \false;
assigns         a[0..n-1];

ensures   size:      \result == CountNotEqual{Old}(a, n, v);
ensures   bound:     0 <= \result <= n;
ensures   remove:    Remove{Old,Here}(a, n, v);
ensures   discard:   NoneEqual(a, \result, v);
ensures   unchanged: Unchanged{Old,Here}(a, \result, n);
*/
size_type remove(value_type* a, size_type n, value_type v);
```

Listing 7.56: Formal specification of `remove`

As in the specification of `remove_copy3` [7.53], the predicate `Remove` [7.52] is used to formalize the correspondence between the elements of the original array and those of the resulting array. In the present case, this correspondence relates the initial contents of the array `a` to the resulting prefix `a[0 .. \result)`. The essential difference lies in the treatment of the output. While `remove_copy3` writes the selected

elements to a separate output array, `remove` performs the transformation in place by compacting the array *a* itself.

7.17.2. Implementation of `remove`

The following listing shows the implementation of `remove` [7.57] together with its ACSL annotations. At the algorithmic level, the implementation closely follows that of `remove_copy3` [7.54]. The main difference is that the output array coincides with the input array, so that selected elements are written back into the array *a* itself.

```

size_type remove(value_type* a, size_type n, value_type v)
{
    size_type k = 0u;

    /*@
    loop invariant size:      k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:    0 <= k <= i <= n;
    loop invariant remove:   Remove{Pre,Here}(a, n, i, v);
    loop invariant discard:  NoneEqual(a, k, v);
    loop invariant interval: IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval: i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged: Unchanged{Pre,Here}(a, k, n);
    loop invariant unchanged: a[k] == At{Pre}(a, k);
    loop assigns    k, i, a[0..n-1];
    loop variant    n-i;
    */
    for (size_type i = 0u; i < n; ++i ) {
        if (a[i] != v) {
            a[k++] = a[i];
            //@ assert size:      k == CountNotEqual{Pre}(a, 0, i+1, v);
            //@ assert update:    a[k-1] == At{Pre}(a, i);
            //@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
            //@ assert remove:    Remove{Pre,Here}(a, n, i, v);
            //@ assert remove:    Remove{Pre,Here}(a, n, i+1, v);
        }
    }

    return k;
}

```

Listing 7.57: Implementation of `remove`

Accordingly, most loop invariants are reused with only minor adaptations. In particular, the invariants governing the result size, the mapping between input and output positions, and the exclusion of the value *v* from the resulting prefix remain unchanged in spirit.

Since the array is modified in place, the annotations must additionally refer to values of array elements in earlier program states. For this purpose, the logic function `At` [7.3] is used in the loop invariant `unchanged` and in the assertion `update`. As discussed earlier, this is a matter of convenience rather than necessity, and the same properties could equally be expressed using the built-in function `\at`.

7.18. The shuffle algorithm

The shuffle algorithm in the C++ Standard Library [15, §28.6.13] rearranges the elements of a given array in a randomized manner. For our purposes, the generic implementation has been adapted to operate on arrays of type `value_type`. The resulting signature reads:

```
void shuffle(value_type* a, size_type n, unsigned short* seed);
```

The argument `seed` represents the state of a simple pseudo-random number generator that is advanced during the execution of `shuffle` and controls the sequence of element exchanges performed by the algorithm.

Figure 7.58 illustrates a representative execution of `shuffle`. In the example shown, the values 1, 2, 3, and 4 occur twice, once, once, and three times, respectively, both before and after the call. This shows that `shuffle` preserves the multiset of elements while changing their relative order.

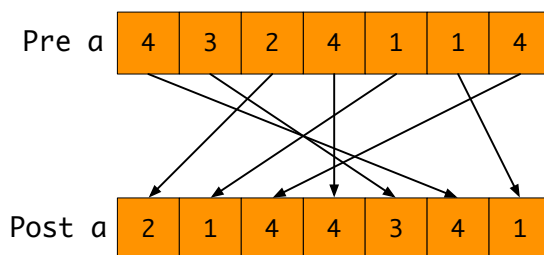


Figure 7.58.: Effects of shuffle

In the following sections, we will concentrate on formally capturing this rearrangement property. Rather than characterizing the result in terms of explicit permutations, we describe the effect of `shuffle` by relating the contents of the input array to those of the output array in a value-oriented manner.

7.18.1. The predicate `MultisetReorder`

The `shuffle` algorithm is the first algorithm in this document for which we must specify a genuine *rearrangement* of the elements of a given array. In contrast to algorithms that modify values or discard elements, `shuffle` is intended to change only the order of the elements.

To express this property precisely, we introduce the notion of a *multiset*. A multiset is a collection of values in which elements may occur with multiplicity, but in which no order is imposed. Two arrays represent the same multiset if and only if, for every possible value, they contain the same number of occurrences of that value.¹²

Accordingly, we say that an array has been reordered between two program states if the multiset of elements contained in the array remains unchanged. That is, a reordering preserves all values together with their multiplicities, while possibly changing their positions.

We use the predicate `MultisetReorder` [7.59] to formalize this notion. The predicate is provided in two overloaded variants and is defined in terms of the logic function `Count` [4.53], which counts the number of occurrences of a given value in a range.

In order to reason effectively about rearranging algorithms, it is not sufficient to introduce `MultisetReorder` alone. One also needs a collection of elementary properties that allow the predicate to be combined, propagated through program fragments, and instantiated for subranges. For

¹²See <http://en.wikipedia.org/wiki/Multiset>.

this purpose, we state a number of lemmas that capture basic structural properties of `MultisetReorder`, such as reflexivity, symmetry, its behavior under range decomposition, and its behavior under the pointer arithmetic by which a subrange is addressed as an array in its own right.

```

/*@
predicate MultisetReorder{K,L}(value_type* a, integer m, integer n) =
  \forall value_type v;
    Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

predicate MultisetReorder{K,L}(value_type* a, integer n) =
  MultisetReorder{K,L}(a, 0, n);

lemma Unchanged_MultisetReorder{K,L}:
  \forall value_type *a, integer k, n;
    Unchanged{K,L}(a, k, n) ==> MultisetReorder{K,L}(a, k, n);

lemma MultisetReorder_DisjointUnion{K,L}:
  \forall value_type *a, integer i, k, n;
    0 <= i <= k <= n ==>
      MultisetReorder{K,L}(a, i, k) ==>
      MultisetReorder{K,L}(a, k, n) ==>
      MultisetReorder{K,L}(a, i, n);

lemma MultisetReorder_Shift{K,L}:
  \forall value_type *a, integer p, n;
    0 <= p ==> 0 <= n ==>
      MultisetReorder{K,L}(a+p, 0, n) ==>
      MultisetReorder{K,L}(a, p, p+n);

lemma MultisetReorder_Symmetric{K,L}:
  \forall value_type *a, integer m, n;
    MultisetReorder{K,L}(a, m, n) ==> MultisetReorder{L,K}(a, m, n);

lemma MultisetReorder_Transitive{K,L,M}:
  \forall value_type *a, integer m, n;
    MultisetReorder{K,L}(a, m, n) ==>
    MultisetReorder{L,M}(a, m, n) ==>
    MultisetReorder{K,M}(a, m, n);
*/

```

Listing 7.59: `MultisetReorder` (ACSL)

These lemmas are not specific to shuffle. Rather, they form a small reasoning toolkit that will be reused throughout the verification of algorithms whose effect consists solely in rearranging existing elements. They will play a central role in the verification of the heap-related algorithms in Chapter 9 and of the sorting algorithms in Chapter 10.

7.18.2. Formal specification of shuffle

The shuffle algorithm preserves the *multiset represented by the input array* (see Section 7.18.1), while modifying only the order of its elements. This essential property is formalized with the help of the predicate `MultisetReorder` [7.59], which expresses that the number of occurrences of each value in the range $a[0 \dots n)$ is unchanged upon termination.

Accordingly, the specification of `shuffle` requires that the range $a[0 \dots n)$ be valid for both reading and writing.

The array `seed` represents the mutable state of the pseudo-random number generator used by the implementation. By allowing assignments to `seed` in the `assigns`-clause, the specification reflects that successive calls to `shuffle` may produce different reorderings of the same input array.

Overall, the specification states exactly one semantic property of the resulting array: it is a rearrangement of the original array and nothing more. No assumption is made about which particular rearrangement is produced in any given call.

```
/*@
  requires   valid:    \valid(a + (0..n-1));
  requires   valid:    \valid(seed + (0..2));
  requires   sep:      \separated(a + (0..n-1), seed + (0..2));

  terminates          \true;
  exits               \false;
  assigns             a[0..n-1];
  assigns             seed[0..2];

  ensures        reorder: MultisetReorder{Old,Here}(a,n);
*/
void shuffle(value_type* a, size_type n, unsigned short* seed);
```

Listing 7.60: Formal specification of shuffle

Ideally, one would like to express that repeated calls to `shuffle` produce reorderings that are random in a well-defined sense. The informal specification of `shuffle` [15, §28.6.13] states that each possible ordering of the elements should occur with equal probability. Such a requirement, however, is inherently a property of sequences of executions rather than of a single function call. At present, ACSL does not provide mechanisms for specifying or verifying such temporal or probabilistic properties.

More generally, formalizing randomness in a mathematically precise and verification-friendly way is a nontrivial task. In practice, randomness is often assessed by means of statistical tests applied to sequences of generated values. From a theoretical perspective, one possible characterization is based on Kolmogorov complexity, which informally requires that the generated sequence admits no description substantially shorter than the sequence itself. Such notions, however, are undecidable in general and therefore lie outside the scope of deductive program verification.

7.18.3. Implementation of shuffle

The following listing shows our implementation of the function `shuffle` [7.61]. The algorithm proceeds incrementally and repeatedly calls `swap` [7.7] to exchange pairs of elements, thereby building up a reordering of the input array. The indices of the elements to be swapped are determined using a simple pseudo-random number generator; for details of this source of randomness we refer to the function `random_number` [7.64].

From a verification perspective, the central challenge is to relate the sequence of local swaps performed by the algorithm to the global postcondition stating that the entire array has been reordered. This is achieved by means of two loop invariants, labelled `reorder` and `unchanged`. At the beginning of the i th loop iteration, the prefix $a[0 \dots i)$ has already been rearranged, while the suffix $a[i \dots n)$ is still equal to its initial contents.

```
void shuffle(value_type* a, size_type n, unsigned short* seed)
{
    if (0u < n) {
        /*@
        loop invariant bound:      1 <= i <= n;
        loop invariant reorder:    MultisetReorder{Pre,Here}(a, 0, i);
        loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
        loop assigns    i, a[0..n-1], seed[0..2];
        loop variant    n - i;
        */
        for (size_type i = 1u; i < n; ++i) {
            size_type k = random_number(seed, i) + 1u;

            /*@ assert less: 0 <= k <= i;
            if (k < i) {
                swap(&a[k], &a[i]);
                /*@ assert swapped: ArraySwap{LoopCurrent,Here}(a, k, i, n);
                /*@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, i+1);
                /*@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
            }
            else {
                /*@ assert unchanged: Unchanged{LoopCurrent,Here}(a, i+1);
                /*@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
            }

            /*@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
        }
    }
}
```

Listing 7.61: Implementation of shuffle

In order to make this reasoning accessible to the automatic provers, the implementation contains several auxiliary assertions labelled `reorder`. These assertions make explicit how the current iteration preserves the reordering property across loop steps. They are formulated using the predefined label `LoopCurrent` and serve solely to guide the verification of the loop invariant. In particular, the empty `else`-branch contains only such an assertion. Although operationally redundant, this assertion is crucial for establishing that the `reorder` invariant is maintained in all cases.

To keep the annotations concise, we do not reason directly with the postcondition of `swap` [7.7]. Instead, we introduce the auxiliary predicate `ArraySwap` [7.62], which abstracts the effect of exchanging two array elements in a value-oriented manner. Using this predicate leads to significantly more compact annotations and improves the performance of the automatic provers.

```

/*@
  predicate ArraySwap{K,L}(value_type* a, integer i, integer k, integer n) =
    0 <= i < k < n          &&
    At{K}(a, i) == At{L}(a, k) &&
    At{K}(a, k) == At{L}(a, i) &&
    Unchanged{K,L}(a, 0, i)  &&
    Unchanged{K,L}(a, i+1, k) &&
    Unchanged{K,L}(a, k+1, n);
*/

```

Listing 7.62: `ArraySwap` (ACSL)

On top of this abstraction, we rely on a small collection of lemmas that relate swapping operations to multiset preservation. In particular, the lemma `MultisetSwap_Middle` [7.63] states that swapping the elements $a[i]$ and $a[k]$ constitutes a specific instance of a multiset reordering on the range $a[i \dots k+1]$. This lemma forms the key link between the local effect of a single swap and the global `MultisetReorder` [7.59] property used in the contract of `shuffle`.

```

/*@
  lemma MultisetSwap_Middle{K,L}:
    \forall value_type* a, integer i, k, n;
      ArraySwap{K,L}(a, i, k, n) ==> MultisetReorder{K,L}(a, i, k+1);

  lemma MultisetSwap_FrontMiddle{K,L}:
    \forall value_type* a, integer i, k, n;
      ArraySwap{K,L}(a, i, k, n) ==> MultisetReorder{K,L}(a, 0, k+1);
*/

```

Listing 7.63: `MultisetSwap` (ACSL)

7.19. Verifying a random number generator

In this section, we describe the auxiliary function `random_number` [7.64], which implements a simple pseudo-random number generator used by `shuffle`. Unlike the algorithms studied elsewhere in this document, `random_number` is not part of the C++ Standard Library but serves solely as a supporting component. Accordingly, the structure of this section differs slightly from that of the surrounding algorithm chapters.

As in the case of `shuffle` itself, we do not attempt to formalize statistical properties of randomness. Instead, the specification of `random_number` merely requires that the returned value lies within the prescribed range $[0 \dots n)$. The `assigns`-clause for the array `state` models the dependence of the generator on a mutable internal state and makes explicit that successive calls may yield different results.

In the same listing, we also provide the specification of the auxiliary function `random_init`, which initializes the generator state. Both specifications are shown below.

```
/*@
  requires   pos:      0 < n;
  requires   valid:    \valid(state + (0..2));

  terminates      \true;
  exits           \false;
  assigns         state[0..2];

  ensures      result:  0 <= \result < n;
*/
size_type random_number(unsigned short* state, size_type n);

/*@
  requires   valid:    \valid(state + (0..2));

  terminates      \true;
  exits           \false;
  assigns         state[0..2];
*/
void random_init(unsigned short* state);
```

Listing 7.64: Formal specification of `random_number`

The implementations of `random_number` and `random_init` are given in the following listing. Internally, the generator is based on a custom implementation of the POSIX.1 function `lrand48()`¹³. This generator is a linear congruential generator with a 48-bit state. Its iteration step is given by

$$x_{n+1} = ax_n + c \bmod 2^{48}, \quad (7.12)$$

where $a = 25214903917$ and $c = 11$ are relatively prime integers.

During the computation of Equation (7.12), unsigned arithmetic overflow may occur. This overflow is intentional and benign, since only the lowest 48 bits of the result are retained. However, one of the verification options used throughout this document, `-warn-unsigned-overflow`, instructs Frama-C/WP to assert the absence of such overflows. As a consequence, the verification of `random_number` would fail under the same option settings used for the other algorithms.

¹³See <http://pubs.opengroup.org/onlinepubs/9699919799/functions/lrand48.html>

As an exception, we therefore disable the `-warn-unsigned-overflow` option for this function. This is justified because unsigned overflow is well-defined in C and does not affect the correctness of the computation (cf. [16, §6.2.5, 9]).

```
// see IEEE 1003.1-2008, 2016 Edition for specification
/*@
  requires valid: \valid(seed + (0..2));
  assigns seed[0..2];
  ensures lower: 0 <= \result;
  ensures upper: \result <= 0x7fffffff;
*/
static long my_lrand48(unsigned short* seed)
{
  unsigned long long state = (unsigned long long)seed[0] << 32
                              | (unsigned long long)seed[1] << 16
                              | (unsigned long long)seed[2];
  state = (0x5deece66dull * state + 0xbull) % (1ull << 48);
  //@ assert upper: state < (1ull << 48);
  long result = state / (1ull << 17);
  //@ assert lower: 0 <= result;
  seed[0u] = state >> 32 & 0xffff;
  seed[1u] = state >> 16 & 0xffff;
  seed[2u] = state >> 8 & 0xffff;
  return result;
}

size_type random_number(unsigned short* state, size_type n)
{
  return my_lrand48(state) % n;
}

void random_init(unsigned short* state)
{
  state[0] = 0x243f;
  state[1] = 0x6a88;
  state[2] = 0x85a3;
}

```

Listing 7.65: Implementation of `random_number`

Finally, we rely on the auxiliary ACSL lemma `RandomNumberModulo` [7.66], shown in the following listing, to support the verification of several assertions concerning modular arithmetic in the implementation.

```
/*@
  lemma RandomNumberModulo:
    \forall unsigned long long a;
    (a % (1ull << 48)) < (1ull << 48);
*/

```

Listing 7.66: `C_Bit` (ACSL)

8. Numeric algorithms

The algorithms considered so far operate primarily on the structure of sequences. They compare values, read elements, or copy data from one array to another, but they do not modify values by means of arithmetic operations.

In this chapter, we turn to the so-called *numeric* algorithms of the C++ Standard Library [15, §29.8]. These algorithms combine the elements of one or more sequences by applying arithmetic operations on `value_type` and thereby introduce additional semantic and verification challenges.

To reason about potential arithmetic overflows, we introduce the two constants shown in Listing 8.1, which denote the numeric limits of `value_type` (see also §2). These constants will be used systematically in the specifications to characterize admissible input arrays and to exclude undefined arithmetic behavior.

```
#define VALUE_TYPE_MAX INT_MAX
#define VALUE_TYPE_MIN INT_MIN
```

Listing 8.1: Limits of `value_type`

The following numeric algorithms are considered in this chapter.

- `iota`, which writes sequentially increasing values into an array (§8.1);
- `accumulate`, which computes the sum of the elements in an array (§8.2);
- `inner_product`, which computes the inner product of two arrays (§8.3);
- `partial_sum`, which computes the sequence of partial sums of an array (§8.4);
- `adjacent_difference`, which computes the differences of adjacent elements in an array (§8.5);
- finally, in §8.6, we show that, under appropriate preconditions, the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

The formal specification of these algorithms raises new issues that did not arise in the earlier chapters. In particular, the presence of arithmetic operations requires an explicit treatment of numeric bounds in order to exclude arithmetic overflow and to ensure that the computed results coincide with their mathematical definitions.

8.1. The `iota` algorithm

The `iota` algorithm of the C++ Standard Library [15, §29.8.12] initializes an array with sequentially increasing values, starting from a user-provided initial value. In contrast to algorithms that merely rearrange existing elements, `iota` actively constructs new values and assigns them to the array.

For our purposes, the generic interface of the standard library is specialized to arrays of type `value_type`. The resulting signature reads:

```
void iota(value_type* a, size_type n, value_type v);
```

Starting with the value `v`, the function assigns consecutive values `v`, `v+1`, up to `v+n-1` to the elements of the array `a`. Since `value_type` is instantiated as an integer type, the specification must explicitly account for the possibility of arithmetic overflow when incrementing the initial value `v`. This aspect plays a central role in the formulation of the contract for `iota`.

8.1.1. Formal specification of `iota`

The specification of `iota` relies on the auxiliary predicate `IotaGenerate` [8.2], defined in the following listing.

```
/*@  
  predicate IotaGenerate(value_type* a, integer n, value_type v) =  
    \forall integer i; 0 <= i < n ==> a[i] == v+i;  
*/
```

Listing 8.2: `IotaGenerate` (ACSL)

The predicate `Iota(a, n, v)` expresses the intended effect of `iota` as an arithmetic progression: for every index `i` with $0 \leq i < n$, the element `a[i]` equals `v+i`.

The complete specification of `iota` is shown in the following listing. Its postcondition increment is precisely the assertion `IotaGenerate(a, n, v)`.

```
/*@  
  requires   valid:    \valid(a + (0..n-1));  
  requires   limit:    v + n <= VALUE_TYPE_MAX;  
  
  terminates          \true;  
  exits               \false;  
  assigns             a[0..n-1];  
  
  ensures   increment: IotaGenerate(a, n, v);  
*/  
void iota(value_type* a, size_type n, value_type v);
```

Listing 8.3: Formal specification of `iota`

The contract refers to `VALUE_TYPE_MAX`, the maximum representable value of `value_type` (see Listing 8.1). To rule out arithmetic overflow in the expression `v+i` occurring in `IotaGenerate`, the precondition `limit` requires `v+n <= VALUE_TYPE_MAX`. Since the quantification in `IotaGenerate` ranges over $0 \leq i < n$, we have `v+i < v+n`, and thus the bound `v+n <= VALUE_TYPE_MAX` implies `v+i <= VALUE_TYPE_MAX` for all indices covered by the postcondition. This formulation avoids a special treatment of the case `n==0` and keeps the overflow guard uniform.

The specification of `iota` refers to `VALUE_TYPE_MAX` which is the maximum value of the underlying integer type (see Listing 8.1). In order to avoid integer overflows the sum $v+n$ must not be greater than the constant `VALUE_TYPE_MAX`.

8.1.2. Implementation of `iota`

The following listing shows the implementation of the `iota` function together with its ACSL annotations.

```

void iota(value_type* a, size_type n, value_type v)
{
    /*@
        loop invariant bound:      0 <= i <= n;
        loop invariant value:      v == \at(v, Pre) + i;
        loop invariant increment: IotaGenerate(a, i, \at(v, Pre));

        loop assigns i, v, a[0..n-1];
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        a[i] = v++;
    }
}

```

Listing 8.4: Implementation of `iota`

The implementation follows the straightforward iterative scheme suggested by the informal description of the algorithm. Starting from the initial value v , the loop assigns the current value to the next array element and then increments v before proceeding to the next index.

The loop invariant value relates the local variable v to the iteration count. It states that upon entry to iteration i , the current value of the local variable v equals the initial value supplied to the function, plus the number of iterations already completed. Formally, this relationship is expressed as $v == \text{\texttt{\textbackslash at}(v, Pre)} + i$, where $\text{\texttt{\textbackslash at}(v, Pre)}$ denotes the value of v in the pre-state of the function `iota`.

This invariant establishes the precise link between the evolving program state and the arithmetic progression described by the specification. Combined with the invariant `increment`, which characterizes the contents of the already initialized prefix $a[0 \dots i)$, it allows the verifier to derive the postcondition `IotaGenerate(a, n, \text{\texttt{\textbackslash at}(v, Pre)})` upon loop termination. In particular, the invariant ensures that each assignment $a[i] = v$ produces exactly the value required by the formal specification.

8.2. The accumulate algorithm

The `accumulate` algorithm in the C++ Standard Library [15, §29.8.2] computes the sum of an initial value and the elements of a given array. Conceptually, it performs a left fold over the array, starting from the supplied initial value and successively incorporating each element.

For the purposes of this discussion, we consider the following simplified version of the original interface, specialized to arrays of type `value_type`:

```
value_type  
accumulate(const value_type* a, size_type n, value_type init);
```

The value returned by `accumulate` can be characterized mathematically as the sum of the initial value and all elements of the array. Formally, it equals

$$init + \sum_{i=0}^{n-1} a_i.$$

Here, a_i denotes the element stored at index i of the array a , and n is the length of the array. In particular, if the array is empty, that is, if $n == 0$, the algorithm returns the initial value `init` unchanged.

8.2.1. The logic function `Accumulate`

As in the case of `count` [4.57], we specify `accumulate` by first defining the *logic function* `Accumulate` [8.5], which formalizes summation over an array prefix.

The logic function `Accumulate` maps an array a , a length n , and an initial value `init` to the corresponding mathematical sum. For $n \geq 0$, the following equation characterizes its meaning:

$$\text{Accumulate}(a, n, \text{init}) = \text{init} + \sum_{i=0}^{n-1} a_i. \quad (8.1)$$

Here, a_i denotes the element stored at index i of the array a . The right-hand side is a mathematical sum, and in particular it is not subject to the wrap-around behavior of machine arithmetic.

The specification also introduces the predicate `AccumulateBounds` [8.5]. This predicate is used to express overflow requirements in a compact form. It states that, for every i with $0 \leq i < n$, the partial sums

$$\text{init} + \sum_{k=0}^i a_k \quad (8.2)$$

are representable in the underlying integer type.

The role of `AccumulateBounds` is essential. If any intermediate sum overflowed, the value computed by the C implementation of `accumulate` could differ from the mathematical value defined by `Accumulate`, even when the final result itself lies within bounds.

```

/*@
logic integer
Accumulate{L}(value_type* a, integer n, integer init) =
  n <= 0 ? init : Accumulate(a, n-1, init) + a[n-1];

predicate AccumulateBounds{L}(value_type* a, integer n, value_type init) =
  \forall integer i; 0 <= i <= n ==>
    VALUE_TYPE_MIN <= Accumulate(a, i, init) <= VALUE_TYPE_MAX;

lemma Accumulate_Init:
  \forall value_type *a, init, integer n;
    n <= 0 ==> Accumulate(a, n, init) == init;

lemma Accumulate_Next{L}:
  \forall value_type *a, init, integer n;
    0 <= n ==>
      Accumulate(a, n+1, init) == Accumulate(a, n, init) + a[n];

lemma Accumulate_Unchanged{K,L}:
  \forall value_type *a, init, integer n;
    Unchanged{K,L}(a, n) ==>
      Accumulate{K}(a, n, init) == Accumulate{L}(a, n, init);

lemma Accumulate_Unchanged_Shrink{K,L}:
  \forall value_type *a, init, integer m, n;
    0 <= m <= n ==>
      Unchanged{K,L}(a, n) ==>
        Accumulate{K}(a, m, init) == Accumulate{L}(a, m, init);

lemma AccumulateBounds_Unchanged{K,L}:
  \forall value_type *a, init, integer n;
    Unchanged{K,L}(a, n) ==>
      AccumulateBounds{K}(a, n, init) ==>
        AccumulateBounds{L}(a, n, init);
*/

```

Listing 8.5: Accumulate (ACSL)

8.2.2. AccumulateDefault— a variant of Accumulate

The following listing introduces the logic function `AccumulateDefault` [8.6], a second logic function related to `Accumulate` [8.5]. Unlike `Accumulate`, this variant does not take an explicit initial value as a parameter. Instead, the accumulation is defined to start from the constant value 0. Formally, `AccumulateDefault` is defined as a direct specialization of `Accumulate` with `init` fixed to 0. Consequently, for all $n \geq 0$, the following equation holds:

$$\text{AccumulateDefault}(a, n) = \sum_{i=0}^{n-1} a_i. \quad (8.3)$$

This formulation corresponds to the usual mathematical notion of a prefix sum and avoids the special treatment of the first array element. In particular, the case $n = 0$ is well defined and yields the neutral value 0.

We will use `AccumulateDefault` in the specification of the algorithm `partial_sum` [8.13], where such a canonical initialization is natural.

```

/*@
logic integer
AccumulateDefault{L}(value_type* a, integer n) =
    Accumulate(a, n, (value_type)(0));

predicate AccumulateDefaultBounds{L}(value_type* a, integer n) =
    AccumulateBounds(a, n, (value_type)(0));

lemma AccumulateDefault_Unchanged{K,L}:
    \forall value_type *a, integer n;
        0 <= n ==>
            Unchanged{K,L}(a, n) ==>
                AccumulateDefault{K}(a, n) == AccumulateDefault{L}(a, n);

lemma AccumulateDefault_Zero{L}:
    \forall value_type* a; AccumulateDefault(a, 0) == 0;

lemma AccumulateDefault_One{L}:
    \forall value_type* a; AccumulateDefault(a, 1) == a[0];

lemma AccumulateDefault_Next{L}:
    \forall value_type* a, integer n;
        0 <= n ==>
            AccumulateDefault(a, n+1) == AccumulateDefault(a, n) + a[n];

lemma AccumulateDefaultBounds_Shrink{L}:
    \forall value_type* a, integer m, n;
        0 <= m <= n ==>
            AccumulateDefaultBounds(a, n) ==> AccumulateDefaultBounds(a, m);
*/

```

Listing 8.6: AccumulateDefault (ACSL)

The listing further contains lemmas that characterize the observable behavior of `AccumulateDefault`. In particular, the step lemma expresses how the accumulated value evolves when the array is extended by one element. Finally, the predicate `AccumulateDefaultBounds` specializes `AccumulateBounds` [8.5] to the case of zero initialization and provides corresponding numeric limits for `AccumulateDefault`.

8.2.3. Formal specification of accumulate

The formal specification of `accumulate` [8.7] is expressed in terms of the logic function `Accumulate`, which defines the mathematical meaning of the accumulation process. The predicate `AccumulateBounds` additionally constrains the range of admissible values and characterizes conditions under which numerical overflows cannot occur during the accumulation process. Together, these abstractions allow the contract of `accumulate` to be stated in a direct and compact manner, as shown in the following listing.

```
/*@
requires   valid:   \valid_read(a + (0..n-1));
requires   bounds:  AccumulateBounds(a, n, init);

terminates          \true;
exits               \false;
assigns             \nothing;

ensures    result:  \result == Accumulate(a, n, init);
*/
value_type accumulate(const value_type* a, size_type n, value_type init);
```

Listing 8.7: Formal specification of `accumulate`

8.2.4. Implementation of `accumulate`

The following listing shows an implementation of the `accumulate` function together with its loop annotations.

```
value_type accumulate(const value_type* a, size_type n, value_type init)
{
    /*@
        loop invariant bound:    0 <= i <= n;
        loop invariant partial:  init == Accumulate(a, i, \at(init,Pre));
        loop assigns i, init;
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        //@ assert rte_help: init + a[i] == Accumulate(a, i+1, \at(init,Pre));
        init = init + a[i];
    }

    return init;
}
```

Listing 8.8: Implementation of `accumulate`

The central loop invariant `partial` expresses that upon entering the i -th iteration, the variable `result` represents the accumulated value defined by Equation (8.2) for the prefix $a[0 \dots i]$. The preservation of this invariant relies on the precondition `bounds` of `accumulate` [8.7], which characterizes those input values for which updating the accumulator does not lead to numerical overflow.

8.3. The inner_product algorithm

The `inner_product` algorithm of the C++ Standard Library [15, §29.8.4] computes the *inner product*¹⁴ of two arrays of equal length. Given two input sequences, the algorithm multiplies corresponding elements and accumulates the resulting products.

In our setting, the original signature is adapted to operate on `value_type` arrays and reads as follows:

```
value_type  
inner_product(const value_type* a, const value_type* b, size_type n,  
              value_type init);
```

The result returned by `inner_product` is defined as

$$\text{init} + \sum_{i=0}^{n-1} a[i] \cdot b[i].$$

In particular, for an empty array, the function returns the initial value `init`.

8.3.1. The logic function InnerProduct

As in the case of `accumulate` [8.7] we specify `inner_product` by defining in the following listing the logic function `InnerProduct` [8.9] that formally expresses the summation of the element-wise product of two arrays.

Predicate `ProductBounds` [8.9] expresses that for $0 \leq i < n$ the products

$$a[i] \cdot b[i] \tag{8.4}$$

do not overflow. Predicate `InnerProductBounds`, on the other hand, states that for $0 \leq i < n$ the following sums do not overflow.

$$\text{init} + \sum_{k=0}^i a[k] \cdot b[k] \tag{8.5}$$

These bounds are essential to ensure that any compliant implementation computes a result that coincides with the mathematical value defined by `InnerProduct`. Finally, Lemma `InnerProduct_Unchanged` states that the result of the `InnerProduct` only depends on the values of $a[0 \dots n)$ and $b[0 \dots n)$.

¹⁴Also referred to as the *dot product*, see http://en.wikipedia.org/wiki/Dot_product.

```

/*@
logic integer
InnerProduct{L}(value_type* a, value_type* b, integer n,
                 value_type init) =
  n <= 0 ? init : InnerProduct(a, b, n-1, init) + (a[n-1] * b[n-1]);

predicate ProductBounds(value_type* a, value_type* b, integer n) =
  \forall integer i; 0 <= i < n ==>
    VALUE_TYPE_MIN <= a[i] * b[i] <= VALUE_TYPE_MAX;

predicate InnerProductBounds(value_type* a, value_type* b, integer n,
                             value_type init) =
  \forall integer i; 0 <= i <= n ==>
    VALUE_TYPE_MIN <= InnerProduct(a, b, i, init) <= VALUE_TYPE_MAX;

lemma InnerProduct_Init:
  \forall value_type *a, *b, init, integer n;
    n <= 0 ==> InnerProduct(a, b, n, init) == init;

lemma InnerProduct_Next{L}:
  \forall value_type* a, *b, init, integer n;
    0 <= n ==>
      InnerProduct(a, b, n+1, init) == InnerProduct(a, b, n, init) + a[n] * b[n];

lemma InnerProduct_Unchanged{K,L}:
  \forall value_type *a, *b, init, integer n;
    Unchanged{K,L}(a, n) ==>
    Unchanged{K,L}(b, n) ==>
    InnerProduct{K}(a, b, n, init) == InnerProduct{L}(a, b, n, init);
*/

```

Listing 8.9: InnerProduct (ACSL)

8.3.2. Formal specification of inner_product

The logic function `InnerProduct` [8.9] captures the mathematical definition of the inner product of two arrays and forms the basis of the formal specification of `inner_product` [8.10]. Using this abstraction, the contract of `inner_product` can be stated in a direct and concise form, as shown in the following listing.

Note that the specification does not require the input arrays a and b to be separated. Since the algorithm only reads from both arrays and does not modify either of them, overlapping memory regions do not affect the correctness of the result.

```

/*@
requires   valid:      \valid_read(a + (0..n-1));
requires   valid:      \valid_read(b + (0..n-1));
requires   bounds:     ProductBounds(a, b, n);
requires   bounds:     InnerProductBounds(a, b, n, init);

terminates          \true;
exits               \false;
assigns             \nothing;

ensures   result:      \result == InnerProduct(a, b, n, init);
ensures   unchanged:   Unchanged{Old,Here}(a, n);
ensures   unchanged:   Unchanged{Old,Here}(b, n);
*/
value_type inner_product(const value_type* a, const value_type* b, size_type n,
                        value_type init);

```

Listing 8.10: Formal specification of inner_product

8.3.3. Implementation of inner_product

The following listing shows an implementation of inner_product together with its loop annotations.

```

value_type inner_product(const value_type* a, const value_type* b, size_type n,
                        value_type init)
{
    /*@
        loop invariant bound: 0 <= i <= n;
        loop invariant inner: init == InnerProduct(a, b, i, \at(init,Pre));
        loop assigns i, init;
        loop variant n-i;
    */
    for (size_type i = 0u; i < n; ++i) {
        /*@
            assert rte_help: init + a[i] * b[i] ==
                          InnerProduct(a, b, i+1, \at(init,Pre));
        */
        init = init + a[i] * b[i];
    }

    return init;
}

```

Listing 8.11: Implementation of inner_product

The central loop invariant `inner` expresses that upon entering the i -th iteration, the current value of `init` coincides with the partial inner product defined by Equation (8.5) for the prefix $a[0 \dots i]$ and $b[0 \dots i]$. In each iteration step, this invariant is preserved by extending the sum with the product $a[i] * b[i]$.

The bounds properties stated in the contract of `inner_product` [8.10] underpin the validity of this reasoning. These properties describe conditions under which neither the individual products nor the successive updates of the accumulator `init` lead to arithmetic overflow, thereby ensuring that the computed result agrees with the mathematical definition of the inner product.

8.4. The `partial_sum` algorithm

The `partial_sum` algorithm of the C++ Standard Library [15, §29.8.6] computes prefix sums of a given input array. More precisely, it successively accumulates the elements of the input array and stores the intermediate results in an output array. In our setting, the original signature is adapted to operate on `value_type` arrays and reads as follows:

```
size_type  
partial_sum(const value_type* a, size_type n, value_type* b);
```

After execution of `partial_sum`, the output array $b[0 \dots n)$ satisfies

$$b[i] = \sum_{k=0}^i a[k] \quad (8.6)$$

for all $0 \leq i < n$. That is, each entry $b[i]$ contains the sum of the prefix $a[0 \dots i + 1)$.

8.4.1. The predicate `PartialSum`

The predicate `PartialSum` [8.12] formalizes the relationship between an input array a and an output array b , where each entry $b[i]$ represents the partial sum of a up to index i , as defined in Equation (8.6). For each index i , it states that the value stored in $b[i]$ corresponds to the sum of the prefix $a[0 \dots i + 1)$.

```
/*@  
  predicate PartialSum{L}(value_type* a, integer n, value_type* b) =  
    \forall integer i; 0 <= i < n ==> b[i] == AccumulateDefault(a, i+1);  
  
  lemma PartialSum_Section{K}:  
    \forall value_type *a, *b, integer m, n;  
    0 <= m <= n ==>  
    PartialSum{K}(a, n, b) ==> PartialSum{K}(a, m, b);  
  
  lemma PartialSum_Step{L}:  
    \forall value_type *a, *b, integer n;  
    0 <= n ==>  
    PartialSum(a, n, b) ==>  
    b[n] == AccumulateDefault(a, n+1) ==> PartialSum(a, n+1, b);  
  
  lemma PartialSum_Unchanged{K,L}:  
    \forall value_type *a, *b, integer n;  
    0 <= n ==>  
    PartialSum{K}(a, n, b) ==>  
    Unchanged{K,L}(a, n) ==>  
    Unchanged{K,L}(b, n) ==> PartialSum{L}(a, n, b);  
  
  lemma PartialSum_One{L}:  
    \forall value_type *a, *b, integer n;  
    b[0] == AccumulateDefault(a, 1) ==> PartialSum(a, 1, b);  
*/
```

Listing 8.12: `PartialSum` (ACSL)

Rather than introducing a new logic function, the definition of `PartialSum` is naturally expressed in terms of the logic function `AccumulateDefault` [8.6]. This reuse reflects the fact that `partial_sum` computes a sequence of accumulated values with a fixed initial value.

8.4.2. Formal specification of `partial_sum`

The specification of `partial_sum` [8.13] requires that the input array $a[0 \dots n)$ and the output array $b[0 \dots n)$ are separated, that is, the two arrays do not overlap. This assumption simplifies the specification by ensuring that each value $a[i]$ is read before any corresponding entry of the output array is written.

```
/*@
requires   valid:      \valid_read(a + (0..n-1));
requires   valid:      \valid(b + (0..n-1));
requires   sep:        \separated(a + (0..n-1), b + (0..n-1));
requires   bounds:     AccumulateDefaultBounds(a, n);

terminates           \true;
exits                \false;
assigns              b[0..n-1];

ensures   result:      \result == n;
ensures   partialsum:  PartialSum(a, n, b);
ensures   unchanged:   Unchanged{Old,Here}(a, n);
*/
size_type partial_sum(const value_type* a, size_type n, value_type* b);
```

Listing 8.13: Formal specification of `partial_sum`

This requirement is stricter than that of the original C++ version of `partial_sum`, which permits a and b to refer to the same array and thus allows the computation of partial sums *in place*. From a functional point of view, such in-place execution is well defined, but accommodating it at the specification level would require a more subtle treatment of aliasing and data dependencies.

For the purposes of the present exposition, the separation condition provides a clear and robust contract that avoids these additional complications while still capturing the essential behavior of the algorithm.

8.4.3. Implementation of `partial_sum`

The following listing shows an implementation of `partial_sum` together with its loop annotations.

```
size_type partial_sum(const value_type* a, size_type n, value_type* b)
{
    if (0u < n) {
        //@ assert bounds:      AccumulateDefaultBounds(a, n);
        b[0u] = a[0u];
        //@ assert unchanged:   Unchanged{Pre,Here}(a, n);
        //@ assert bounds:      AccumulateDefaultBounds(a, n);
        //@ assert accumulate:  b[0] == AccumulateDefault(a, 1);
        //@ assert partialsum:  PartialSum(a, 1, b);

        /*@
            loop invariant bound:      1 <= i <= n;
            loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
            loop invariant accumulate: b[i-1] == AccumulateDefault(a, i);
            loop invariant bounds:      AccumulateDefaultBounds(a, n);
            loop invariant partialsum: PartialSum(a, i, b);
            loop assigns i, b[1..n-1];
            loop variant n - i;
        */
        for (size_type i = 1u; i < n; ++i) {
            b[i] = b[i - 1u] + a[i];
            //@ assert unchanged:  Unchanged{LoopCurrent,Here}(b, i);
            //@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, n);
            //@ assert partialsum: b[i] == AccumulateDefault(a, i+1);
            //@ assert bounds:      AccumulateDefaultBounds(a, n);
        }
    }

    return n;
}
```

Listing 8.14: Implementation of `partial_sum`

The implementation computes the output array incrementally. In each iteration, the next partial sum is obtained by adding the current input element $a[i]$ to the previously computed sum stored in $b[i - 1]$. This iterative structure is reflected in the loop invariant `partialsum`, which states that upon entering the i -th iteration, the prefix $b[0 \dots i)$ already contains the correct partial sums as defined by Equation (8.6).

The correctness of the update step relies on the bounds conditions stated in the contract of `partial_sum` [8.13], which ensure that neither the individual additions nor the successive accumulation of values lead to arithmetic overflow. In addition, the separation requirement between the input array a and the output array b guarantees that each input value is read before any overlapping memory location is modified. This avoids aliasing effects and allows the invariant to be maintained in a straightforward manner throughout the loop.

8.5. The adjacent_difference algorithm

The `adjacent_difference` algorithm of the C++ Standard Library [15, §29.8.11] computes the differences between consecutive elements of a given input array. The result is written to an output array of the same length, where the first element is copied unchanged and each subsequent element records the difference to its predecessor.

In our setting, the original signature is adapted to operate on `value_type` arrays and reads as follows:

```
size_type  
adjacent_difference(const value_type* a, size_type n, value_type* b);
```

After execution of `adjacent_difference`, the output array $b[0 \dots n)$ satisfies

$$\begin{aligned} b[0] &= a[0], \\ b[i] &= a[i] - a[i-1] \quad \text{for } 1 \leq i < n. \end{aligned} \tag{8.7}$$

8.5.1. The predicate `AdjacentDifference`

The predicate `AdjacentDifference` [8.16] forms the core of the specification of `adjacent_difference` and formalizes the relationship between the input array a and the output array b . As a preparatory step, we introduce the logic function `Difference` [8.15].

This function defines the difference between two adjacent elements of an array. It primarily serves as a convenient abstraction for expressing and structuring properties related to adjacent differences and is reused in several subsequent assertions.

```
/*@  
  logic integer  
  Difference{L}(value_type* a, integer n) =  
    n <= 0 ? a[0] : a[n] - a[n-1];  
  
  lemma Difference_Zero{L}:  
    \forall value_type *a; Difference(a, 0) == a[0];  
  
  lemma Difference_Next{L}:  
    \forall value_type *a, integer n;  
    0 < n ==> Difference(a, n) == a[n] - a[n-1];  
  
  lemma Difference_Unchanged{K,L}:  
    \forall value_type *a, integer n;  
    0 <= n ==> Unchanged{K,L}(a, n+1) ==>  
    Difference{K}(a, n) == Difference{L}(a, n);  
  
  lemma Difference_UnchangedRange{K,L}:  
    \forall value_type *a, integer n;  
    0 <= n ==> Unchanged{K,L}(a, n) ==>  
    \forall integer i; 0 <= i < n ==>  
    Difference{K}(a, i) == Difference{L}(a, i);  
*/
```

Listing 8.15: `Difference` (ACSL)

Building on this logic function, we define the predicate `AdjacentDifference`, which states that the output array b contains the differences of adjacent elements of the input array a . To exclude arithmetic overflows when computing expressions of the form $a[i] - a[i-1]$, the inputs are further restricted by the predicate

AdjacentDifferenceBounds. The lemmas record fundamental properties of AdjacentDifference and will later be used to establish and maintain loop invariants.

```

/*@
  predicate AdjacentDifference{L}(value_type* a, integer n, value_type* b) =
    \forall integer i; 0 <= i < n ==> b[i] == Difference(a, i);

  predicate AdjacentDifferenceBounds(value_type* a, integer n) =
    \forall integer i; 1 <= i < n ==>
      VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;

  lemma AdjacentDifference_Step{K,L}:
    \forall value_type *a, *b, integer n;
      AdjacentDifference{K}(a, n, b) ==>
        Unchanged{K,L}(b, n) ==>
        Unchanged{K,L}(a, n+1) ==>
        \at(b[n],L) == Difference{L}(a, n) ==>
        AdjacentDifference{L}(a, n+1, b);

  lemma AdjacentDifference_Section{K}:
    \forall value_type *a, *b, integer m, n;
      0 <= m <= n ==>
        AdjacentDifference{K}(a, n, b) ==>
        AdjacentDifference{K}(a, m, b);
*/

```

Listing 8.16: AdjacentDifference (ACSL)

8.5.2. Formal specification of adjacent_difference

The predicates AdjacentDifference [8.16] and AdjacentDifferenceBounds form the basis of the formal specification of adjacent_difference [8.17]. Together, these predicates characterize the functional relationship between the input and output arrays and restrict the admissible inputs so as to exclude arithmetic overflows when computing adjacent differences. Using these abstractions, the contract of adjacent_difference can be stated in a compact and direct form.

```

/*@
  requires valid: \valid_read(a + (0..n-1));
  requires valid: \valid(b + (0..n-1));
  requires sep: \separated(a + (0..n-1), b + (0..n-1));
  requires bounds: AdjacentDifferenceBounds(a, n);

  terminates \true;
  exits \false;
  assigns b[0..n-1];

  ensures result: \result == n;
  ensures difference: AdjacentDifference(a, n, b);
  ensures unchanged: Unchanged{Old,Here}(a, n);
*/
size_type adjacent_difference(const value_type* a, size_type n, value_type* b);

```

Listing 8.17: Formal specification of adjacent_difference

As in the specification of partial_sum [8.13], we require that the input array $a[0 \dots n)$ and the output array $b[0 \dots n)$ are separated. This assumption avoids aliasing between reads from a and writes to b and thereby simplifies both the specification and the subsequent verification.

8.5.3. Implementation of adjacent_difference

The following listing shows an implementation of adjacent_difference together with its loop annotations.

```
size_type adjacent_difference(const value_type* a, size_type n, value_type* b)
{
    if (0u < n) {
        b[0u] = a[0u];

        /*@
            loop invariant bound:      1 <= i <= n;
            loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
            loop invariant difference: AdjacentDifference(a, i, b);
            loop assigns i, b[1..n-1];
            loop variant n - i;
        */
        for (size_type i = 1u; i < n; ++i) {
            //@ assert bound: VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;
            b[i] = a[i] - a[i - 1u];
            //@ assert unchanged: Unchanged{LoopCurrent,Here}(b, i);
            //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, i+1);
            //@ assert difference: AdjacentDifference(a, i+1, b);
        }
    }

    return n;
}
```

Listing 8.18: Implementation of adjacent_difference

The implementation proceeds sequentially through the input array and computes each adjacent difference independently. The central loop invariant difference expresses that upon entering the i -th iteration, the prefix $b[0 \dots i)$ already satisfies the predicate `AdjacentDifference` [8.16] with respect to the corresponding prefix of the input array a .

The verification of this invariant relies on two complementary ingredients. First, the assertions `bound` and `difference` record local properties of the current iteration state, in particular the absence of arithmetic overflow when computing differences. Second, the lemmas `AdjacentDifference_Step` [8.16] and `AdjacentDifference_Section` provide the logical means to extend the predicate `AdjacentDifference` from one iteration to the next and to reason about ranges.

Together, these elements allow the loop invariant difference to be established initially and preserved throughout the loop, thereby ensuring the correctness of the implementation with respect to the formal specification.

8.6. Inverting `partial_sum` and `adjacent_difference`

In this section, we show that, under suitable preconditions, `adjacent_difference` and `partial_sum` are inverse to each other. More precisely,

- applying `adjacent_difference` to the result of `partial_sum` recovers the original input sequence, and
- applying `partial_sum` to the result of `adjacent_difference` reconstructs the corresponding prefix sums.

The purpose of this section is therefore not to introduce new algorithms, but to relate the previously specified ones and to make their algebraic connection explicit. Accordingly, the presentation focuses on logical properties and their verification rather than on implementation details.

8.6.1. Inverting `partial_sum`

Let $a[0 \dots n)$ denote the input array and $b[0 \dots n)$ the output array produced by `partial_sum`. By definition of the algorithm, the entries of b are given by

$$\begin{aligned} b[0] &= a[0], \\ b[1] &= a[0] + a[1], \\ &\vdots \\ b[n-1] &= a[0] + a[1] + \dots + a[n-1]. \end{aligned}$$

If we now apply the algorithm `adjacent_difference` to the array $b[0 \dots n)$, we obtain an output array $a'[0 \dots n)$ whose entries satisfy

$$\begin{aligned} a'[0] &= b[0] &= a[0], \\ a'[1] &= b[1] - b[0] &= a[1], \\ &\vdots \\ a'[n-1] &= b[n-1] - b[n-2] &= a[n-1]. \end{aligned}$$

Thus, under ideal arithmetic, the original input array a is recovered.

Before formalizing this argument in ACSL, we introduce the predicate `DefaultBounds` [8.19], which will be used to capture the absence of arithmetic overflow throughout both computations.

```
/*@
  predicate DefaultBounds{L}(value_type* a, integer n) =
    \forall integer i; 0 <= i < n ==>
      VALUE_TYPE_MIN <= a[i] <= VALUE_TYPE_MAX;
*/
```

Listing 8.19: `DefaultBounds` (ACSL)

The lemma `PartialSum_Inverse`, shown in the following listing, formalizes the above reasoning. It states, at the logical level, that under the appropriate bounds assumptions, the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

```

/*@
lemma PartialSum_Inverse:
  \forall value_type *a, *b, integer n;
    0 <= n ==>
    PartialSum(a, n, b) ==>
    AdjacentDifference(b, n, a);

lemma AdjacentDifference_Inverse:
  \forall value_type *a, *b, integer n;
    0 <= n ==>
    AdjacentDifference(a, n, b) ==>
    PartialSum(b, n, a);

lemma AdjacentDifference_InverseBounds:
  \forall value_type *a, *b, integer n;
    0 <= n ==>
    DefaultBounds(a, n) ==>
    AdjacentDifference(a, n, b) ==>
    AccumulateDefaultBounds(b, n);
*/

```

Listing 8.20: NumericInverse (ACSL)

The following listing presents the C function `partial_sum_inv`, including both its contract and its implementation. This function first invokes `partial_sum` and subsequently applies `adjacent_difference` to the resulting array.

```

/*@
requires   valid:    \valid(a + (0..n-1));
requires   valid:    \valid(b + (0..n-1));
requires   sep:      \separated(a + (0..n-1), b + (0..n-1));
requires   bounds:   AccumulateDefaultBounds(a, n);
requires   bounds:   DefaultBounds(a, n);

terminates      \true;
exits           \false;
assigns         a[0..n-1], b[0..n-1];

ensures   unchanged:  Unchanged{Pre,Here}(a, n);
*/
void partial_sum_inv(value_type* a, size_type n, value_type* b)
{
  partial_sum(a, n, b);
  //@ assert difference: AdjacentDifference(b, n, a);
  adjacent_difference(b, n, a);
}

```

Listing 8.21: Implementation of `partial_sum_inv`

The contract of `partial_sum_inv` formulates preconditions that ensure that neither arithmetic overflow (property `bounds`) nor unintended aliasing between arrays (property `sep`) can occur during the computation. Under these assumptions, Frama-C is able to verify that the final call to `adjacent_difference` restores exactly the original contents of the array `a[0 .. n)` supplied to the initial call to `partial_sum`.

8.6.2. Inverting adjacent_difference

Let $a[0 \dots n)$ denote the input array and $b[0 \dots n)$ the output array produced by `adjacent_difference`. By definition of the algorithm, the entries of b are given by

$$\begin{aligned} b[0] &= a[0], \\ b[1] &= a[1] - a[0], \\ &\vdots \\ b[n-1] &= a[n-1] - a[n-2]. \end{aligned}$$

If we now apply the algorithm `partial_sum` to the array $b[0 \dots n)$, we obtain an output array $a'[0 \dots n)$ satisfying

$$\begin{aligned} a'[0] &= b[0] &= a[0], \\ a'[1] &= b[0] + b[1] &= a[1], \\ &\vdots \\ a'[n-1] &= b[0] + b[1] + \dots + b[n-1] &= a[n-1]. \end{aligned}$$

Thus, under ideal arithmetic, applying `partial_sum` to the result of `adjacent_difference` reconstructs the original input array a . The lemma `AdjacentDifference_Inverse` [8.20] formally captures this property by stating that the two algorithms are inverse to each other under appropriate preconditions.

The following listing presents the C function `adjacent_difference_inv`. It invokes the algorithm `adjacent_difference` and then applies `partial_sum` to the resulting array. Its contract formulates preconditions that ensure that neither arithmetic overflow (property `bounds`) nor unintended aliasing between arrays (property `sep`) can occur during the computation.

```

/*@
requires    size:      0 <= n;
requires    valid:     \valid(a + (0..n-1));
requires    valid:     \valid(b + (0..n-1));
requires    sep:       \separated(a + (0..n-1), b + (0..n-1));
requires    bounds:    DefaultBounds(a, n);
requires    bounds:    AdjacentDifferenceBounds(a, n);

terminates      \true;
exits           \false;
assigns         a[0..n-1], b[0..n-1];

ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void adjacent_difference_inv(value_type* a, size_type n, value_type* b)
{
    adjacent_difference(a, n, b);
    partial_sum(b, n, a);
}

```

Listing 8.22: Implementation of `adjacent_difference_inv`

The lemma `Unchanged_Transitive` [7.2] is additionally used to facilitate automatic verification. Moreover, the lemma `AdjacentDifference_InverseBounds` [8.20] is used to establish the bounds precondition required for the subsequent call to `partial_sum`.

Part IV.

Heaps and Sorting Algorithms

Introduction

This part is devoted to algorithms that reorder the elements of a sequence according to a specified ordering relation. In contrast to the algorithmic patterns studied earlier, these procedures fundamentally transform the structure of the data by permuting elements, while preserving their values. As such, they combine mutation with global ordering properties.

Chapter 9 introduces heap-based algorithms. Heaps provide a structured representation of partially ordered data and serve both as an abstract data structure and as a foundation for efficient algorithms. From a verification perspective, heaps give rise to characteristic structural invariants that relate local ordering constraints to global correctness properties.

Building on these ideas, Chapter 10 studies sorting algorithms and well-known classical implementations. Sorting serves as a canonical example of algorithms that impose a total order on a sequence. The verification of sorting algorithms requires reasoning not only about mutation and ordering, but also about permutation properties and the preservation of multisets of values.

Together, these chapters address algorithms whose primary purpose is the reorganization of data. They bring together mutation, ordering, and global structural invariants, and they form a natural culmination of the algorithmic developments presented in the preceding parts.

9. Heap Algorithms

In this chapter we study the heap algorithms of the C++ Standard Library [15, p. 28.7.7] and discuss their formal verification in some detail. The exposition and verification approach adopted here are largely based on the bachelor thesis of one of the authors [20].

A heap is an array-based data structure that supports efficient access to its largest element. At any time, this element is stored at the beginning of the array and can be removed or replaced in logarithmic time. These properties make heaps a natural foundation for priority queues.

The heap algorithms provided by the C++ Standard Library can be summarized as follows:

- `make_heap()` rearranges an array into a heap.
- `push_heap()` and `pop_heap()` update an existing heap after inserting or removing an element.
- `sort_heap()` transforms a heap into an array that is ordered in increasing order.

These algorithms transform arrays; predicate-style algorithms such as `is_heap()` are also discussed.

Figure 9.1 provides an overview of the heap algorithms by means of a concrete example. For illustration purposes, the array is interpreted as a binary tree in the standard way; the underlying array-to-tree mapping is discussed in §9.1. Algorithms that stand in a typical caller–callee relationship in common implementations are shown in the same color.

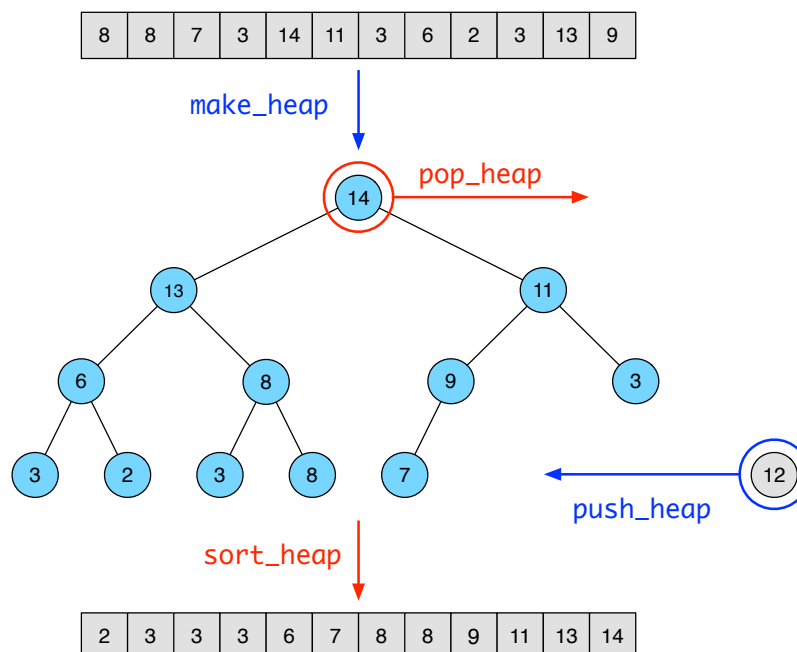


Figure 9.1.: Overview of heap algorithms

At a high level, the material covered in Figure 9.1 is organized as follows.

- In §9.1 we briefly recapitulate the basic concepts underlying heaps.
- In §9.2 we show how these concepts can be expressed in ACSL by means of logic functions and predicates.
- In §9.3 we verify two auxiliary functions that are used by several heap algorithms.
- The functions `is_heap_until` and `is_heap`, introduced in §9.4 and §9.5, allow one to check at run time whether a given array satisfies the heap property.
- In §9.6 we address the specific challenges that heap algorithms pose for formal reasoning. Although these algorithms preserve the multiset of elements globally, they may induce temporary local changes in multiplicity during intermediate steps. We therefore introduce predicates and lemmas that make such transient reordering behavior explicit while capturing multiset preservation as a global invariant.
- The algorithm `push_heap`, discussed in §9.7, inserts an element into a heap while preserving the heap property.
- The auxiliary function `heap_sift_down` from §9.8 repairs a heap whose defect is confined to a single node. It is not part of the C++ Standard Library, but it is the operation on which both `pop_heap` and `make_heap` rest.
- Conversely, the algorithm `pop_heap` from §9.9 removes an element from a heap, again preserving the heap property.
- The algorithm `make_heap` from §9.10 rearranges an arbitrary array into a heap.
- Finally, the algorithm `sort_heap` from §9.11 transforms a heap into an array that is ordered in increasing order.

We begin in §9.1 with a precise definition of heaps. The ACSL logic functions and predicates that formalize the fundamental heap properties are then introduced in §9.2.

9.1. Basic heap concepts

The informal description of heaps given at the beginning of this chapter necessarily remains imprecise. It highlights the intended behavior of heap operations but does not yet identify the exact and verifiable properties that an array must satisfy in order to qualify as a heap.

A more explicit characterization can be found, for example, in the Apache C++ Standard Library User's Guide:¹⁵

A heap is a binary tree in which every node is larger than the values associated with either child. A heap, and a binary tree in general, can be stored very efficiently in a vector by placing the children of node i at positions $2i + 1$ and $2i + 2$.

This description already suggests that the structure of a heap is determined entirely by simple arithmetic relations between array indices. More precisely, if a heap is represented as a contiguous array indexed

¹⁵See <http://stdcxx.apache.org/doc/stdlibug/14-7.html>

from 0, the positions of parent and child nodes are given by the following functions.

$$\text{left child of index } i \qquad \text{child}_l : i \mapsto 2i + 1 \qquad (9.1)$$

$$\text{right child of index } i \qquad \text{child}_r : i \mapsto 2i + 2 \qquad (9.2)$$

and

$$\text{parent of index } i \qquad \text{parent} : i \mapsto \frac{i - 1}{2} \qquad (9.3)$$

These functions are not independent. For all integers i , they satisfy the following relations. Note that in ACSL integer division rounds towards zero (cf. [9, §2.2.4]).

$$\text{parent}(\text{child}_l(i)) = i \qquad (9.4)$$

$$\text{parent}(\text{child}_r(i)) = i \qquad (9.5)$$

To illustrate the usefulness of heaps, we consider on the following pages the following multiset of integers, using the notion of multisets introduced in Section 7.18.1.

$$X = \{2, 3, 3, 3, 6, 7, 8, 8, 9, 11, 13, 14\} \qquad (9.6)$$

Figure 9.2 illustrates how the multiset from Equation (9.6) can be organized as a binary tree that respects the parent–child relations of a heap.

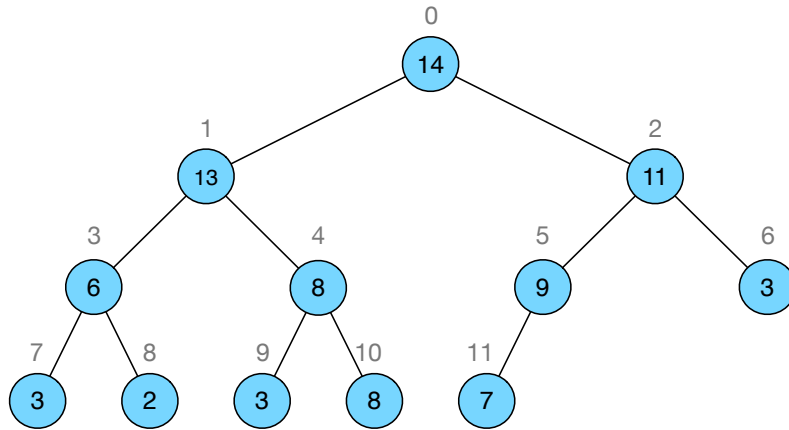


Figure 9.2.: Tree representation of the multiset X

The numbers shown outside the nodes in Figure 9.2 denote the array indices at which the corresponding values are stored. Using these indices, the tree representation can be flattened into a contiguous array, as shown in Figure 9.3.

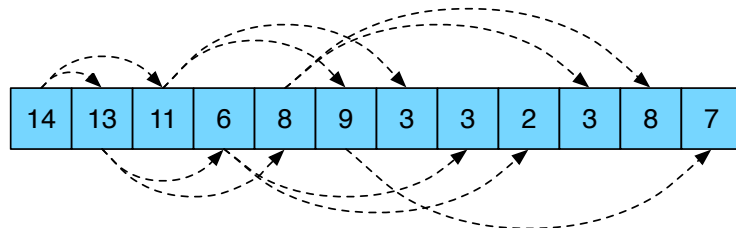


Figure 9.3.: Underlying array representation of the heap

At this point, it is worth emphasizing that the heap property constrains only local parent–child relations. It does not impose a unique global arrangement of the elements. In particular, a given multiset may admit several distinct heap realizations.

Consequently, a multiset does not determine a unique heap. Different arrangements of the same elements may satisfy the heap property, while inducing different tree shapes and array layouts. Figure 9.4 illustrates this by showing an alternative heap structure for the same multiset X .

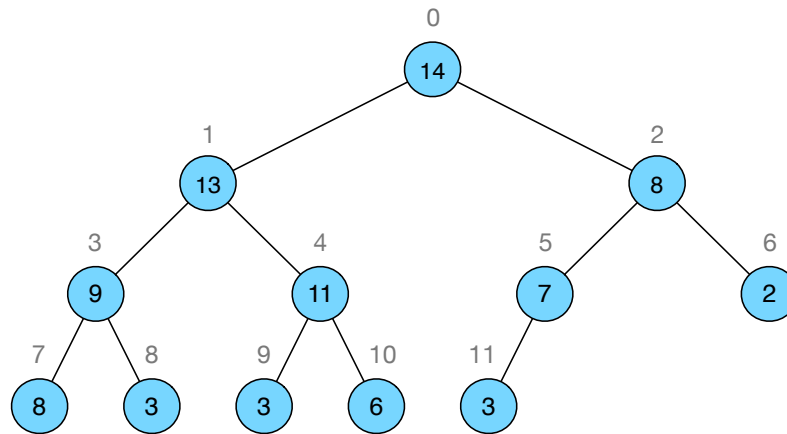


Figure 9.4.: An alternative heap representation of the multiset X

The corresponding array representation is given in Figure 9.5. Although the ordering of elements in the array differs from that in Figure 9.3, both arrays encode valid heaps and represent the same multiset.

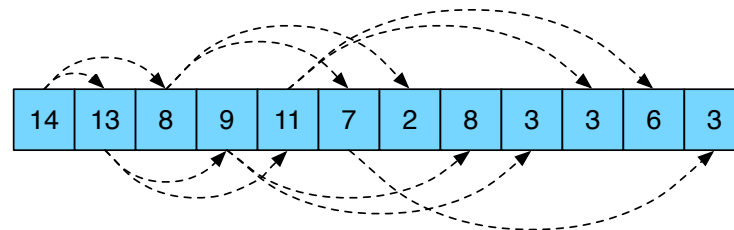


Figure 9.5.: Underlying array of the alternative heap representation

9.2. Representation of heap concepts in ACSL

We now formalize the index relations introduced in Section 9.1 using ACSL. The following listing defines three logic functions, `HeapLeft` [9.6], `HeapRight`, and `HeapParent`, which correspond directly to the mathematical definitions given in Equations (9.1), (9.2), and (9.3), respectively.

```
/*@
  logic integer HeapLeft(integer i) = 2*i + 1;

  logic integer HeapRight(integer i) = 2*i + 2;

  logic integer HeapParent(integer i) = (i-1) / 2;

  lemma HeapParent_Zero: HeapParent(0) == 0;

  lemma Heap_ParentLeft:
    \forall integer p; 0 <= p ==> HeapParent(HeapLeft(p)) == p;

  lemma Heap_ParentRight:
    \forall integer p; 0 <= p ==> HeapParent(HeapRight(p)) == p;

  lemma Heap_ParentChild:
    \forall integer c, p;
      0 < c ==> HeapParent(c) == p ==>
      (c == HeapLeft(p) || c == HeapRight(p));

  lemma Heap_Childs:
    \forall integer a, b;
      0 < a ==> 0 < b ==>
      HeapParent(a) == HeapParent(b) ==>
      (a == b || a+1 == b || a == b+1);

  lemma Heap_ParentBounds:
    \forall integer c; 0 < c ==> 0 <= HeapParent(c) < c;

  lemma Heap_ChildBounds:
    \forall integer p; 0 <= p ==> p < HeapLeft(p) < HeapRight(p);
*/
```

Listing 9.6: HeapNodes (ACSL)

In addition to these definitions, the listing contains several ACSL lemmas that capture essential algebraic properties of the index functions. In particular, the lemmas establish that

- the function `HeapParent` satisfies the identities (9.4) and (9.5), that is, applying `HeapParent` to either the left or right child of an index recovers the original index, and
- the function `HeapParent` acts as a *left inverse* of both `HeapLeft` and `HeapRight`, in the sense that composing `HeapParent` with either child function yields the identity on indices.¹⁶

These properties make explicit the close correspondence between the arithmetic index calculations used in heap implementations and their abstract tree-theoretic interpretation. They will be used repeatedly in subsequent specifications and proofs to relate array indices to parent–child relations in heaps.

Although the stated properties are mathematically elementary, it is important to record them explicitly as lemmas, as §4.3.1 explains for `Find`. Automated provers do not, in general, infer such relationships between user-defined logic functions and integer arithmetic without guidance. By making these properties

¹⁶See the section *Left and right inverses* at http://en.wikipedia.org/wiki/Inverse_function.

available as named lemmas, we ensure that subsequent proofs can rely on them uniformly when reasoning about parent–child relations and array indices in heap algorithms.

Building on these basic index definitions, we now introduce the predicate `Heap` [9.7], which formalizes the heap property for a range. Intuitively, this predicate expresses that for every valid index, the value stored at that index dominates the values stored at its children, whenever such children exist.

A fundamental consequence of this property is that the element stored at index 0 of a (maximum) heap is the largest element of the heap. This fact is stated explicitly by the lemma `Heap_Maximum` [9.7], which relates the predicate `Heap` [9.7] to the auxiliary predicate `MaxElement` [5.3].

```

/*@
predicate Heap{L}(value_type* a, integer n) =
  \forall integer i; 0 < i < n ==> a[i] <= a[HeapParent(i)];

lemma Heap_Maximum{L}:
  \forall value_type* a, integer n;
    0 < n ==> Heap(a, n) ==> MaxElement(a, n, 0);

lemma Heap_Shrink{L}:
  \forall value_type* a, integer m, n;
    0 <= m <= n ==> Heap(a, n) ==> Heap(a, m);

lemma Heap_Unchanged{K,L}:
  \forall value_type* a, integer n;
    0 <= n ==> Unchanged{K,L}(a, n) ==> Heap{K}(a, n) ==> Heap{L}(a, n);

predicate
HeapCompatible{L}(value_type* a, integer n, integer m, value_type v) =
  (0 <= m < n) &&
  (0 <= HeapParent(m) ==> v <= a[HeapParent(m)]) &&
  (HeapLeft(m) < n ==> a[HeapLeft(m)] <= v) &&
  (HeapRight(m) < n ==> a[HeapRight(m)] <= v);

lemma HeapCompatible_Update{K,L}:
  \forall value_type* a, v, integer m, n;
    0 <= m < n ==>
    Heap{K}(a, n) ==>
    HeapCompatible{K}(a, n, m, v) ==>
    ArrayUpdate{K,L}(a, n, m, v) ==> Heap{L}(a, n);
*/

```

Listing 9.7: Heap (ACSL)

The lemmas `Heap_Shrink` and `Heap_Unchanged` capture simple but frequently used reasoning principles. They allow the heap property to be transferred from a heap to a related (sub-)array, either by restricting the considered index range or by observing that modifications outside the relevant range do not affect the heap property.

Finally, the predicate `HeapCompatible` [9.7] characterizes the conditions under which updating a single element of a heap preserves the heap property. Such an update is modeled using the generic predicate `ArrayUpdate` [7.4], which expresses that exactly one array position is modified while all others remain unchanged. Together with the lemma `HeapCompatible_Update` [9.7], this yields a local reasoning principle for heap modifications. These results will play a central role in the verification of the algorithms `push_heap` [9.26] and `heap_sift_down` [9.31], where heap updates are realized by a sequence of local exchanges.

9.2.1. Forests of heaps

The predicate `Heap` [9.7] demands that *every* node dominate its children. The algorithm `heap_sift_down` [9.31] of §9.8 needs a weaker notion: it starts from an array in which a single node, and only that node, may violate the heap property. The predicate `HeapForest` [9.8] provides it. The expression `HeapForest(a, m, n)` demands the heap property of every parent–child pair whose *parent* index is at least m . Since a child carries a larger index than its parent, these are exactly the pairs that lie entirely within $a[m \dots n)$, and every index of that range therefore heads a subtree in which the heap property holds throughout. In other words, each index of $a[m \dots n)$ is the root of a heap—the range is a *forest* of heaps. In particular, `HeapForest(a, 1, n)` makes the subtrees rooted at the indices 1 and 2 heaps and leaves the root 0 unconstrained: an array that is a heap except possibly at its root.

The special case $m == 0$ constrains every node and thus coincides with `Heap` [9.7]; the lemma `HeapForest_Heap` [9.8] states this connection, which is how the algorithms of this chapter return from the weaker notion to the familiar one.

```

/*@
  predicate HeapForest{L}(value_type* a, integer m, integer n) =
    \forall integer c;
      0 < c < n && m <= HeapParent(c) ==> a[c] <= a[HeapParent(c)];

  lemma HeapForest_Heap{L}:
    \forall value_type* a, integer n;
      HeapForest(a, 0, n) ==> Heap(a, n);

  lemma HeapForest_Maximum{L}:
    \forall value_type* a, integer m, c, n;
      0 <= m <= c < n ==>
        HeapForest(a, m, n) ==>
        HeapAncestor(m, c) ==> a[c] <= a[m];

  predicate
  HeapForestCompatible{L}(value_type* a, integer n, integer root, integer p,
                          value_type v) =
    (root <= p < n) &&
    (0 < p && root <= HeapParent(p) ==> v <= a[HeapParent(p)]) &&
    (HeapLeft(p) < n ==> a[HeapLeft(p)] <= v) &&
    (HeapRight(p) < n ==> a[HeapRight(p)] <= v);

  lemma HeapForestCompatible_Update{K,L}:
    \forall value_type* a, integer n, root, p, value_type v;
      HeapForest{K}(a, root, n) ==>
      HeapForestCompatible{K}(a, n, root, p, v) ==>
      ArrayUpdate{K,L}(a, n, p, v) ==>
      HeapForest{L}(a, root, n);
*/

```

Listing 9.8: HeapForest (ACSL)

The predicate `HeapForestCompatible` [9.8] and the lemma `HeapForestCompatible_Update` [9.8] generalize `HeapCompatible` [9.7] and `HeapCompatible_Update` [9.7] in the same manner, and serve the same purpose: they justify a single array update from local conditions alone.

The lemma `HeapForest_Maximum` [9.8] generalizes `Heap_Maximum` [9.7]. In a heap the maximum resides at index 0; under `HeapForest` [9.8] the node m dominates only those nodes that are reachable from it by descending through the tree. That relation is the transitive closure of `HeapParent` [9.6] and is defined inductively.

```

/*@
  inductive HeapAncestor(integer m, integer c) {
    case HeapAncestor_Refl:
      \forall integer m;
        0 <= m ==> HeapAncestor(m, m);

    case HeapAncestor_Step:
      \forall integer m, c;
        0 < c ==> HeapAncestor(m, HeapParent(c)) ==> HeapAncestor(m, c);
  }

  lemma HeapAncestor_Bounds:
    \forall integer m, c;
      HeapAncestor(m, c) ==> 0 <= m <= c;

  lemma HeapAncestor_Root:
    \forall integer c;
      0 <= c ==> HeapAncestor(0, c);
*/

```

Listing 9.9: Inductive definition HeapAncestor

The base case relates every node to itself and the step case adds one descent, so that `HeapAncestor(m, c)` holds exactly if `m` lies on the path from `c` to the root. The lemma `HeapAncestor_Root` [9.9] records the consequence that the root is an ancestor of every node, and `HeapAncestor_Bounds` [9.9] that an ancestor never has the larger index. Both are proved interactively with Coq: reasoning about an inductive predicate requires an induction, which the automatic provers do not perform.

9.3. The auxiliary functions `heap_parent` and `heap_child`

This section introduces two auxiliary functions that support the implementation of heap algorithms. We begin with the function `heap_parent` [9.10], which serves as the C counterpart of the ACSL logic function `HeapParent` [9.6].

The correspondence is not exact. While `HeapParent` is defined for all integer indices, the executable function `heap_parent` deliberately avoids the boundary case of index 0, for which a parent node does not exist. This restriction reflects the intended use of `heap_parent` in heap algorithms, where it is applied only to valid child indices.

The interface of `heap_parent` is shown in the following listing. It specifies the expected behavior and the admissible range of inputs.

```

/*@
  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures   parent: \result == HeapParent(child);
*/
size_type heap_parent(size_type child);

```

Listing 9.10: Formal specification of `heap_parent`

The corresponding implementation is given next. It follows the specification closely and reduces the parent computation to a simple arithmetic expression.

```

size_type heap_parent(size_type child)
{
    return (0u < child) ? (child - 1u) / 2u : 0u;
}

```

Listing 9.11: Implementation of heap_parent

We do not provide exact C counterparts for the logic functions HeapLeft [9.6] and HeapRight. Indeed, throughout the development we encounter only a single situation — in the implementation of heap_sift_down [9.31] — where such functions would appear to be useful.

Even in this case, however, direct access to the left and right child indices is not what is required. Instead, the algorithm needs to determine, for a given parent index p , a child index c at which the maximum of the two values $a[\text{HeapLeft}(p)]$ and $a[\text{HeapRight}(p)]$ is stored. This task is performed by the auxiliary function heap_child [9.12].

```

/*@
requires    bound:    0 <= p < n;
requires    valid:    \valid(a + (0..n-1));

terminates    \true;
exits         \false;
assigns       \nothing;

ensures    bound:    p < \result <= n;
ensures    parent:   \result < n      ==> p == HeapParent(\result);
ensures    parent:   \result < n-1    ==> HeapLeft(p) < n-1;
ensures    parent:   \result < n-1    ==> HeapRight(p) < n;
ensures    left:     HeapLeft(p) < n ==> \result < n;
ensures    right:    HeapRight(p) < n ==> \result < n;
ensures    max:      HeapLeft(p) < n ==> a[HeapLeft(p)] <= a[\result];
ensures    max:      HeapRight(p) < n ==> a[HeapRight(p)] <= a[\result];
ensures    none:     \result == n     ==> n <= HeapLeft(p);
ensures    none:     \result == n     ==> n <= HeapRight(p);
*/
size_type heap_child(const value_type* a, size_type n, size_type p);

```

Listing 9.12: Formal specification of heap_child

Note that the implementation of `heap_child` [9.13] explicitly accounts for the possibility that the computation of child indices may overflow. If such an overflow occurs, the function returns the value `n`, which signals that no valid child index is available.

```
size_type heap_child(const value_type* a, size_type n, size_type p)
{
    if (p + 1u <= n - p - 1u) {
        const size_type left  = 2u * p + 1u;
        const size_type right = left + 1u;

        if (right < n) {
            // case of two children: select child with maximum value
            return a[right] <= a[left] ? left : right;
        }
        else {
            // at most one child that comes before n-1 can exist
            return left;
        }
    }
    else {
        return n;
    }
}
```

Listing 9.13: Implementation of `heap_child`

9.4. The `is_heap_until` algorithm

The algorithm `is_heap_until` of the C++ Standard Library [15, §28.7.7.5] operates on generic sequences and determines the largest prefix that satisfies the heap property. For the purposes of this chapter, we adapt the generic formulation to arrays of type `value_type`. The resulting interface is given by the following signature.

```
size_type is_heap_until(const value_type* a, size_type n);
```

Given an array `a` of length `n`, the function `is_heap_until` returns the smallest index at which the heap property is violated. Equivalently, it returns the length of the largest prefix $a[0 \dots k)$ that forms a heap in the sense formalized in Section 9.1. If the entire array satisfies the heap property, the function returns `n`.

In particular, when applied to the array shown in Figure 9.3, the function `is_heap_until` returns the value `n`, indicating that the array represents a valid heap.

9.4.1. Formal specification of `is_heap_until`

The formal specification of `is_heap_until` is given in the following listing. The return value `\result` denotes the boundary of the maximal heap prefix: the range $a[0 \dots \text{\result})$ satisfies the heap predicate `Heap` [9.7].

In addition, the postcondition `last` characterizes the returned index as the *first* position at which the heap property fails. It suffices to state this for the shortest prefix that exceeds the returned boundary: unless `\result` has reached `n`, appending the single element $a[\text{\result}]$ to the prefix already destroys the predicate `Heap` [9.7]. Lemma `Heap_Shrink` then carries the failure to every longer prefix, so that `\result` is maximal and not merely a valid heap boundary.

```
/*@
requires    valid:  \valid_read(a + (0..n-1));

terminates    \true;
exits         \false;
assigns       \nothing;

ensures    bound:  0 <= \result <= n;
ensures    heap:   Heap(a, \result);
ensures    last:   \result < n ==> !Heap(a, \result + 1);
*/
size_type is_heap_until(const value_type* a, size_type n);
```

Listing 9.14: Formal specification of `is_heap_until`

9.4.2. Implementation of `is_heap_until`

The following listing presents an implementation of the function `is_heap_until`.

```
size_type is_heap_until(const value_type* a, size_type n)
{
    size_type parent = 0u;

    /*@
    loop invariant bound:    0 <= parent < child <= n+1;
    loop invariant parent:   parent == HeapParent(child);
    loop invariant heap:     Heap(a, child);
    loop assigns child, parent;
    loop variant n - child;
    */
    for (size_type child = 1u; child < n; ++child) {
        if (a[parent] < a[child]) {
            return child;
        }

        if ((child % 2u) == 0u) {
            ++parent;
        }
    }

    return n;
}
```

Listing 9.15: Implementation of `is_heap_until`

The algorithm starts at index 1, which is the smallest index at which a child node may occur in the array-based representation of a heap. For each such index, the algorithm determines the corresponding parent index and checks whether the heap property is satisfied, that is, whether the value stored at the parent index is greater than or equal to the value stored at the child index.

If this condition is violated for some child index *i*, the algorithm terminates immediately and returns *i*, which marks the first position at which the heap property fails. If no such violation is encountered and the end of the array is reached, the algorithm returns the value *n*, indicating that the entire array satisfies the heap property.

9.5. The `is_heap` algorithm

The algorithm `is_heap` of the C++ Standard Library [15, §28.7.7.5] determines whether a given sequence satisfies the heap property. In the standard library, the algorithm is defined for generic iterator ranges. For the purposes of this chapter, we specialize the algorithm to operate on arrays of type `value_type`. The resulting interface is given by the following signature.

```
bool is_heap(const value_type* a, size_type n);
```

Given an array `a` of length `n`, the function `is_heap` returns **true** precisely if the array represents a heap in the sense defined in Section 9.1. That is, all parent–child relations induced by the array indices satisfy the heap property. In particular, when applied to the array shown in Figure 9.3, the function `is_heap` returns **true**.

9.5.1. Formal specification of `is_heap`

The formal specification of `is_heap` is shown in the following listing. The contract expresses a direct correspondence between the return value of the function and the abstract heap predicate. More precisely, the function returns **true** if and only if the input array satisfies the predicate `Heap` [9.7].

```
/*@  
  requires    valid:  \valid_read(a +(0..n-1));  
  
  terminates      \true;  
  exits          \false;  
  assigns        \nothing;  
  
  ensures    heap:  \result <==> Heap(a, n);  
*/  
bool is_heap(const value_type* a, size_type n);
```

Listing 9.16: Formal specification of `is_heap`

9.5.2. Implementation of `is_heap`

The implementation of `is_heap` is shown in the following listing. Rather than rechecking the heap property directly, the function delegates the actual work to `is_heap_until` [9.14].

```
bool is_heap(const value_type* a, size_type n)  
{  
  return is_heap_until(a, n) == n;  
}
```

Listing 9.17: Implementation of `is_heap`

The implementation simply tests whether the index returned by `is_heap_until` coincides with the array length `n`. This design emphasizes the close conceptual relationship between the two algorithms: `is_heap` acts as a Boolean wrapper around `is_heap_until`, trading detailed positional information for a simple yes-or-no answer.

9.6. Reorderings and temporary changes in multiplicity

Heap algorithms present a particular challenge for formal reasoning: while they do not alter the multiset of values as a whole, they may produce temporary changes in the multiplicity of values at specific positions in an array during intermediate steps. In other words, the count of a particular value at a given position can vary for a short period even though the overall collection of elements remains the same up to permutation.

To support rigorous specification and verification of such behavior we introduce a collection of predicates and supporting lemmas that make it possible to describe temporary local changes and the global invariant of multiset preservation.

9.6.1. Formalizing temporary changes in multiplicity

The predicate `MultisetAdd` [9.18] expresses that the number of occurrences of a specific value in an array has increased by one between two program points `K` and `L`.

```
/*@
predicate MultisetAdd{K,L}(value_type* a, integer n, value_type v) =
    Count{L}(a, 0, n, v) == Count{K}(a, 0, n, v) + 1;

predicate MultisetMinus{K,L}(value_type* a, integer n, value_type v) =
    MultisetAdd{L,K}(a, n, v);

predicate MultisetRetain{K,L}(value_type* a, integer n, value_type v) =
    Count{K}(a, 0, n, v) == Count{L}(a, 0, n, v);

lemma MultisetAdd_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
        0 <= m < n ==>
        At{K}(a, m) != v ==> At{L}(a, m) == v ==>
        MultisetReorder{K,L}(a, 0, m) ==>
        MultisetReorder{K,L}(a, m+1, n) ==> MultisetAdd{K,L}(a, n, v);

lemma MultisetMinus_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
        0 <= m < n ==>
        At{K}(a, m) == v ==> At{L}(a, m) != v ==>
        MultisetReorder{K,L}(a, 0, m) ==>
        MultisetReorder{K,L}(a, m+1, n) ==> MultisetMinus{K,L}(a, n, v);

lemma MultisetRetain_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
        0 <= m < n ==>
        At{K}(a, m) != v ==> At{L}(a, m) != v ==>
        MultisetReorder{K,L}(a, 0, m) ==>
        MultisetReorder{K,L}(a, m+1, n) ==> MultisetRetain{K,L}(a, n, v);
*/
```

Listing 9.18: MultisetOperations (ACSL)

The predicate `MultisetMinus`, in contrast, expresses that the number of occurrences of a specific value in an array has decreased by one between the program points `K` and `L`. For convenience, `MultisetMinus` is defined by invoking `MultisetAdd` with the labels `K` and `L` exchanged.

Finally, the predicate `MultisetRetain` expresses that the number of occurrences of a given value remains unchanged between two program points. Together, these predicates make it possible to describe local and temporary changes in multiplicity while reasoning about the evolution of an array during heap operations.

To support automated reasoning, we also provide a number of lemmas that characterize conditions under which the respective predicates hold. These lemmas are formulated using the predicate `MultisetReorder` [7.59] and the logic function `At` [7.3].

9.6.2. Simple properties of temporary changes in multiplicity

The predicate `MultisetRetainRest` [9.19] builds on the predicate `MultisetRetain` [9.18] to express that, between two program points `K` and `L`, all values of an array occur with the same multiplicity, except possibly for two distinguished values `u` and `v`. In other words, only the occurrences of `u` and `v` are allowed to change, while the multiplicities of all other values are preserved.

```

/*@
  predicate MultisetRetainRest{K,L}(value_type* a, integer n, value_type v,
    value_type w) =
    \forall value_type x;
      x != v ==> x != w ==> MultisetRetain{K,L}(a, n, x);

  lemma MultisetRetainRest_UnchangedFirst{K,L,M}:
    \forall value_type *a, u, v, integer n;
      Unchanged{K,L}(a, n) ==>
      MultisetRetainRest{L,M}(a, n, u, v) ==>
      MultisetRetainRest{K,M}(a, n, u, v);

  lemma MultisetRetainRest_UnchangedSecond{K,L,M}:
    \forall value_type *a, u, v, integer n;
      MultisetRetainRest{K,L}(a, n, u, v) ==>
      Unchanged{L,M}(a, n) ==>
      MultisetRetainRest{K,M}(a, n, u, v);

  lemma Multiset_AddMinusRetain{K,L,M}:
    \forall value_type *a, u, integer n;
      MultisetAdd{K,L}(a, n, u) ==>
      MultisetMinus{L,M}(a, n, u) ==> MultisetRetain{K,M}(a, n, u);

  lemma Multiset_MinusAddRetain{K,L,M}:
    \forall value_type *a, u, integer n;
      MultisetMinus{K,L}(a, n, u) ==>
      MultisetAdd{L,M}(a, n, u) ==> MultisetRetain{K,M}(a, n, u);

  lemma Multiset_AddMinusRetainReorder{K,L,M}:
    \forall value_type *a, u, v, integer n;
      u != v ==>
      MultisetAdd{K,L}(a, n, u) ==>
      MultisetMinus{K,L}(a, n, v) ==>
      MultisetRetainRest{K,L}(a, n, u, v) ==>
      MultisetAdd{L,M}(a, n, v) ==>
      MultisetMinus{L,M}(a, n, u) ==>
      MultisetRetainRest{L,M}(a, n, v, u) ==> MultisetReorder{K,M}(a, n);
*/

```

Listing 9.19: `MultisetRetainRest` (ACSL)

The first two lemmas state that the predicate is insensitive to a stretch of the program in which the array does not change at all, whether that stretch precedes or follows the temporary change. They carry no reasoning of their own, but they spare the automatic provers the step that is expensive here: `MultisetRetainRest` [9.19] quantifies over all values, so re-establishing it at a later program point means instantiating that quantifier, while transporting it across an unchanged stretch does not.

The remaining lemmas capture simple cancellation properties of temporary changes in multiplicity. They express conditions under which an increase in the number of occurrences of a value at one program point, described by `MultisetAdd` [9.18], is compensated by a corresponding decrease at a later program point, described by `MultisetMinus`. Taken together, these lemmas allow local changes in multiplicity between intermediate program points to be reconciled with the preservation of the overall multiset.

9.6.3. Combining temporary changes in multiplicity

Temporary changes in multiplicity occur so frequently in the central heap algorithms `push_heap` [9.26] and `heap_sift_down` [9.31] that it is useful to capture them by a dedicated predicate. For this purpose, we introduce the predicate `MultisetParity` [9.20].

```

/*@
predicate MultisetParity{K,L}(value_type* a, integer n, value_type u, value_type
    v) =
    MultisetAdd{K,L}(a, n, u)    &&
    MultisetMinus{K,L}(a, n, v)  &&
    MultisetRetainRest{K,L}(a, n, u, v);

lemma MultisetParity_UnchangedFirst{K,L,M}:
    \forall value_type *a, u, v, integer n;
    u != v ==>
    Unchanged{K,L}(a, n) ==>
    MultisetParity{L,M}(a, n, u, v) ==> MultisetParity{K,M}(a, n, u, v);

lemma MultisetParity_UnchangedSecond{K,L,M}:
    \forall value_type *a, u, v, integer n;
    u != v ==>
    MultisetParity{K,L}(a, n, u, v) ==>
    Unchanged{L,M}(a, n) ==> MultisetParity{K,M}(a, n, u, v);

lemma MultisetParity_MultisetReorder{K,L,M}:
    \forall value_type *a, u, v, integer n;
    u != v ==>
    MultisetParity{K,L}(a, n, u, v) ==>
    MultisetParity{L,M}(a, n, v, u) ==> MultisetReorder{K,M}(a, n);

lemma MultisetParity_Combined{K,L,M}:
    \forall value_type *a, u, v, w, integer n;
    u != v ==>
    u != w ==>
    v != w ==>
    MultisetParity{K,L}(a, n, u, v) ==>
    MultisetParity{L,M}(a, n, w, u) ==> MultisetParity{K,M}(a, n, w, v);
*/

```

Listing 9.20: `MultisetParity` (ACSL)

The predicate `MultisetParity` [9.20] describes a very specific pattern of local change between two program points. Given two distinct values, it expresses that

- the number of occurrences of the first value increases by one,
- the number of occurrences of the second value decreases by one, and
- the multiplicities of all other values remain unchanged.

The term *parity* reflects the fact that the total number of elements is preserved, and that the two changes compensate each other. Such paired changes arise naturally in heap algorithms, for example when a value is moved upward or downward in the array and temporarily replaces another value.

Using the predicate `MultisetParity` [9.20], we can formulate a number of lemmas that describe how temporary changes in multiplicity interact with reorderings. For example, the situation in which two consecutive parity changes cancel each other is captured by the lemma `MultisetParity_MultisetReorder`. As a result, the net effect between the initial and final program points is a pure reordering of the array, that is, preservation of the overall multiset.

9.6.4. How do temporary changes in multiplicity arise?

The simplest way to create a temporary change in multiplicity is to update a single array element with a different value. Such an update removes one occurrence of the original value and introduces one occurrence of the new value, thereby producing a compensating pair of changes and hence a *parity change* in the multiset of array elements.

Accordingly, and in analogy to the predicate `ArrayUpdate` [7.4], we introduce in the following Listing the predicate `MultisetUpdate`. This predicate characterizes array updates that give rise to a parity change in the sense formalized by `MultisetParity` [9.20].

```

/*@
  predicate MultisetUpdate{K,L}(value_type* a, integer n, integer i, value_type v)
    =
    \let u = At{K}(a, i);
    u != v                                &&
    0 <= i < n                            &&
    MultisetReorder{K,L}(a, 0, i)         &&
    MultisetReorder{K,L}(a, i+1, n)       &&
    MultisetParity{K,L}(a, n, v, u);

  lemma ArrayUpdate_MultisetUpdate{K,L}:
    \forall value_type *a, v, integer n, i;
      ArrayUpdate{K,L}(a, n, i, v) ==> MultisetUpdate{K,L}(a, n, i, v);
*/

```

Listing 9.21: `MultisetUpdate` (ACSL)

Lemma `ArrayUpdate_MultisetUpdate` [9.21] makes this relationship precise. It formalizes the fact that updating an array element with a different value induces a temporary change in multiplicity, while preserving the total number of elements.

9.6.5. What a reordering cannot do

Once the temporary changes have cancelled out, what remains is a reordering, and the essential property of a reordering is a negative one: it introduces no value that was not there before.

```

/*@
  lemma MultisetReorder_SomeEqual{K,L}:
    \forall value_type *a, integer n, i;
      0 < n ==>
      0 <= i < n ==>
      MultisetReorder{K,L}(a, n) ==>
      SomeEqual{K}(a, n, \at(a[i],L));
*/

```

Listing 9.22: MultisetReorderLemmas (ACSL)

Lemma `MultisetReorder_SomeEqual` [9.22] states this. If a range is reordered, then every value found in it afterwards already occurred in it beforehand, which the predicate `SomeEqual` [4.4] expresses. The lemma is the technical foundation for transferring bounds across a reordering: an upper bound of the range before the reordering is still an upper bound afterwards, because no larger value can have appeared. It is used in this way by `pop_heap` [9.35] in §9.9 and again, in the dual form for lower bounds, by `partial_sort` [10.9] in §10.4.

9.7. The push_heap algorithm

The push_heap algorithm extends an existing heap by one additional element. More precisely, it assumes that the first $n - 1$ elements of an array of length n already form a heap, and it incorporates the element $a[n-1]$ into this structure.

In the C++ Standard Library [15, §28.7.7.1], push_heap operates on a range of random access iterators. In contrast, our variant is specialized to arrays of value_type. Accordingly, we use the following signature:

```
void push_heap(value_type* a, size_type n);
```

The algorithm requires that n is greater than or equal to 1 and that the subarray $a[0 \dots n - 1)$ forms a heap. It then rearranges the elements of $a[0 \dots n)$ such that the entire array satisfies the heap property. In this sense, the algorithm *pushes* the element $a[n - 1]$ onto the given heap.

9.7.1. Formal specification of push_heap

The following listing presents our formal specification of push_heap. A central role is played by the postcondition reorder, which states that push_heap is not allowed to change the multiplicity of the array elements.

Without this postcondition, the specification would be far too weak. For instance, an implementation that assigns the value 0 to every array element would trivially satisfy the postcondition heap, yet would clearly violate the intended semantics of the algorithm.

```
/*@
  requires  nonempty:  0 < n;
  requires  valid:     \valid(a + (0..n-1));
  requires  heap:      Heap(a, n-1);

  terminates      \true;
  exits          \false;
  assigns        a[0..n-1];

  ensures      heap:   Heap(a, n);
  ensures      reorder: MultisetReorder{Old,Here}(a, n);
*/
void push_heap(value_type* a, size_type n);
```

Listing 9.23: Formal specification of push_heap

Pushing an element onto a heap typically requires the rearrangement of several array elements, as illustrated in Figures 9.24 and 9.25. It is therefore essential to express that push_heap only permutes the elements of the array, without introducing or removing values. To formalize this requirement, we reuse the predicate MultisetReorder [7.59], which precisely captures the notion that the result array is a reordering of the original one.

9.7.2. Implementation of push_heap

The following two figures illustrate the effect of `push_heap` on an array. The array is represented as a tree whose nodes are colored blue or grey, indicating heap and non-heap nodes, respectively. Figure 9.24 shows the heap from Figure 9.2 together with the additional element 12 that is to be pushed onto the heap. At this stage, the element 12 is the last element of the array and is not yet part of the heap.

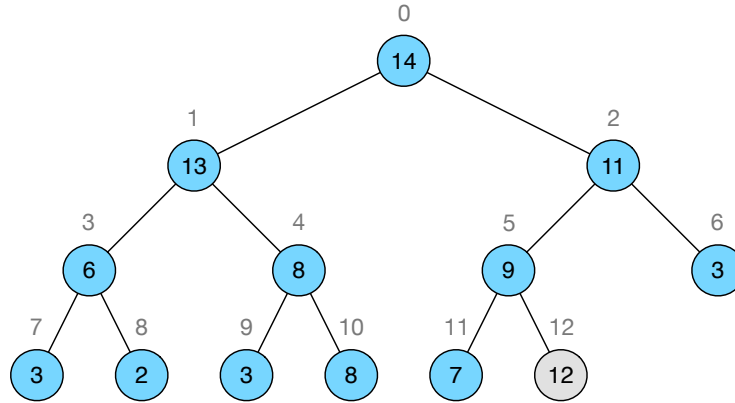


Figure 9.24.: Example heap before the call of `push_heap`

Figure 9.25 shows the array after the call of `push_heap`. All nodes are now colored blue, indicating that they belong to the heap. The dashed nodes mark those heap elements whose positions have changed during the execution of the algorithm. The element to be pushed has reached its correct position in the heap. The arrows indicate the *cyclic reordering* of array elements that leads to this final configuration.

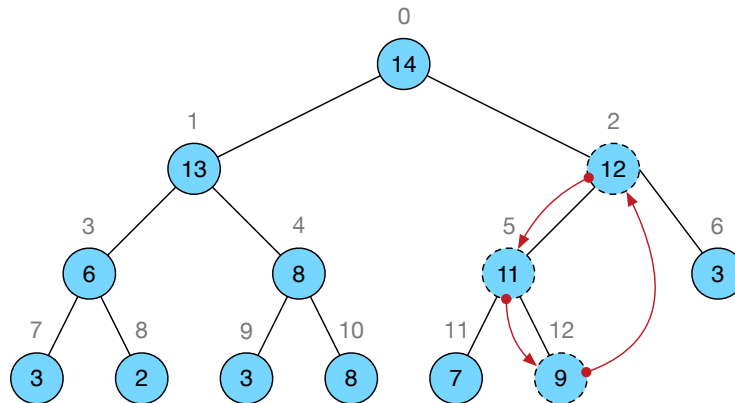


Figure 9.25.: Example heap after the call of `push_heap`

Verifying our implementation of `push_heap` [9.26] is a non-trivial task. To structure the discussion, we divide the algorithm into three conceptual parts. The *main act* is the **for** loop together with its invariants, the *prologue* everything that precedes it, and the *epilogue* everything from the ghost label `Epilogue` onwards; the assertions of the epilogue refer to that label.

The heap property required by the specification of `push_heap` [9.23] can already be established in the prologue. In contrast, the reorder property holds only at the function boundaries and is temporarily violated while `push_heap` manipulates the array. More precisely, the reorder property is lost in the prologue and replaced by a slightly more general invariant that is maintained throughout the main act. From this invariant, the original reorder property is recovered in the epilogue.

The effect of each stage on the underlying array will be illustrated by figures showing the array in tree form. All figures are based on the `push_heap` example introduced in Figure 9.24.

```

void push_heap(value_type* a, size_type n)
{
    if (1u < n) { // otherwise nothings needs to be done
        size_type c = n - 1u;
        size_type p = heap_parent(c);
        //@ assert parent: p == HeapParent(c);

        if (a[p] < a[c]) {
            const value_type v = a[c];
            a[c] = a[p];
            //@ assert update: ArrayUpdate{Pre,Here}(a, n, c, a[p]);
            //@ assert heap: Heap(a, n);
            //@ assert reorder: MultisetParity{Pre,Here}(a, n, a[p], v);

            /*@
                loop invariant bound:      0 <= c < n-1;
                loop invariant heap:      Heap(a, n);
                loop invariant less:      a[c] < v;
                loop invariant parent:    p == HeapParent(c);
                loop invariant reorder:   MultisetParity{Pre,Here}(a, n, a[c], v);
                loop invariant unchanged: Unchanged{Pre,Here}(a, c);
                loop assigns              c, p, a[0..n-1];
                loop variant              c;
            */
            for (c = p, p = heap_parent(c); 0u < c && a[p] < v;
                c = p, p = heap_parent(c)) {
                //@ ghost value_type ac = a[c];
                if (a[c] < a[p]) {
                    a[c] = a[p];
                    //@ assert update: ArrayUpdate{LoopCurrent,Here}(a, n, c, a[p]);
                    //@ assert update: MultisetUpdate{LoopCurrent,Here}(a, n, c, a[p]);
                    //@ assert bound: 0 <= c < n;
                    //@ assert less: ac < a[c] < v;
                    //@ assert reorder: MultisetParity{Pre,Here}(a, n, a[c], v);
                }
            }

            //@ ghost Epilogue: ;
            //@ assert heap: 0 == c || v <= a[HeapParent(c)];
            //@ ghost value_type ac = a[c];
            //@ assert update: ac == At{Epilogue}(a, c) < v;
            //@ assert reorder: MultisetParity{Pre,Here}(a, n, ac, v);
            a[c] = v;
            //@ assert unchanged: Unchanged{Epilogue,Here}(a, 0, c);
            //@ assert unchanged: Unchanged{Epilogue,Here}(a, c+1, n);
            //@ assert update: At{Here}(a, c) == v;
            //@ assert update: ArrayUpdate{Epilogue,Here}(a, n, c, v);
            //@ assert heap: HeapCompatible(a, n, c, v);
            //@ assert heap: Heap(a, n);
            //@ assert update: MultisetUpdate{Epilogue,Here}(a, n, c, v);
            //@ assert reorder: MultisetParity{Epilogue,Here}(a, n, v, ac);
            //@ assert reorder: MultisetReorder{Pre,Here}(a, n);
        }
    }
}

```

Listing 9.26: Implementation of push_heap

9.7.2.1. Prologue

In the prologue, the algorithm first checks that the initial heap is nonempty and initializes the required auxiliary variables. It then compares the value $a[c]$, which is the element to be pushed onto the heap and the last element of the array, with its parent node. If $a[c]$ already satisfies the heap condition with respect to its parent, no further action is required at this stage.

Otherwise, the value $a[c]$ is temporarily stored in the variable v , and the parent value $a[p]$ is assigned to $a[c]$. This assignment is performed only if the two values differ. As a consequence, its effect can be described precisely using the predicate `ArrayUpdate` [7.4].

Figure 9.27 illustrates the main effect of the prologue. Here, and in all subsequent figures, the currently active node is highlighted.

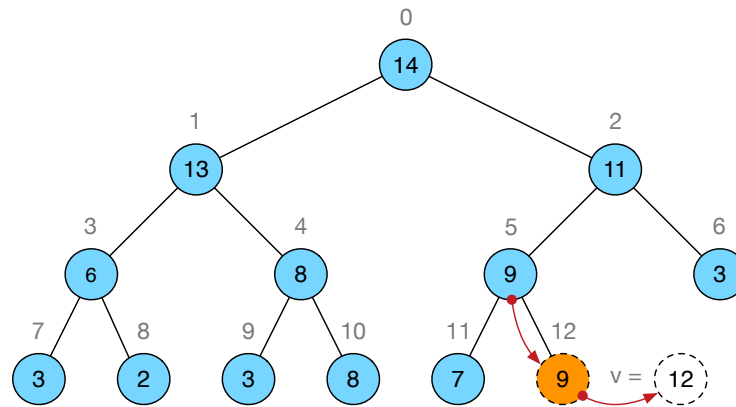


Figure 9.27.: Heap after the prologue of `push_heap`

At this point, two essential properties have been established.

1. The array $a[0 \dots n)$ satisfies the heap property.
2. With respect to the multiplicities of values in $a[0 \dots n)$, the following changes have occurred:
 - the original value $a[c]$ occurs one time less,
 - the original value $a[p]$ occurs one time more,
 - all other values occur with the same multiplicity as before.

The first property is captured by the assertion `heap`. The temporary imbalance in element multiplicities described by the second property is expressed using the predicate `MultisetParity` [9.20] in the assertion `reorder`.

9.7.2.2. Main act

Figure 9.28 shows the effect of the main act of `push_heap` on the example heap. In this example, only a single iteration of the loop is required until a node is reached whose parent value $a[p]$ is greater than or equal to v . Assignments performed during the iteration are indicated by dashed arrows.

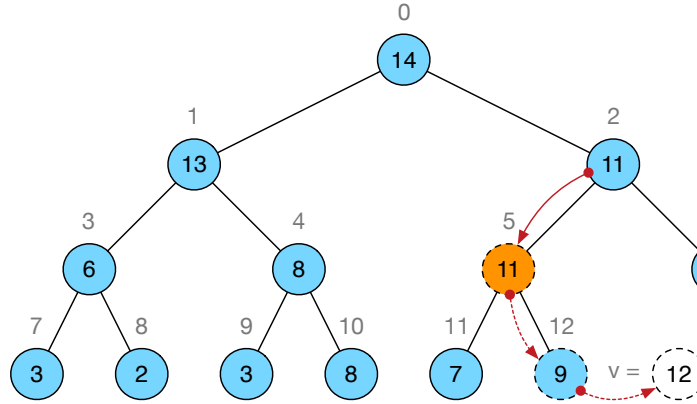


Figure 9.28.: Heap after the main act of `push_heap`

In the main act, the algorithm starts at the parent position reached in the prologue. This index is stored in the variable c , referring to the current *child* position. Compared to the pre-state of `push_heap`, the array $a[0 \dots n)$ now satisfies the following multiplicity relation:

- the value v occurs one time less,
- the value $a[c]$ occurs one time more,
- all other values occur with the same multiplicity as in the pre-state.

As long as the index c is not the root of the heap and its (necessarily existing) parent value $a[p]$ is strictly less than v , no admissible position for v has yet been found without violating the heap property.

Within the loop body, the following cases are distinguished.

- If $a[c]$ is strictly less than $a[p]$, the parent value $a[p]$ is copied to $a[c]$. This assignment preserves the heap property of the array. With respect to multiplicities, the value $a[p]$ now occurs one time more than in the pre-state, the overwritten value $a[c]$ occurs as often as in the pre-state, and the value v continues to occur one time less. The next iteration proceeds with c updated to p .

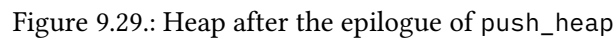
At the level of specification, these effects can be decomposed into elementary updates captured by the predicates `ArrayUpdate` [7.4] and `MultisetUpdate` [9.21]. The lemma `MultisetParity_Combined` [9.20] indicates how these local changes combine into the maintained global relation. This discussion is intended to convey the structure of the invariants, not to present a detailed proof.

- Otherwise, since c is a child of p , we necessarily have $a[c]$ equal to $a[p]$. In this case no assignment is required, and the next iteration proceeds after updating c to p .

At the beginning of each iteration, the same multiplicity relation as above holds. Compared to the pre-state of `push_heap`, the array $a[0 \dots n)$

- contains the value v one time less,
- contains the value $a[c]$ one time more,
- while all other values are unchanged in their multiplicity.

Figure 9.29 shows the final assignment performed by `push_heap` and the resulting configuration of the heap. The figure highlights the completion of the cycle initiated in Figure 9.27. It also makes clear that the value `v` behaves like an additional element temporarily circulating through the array during this cycle.



At the same time, this final assignment corrects the temporary imbalance in value multiplicities that was introduced earlier. The value v , which up to now occurred one time less than in the pre-state, is reinserted into the array, while the value currently stored at $a[c]$ loses its extra occurrence. As a result, the desired reorder property of `push_heap` is restored.

216

9.8. The auxiliary function `heap_sift_down`

The C++ Standard Library does not expose the function discussed in this section. It is nevertheless the operation on which the heap algorithms rest: it repairs a heap whose defect is confined to a single node. Every implementation therefore has it internally, under a name reserved to the implementation: `libc++` calls it `__sift_down` and `libstdc++` calls it `__adjust_heap`. It is therefore not one of the algorithms that this document mirrors, and its name carries the prefix `heap_` for that reason, as do those of `heap_parent` and `heap_child`.

Assume that the array $a[0 \dots n)$ would satisfy the heap property if the element at index `root` were disregarded, that is, every node other than `root` already dominates its children. The function `heap_sift_down` restores the heap property by moving the value stored at `root` downwards along a path of the tree until it reaches a position where it dominates its own children. Accordingly, the signature of `heap_sift_down` reads

```
void heap_sift_down(value_type* a, size_type n, size_type root);
```

9.8.1. Formal specification of `heap_sift_down`

Expressing the precondition of `heap_sift_down` requires a heap property that starts partway into the array. The predicate `Heap` [9.7] cannot state it, as it demands that *every* node dominate its children. This is the purpose of the predicate `HeapForest` [9.8]: the expression `HeapForest(a, m, n)` demands that each index of $a[m \dots n)$ be the root of a heap.

```
/*@
  requires  valid:    \valid(a + (0..n-1));
  requires  bounds:   0 <= root < n;
  requires  forest:   HeapForest(a, root + 1, n);

  terminates      \true;
  exits           \false;
  assigns         a[0..n-1];

  ensures  forest:   HeapForest(a, root, n);
  ensures  reorder:  MultisetReorder{Old,Here}(a, n);
*/
void heap_sift_down(value_type* a, size_type n, size_type root);
```

Listing 9.30: Formal specification of `heap_sift_down`

The precondition `forest` therefore states exactly the assumption described above, namely that all nodes below `root` are already in order. The postcondition `forest` extends this property to `root` itself, and the postcondition `reorder` records that `heap_sift_down` only permutes the elements of the array.

Note that the contract does not mention the predicate `Heap` [9.7] at all. A caller obtains it from the special case `HeapForest(a, 0, n)` through the lemma `HeapForest_Heap` [9.8].

9.8.2. Implementation of `heap_sift_down`

The implementation follows the path from `root` towards the leaves. As in the case of `push_heap` [9.26], we subdivide the discussion into three parts: the prologue up to the loop annotation, the main act of the `for` loop and its invariants, and the epilogue from the ghost label `Epilogue` onwards.

```

void heap_sift_down(value_type* a, size_type n, size_type root)
{
    value_type v = a[root];
    size_type p = root;
    size_type c = heap_child(a, n, p);

    if (c < n && v < a[c]) {
        a[p] = a[c];

        //@ assert update:    ArrayUpdate{Pre,Here}(a, n, root, a[c]);
        //@ assert forest:    HeapForest(a, root, n);
        //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);

        p = c;
        c = heap_child(a, n, p);

        /*@
        loop invariant bounds:    root <= p < c <= n;
        loop invariant ancestor:  HeapAncestor(root, p);
        loop invariant parent:    c < n ==> p == HeapParent(c);
        loop invariant child:     c < n ==> HeapLeft(p) < n;
        loop invariant child:     c == n ==> n <= HeapLeft(p);
        loop invariant child:     HeapLeft(p) < n ==> a[HeapLeft(p)] <= a[c];
        loop invariant child:     HeapRight(p) < n ==> a[HeapRight(p)] <= a[c];
        loop invariant unchanged: Unchanged{LoopEntry,Here}(a, p, n);
        loop invariant update:    a[p] == a[HeapParent(p)];
        loop invariant reorder:    MultisetParity{Pre,Here}(a, n, a[p], v);
        loop invariant forest:    HeapForest(a, root, n);
        loop invariant bound:     v < a[p];
        loop assigns              p, c, a[0..n-1];
        loop variant              n - p;
        */
        for (; c < n && v < a[c]; p = c, c = heap_child(a, n, p)) {
            //@ ghost value_type ac = a[c];
            if (a[c] < a[p]) {
                a[p] = a[c];
                //@ assert update:    ArrayUpdate{LoopCurrent,Here}(a, n, p, ac);
                //@ assert update:    MultisetUpdate{LoopCurrent,Here}(a, n, p, ac);
                //@ assert update:    a[c] == At{LoopCurrent}(a, c);
                //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
            }

            //@ assert progress:    p < c;
        }

        //@ ghost Epilogue: ;
        //@ assert compatible:    HeapForestCompatible(a, n, root, p, v);
        a[p] = v;
        //@ assert unchanged:    Unchanged{Epilogue,Here}(a, 0, p);
        //@ assert unchanged:    Unchanged{Epilogue,Here}(a, p+1, n);
        //@ assert update:    ArrayUpdate{Epilogue,Here}(a, n, p, v);
        //@ assert update:    MultisetUpdate{Epilogue,Here}(a, n, p, v);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
        //@ assert forest:    HeapForest(a, root, n);
    }
}

```

Listing 9.31: Implementation of heap_sift_down

The variable v holds the value that is being sifted down, p the position it currently occupies, and c the child of p that holds the larger of the two child values, as computed by `heap_child` [9.13]. As long as that child exceeds v , the child value is moved up to p and the descent continues. The value v itself is written only once, at the end, when its final position is known. This is the same technique that `push_heap` [9.26] employs in the opposite direction, and it explains why the array does not hold a heap during the loop: the position p carries a duplicate of the value that was moved up from it, while v is held outside the array.

9.8.2.1. Prologue

The prologue determines the larger child of `root` and decides whether there is anything to do at all. If `root` has no child inside $a[0 \dots n)$, the function `heap_child` [9.13] returns `n` and the guard fails; if it has one but already dominates it, the guard fails as well. In both cases the array is a heap from `root` downwards to begin with, and `heap_sift_down` returns without writing anything.

Otherwise the first descent step is performed, and it is performed *outside* the loop. The reason is the loop invariant update, which states that p holds a duplicate of the value that has just been lifted out of it into its parent. At $p == \text{root}$ nothing has been lifted yet, so the invariant cannot hold there; peeling one iteration off establishes it before the loop is entered. The assignment moves the larger child up to `root`, which the assertion update describes by means of the predicate `ArrayUpdate` [7.4]. The assertion forest then records something that may be surprising: the heap property already holds again from `root` downwards. What is broken at this point is not the ordering but the multiset — the position the descent has moved to now holds its value twice, and v is not in the array at all, which the assertion `reorder` states.

9.8.2.2. Main act

When referring to a *child node* here we implicitly mean the child — among the at most two children of a node — that carries the larger value. Copying that larger value into the parent preserves the heap property, whereas copying the smaller one would not. The loop continues as long as the child index stays inside the range and the value stored there is strictly greater than v ; when either fails, the position for v has been found.

The loop invariants fall into three groups. The invariants `bounds`, `parent` and `child` place p and c in the tree: they record that the descent moves forward, that c is a child of p while it stays inside the range, and that the value at c dominates both children of p . The invariants `unchanged` and `bound` describe what has not happened: nothing from p onwards has changed since the loop was entered, and v is still smaller than the value sitting at p , so the descent is not yet finished. The invariants `update`, `reorder`, `forest` and `ancestor` carry the state of the repair, and are discussed below.

The invariant `reorder` accounts for the transient defect by means of the predicate `MultisetParity` [9.20] discussed in §9.6: the multiset of the array differs from the one of the pre-state only in that v has been replaced by $a[p]$. Writing v back into position p in the epilogue repairs precisely this difference.

The invariant `forest` maintains the heap property for all nodes at or below `root` except p , and the invariant `ancestor` records that p is reached from `root` by repeatedly descending to a child. The latter is expressed by the inductive predicate `HeapAncestor` [9.9]. It is what allows the lemma `HeapForest_Maximum` [9.8] to conclude that the value at `root` still dominates the value at p , however far the descent has proceeded.

The assignment in the loop body is guarded by a comparison of $a[c]$ with $a[p]$, and that guard is easy to mistake for an optimization. It is not one. The invariant `forest` makes $a[c]$ at most $a[p]$, so the guard fails exactly when the two values are equal — in which case the assignment would not change the array. That case has to be excluded, because `ArrayUpdate` [7.4] and `MultisetUpdate` [9.21] both require the overwritten value to differ from the new one, and both would be false for an assignment that changes

nothing. This is why the assertions `update` and `reorder` sit inside the conditional rather than after it, and `push_heap` [9.26] carries the same guard for the same reason.

Finally, the loop variant $n - p$ decreases because each iteration moves p to one of its children, which the assertion progress makes explicit.

9.8.2.3. Epilogue

When the loop terminates, the position p is the one at which v belongs, and writing it there completes the repair. That this single assignment restores the heap property is not obvious from the assignment alone, and it is exactly the situation the predicate `HeapForestCompatible` [9.8] was introduced for. The assertion `compatible` states its four local conditions: that p lies in the range, that v does not exceed the value at the parent of p , and that it is not exceeded by either child of p . Together with `ArrayUpdate` [7.4], the lemma `HeapForestCompatible_Update` [9.8] turns them into the postcondition `forest`, so the heap property is recovered from local conditions alone, without any further reasoning about the path the descent took.

The assertion `reorder` after the assignment closes the multiset argument in the same way: the parity recorded by the invariant is cancelled by writing v back, which leaves a plain reordering and hence the postcondition `reorder`.

9.9. The pop_heap algorithm

The pop_heap algorithm removes the largest element from a heap. More precisely, it moves the first element of the array, which contains the maximum value of the heap, to the end of the underlying sequence.

In the C++ Standard Library [15, §28.7.7.2], pop_heap operates on a range of random access iterators. In contrast, our variant is specialized to arrays of value_type. Accordingly, we use the following signature:

```
void pop_heap(value_type* a, size_type n);
```

The pop_heap algorithm requires that n is greater than or equal to 1 and that the array $a[0 \dots n)$ forms a heap. It then rearranges the elements of $a[0 \dots n)$ such that the following properties hold:

- $a[n-1] = \text{old}(a[0])$, that is, the largest element of the original heap is moved to the end of the array,
- the subarray $a[0 \dots n-1)$ satisfies the heap property.

In this sense, the algorithm *pops* the largest element from the heap.

9.9.1. Formal specification of pop_heap

On the basis of the preceding semi-formal description, we now present the formal specification of pop_heap.

```
/*@  
  requires  nonempty: 0 < n;  
  requires  valid:    \valid(a + (0..n-1));  
  requires  heap:     Heap(a, n);  
  
  terminates \true;  
  exits      \false;  
  assigns    a[0..n-1];  
  
  ensures   heap:     Heap(a, n-1);  
  ensures   result:   a[n-1] == \old(a[0]);  
  ensures   max:      MaxElement(a, n, n-1);  
  ensures   reorder:  MultisetReorder{Old,Here}(a, n);  
*/  
void pop_heap(value_type* a, size_type n);
```

Listing 9.32: Formal specification of pop_heap

The contract expresses three essential aspects of the algorithm. First, it states that the last element of the array $a[n-1]$ contains the value that was stored at the root of the heap in the pre-state. Second, it requires that the prefix $a[0 \dots n-1)$ forms a heap after the function call. Third, it enforces that the function only reorders existing elements of the array and does not introduce or remove values.

As in the case of push_heap, the last requirement is crucial. Without an explicit reordering condition, implementations that destroy or duplicate values could satisfy the heap-related postconditions while violating the intended semantics of the algorithm.

9.9.2. Implementation of pop_heap

In an abstract sense, the algorithm `pop_heap` is closely related to `push_heap`, but proceeds in the opposite direction. In the case of `push_heap`, we start at the last array element and climb upwards through the tree until a suitable position for the new value is found. At each step, the value of the current parent node is moved downwards to the position from which we just ascended.

For `pop_heap` this process is reversed. One starts at the root of the tree and descends by repeatedly selecting an appropriate child node, lifting the value of that child to the current node, until a node is reached at which the displaced value can be inserted without violating the heap property. That descent is precisely the task of `heap_sift_down` [9.30], and so `pop_heap` carries no loop of its own. It exchanges the root with the last array element and then repairs the remaining prefix by a single call.

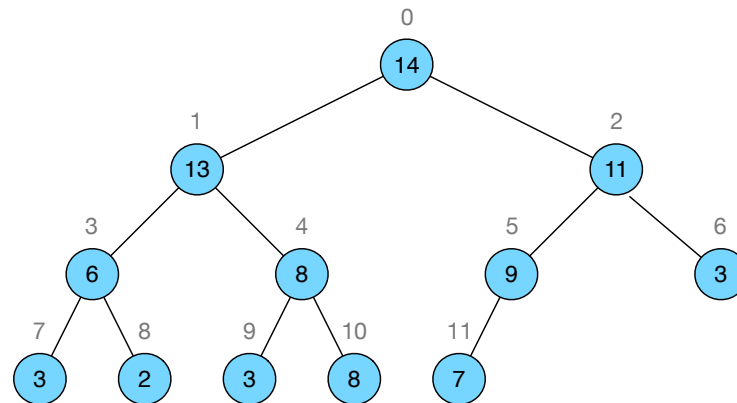


Figure 9.33.: Heap before the call of `pop_heap`

Figure 9.34 depicts the state of the array after the execution of `pop_heap`. The maximum element, originally stored at the root, has been moved to the last array position and is no longer part of the heap. Conversely, the last array element has been inserted into the heap by descending along a suitable path. The dashed nodes indicate those heap positions whose values have changed during the operation. The arrows visualize the *cyclic reordering* of array elements that implements this transformation.

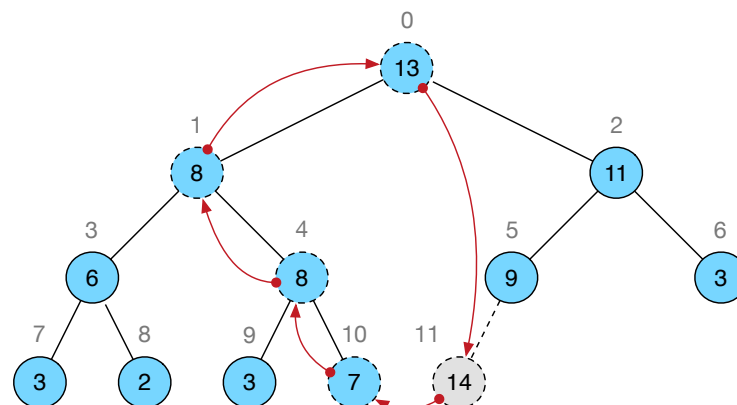


Figure 9.34.: Heap after the call of `pop_heap`

```

void pop_heap(value_type* a, size_type n)
{
    if (1u < n) {
        //@ assert max: MaxElement(a, n, 0);
        if (a[n - 1u] < a[0u]) { // otherwise a[0] == a[n-1] and nothing to be done
            //@ ghost Before: ;
            swap(a, a + n - 1u);
            //@ assert swapped: ArraySwap{Before,Here}(a, 0, n-1, n);
            //@ assert reorder: MultisetReorder{Pre,Here}(a, n);
            //@ assert max:      UpperBound(a, n-1, a[n-1]);
            //@ assert forest:   HeapForest(a, 1, n-1);

            //@ ghost BeforeSift: ;
            heap_sift_down(a, n - 1u, 0u);

            //@ assert unchanged: Unchanged{BeforeSift,Here}(a, n-1, n);
            //@ assert reorder:   MultisetReorder{Pre,Here}(a, n);
            //@ assert heap:      Heap(a, n-1);
            //@ assert max:      SomeEqual{BeforeSift}(a, n-1, a[0]);
            //@ assert max:      a[0] <= a[n-1];
            //@ assert max:      UpperBound(a, n, a[n-1]);
        }
    }
}

```

Listing 9.35: Implementation of pop_heap

The exchange is the entire preparation. After it the maximum resides at index $n-1$, outside the heap proper, and the prefix $a[0 \dots n-1]$ satisfies the heap property everywhere except possibly at its root: each of its nodes from index 1 on still heads a heap. This is exactly $\text{HeapForest}(a, 1, n-1)$, which the assertion forest records and which is the precondition of heap_sift_down [9.30] for $\text{root} == 0$.

The remaining assertions carry the postconditions of pop_heap across the call. The predicate ArraySwap [7.62] describes the exchange, and the lemma $\text{MultisetSwap_FrontMiddle}$ [7.63] turns it into a reordering of the whole array; composed with the reordering that heap_sift_down [9.30] guarantees for the prefix, it establishes reorder. The postcondition heap follows from the postcondition of heap_sift_down [9.30] through the lemma HeapForest_Heap [9.8].

The postcondition max needs one more step. The exchange makes the value at index $n-1$ an upper bound of the prefix, and heap_sift_down [9.30] leaves that position untouched, but it does move the prefix around. The lemma $\text{MultisetReorder_SomeEqual}$ [9.22] closes the gap: reordering a range introduces no new values, so the value that ends up at index 0 already occurred in the prefix beforehand and is therefore bounded by $a[n-1]$. Since the prefix is a heap again, Heap_Maximum [9.7] makes $a[0]$ an upper bound of it, and the bound extends to the whole array.

9.10. The make_heap algorithm

Whereas in the C++ Standard Library [15, §28.7.7.3] `make_heap` operates on a pair of generic random access iterators, our version operates on an array of `value_type`. Accordingly, the signature of `make_heap` reads

```
void make_heap(value_type* a, size_type n);
```

The function `make_heap` rearranges the elements of the array $a[0 \dots n)$ such that they satisfy the heap property.

As an example, consider the array shown in Figure 9.36. When interpreted as a binary tree via the usual array-to-heap mapping, the ordering of its elements violates the heap property. Executing `make_heap` on this array rearranges the elements so that the heap property holds.

8	8	7	3	14	11	3	6	2	3	13	9
---	---	---	---	----	----	---	---	---	---	----	---

Figure 9.36.: Array before the call of `make_heap`

The result can be viewed as an array in heap form, as shown in Figure 9.3. As already noted, this arrangement is not unique: the same multiset of elements may admit several different array representations that all satisfy the heap property.

9.10.1. Formal specification of `make_heap`

The following listing presents the formal specification of `make_heap`.

```
/*@
requires    valid:    \valid(a + (0..n-1));

terminates      \true;
exits           \false;
assigns         a[0..n-1];

ensures    heap:      Heap(a, n);
ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void make_heap(value_type* a, size_type n);
```

Listing 9.37: Formal specification of `make_heap`

As in the case of `push_heap`, the specification of `make_heap` must establish two essential properties. First, the resulting array $a[0 \dots n)$ must satisfy the heap property, that is, it must represent a heap of size n . Second, the operation must preserve the multiset of array elements with respect to the pre-state.

These requirements are captured by the postconditions `heap` and `reorder`, respectively. The `reorder` postcondition relies on the predicate `MultisetReorder` [7.59] to formalize that `make_heap` only permutes the elements of the array and does not create or remove values.

9.10.2. Implementation of make_heap

The implementation of `make_heap`, shown in the following listing, builds the heap from the bottom upwards. A node without children is a heap of its own, so the leaves of the tree need no attention at all. The loop therefore visits the internal nodes only, from the last one down to the root, and applies `heap_sift_down` [9.30] to each of them.

```
void make_heap(value_type* a, size_type n)
{
    if (1u < n) {
        size_type root = heap_parent(n - 1u) + 1u;

        /*@
        loop invariant bounds:      0 <= root <= HeapParent(n - 1u) + 1;
        loop invariant forest:      HeapForest(a, root, n);
        loop invariant reorder:     MultisetReorder{Pre,Here}(a, n);
        loop assigns                root, a[0..n-1];
        loop variant                root;
        */
        while (0u < root) {
            --root;
            //@ assert forest_next:  HeapForest(a, root + 1u, n);
            heap_sift_down(a, n, root);
            //@ assert forest:       HeapForest(a, root, n);
            //@ assert reorder_step: MultisetReorder{LoopCurrent,Here}(a, n);
            //@ assert reorder:      MultisetReorder{Pre,Here}(a, n);
        }

        //@ assert forest_zero:     HeapForest(a, 0, n);
    }

    //@ assert heap:               Heap(a, n);
}
```

Listing 9.38: Implementation of `make_heap`

The last internal node is the parent of the last array element, which is why the loop starts at `heap_parent(n-1)`. The variable `root` is set to that index increased by one and decremented at the beginning of each iteration; this keeps the loop condition $0 < \text{root}$ free of the boundary case that a decrement below zero would raise for an unsigned type. Since the loop treats an array of length one as a heap without entering the body at all, the whole construction is guarded by $1 < n$.

When `heap_sift_down` [9.30] is called for the node `root`, all nodes below it have already been treated: each of them is the root of a heap, so that the nodes from `root` onwards form a forest of heaps. This is what the loop invariant `forest` states, and it is precisely the precondition of `heap_sift_down` [9.30]. Its postcondition extends the property to `root` itself, so that the forest grows by one tree per iteration and after the final one `HeapForest(a, 0, n)` holds, from which the lemma `HeapForest_Heap` [9.8] yields the postcondition `heap` of `make_heap`. The invariant `reorder` is obtained by composing the corresponding postcondition of `heap_sift_down` [9.30] with the reordering already accumulated, as the two assertions after the call make explicit.

This bottom-up construction is not merely an alternative formulation of the previous one, which inserted the elements one by one with `push_heap` [9.23]. Sifting down from the last internal node visits the deep nodes, of which there are many, only over short distances, and the long paths only for the few nodes near the root. The result is a construction that performs $O(n)$ operations, whereas the incremental one needs $O(n \log n)$. The specification of `make_heap` is unaffected by the change: the contract shown above is the

same as before. Note that this does not improve `heap_sort`, whose overall $O(n \log n)$ bound is determined by the n calls of `pop_heap` rather than by the construction of the heap.

9.11. The `sort_heap` algorithm

The algorithm `sort_heap` completes the heap-based sorting process by transforming a heap into a fully ordered sequence. Whereas in the C++ Standard Library [15, §28.7.7.4] `sort_heap` operates on a range of random access iterators, our version works directly on an array of `value_type`. Accordingly, we use the following signature for `sort_heap`.

```
void sort_heap(value_type* a, size_type n);
```

The function `sort_heap` expects that the array $a[0 \dots n)$ satisfies the heap property. It then rearranges the elements of this heap into increasing order, thereby producing a sorted array. Thus, applying `sort_heap` to the heap shown in Figure 9.3 yields the increasing sequence displayed in Figure 9.39.

2	3	3	3	6	7	8	8	9	11	13	14
---	---	---	---	---	---	---	---	---	----	----	----

Figure 9.39.: Array after the call of `sort_heap`

9.11.1. Formal specification of `sort_heap`

The following listing presents our formal specification of `sort_heap`. The specification must ensure that, upon termination, the array $a[0 \dots n)$ is sorted in increasing order. Moreover, the algorithm must not introduce or remove elements; it may only permute them. These requirements are expressed by the postconditions `increasing` and `reorder`, respectively.

```
/*@
requires   valid:      \valid(a + (0..n-1));
requires   heap:       Heap(a, n);

terminates      \true;
exits          \false;
assigns        a[0..n-1];

ensures   reorder:    MultisetReorder{Old,Here}(a, n);
ensures   increasing: Increasing(a, n);
*/
void sort_heap(value_type* a, size_type n);
```

Listing 9.40: Formal specification of `sort_heap`

The specification effort is relatively modest. Since `sort_heap` merely rearranges the elements of the input heap, we can directly reuse the established predicates for sortedness and multiset preservation without introducing additional auxiliary logic.

9.11.2. Implementation of sort_heap

The implementation of `sort_heap` is comparatively simple. It relies on `pop_heap` [9.32] to perform the essential work of removing the maximum element from the heap and relocating it within the array. The algorithm repeatedly calls `pop_heap` in order to remove the maximum element from the current heap and place it at the end of the array. With each iteration, the heap stored in the prefix of the array shrinks, while the suffix grows into an increasing sequence.

The loop invariants describe this decomposition of the array into two parts. The first i elements form a heap and are characterized by the invariant `heap`. The remaining elements $a[i \dots n)$ are already arranged in increasing order and are captured by the invariant `increasing`.

```
void sort_heap(value_type* a, size_type n)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant heap:      Heap(a, i);
    loop invariant lower:      LowerBound(a, i, n, a[0]);
    loop invariant reorder:    MultisetReorder{Pre,Here}(a, 0, n);
    loop invariant increasing: WeaklyIncreasing(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant i;
    */
    for (size_type i = n; i > 1u; --i) {
        //@ ghost Before: ;
        pop_heap(a, i);
        //@ assert lower:      LowerBound(a, i, n, a[i-1]);
        //@ assert reorder:    MultisetReorder{Before,Here}(a, 0, i);
        //@ assert unchanged:  Unchanged{Before,Here}(a, i, n);
    }

    //@ assert increasing: Increasing(a, n);
}
```

Listing 9.41: Implementation of `sort_heap`

As already noted in the introduction of Chapter 6, the loop annotation `increasing` is expressed using the predicate `WeaklyIncreasing` [6.2]. Consequently, upon loop termination we only obtain the property `WeaklyIncreasing(a, n)`. The final postcondition `increasing`, which relies on the predicate `Increasing` [6.1], is then derived by an application of the lemma `WeaklyIncreasing_Increasing` [6.3].

The preservation of the multiset property `reorder` follows from the specification of `pop_heap`. After each call to `pop_heap`, we introduce assertions relating the current state to the ghost label `Before`. These assertions establish

- `MultisetReorder{Before,Here}(a, 0, i)`, and
- `Unchanged{Before,Here}(a, i, n)`.

Together, they imply the loop invariant `MultisetReorder{Pre,Here}(a, 0, n)`, which suffices to prove the final postcondition `reorder`.

10. Sorting Algorithms

Sorting algorithms provide a classical and well-studied domain in which many fundamental issues of computer science can be illustrated; see, for example, the seminal textbook by Knuth [21]. Accordingly, we consider a selection of representative sorting algorithms. In this chapter, we focus on algorithms from the C++ Standard Library [15, §28.7] that operate on linear arrays.

Following [22], we employ (possibly C-rephrased) interfaces of C++ Standard Library algorithms wherever feasible in order to realize the different algorithmic paradigms in a uniform manner.

The algorithms fall into three groups. The first two only inspect an array and leave it as it is.

- The functions `is_sorted_until` and `is_sorted`, introduced in §10.2 and §10.3, allow one to check at run time whether a given array is already arranged in non-decreasing order. The former returns the largest prefix that is so arranged, whereas the latter answers the question for the array as a whole.

The next five sort an array, and differ in the strategy by which they do it.

- `partial_sort` in §10.4 rearranges an array into two contiguous parts. All elements of the first part are less than or equal to those of the second part. Moreover, the first part is sorted, whereas the order of the remaining elements is left unspecified.
- `bubble_sort` in §10.5 presents the simple and well-known *bubble sort* algorithm.
- `selection_sort` in §10.6 discusses the classical *selection sort* algorithm.
- `insertion_sort` in §10.7 describes the equally well-known *insertion sort* algorithm.
- `heap_sort` in §10.8 introduces the comparatively efficient *heap sort* algorithm, which builds upon the heap algorithms studied in Chapter 9.

The last three belong together. Neither `merge` nor `inplace_merge` is a sorting algorithm: each combines two ranges that are ordered already into a single ordered range, and differs from the other only in where it puts the result. `merge_sort` is what one builds out of the first of them, and it is presented last for that reason.

- `merge` in §10.9 writes the combined range into a third array, which must be disjoint from the two it reads.
- `inplace_merge` in §10.10 combines two ranges that lie next to each other in one array, and needs no second array at all.
- `merge_sort` in §10.11 builds the *merge sort* algorithm on top of `merge`, sorting the two halves of an array recursively and combining them in a single merge step.

With respect to asymptotic complexity, `heap_sort` achieves an upper bound of $\mathcal{O}(n \cdot \log(n))$, owing to the efficiency of the underlying heap data structure. `merge_sort` attains the same bound by a different route, halving the input at each step and paying a linear merge for each level of the recursion, at the price of an auxiliary array of the size of the input. In contrast, both `selection_sort` and `insertion_sort` require $\mathcal{O}(n^2)$ time in the average case as well as in the worst case. The two merging algorithms show what that

auxiliary array is for. merge moves each element exactly once and needs a second array of the same size; `inplace_merge` needs no second array and, in the worst case, moves a number of elements quadratic in n .

Two of the algorithms above are merging algorithms in a thin disguise. `insertion_sort` extends an ordered prefix $a[0 \dots i)$ by the single element $a[i]$, which is to combine $a[0 \dots i)$ with the one-element range $a[i \dots i + 1)$; `merge_sort` combines two halves of equal length. It is the same operation in both cases, and only the split differs: the most unbalanced split gives a quadratic algorithm, the balanced one $\mathcal{O}(n \cdot \log(n))$. The implementations differ in more than the split — `insertion_sort` finds the position by binary search with `upper_bound` [6.8], where `inplace_merge` scans — and `insertion_sort` does not call `inplace_merge`. What the two share is the rotation by which room is made, and with it the lemmas that describe it.

The sort algorithm of the C++ Standard Library is not treated here, since it typically relies on *introspection sort*, a sophisticated hybrid that combines several classical algorithms.¹⁷ We plan to address this and further sorting-related algorithms in future revisions.

¹⁷See <https://en.wikipedia.org/wiki/Introsort>

10.1. Common contract of sorting algorithms

Most of the algorithms discussed in this chapter share the same formal contract. They differ not in their specification, but in the algorithmic strategy used to establish it. Consequently, Listing 10.1 states the common specification in the form of a generic sorting routine.

```
/*@
  requires valid: \valid(a + (0..n-1));

  assigns a[0..n-1];

  ensures increasing: Increasing(a, n);
  ensures reorder:   MultisetReorder{Old, Here}(a, n);
*/
void generic_sort(value_type* a, size_type n);
```

Listing 10.1: Common specification of sorting algorithms

The contract expresses two fundamental properties. First, the array $a[0 \dots n)$ is arranged in increasing order upon termination. Second, the algorithm performs a pure reordering of elements: no values are created or lost.

Five algorithms carry this contract: `bubble_sort`, `selection_sort`, `insertion_sort`, `heap_sort` and `merge_sort`, the last with the clauses added that its auxiliary array requires. The other five stand to it in four different ways.

- `is_sorted_until` and `is_sorted` change nothing, and their contracts accordingly say `assigns \nothing`. They report on the order an array already has, and their postconditions speak of `\result` rather than of the array.
- `partial_sort` weakens the ordering postcondition. Only the prefix $a[0 \dots m)$ is ordered, and Partition [10.7] states where the remaining elements stand relative to it. The postcondition `reorder` is the one above.
- `merge` has a different contract altogether. It relates three arrays in a single program state, whereas `MultisetReorder` compares one array in two states, so `MultisetUnion` takes its place.
- `inplace_merge` carries the contract above exactly, and adds preconditions: the two ranges it is to combine must be ordered on entry.

Throughout this chapter we adopt the convention established in Section 6.2. Global ordering requirements are stated in contracts using `Increasing`, whereas loop invariants and intermediate assertions are phrased in terms of `WeaklyIncreasing`. Whenever it is necessary to pass from `WeaklyIncreasing` to `Increasing`, we appeal to Lemma `WeaklyIncreasing_Increasing`.

10.2. The `is_sorted_until` algorithm

The algorithm `is_sorted_until` of the C++ Standard Library [15, §28.7.1.5] operates on generic sequences and determines the largest prefix that is in increasing order. For the purposes of this chapter, we adapt the generic formulation to arrays of type `value_type`. The resulting interface is given by the following signature.

```
size_type is_sorted_until(const value_type* a, size_type n);
```

Given an array a of length n , the function `is_sorted_until` returns the smallest index at which the ordering is violated. Equivalently, it returns the length of the largest prefix $a[0 \dots k)$ that is in increasing order. If the entire array is ordered, the function returns n .

The reader may notice the deliberate parallel to `is_heap_until` [9.14] in Section 9.4. Both algorithms locate the boundary of the largest prefix satisfying a given property, and in both cases the corresponding Boolean algorithm is obtained by comparing that boundary with the array length.

10.2.1. Formal specification of `is_sorted_until`

The formal specification of `is_sorted_until` is given in the following listing. The return value `\result` denotes the boundary of the maximal increasing prefix: the range $a[0 \dots \text{\result})$ satisfies the predicate `Increasing` [6.1].

In addition, the postcondition `last` characterizes the returned index as the *first* position at which the ordering fails. It suffices to state this for the shortest prefix that exceeds the returned boundary: unless `\result` has reached n , appending the single element $a[\text{\result}]$ to the prefix already destroys the predicate `Increasing` [6.1].

```

/*@
  requires    valid:      \valid_read(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            \nothing;

  ensures    bound:      0 <= \result <= n;
  ensures    increasing: Increasing(a, \result);
  ensures    last:       \result < n ==> !Increasing(a, \result + 1);
*/
size_type is_sorted_until(const value_type* a, size_type n);

```

Listing 10.2: Formal specification of `is_sorted_until`

Nothing is lost by confining the postcondition to that one prefix, because `Increasing` is inherited by prefixes: once the ordering fails for some prefix, it fails for every longer prefix as well. This is precisely the content of Lemma `Increasing_Shrink`, introduced in Section 6.1, and a caller that needs the stronger reading obtains it from the lemma in a single step. Section 10.3 shows such a caller.

10.2.2. Implementation of `is_sorted_until`

The following listing presents an implementation of the function `is_sorted_until`.

```

size_type is_sorted_until(const value_type* a, size_type n)
{
    /*@
        loop invariant bound:      1 <= i <= n+1;
        loop invariant increasing: WeaklyIncreasing(a, i);
        loop assigns i;
        loop variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
        if (a[i - 1u] > a[i]) {
            return i;
        }
    }

    return n;
}

```

Listing 10.3: Implementation of `is_sorted_until`

The algorithm proceeds sequentially and checks only adjacent pairs. If at some index i the element $a[i-1]$ is found to be greater than its successor $a[i]$, the algorithm terminates immediately and returns i , which marks the first position at which the ordering fails. This linear strategy is sufficient, since any violation of the global ordering property implies the existence of an adjacent pair $a[i-1] > a[i]$.

The loop starts at index 1, which is the smallest index at which an adjacent pair is available. As a consequence, the case $n == 0$ requires no special treatment: the loop body is never entered and the function returns 0, which is the length of the empty prefix.

Following the convention established earlier, the predicate `WeaklyIncreasing` [6.2] is used as the loop invariant `increasing`. The loop invariant is preserved by each iteration and falsified as soon as a counterexample is encountered. After loop termination, the desired postcondition `increasing` is obtained from the corresponding loop invariant and Lemma `WeaklyIncreasing_Increasing` [6.3].

Maintaining the stronger predicate `Increasing` as a loop invariant is possible, but leads to heavier annotations without additional benefit.

The postcondition `last` requires no loop invariant of its own. At the point of the premature return, the elements $a[i-1]$ and $a[i]$ already contradict the increasing property of the prefix $a[0 \dots i+1)$, and no information carried over from earlier iterations is needed to see this. The loop invariant `bound` confines the index to the range $1 \leq i \leq n+1$, the upper limit being attained only in the case $n == 0$. Its lower limit is what makes $a[i-1]$ an admissible witness.

10.3. The `is_sorted` algorithm

Our version of the `is_sorted` algorithm differs slightly from its counterpart in the C++ Standard Library [15, §28.7.1.5]. In particular, we consider a variant that operates directly on arrays of `value_type` and is therefore given the following signature.

```

bool is_sorted(const value_type* a, size_type n);

```

The function returns **true** if and only if the array $a[0 \dots n)$ is in increasing order. Otherwise, it returns **false**.

10.3.1. Formal specification of `is_sorted`

The following listing presents the ACSL specification of `is_sorted`. The contract characterizes sortedness by means of the predicate `Increasing` [6.1], which expresses the required global ordering property of the array.

```
/*@
  requires    valid:    \valid_read(a + (0..n-1));

  terminates      \true;
  exits           \false;
  assigns         \nothing;

  ensures    result:    \result <==> Increasing(a, n);
*/
bool is_sorted(const value_type* a, size_type n);
```

Listing 10.4: Formal specification of `is_sorted`

10.3.2. Implementation of `is_sorted`

The implementation of `is_sorted` is shown in the following listing. Rather than scanning the array itself, the function delegates the actual work to `is_sorted_until` [10.2].

```
bool is_sorted(const value_type* a, size_type n)
{
  return is_sorted_until(a, n) == n;
}
```

Listing 10.5: Implementation of `is_sorted`

The implementation simply tests whether the index returned by `is_sorted_until` coincides with the array length `n`. This design emphasizes the close conceptual relationship between the two algorithms: `is_sorted` acts as a Boolean wrapper around `is_sorted_until`, trading detailed positional information for a simple yes-or-no answer.

It also shows how the postcondition `result` follows from the contract of `is_sorted_until` alone. The postcondition `increasing` of `is_sorted_until` yields the implication from left to right, whereas the postcondition `last`, which establishes the maximality of the returned index, yields the converse. Without the latter, a return value smaller than `n` would not permit any conclusion about the array as a whole.

The converse direction is also the place where Lemma `Increasing_Shrink` earns its keep. The postcondition `last` denies the increasing property of a single prefix, namely $a[0 \dots \text{result} + 1)$, whereas the postcondition `result` of `is_sorted` speaks about the whole array. The lemma bridges the two: were the array increasing, so would be that prefix.

10.4. The `partial_sort` algorithm

Compared to its counterpart in the C++ Standard Library [15, §28.7.1.3], our version of the `partial_sort` algorithm has the following signature.

```
void partial_sort(value_type* a, size_type m, size_type n);
```

The algorithm *reorders* the array a so that it is partitioned at position m . Every element of the left part $a[0 \dots m)$ is less than or equal to every element of the right part $a[m \dots n)$. In addition, the left part is *sorted* in increasing order. No ordering guarantees are provided for the elements of the right part.

In contrast to a full sorting algorithm, `partial_sort` does not establish a global ordering of the entire array. It guarantees only that the first m elements are the m smallest elements of the array and that they appear in increasing order. This makes `partial_sort` particularly suitable when only a prefix of the sorted sequence is required. Figure 10.6 illustrates the effect of a call `partial_sort(a, m, n)` using a bar chart. In the post-state, the left and right parts of the array are highlighted in green and orange, respectively.

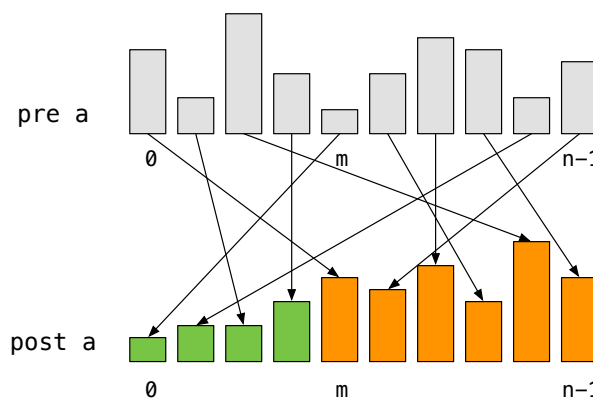


Figure 10.6.: Effects of `partial_sort`

10.4.1. The predicate `Partition`

We begin by introducing the overloaded predicate `Partition` [10.7], which captures the essence of a partition at a position within a range. It states a *cross-ordering* property: every element in the left part $a[m \dots p)$ is less than or equal to every element in the right part $a[p \dots n)$. No ordering is required *within* either part.

```
/*@
predicate Partition{L}(value_type* a, integer m, integer p, integer n) =
  \forall integer i, k; m <= i < p <= k < n ==> a[i] <= a[k];

predicate Partition{L}(value_type* a, integer p, integer n) =
  Partition{L}(a, 0, p, n);
*/
```

Listing 10.7: `Partition` (ACSL)

The arguments follow the convention of `Increasing`, `Unchanged` and `Rotate`: the outer indices m and n delimit the range under consideration, and p is the position at which it is split. The two-index form

`Partition(a,p,n)` abbreviates the case where the range starts at 0, that is, a split at position p within the prefix $a[0 \dots n)$. It is the form used throughout this section, where the split is the parameter m of `partial_sort`.

10.4.2. Auxiliary lemmas for Partition

The lemmas collected in the following listing support the reasoning about partitioning properties under reordering operations. They are stated independently of any particular algorithmic context and are later instantiated in the verification of `partial_sort`. Their technical foundation is the lemma `MultisetReorder_SomeEqual` [9.22] of §9.6, which states that reordering a range introduces no value that was not already present in it.

```

/*@
  lemma MultisetReorder_LowerBound{K,L}:
    \forallall value_type* a, integer n, value_type v;
      0 <= n ==>
      MultisetReorder{K,L}(a, n) ==>
      LowerBound{K}(a, n, v) ==>
      LowerBound{L}(a, n, v);

  lemma MultisetReorder_UpperBound{K,L}:
    \forallall value_type* a, integer n, value_type v;
      0 <= n ==>
      MultisetReorder{K,L}(a, n) ==>
      UpperBound{K}(a, n, v) ==>
      UpperBound{L}(a, n, v);

  lemma MultisetReorder_PartitionLowerBound{K,L}:
    \forallall value_type* a, integer m, n;
      0 < m <= n ==>
      MultisetReorder{K,L}(a, 0, m) ==>
      Partition{K}(a, m, n) ==>
      Unchanged{K,L}(a, m, n) ==>
      LowerBound{L}(a, m, n, \at(a[0],L));
*/

```

Listing 10.8: PartitionLemmas (ACSL)

- Lemma `MultisetReorder_LowerBound` states that a lower bound v of a range $a[0 \dots n)$ remains a lower bound after the range is reordered.
- Dually, lemma `MultisetReorder_UpperBound` asserts that reordering preserves upper bounds.
- Lemma `MultisetReorder_PartitionLowerBound` addresses a more specific situation. If the array is partitioned at position m and the left part $a[0 \dots m)$ is reordered while the right part $a[m \dots n)$ remains unchanged, then the first element of the left part remains a lower bound of the right part. This lemma will later be used to maintain partitioning properties across reordering steps.

10.4.3. Formal specification of `partial_sort`

The formal specification of the function `partial_sort` is given in the following listing. It reflects directly the informal description of the algorithm. The predicate `Partition` [10.7] expresses the partitioning property at position m , while the predicate `Increasing` [6.1] captures the required ordering of the left part $a[0 \dots m)$. As usual, the predicate `MultisetReorder` [7.59] is used to state that `partial_sort` merely reorders the elements of the array without changing their multiplicities.

```

/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   split:      0 <= m <= n;

  terminates          \true;
  exits              \false;
  assigns            a[0..n-1];

  ensures   reorder:    MultisetReorder{Old,Here}(a, n);
  ensures   partition:  Partition(a, m, n);
  ensures   increasing: Increasing(a, m);
*/
void partial_sort(value_type* a, size_type m, size_type n);

```

Listing 10.9: Formal specification of `partial_sort`

10.4.4. Implementation of `partial_sort`

The implementation of `partial_sort` is shown for better readability in two listings. Listing 10.10 shows the overall structure of the algorithm together with the scan loop. Listing 10.12 contains the loop body and the finalization step in full detail, including the assertions used to support the verification.

10.4.4.1. Prologue: building the initial heap

The algorithm begins by calling `make_heap` [9.37] to rearrange the left part $a[0 \dots m)$ into a heap. This representation makes the maximal element of the left part readily available at position $a[0]$. It will serve as the separating element during the subsequent scan of the right part.

10.4.4.2. Main act: scanning and maintaining the invariants

The right part $a[m \dots n)$ is scanned from left to right. Whenever the current element $a[i]$ is not smaller than the heap maximum $a[0]$, the scan proceeds without modifying the array. Otherwise, execution enters the then branch of the loop, where the current element is exchanged with the heap maximum by a suitable combination of `pop_heap` [9.32] and `push_heap` [9.23]. Throughout the scan loop, the following invariants are maintained:

- the left part $a[0 \dots m)$ forms a heap (invariant heap);
- the heap maximum $a[0]$ separates the left part from the already scanned right part $a[m \dots i)$ (invariants upper and lower);
- the suffix $a[i \dots n)$ has not yet been modified (invariant unchanged); and
- all modifications of $a[0 \dots i)$ are permutations (invariant reorder).

```

void partial_sort(value_type* a, size_type m, size_type n)
{
    if (m > 0u) {
        make_heap(a, m);

        //@ assert unchanged: Unchanged{Pre,Here}(a, m, n);
        /*@
            loop invariant bound:      m <= i <= n;
            loop invariant heap:      Heap(a, m);
            loop invariant upper:      UpperBound(a, 0, m, a[0]);
            loop invariant lower:      LowerBound(a, m, i, a[0]);
            loop invariant reorder:    MultisetReorder{Pre,Here}(a, i);
            loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
            loop assigns              i, a[0..n-1];
            loop variant              n-i;
        */
        for (size_type i = m; i < n; ++i) {
            if (a[i] < a[0u]) {
                //@ ghost BeforePop: ;
                pop_heap(a, m);
                //@ assert lower:      a[0] <= a[m-1];
                //@ assert lower:      a[i] < a[m-1];
                //@ assert lower:      LowerBound(a, m, i, a[m-1]);
                //@ assert upper:      UpperBound(a, 0, m-1, a[0]);
                //@ assert upper:      UpperBound(a, 0, m, a[m-1]);
                //@ assert partition: Partition(a, m, i);
                //@ assert unchanged: Unchanged{BeforePop,Here}(a, m, i);
                //@ assert reorder:    MultisetReorder{BeforePop,Here}(a, m);
                //@ assert reorder:    MultisetReorder{Pre,Here}(a, i);
                //@ assert unchanged: Unchanged{Pre,Here}(a, i, n);

                //@ ghost BeforeSwap: ;
                //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
                swap(a + m - 1u, a + i);
                //@ assert swapped:    ArraySwap{BeforeSwap,Here}(a, m-1, i, n);
                //@ assert unchanged: Unchanged{BeforeSwap,Here}(a, m-1);
                //@ assert unchanged: Unchanged{Pre,Here}(a, i+1, n);
                //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, m-1, i+1);
                //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, i+1);
                //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
                //@ assert lower:      a[m-1] < a[i];
                //@ assert partition: Partition(a, m, i+1);
                //@ assert upper:      UpperBound(a, 0, m-1, a[0]);

                //@ ghost BeforePush: ;
                push_heap(a, m);
                //@ assert heap:      Heap(a, m);
                //@ assert unchanged: Unchanged{BeforePush,Here}(a, m, i+1);
                //@ assert unchanged: Unchanged{BeforePush,Here}(a, i+1, n);
                //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
                //@ assert upper:      UpperBound(a, 0, m, a[0]);
                //@ assert lower:      LowerBound(a, m, i+1, a[0]);
            }
        }
    }
}

```

Listing 10.10: Implementation of partial_sort (1): overall structure and scan loop

The intermediate states arising in the non-trivial case of the loop are illustrated in Figure 10.11.

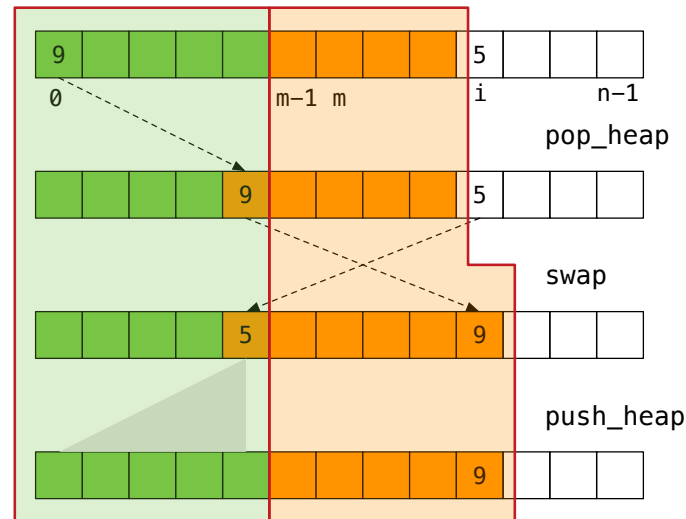


Figure 10.11.: An iteration of `partial_sort`

The elements considered so far are shown using the same coloring convention as in Figure 10.6. In particular, the heap is highlighted in green. The transparent red overlay indicates the ranges to which the predicate `Partition` applies in the respective states. For concreteness, the figure assumes initial values 9 and 5 at positions $a[0]$ and $a[i]$. In the general reasoning below, these values are denoted by p and q , respectively.

After the calls to `pop_heap` and `swap`, the value p resides at index i , while q is placed at index $m-1$. At this point, the following relations hold:

1. $q < p \leq a[k]$ for all $m \leq k < i$;
2. $q < p = a[i]$;
3. $a[j] \leq p \leq a[k]$ for all $0 \leq j < m - 1$ and all $m \leq k < i$; and
4. $a[j] \leq p = a[i]$ for all $0 \leq j < m - 1$.

Combining these inequalities, we obtain

$$a[j] \leq p \leq a[k] \quad \text{for all } 0 \leq j < m \text{ and all } m \leq k < i + 1.$$

Hence, `Partition(a, m, i+1)` holds, even though no distinguished separating element can be identified at this stage.

The subsequent call to `push_heap` only reorders elements within the left part and therefore preserves this partitioning property. At the same time, the heap property is restored, making $a[0]$ again an upper bound of the left part and thus a separating element between $a[0 \dots m)$ and $a[m \dots i + 1)$. Consequently, the invariants upper and lower are re-established. They are later used to prove the postcondition partition.

10.4.4.3. Epilogue: sorting the heap prefix

After termination of the scan loop (see Listing 10.12), the invariant partition guarantees that the array $a[0 \dots n]$ is partitioned at position m . Consequently, the left part $a[0 \dots m)$ contains exactly the m smallest elements of the array. At this stage, however, these elements are still arranged as a heap rather than in increasing order.

```
//@ assert partition: Partition(a, m, n);
//@ ghost BeforeSort: ;
sort_heap(a, m);
//@ assert unchanged: Unchanged{BeforeSort,Here}(a, m, n);
//@ assert reorder:   MultisetReorder{BeforeSort,Here}(a, m);
//@ assert reorder:   MultisetReorder{Pre,Here}(a, n);
//@ assert partition: Partition(a, m, n);
}
}
```

Listing 10.12: Implementation of `partial_sort` (2)

The final call to `sort_heap` transforms this heap into an increasingly ordered range. To reason about this call, a ghost label `BeforeSort` marks the program state immediately preceding it. This allows the post-state of `sort_heap` to be related both to the pre-state of the call and to the initial pre-state of `partial_sort`.

The predicate `Unchanged` [7.1] records that the right part $a[m \dots n]$ is not affected by the call to `sort_heap`. Moreover, the predicate `MultisetReorder` [7.59] is instantiated twice: first to express that `sort_heap` only reorders the left part, and second to re-establish that the entire array is a permutation of its initial contents. Since `sort_heap` preserves the multiset of the left part and leaves the right part unchanged, the partitioning property `Partition(a,m,n)` remains valid.

10.4.5. Remarks on specification style and complexity

Relative to its size, the `partial_sort` algorithm performs a considerable number of procedure calls. In this respect, it is closer to real-world software than many of the algorithms presented in this tutorial. For most procedure calls, the callee's contract is instantiated with the actual parameters and recorded as assertions, which may refer to the pre-state of the call by means of *ghost labels*. By contrast, explicit `assert` clauses are used to document local reasoning steps that are required for subsequent verification arguments.

The worst-case time complexity of the implementation is $\mathcal{O}((n + m) \cdot \log m)$. An implementation that ignores the parameter `m` and simply sorts the entire array $a[0 \dots n]$ also satisfies the contract of `partial_sort` [10.9], but has worst-case complexity $\mathcal{O}(n \cdot \log n)$. Elementary arithmetic shows that `partial_sort` is asymptotically more efficient if and only if

$$\log m < \frac{n}{m} \log \left(\frac{n}{m} \right),$$

that is, whenever n is sufficiently larger than m .

10.5. The bubble_sort algorithm

The bubble_sort algorithm is the first sorting algorithm whose implementation we verify in this chapter.¹⁸ From an algorithmic point of view, bubble_sort is simple and not particularly efficient. Its importance in our context lies elsewhere: its control structure is elementary, and the underlying idea can be expressed using straightforward loop invariants. This makes bubble_sort a suitable introductory example for the verification of sorting algorithms.

Algorithmically, bubble_sort repeatedly scans the array $a[0 \dots n]$ from left to right. During these scans it maintains a right-aligned suffix $a[n - i \dots n]$ that is already in increasing order and whose length grows monotonically. This suffix is established by comparing adjacent elements and swapping them whenever they appear in the wrong order, thereby moving larger values step by step towards the end of the array.

10.5.1. Formal specification of bubble_sort

The following listing instantiates the generic sorting contract from Listing 10.1 for the bubble_sort algorithm. As expected, the specification does not reflect any algorithm-specific details, but merely states the common correctness properties of sorting.

```
/*@
  requires    valid:      \valid(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            a[0..n-1];

  ensures      reorder:    MultisetReorder{Old,Here}(a, n);
  ensures      increasing: Increasing(a, n);
*/
void bubble_sort(value_type* a, size_type n);
```

Listing 10.13: Formal specification of bubble_sort

10.5.2. Implementation of bubble_sort

The implementation of bubble_sort is shown in the following listing. As is typical for this algorithm, it consists of two nested loops.

We first address the verification of the postcondition increasing. To this end, the *outer loop* is annotated with the invariant increasing, which states that the suffix $a[n - i + 1 \dots n]$ is already in increasing order. A crucial ingredient in establishing this property is the observation that the first element $a[n - i + 1]$ of the sorted suffix is an upper bound for *all* elements to its left. This fact is captured by the outer-loop invariant upper. To justify this claim, we exploit a property of the *inner loop*: at iteration index j , the element $a[j]$ is the maximum of the subrange $a[0 \dots j + 1]$. This property is formalized by the loop invariant max of the inner loop.

The invariants increasing and upper are also stated for the inner loop. This ensures that the outer loop may rely on these properties being preserved throughout the execution of the inner loop.

¹⁸See https://en.wikipedia.org/wiki/Bubble_sort.

```

void bubble_sort(value_type* a, size_type n)
{
    if (0 < n) {
        /*@
        loop invariant bound:      1 <= i <= n;
        loop invariant increasing: WeaklyIncreasing(a, n-i+1, n);
        loop invariant upper:      1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
        loop invariant reorder:    MultisetReorder{Pre,Here}(a, n);
        loop assigns i, a[0..n-1];
        loop variant n-i;
        */
        for (size_type i = 1u; i < n; ++i) {
            /*@
            loop invariant bound:      0 <= j <= n-i;
            loop invariant increasing: WeaklyIncreasing(a, n-i+1, n);
            loop invariant upper:      1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
            loop invariant max:        MaxElement(a, j+1, j);
            loop invariant reorder:    MultisetReorder{LoopEntry,Here}(a, j+1);
            loop invariant unchanged:  Unchanged{LoopEntry,Here}(a, j+1, n);
            loop assigns j, a[0..n-1];
            loop variant n-j;
            */
            for (size_type j = 0u; j < n - i; ++j) {
                if (a[j] > a[j + 1u]) {
                    /*@ assert max:      MaxElement(a, j+1, j);
                    /*@ assert reorder:    MultisetReorder{LoopEntry,Here}(a, 0, j+1);
                    /*@ assert reorder:    MultisetReorder{LoopEntry,Here}(a, 0, j+2);
                    swap(&a[j], &a[j + 1u]);
                    /*@ assert max:      MaxElement(a, j+2, j+1);
                    /*@ assert swapped:    a[j+1] == At{LoopCurrent}(a, j);
                    /*@ assert swapped:    a[j] == At{LoopCurrent}(a, j+1);
                    /*@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, 0, j);
                    /*@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, j+2, n);
                    /*@ assert swapped:    ArraySwap{LoopCurrent,Here}(a, j, j+1, n);
                    /*@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, j, j+2);
                }
            }
        }
    }

    /*@ assert increasing: Increasing(a, n);
}

```

Listing 10.14: Implementation of bubble_sort

Here are a few remarks about the verification of the postcondition reorder.

- Within each iteration of the outer loop, elements of the unsorted prefix $a[0 \dots n)$ are swapped with their immediate neighbours.
- More precisely, at iteration j of the inner loop, the subrange $a[0 \dots j + 1)$ may be rearranged, while the subrange $a[j + 1 \dots n)$ remains unchanged.

Together, these facts suffice to establish the loop invariant reorder for the outer loop, and hence the desired postcondition.

10.6. The selection_sort algorithm

Next we consider the sorting algorithm `selection_sort`.¹⁹ The algorithm proceeds by growing an increasingly ordered prefix from left to right. In each iteration, it selects the minimum element of the remaining suffix and swaps it with the first element of that suffix.

As a consequence of this strategy, after each iteration the array is partitioned into two contiguous ranges. The prefix is in increasing order, and every element in it is less than or equal to every element in the remaining suffix. This invariant captures the essence of `selection_sort` and will later be reflected directly in the loop annotations.

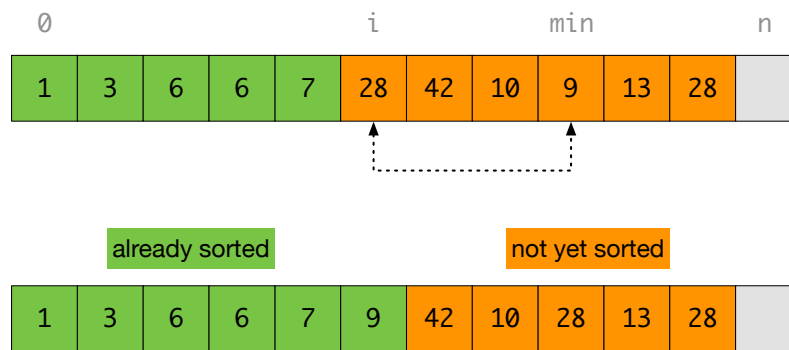


Figure 10.15.: An iteration of `selection_sort`

Figure 10.15 illustrates a typical iteration. The minimum element 9 of the unsorted suffix, located at position `min`, is swapped with the element 28 at position `i`. This operation extends the increasingly ordered prefix by one array cell.

10.6.1. Formal specification of `selection_sort`

The following listing instantiates the generic sorting contract from Listing 10.1 for the `selection_sort` algorithm. As expected, the specification does not reflect any algorithm-specific details, but merely states the common correctness properties of sorting.

```
/*@
  requires   valid:      \valid(a + (0..n-1));

  terminates      \true;
  exits        \false;
  assigns       a[0..n-1];

  ensures reorder:  MultisetReorder{Old,Here}(a, n);
  ensures increasing: Increasing(a, n);
*/
void selection_sort(value_type* a, size_type n);
```

Listing 10.16: Formal specification of `selection_sort`

¹⁹See https://en.wikipedia.org/wiki/Selection_sort.

10.6.2. Implementation of selection_sort

The implementation of `selection_sort` is shown in the following listing. The algorithm iterates over the array from left to right and, in each iteration, selects the minimum element of the remaining suffix using `min_element` [5.15]. This element is then swapped into its final position at the boundary between the already sorted prefix and the unsorted remainder.

```
void selection_sort(value_type* a, size_type n)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant reorder:    MultisetReorder{Pre,Here}(a, n);
    loop invariant increasing: WeaklyIncreasing(a, i);
    loop invariant lower:      0 < i ==> LowerBound(a, i, n, a[i-1]);
    loop assigns   i, a[0..n-1];
    loop variant   n - i;
    */
    for (size_type i = 0u; i < n; ++i) {
        const size_type sel = i + min_element(a + i, n - i);

        if (i < sel) {
            //@ ghost Before: ;
            swap(a + sel, a + i);
            //@ assert unchanged: Unchanged{Before,Here}(a, i+1, sel);
            //@ assert unchanged: Unchanged{Before,Here}(a, sel+1, n);
            //@ assert swapped: ArraySwap{Before,Here}(a, i, sel, n);
        }

        //@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, n);
        //@ assert reorder: MultisetReorder{Pre,Here}(a, n);
    }

    //@ assert increasing: Increasing(a, n);
}
```

Listing 10.17: Implementation of `selection_sort`

The loop invariants `increasing` and `lower` formalize the central ordering argument. They establish that the prefix $a[0 \dots i]$ is in increasing order and that its last element $a[i - 1]$ is a lower bound for all elements in the remaining suffix $a[i \dots n]$. Together, these properties ensure that the prefix grows monotonically and never needs to be modified again.

Because the call to `min_element` operates on a shifted address, its use requires the application of the *shift lemmas* collected in [6.14], which allow us to relate properties of the suffix back to the original array.

The loop invariant `reorder` expresses that `selection_sort` only rearranges the elements of the array. Although this is intuitively clear—since `swap` [7.7] is the only operation that modifies a —its formal verification requires an explicit account of the effect of swapping.

In the current code, this effect is expressed using ghost code rather than a statement contract. A ghost label `Before` is placed immediately before the call to `swap`, and the predicate `swapped` is asserted afterwards. Together, these annotations relate the states at `Before` and `Here` via `ArraySwap{Before,Here}(a, i, sel, n)`, which suffices to apply Lemma `MultisetSwap_Middle` [7.63] and thus establish the required `MultisetReorder` [7.59] property.

10.7. The insertion_sort algorithm

As another simple comparison-based sorting algorithm studied in this chapter, we consider the algorithm `insertion_sort`.²⁰ Like `selection_sort`, the algorithm `insertion_sort` traverses the given array $a[0 \dots n)$ from left to right and maintains a left-adjusted prefix $a[0 \dots i)$ that is already in increasing order.

The defining idea of `insertion_sort` is how this sorted prefix is extended. In the i th iteration, the next element $a[i]$ is inserted into its appropriate position within the sorted prefix, thereby increasing the sorted range by one element. Conceptually, this insertion is achieved by shifting the elements of the prefix to make room for $a[i]$ at its correct position. Figure 10.18 illustrates a single iteration of this process.

This step is a merge. Extending $a[0 \dots i)$ by $a[i]$ combines two ordered ranges, the second of which happens to hold a single element, and §10.10 treats the general case under the name `inplace_merge`. `insertion_sort` does not call it: it finds the position by binary search, where `inplace_merge` scans. The two share the rotation that makes room, and hence the lemmas discussed below.

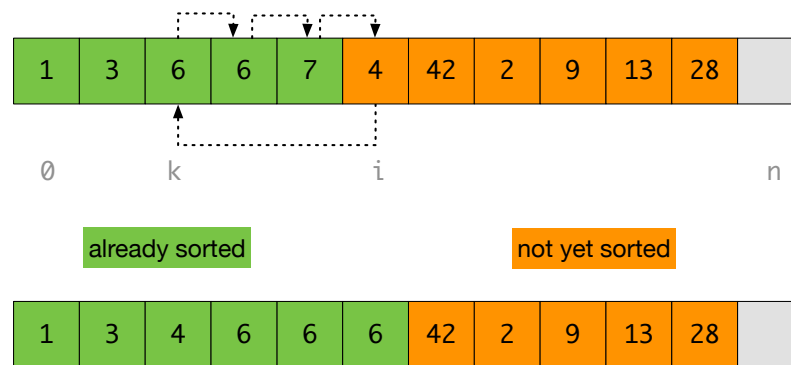


Figure 10.18.: An iteration of `insertion_sort`

10.7.1. Formal specification of `insertion_sort`

The next listing gives the formal specification of the `insertion_sort` algorithm. It is obtained by instantiating the generic sorting contract from Listing 10.1.

```

/*@
  requires    valid:      \valid(a + (0..n-1));

  terminates      \true;
  exits          \false;
  assigns        a[0..n-1];

  ensures      reorder:    MultisetReorder{Old,Here}(a, n);
  ensures      increasing: Increasing(a, n);
*/
void insertion_sort(value_type* a, size_type n);

```

Listing 10.19: Formal specification of `insertion_sort`

As for the other sorting algorithms considered in this chapter, the contract is deliberately free of algorithm-specific details and captures only the essential correctness properties that characterize sorting.

²⁰See https://en.wikipedia.org/wiki/Insertion_sort.

10.7.2. Implementation of insertion_sort

The implementation of `insertion_sort` is shown in the next listing. In each iteration, the appropriate insertion position for the current element `a[i]` is determined by a call to `upper_bound` [6.8], which returns the rightmost index at which `a[i]` can be inserted without violating the ordering. The insertion itself is then realized by a call to `rotate` [7.32], which performs a circular shift of the affected subrange. This decomposition separates the identification of the insertion position from the actual rearrangement of elements, resulting in a modular implementation. As we will see shortly, this modular structure also simplifies the verification effort, since it allows us to rely on the contracts of these auxiliary operations.

```

void insertion_sort(value_type* a, size_type n)
{
    /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant reorder:    MultisetReorder{Pre,Here}(a, 0, i);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
    loop invariant increasing: WeaklyIncreasing(a, i);
    loop assigns    i, a[0..n-1];
    loop variant    n - i;
    */
    for (size_type i = 0u; i < n; ++i) {
        const size_type k = upper_bound(a, i, a[i]);
        /*@ assert bound:      0 <= k <= i;
        /*@ ghost Before: ;
        /*@ assert lower:      StrictLowerBound(a, k, i, a[i]);
        /*@ assert upper:      UpperBound(a, k, a[i]);
        rotate(a + k, i - k, i + 1u - k);
        /*@ assert rotate:     Rotate{Before,Here}(a, k, i, i+1);
        /*@ assert upper:      UpperBound(a, k, a[k]);
        /*@ assert increasing: WeaklyIncreasing(a, k+1);
        /*@ assert increasing: WeaklyIncreasing(a, k+1, i+1);
        /*@ assert increasing: WeaklyIncreasing(a, i+1);
        /*@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, 0, k);
        /*@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, k, i+1);
        /*@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, 0, i+1);
        /*@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
    }

    /*@ assert increasing: Increasing(a, n);
}

```

Listing 10.20: Implementation of `insertion_sort`

The effects of the call to `rotate` are formally captured by a version of the predicate `Rotate` [7.28], which was introduced in Equation (7.3). Using the labels *Before* and *Here* to denote the states before and after the call, respectively, and abbreviating them by *B* and *H*, the corresponding assertion states the following: The former prefix $a[k \dots i]$ becomes the suffix of the rotated slice, that is,

$$a^H[k + 1 \dots i + 1] = a^B[k \dots i],$$

while the former suffix $a[i] = a[i \dots i + 1]$ becomes the new prefix,

$$a^H[k \dots k + 1] = a^B[i \dots i + 1],$$

or, equivalently,

$$a^H[k] = a^B[i].$$

For the parts of the array preceding and following the rotated slice, no changes occur:

$$\begin{aligned} a^H[0 \dots k) &= a^B[0 \dots k), \\ a^H[i + 1 \dots n) &= a^B[i + 1 \dots n). \end{aligned}$$

When `rotate` was originally implemented and verified, its later use within `insertion_sort` was not yet anticipated. As a consequence, certain properties required in the present context — in particular the fact that a rotation constitutes a *reordering* of the affected elements — are not stated explicitly in its contract. One conceivable remedy would be to strengthen the contract of `rotate` [7.32] by adding the properties needed for `insertion_sort` and to repeat its verification. Such an approach, however, is inappropriate if `rotate` is regarded as part of a pre-verified library: in that case, its implementation may no longer be available for re-verification.

We therefore adopt a different strategy. Rather than modifying the existing specification, we show that the original contract of `rotate` already *implies* all properties required for `insertion_sort`. To make this implication explicit, we introduce and prove a small number of auxiliary lemmas that derive the needed properties from the contract of `rotate`. This argument is a direct application of the implication rule of the Hoare calculus (see §3.4).

10.7.2.1. Preservation of Order Under Elementwise Equality

Lemma `Increasing_Equal` [6.3] assumes that a range $a[m \dots n)$ is ordered and states that any elementwise equal range $a[m + p \dots n + p)$ is ordered as well. This lemma is used to show that the call to `rotate` [7.32] preserves the order of those elements that are shifted upwards (see Figure 10.18).

10.7.2.2. Auxiliary Lemmas for Reasoning About Rotations

In addition, we rely on three further lemmas, listed below, which make the required logical implications explicit and assist the provers in establishing the desired properties of rotations.

```

/*@
lemma Rotate_Shift{K,L}:
  \forall value_type *a, integer k, m, p, n;
    Rotate{K,L}(a+k, m, p, n) ==> Rotate{K,L}(a, m+k, p+k, n+k);

lemma RotateOne_StrictLowerBound{K,L}:
  \forall value_type* a, integer m, n;
    StrictLowerBound{K}(a, m, n, \at(a[n],K)) ==>
    Rotate{K,L}(a, m, n, n+1) ==>
    StrictLowerBound{L}(a, m+1, n+1, \at(a[m],L));

lemma RotateOne_MultisetReorder{K,L}:
  \forall value_type* a, integer m, n;
    0 <= m <= n ==>
    Rotate{K,L}(a, m, n, n+1) ==>
    MultisetReorder{K,L}(a, m, n+1);
*/

```

Listing 10.21: RotateLemmas (ACSL)

Lemma `Rotate_Shift` [10.21] relates rotations applied to ranges accessed through pointer offsets to corresponding rotations on the underlying base array. It allows us to transfer properties established for a rotated slice $a[m + k \dots n + k)$ back to the corresponding slice $a[m \dots n)$.

The remaining two lemmas deal with rotations by exactly one element.

- To show that a rotation by one position preserves the strict lower-bound relationship required by the loop invariant, we rely on Lemma `RotateOne_StrictLowerBound`. More precisely, if the range $a[k \dots i)$ has $a[i]$ as a strict lower bound in the state before the call to `rotate`, then after the call the same range has $a[k]$ as a strict lower bound. This statement captures precisely the effect of using `rotate` as a *circular shift by one position* at the call site.
- Lemma `RotateOne_MultisetReorder` establishes that a rotation by one element merely reorders the range to which it is applied. Together with the assertions relating the states of the array before and after the call to `rotate`, this lemma allows us to conclude that the multiset of array elements is preserved, as required by the loop invariant of `insertion_sort`.

Unlike the preceding lemma, this result could, in principle, be generalized to rotations by an arbitrary number of positions; however, the specialized formulation suffices for the present use case.

10.8. The heap_sort algorithm

The heap_sort algorithm²¹ is entirely built on top of the heap primitives introduced in Chapter 9. In particular, it combines a call to make_heap, repeated applications of pop_heap, and a final call to sort_heap to transform an array into increasing order. This process is already suggested by Figure 9.1, which highlights the interaction between the individual heap algorithms and their effect on the array state.

10.8.1. Formal specification of heap_sort

The following listing presents the formal specification of the heap_sort algorithm. It is derived by instantiating the generic sorting contract from Listing 10.1. As in the preceding sorting algorithms, the specification abstracts from algorithm-specific mechanisms and records only the fundamental properties that any correct sorting procedure must satisfy.

```
/*@
  requires    valid:      \valid(a + (0..n-1));

  terminates          \true;
  exits              \false;
  assigns            a[0..n-1];

  ensures      reorder:    MultisetReorder{Old,Here}(a, n);
  ensures      increasing: Increasing(a, n);
*/
void heap_sort(value_type* a, size_type n);
```

Listing 10.22: Formal specification of heap_sort

10.8.2. Implementation of heap_sort

The implementation of heap_sort is shown in the next listing. Conceptually, the algorithm proceeds in two phases. First, the input array $a[0 \dots n]$ is rearranged into a heap by a call to make_heap [9.37], establishing the heap structure required for efficient access to the maximum element. Second, a call to sort_heap [9.40] repeatedly removes this maximum element and places it at the end of the array, thereby transforming the heap into an increasingly ordered range.

```
void heap_sort(value_type* a, size_type n)
{
    make_heap(a, n);
    sort_heap(a, n);
}
```

Listing 10.23: Implementation of heap_sort

From a verification perspective, heap_sort is particularly instructive, as it combines non-trivial structural invariants of heaps with the global ordering and multiset preservation properties common to all sorting algorithms.

²¹See <https://en.wikipedia.org/wiki/Heapsort>.

10.9. The merge algorithm

The algorithm `merge` is taken from the C++ standard library [15, p. 28.7.5]. Unlike the sorting algorithms studied in the surrounding sections, `merge` is not a sorting algorithm in its own right. Instead, it combines two already ordered sequences into a single ordered sequence and thus serves as a fundamental building block for sorting algorithms, most directly for `merge_sort` in §10.11. The following section presents `inplace_merge`, which performs the same combination within a single array and so needs no output array at all.

Our version of `merge` has the following signature.

```
void merge(const value_type* a, size_type m,
           const value_type* b, size_type n, value_type* c);
```

The function `merge` takes two input arrays $a[0 \dots m)$ and $b[0 \dots n)$, both of which are assumed to be ordered with respect to the ordering on `value_type`. It produces an output array that contains exactly the elements of both inputs, arranged in ordered form.

The array starting at `c` must designate a memory region that is disjoint from the input arrays and large enough to store exactly $m + n$ elements. Upon termination, the array $c[0 \dots m + n)$ is ordered and contains each element of a and b exactly once.

If two elements from different input arrays compare equal, the element originating from a precedes the element originating from b . Moreover, the relative order of equal elements within each individual input array is preserved. In this sense, `merge` is a *stable* combining operation. Neither property appears among the postconditions, and the reason is discussed after Open Issue 10.9.1.

10.9.1. The predicate `MultisetUnion`

To say what `merge` produces, the postcondition has to relate three arrays in a single program state: the output $c[0 \dots m + n)$ must consist of exactly the elements of $a[0 \dots m)$ and $b[0 \dots n)$ taken together, each with its multiplicity. The predicate `MultisetReorder` [7.59], which serves this purpose for the sorting algorithms, cannot be used here. It compares *one* array in *two* states, whereas `merge` requires *three* arrays to be compared in *one* state. The logic function `Count` [4.53] is itself parameterized by a program state, so the property is nevertheless expressible, and we introduce it as the predicate `MultisetUnion` [10.24].

```
/*@
  predicate MultisetUnion{L}(value_type* a, integer m,
                             value_type* b, integer n, value_type* c) =
    \forall value_type v;
      Count{L}(c, 0, m+n, v) == Count{L}(a, 0, m, v) + Count{L}(b, 0, n, v);
*/
```

Listing 10.24: `MultisetUnion` (ACSL)

The definition states that every value occurs in the output as often as it occurs in the two input arrays together. No ordering enters the definition. `MultisetUnion` is thus precisely the part of the intended behavior of `merge` that the ordering postcondition does not already cover.

10.9.2. Formal specification of merge

Listing 10.25 shows the formal specification of merge. It requires the input arrays $a[0 \dots m)$ and $b[0 \dots n)$ to be valid, non-overlapping, and ordered in increasing order, and the output array c to be valid for exactly $m + n$ elements and disjoint from the inputs.

```
/*@
  requires bound:      m + n <= SIZE_TYPE_MAX;
  requires valid:      \valid_read(a + (0..m-1));
  requires valid:      \valid_read(b + (0..n-1));
  requires valid:      \valid(c + (0..m+n-1));
  requires sep:        \separated(a + (0..m-1), c + (0..m+n-1));
  requires sep:        \separated(b + (0..n-1), c + (0..m+n-1));
  requires increasing: Increasing(a, m);
  requires increasing: Increasing(b, n);

  terminates           \true;
  exits                \false;
  assigns              c[0 .. m+n-1];

  ensures increasing:   Increasing(c, m + n);
  ensures multiset:     MultisetUnion(a, m, b, n, c);
  ensures unchanged:    Unchanged{Old,Here}(a, m);
  ensures unchanged:    Unchanged{Old,Here}(b, n);
*/
void merge(const value_type* a, size_type m,
           const value_type* b, size_type n, value_type* c);
```

Listing 10.25: Formal specification of merge

Two postconditions describe the output array. The postcondition `increasing` states that the array $c[0 \dots m + n)$ is ordered, and the postcondition `multiset` states that it holds exactly the elements of the two input arrays. Neither would say much on its own: the first is satisfied by any ordered array of the right length, the second by any arrangement of the right elements. Together they determine c uniquely, because among all arrangements of a given multiset exactly one is increasing. The two postconditions unchanged record that merge leaves its input arrays untouched.

What remains outside the specification are the two *stability* properties described at the beginning of this section. They are collected in the following open issue.

Open Issue 10.9.1 (Completeness of the postcondition of merge).

The following properties, which are stated informally, are not yet captured by the formal postconditions:

- *if two elements from different input arrays are equal, the element originating from a precedes the element originating from b ;*
- *the relative order of equal elements within each input array is preserved.*

Both properties are absent from the contract for a reason that is worth spelling out, because it is not a shortage of proof effort. The type `value_type` is `int` (see §2), so two elements that compare equal *are* the same value, and the output array $c[0 \dots m + n)$ records nothing about the array a value came from. A contract constrains the pre-state and the post-state of a call, and the two possible tie-breaks lead to the same post-state. For $a = (1, 3)$ and $b = (1, 4)$, taking the 1 from a first and taking it from b first both produce $c = (1, 1, 3, 4)$. This is the uniqueness observed above, seen from another side: merge's output is a function of its inputs alone, which is also why the test of merge may compare our result with that of `std::merge` element by element instead of merely checking that it is ordered.

The comparison with `remove_copy3` in §7.16 is instructive, since the corresponding property is formalized there, by the predicate `Remove` [7.52] and the logic function `IndexOfNotEqual`, for the same element type and without any additional machinery. The difference does not lie in the means of expression. It lies in what the algorithms produce: copying the elements of $(1, 2)$ that differ from 3 in the opposite order yields $(2, 1)$, an output that a predicate relating input and output indices can rule out, whereas both tie-breaks of `merge` yield the very same array. An index-based characterization of c could be written just as well, and would hold of either version, adding nothing to `increasing` and `multiset`.

Two changes would give the two properties content. The ordering on `value_type` could be a strict weak ordering instead of the strict partial order to which we restrict ourselves in §5.1, so that distinct elements may compare equivalent, as they do for a structure ordered on one of its fields; stability then concerns two elements a reader can tell apart. Alternatively, ghost state could record for each position of c which input it was taken from, at the price of specifying how `merge` proceeds rather than what it returns, which no other contract in this document does. The open issue is therefore recorded against the element type rather than against the specification, and will be taken up when the document admits an element type of the first kind.

10.9.3. More Lemmas on `WeaklyIncreasing`

The following listing introduces several auxiliary lemmas about `WeaklyIncreasing` [6.2] that are repeatedly used in the formal verification of `merge`. Each lemma captures a simple but frequently needed reasoning step about weakly increasing ranges.

- Lemma `WeaklyIncreasing_Shrink` [10.26] expresses stability under restriction: any sub-range of a weakly increasing array is itself weakly increasing.
- Lemma `WeaklyIncreasing_AddElement` characterizes extension by one element. It states that appending an element preserves the weakly increasing property, provided the new element is greater than or equal to the last element of the existing range.
- Lemma `WeaklyIncreasing_Shift` captures invariance under pointer arithmetic. It allows rewriting `WeaklyIncreasing` when ranges are accessed through shifted pointers.
- Lemma `WeaklyIncreasing_Unchanged` states that the weakly increasing property is preserved between two program points whenever the corresponding range remains unchanged.
- Lemma `WeaklyIncreasing_Equal` formalizes preservation under element-wise equality. If two ranges are pointwise equal, then `WeaklyIncreasing` holds for one if and only if it holds for the other.
- Lemma `WeaklyIncreasing_Join` provides sufficient conditions for concatenating two consecutive weakly increasing ranges. The combined range is weakly increasing if both sub-ranges are weakly increasing and the last element of the first does not exceed the first element of the second.

```

/*@
lemma WeaklyIncreasing_Shrink{L}:
  \forallall value_type *a, integer m, n, p, q;
    m <= p <= q <= n      ==>
    WeaklyIncreasing(a, m, n) ==> WeaklyIncreasing(a, p, q);

lemma WeaklyIncreasing_AddElement{L}:
  \forallall value_type *a, integer n;
    1 < n      ==>
    a[n-2] <= a[n-1] ==>
    WeaklyIncreasing(a, n-1) ==> WeaklyIncreasing(a, n);

lemma WeaklyIncreasing_Shift{L}:
  \forallall value_type *a, integer m, n;
    WeaklyIncreasing(a + m, 0, n) <==> WeaklyIncreasing(a, m, n + m);

lemma WeaklyIncreasing_Equal{K,L}:
  \forallall value_type *a, *b, integer m, n;
    Equal{K,L}(a, m, n, b) ==>
    WeaklyIncreasing{K}(a, m, n) ==> WeaklyIncreasing{L}(b, m, n);

lemma WeaklyIncreasing_Unchanged{K,L}:
  \forallall value_type *a, integer m, n;
    Unchanged{K,L}(a, m, n) ==>
    WeaklyIncreasing{K}(a, m, n) ==> WeaklyIncreasing{L}(a, m, n);

lemma WeaklyIncreasing_Join{L}:
  \forallall value_type *a, integer m, n;
    0 < m < n      ==>
    WeaklyIncreasing(a, m) ==>
    WeaklyIncreasing(a, m, n) ==>
    a[m-1] <= a[m] ==> WeaklyIncreasing(a, n);
*/

```

Listing 10.26: WeaklyIncreasingLemmas (ACSL)

10.9.4. Auxiliary lemmas for MultisetUnion

The lemmas in the following listing carry MultisetUnion through the steps of the algorithm. They are stated independently of any particular algorithmic context and are later instantiated in the verification of merge.

- Lemma Equal_Shift relates the two forms of Equal [4.31]: a range compared through shifted pointers is the same as the corresponding index range of the unshifted arrays. It is needed because copy states its postcondition in the shifted form.
- Lemma Count_Between generalizes Count_Equal [4.53] from one array to two. If a range of a agrees element-wise with a range of b , then every value occurs equally often in both.
- Lemma Count_Tail is the instance of Count_Between that the epilogue of merge needs, with the index arithmetic of that situation already carried out.
- Lemma MultisetUnion_Empty is the base case: two empty inputs and an empty output.

```

/*@
lemma Equal_Shift{K,L}:
  \forall value_type *a, *c, integer i, j, m;
    0 <= i <= m ==> 0 <= j ==>
    Equal{K,L}(a+i, m-i, c+j) ==>
    Equal{K,L}(a, i, m, c, j);

lemma Count_Between{K,L}:
  \forall value_type *a, *b, v, integer m, n, p;
    0 <= m <= n ==>
    Equal{K,L}(a, m, n, b, p) ==>
    Count{K}(a, m, n, v) == Count{L}(b, p, p + (n-m), v);

lemma Count_Tail{L}:
  \forall value_type *a, *c, v, integer i, j, m;
    0 <= i <= m ==> 0 <= j ==>
    Equal{L,L}(a, i, m, c, i+j) ==>
    Count{L}(c, i+j, m+j, v) == Count{L}(a, i, m, v);

lemma MultisetUnion_Empty{L}:
  \forall value_type *a, *b, *c;
    MultisetUnion{L}(a, 0, b, 0, c);

lemma MultisetUnion_Unchanged{K,L}:
  \forall value_type *a, *b, *c, integer m, n;
    0 <= m ==> 0 <= n ==>
    Unchanged{K,L}(a, 0, m) ==>
    Unchanged{K,L}(b, 0, n) ==>
    Unchanged{K,L}(c, 0, m+n) ==>
    MultisetUnion{K}(a, m, b, n, c) ==>
    MultisetUnion{L}(a, m, b, n, c);

lemma MultisetUnion_Symmetric{L}:
  \forall value_type *a, *b, *c, integer m, n;
    MultisetUnion{L}(a, m, b, n, c) ==> MultisetUnion{L}(b, n, a, m, c);

lemma MultisetUnion_Left{L}:
  \forall value_type *a, *b, *c, integer i, j;
    0 <= i ==> 0 <= j ==>
    At{L}(c, i+j) == At{L}(a, i) ==>
    MultisetUnion{L}(a, i, b, j, c) ==>
    MultisetUnion{L}(a, i+1, b, j, c);

lemma MultisetUnion_Right{L}:
  \forall value_type *a, *b, *c, integer i, j;
    0 <= i ==> 0 <= j ==>
    At{L}(c, i+j) == At{L}(b, j) ==>
    MultisetUnion{L}(a, i, b, j, c) ==>
    MultisetUnion{L}(a, i, b, j+1, c);

lemma MultisetUnion_TailLeft{L}:
  \forall value_type *a, *b, *c, integer i, j, m;
    0 <= i <= m ==> 0 <= j ==>
    Equal{L,L}(a, i, m, c, i+j) ==>
    MultisetUnion{L}(a, i, b, j, c) ==>
    MultisetUnion{L}(a, m, b, j, c);
*/

```

Listing 10.27: MultisetUnionLemmas (ACSL)

- Lemma `MultisetUnion_Unchanged` transports `MultisetUnion` from one program state to another in which none of the three arrays has changed over the ranges concerned.
- Lemma `MultisetUnion_Symmetric` states that the roles of the two input arrays may be exchanged. It halves the work: every step that appends to the output from b is the corresponding step for a , read through this lemma.
- Lemmas `MultisetUnion_Left` and `MultisetUnion_Right` are the step of the scan loop. Appending to the output one element that is taken from an input array extends `MultisetUnion` by that element.
- Lemma `MultisetUnion_TailLeft` is the step of the epilogue. Appending a whole remaining block of one input array to the output extends `MultisetUnion` by that block.

10.9.5. Implementation of merge

The implementation of merge, shown in Listings 10.28 to 10.31, follows the standard merge procedure. Two read positions traverse the input arrays a and b , while a write position advances through the output array c . At each step, the smaller of the two current input elements is written to the next position of c . In this way, an increasing prefix of the output array is constructed incrementally.

```

void merge(const value_type* a, size_type m,
           const value_type* b, size_type n, value_type* c)
{
    //@ assert increasing: WeaklyIncreasing(a, 0, m);
    size_type i = 0;
    size_type j = 0;
    size_type x = 0;

    if (0 < m || 0 < n) {
        /*@ loop invariant bound:      0 <= i <= m;
           loop invariant bound:      0 <= j <= n;
           loop invariant index:      x == i+j;
           loop invariant bound:      0 <= x <= m+n-1;
           loop invariant upper:      i < m ==> UpperBound(c, 0, x, a[i]);
           loop invariant upper:      j < n ==> UpperBound(c, 0, x, b[j]);
           loop invariant increasing: WeaklyIncreasing(c, x);
           loop invariant multiset:   MultisetUnion(a, i, b, j, c);
           loop assigns i, j, x, c[0 .. m+n-1];
           loop variant (m+n) - (i+j);
        */
    }
}

```

Listing 10.28: Implementation of merge (1): signature and loop contract

The main `while` loop is responsible for this incremental construction. Its loop invariants bound constrain the three positions to their ranges, and the invariant index records that the write position is the sum of the two read positions, $x = i + j$. The invariant `increasing` states that the already written prefix $c[0 \dots x)$ is weakly increasing, and the two invariants `upper` state that every element written so far is bounded by the next unread element of either input array. Together they ensure that writing that element preserves the ordering of the prefix. The invariant `multiset` states that the prefix written so far consists of exactly the elements already read from the two inputs. Several assertions appear in the loop body. These assertions do not affect the algorithmic behavior but serve to make intermediate reasoning steps explicit for the automatic provers. The ghost label `Step` marks the state at the beginning of an iteration, so that the assertions can say which arrays the single write has left unchanged. That is what Lemma `MultisetUnion_Unchanged` needs in order to carry the invariant across the write, after which Lemma `MultisetUnion_Left` or `MultisetUnion_Right` extends it by the element just appended.

```

while (i < m && j < n) {
  //@ ghost Step: ;
  if (a[i] <= b[j]) {
    c[x++] = a[i++];
    //@ assert unchanged:    Unchanged{Step,Here}(a, 0, i-1);
    //@ assert upper:        i < m ==> UpperBound(c, 0, x, a[i]);
    //@ assert upper:        j < n ==> UpperBound(c, 0, x, b[j]);
    //@ assert unchanged:    Unchanged{Step,Here}(b, 0, j);
    //@ assert unchanged:    Unchanged{Step,Here}(c, 0, (i-1)+j);
    //@ assert increasing:    WeaklyIncreasing(c, 0, x);
    //@ assert multiset:      MultisetUnion(a, i-1, b, j, c);
    //@ assert equal:         At(c, (i-1)+j) == At(a, i-1);
    //@ assert multiset:      MultisetUnion(a, i, b, j, c);
  }
  else {
    c[x++] = b[j++];
    //@ assert unchanged:    Unchanged{Step,Here}(b, 0, j-1);
    //@ assert unchanged:    Unchanged{Step,Here}(a, 0, i);
    //@ assert upper:        j < n ==> UpperBound(c, 0, x, b[j]);
    //@ assert upper:        i < m ==> UpperBound(c, 0, x, a[i]);
    //@ assert unchanged:    Unchanged{Step,Here}(c, 0, i+(j-1));
    //@ assert increasing:    WeaklyIncreasing(c, 0, x);
    //@ assert multiset:      MultisetUnion(a, i, b, j-1, c);
    //@ assert equal:         At(c, i+(j-1)) == At(b, j-1);
    //@ assert multiset:      MultisetUnion(a, i, b, j, c);
  }

  //@ assert increasing:    WeaklyIncreasing(c, 0, x);
}

```

Listing 10.29: Implementation of merge (2): the scan loop

After termination of the **while** loop, at least one of the input arrays has been completely processed. At this point, the algorithm enters its epilogue, starting at the ghost label Epilogue (see Listings 10.30 and 10.31). The remaining elements of the other input array are copied unchanged to the output array *c*.

Since these elements are already ordered and all previously written elements form a weakly increasing prefix, the entire output array is weakly increasing. The postcondition *increasing* then follows by Lemma *WeaklyIncreasing_Increasing* [6.3].

The postcondition *multiset* follows a parallel course. The call to *copy* leaves the copied block element-wise equal to its source, which Lemma *Equal_Shift* restates in index form, and Lemma *MultisetUnion_Tailleft* then extends *MultisetUnion* by the whole block at once. Only the branch that empties *a* is treated in this way. The other branch exchanges the roles of the two input arrays by Lemma *MultisetUnion_Symmetric*, applies the same lemma, and exchanges them back.

```

/*@ ghost Epilogue: ;
/*@ assert index:      x == i+j;
/*@ assert index:      i == m ^^ j == n;
/*@ assert index:      i < m ^^ j < n;
/*@ assert increasing: WeaklyIncreasing(c, 0, x);
/*@ assert unchanged:  Unchanged{Pre,Here}(a, 0, m);
/*@ assert increasing: WeaklyIncreasing(a, 0, m);
/*@ assert multiset:    MultisetUnion(a, i, b, j, c);

if (i < m) {
  /*@ assert upper:      0 < x ==> c[x-1] <= a[i];
  /*@ assert increasing: WeaklyIncreasing(a, i, m);
  /*@ assert increasing: WeaklyIncreasing(a+i, 0, m-i);

  copy(a + i, m - i, c + x);
  /*@ assert unchanged:  Unchanged{Pre,Here}(a, 0, m);
  /*@ assert equal:      c[x] == At{Epilogue}(a, i);
  /*@ assert equal:      a[i] == At{Epilogue}(a, i);
  /*@ assert equal:      c[x] == a[i];
  /*@ assert equal:      Equal{Epilogue,Here}(a+i, m-i, c+x);
  /*@ assert index:      m-i+x == m+n;
  /*@ assert unchanged:  Unchanged{Pre,Here}(b, 0, n);
  /*@ assert increasing: WeaklyIncreasing(c+x, 0, m-i);
  /*@ assert increasing: WeaklyIncreasing(c, x, m+n);
  /*@ assert unchanged:  Unchanged{Epilogue,Here}(a, 0, i);
  /*@ assert unchanged:  Unchanged{Epilogue,Here}(b, 0, j);
  /*@ assert unchanged:  Unchanged{Epilogue,Here}(c, 0, i+j);
  /*@ assert multiset:    MultisetUnion(a, i, b, j, c);
  /*@ assert unchanged:  Unchanged{Epilogue,Here}(a, 0, m);
  /*@ assert equal:      Equal{Epilogue,Here}(a, i, m, c, i+j);
  /*@ assert equal:      Equal{Here,Here}(a, i, m, c, i+j);
  /*@ assert multiset:    MultisetUnion(a, m, b, j, c);
}

```

Listing 10.30: Implementation of merge (3): the epilogue, first branch

```

else {
    //@ assert upper:      0 < x ==> c[x-1] <= b[j];
    //@ assert unchanged:  Unchanged{Pre,Here}(a, 0, m);
    //@ assert unchanged:  Unchanged{Pre,Here}(b, 0, n);
    //@ assert increasing:  WeaklyIncreasing(b, 0, n);
    //@ assert increasing:  WeaklyIncreasing(b+j, 0, n-j);

    copy(b + j, n - j, c + x);
    //@ assert unchanged:  Unchanged{Epilogue,Here}(b, 0, n);
    //@ assert equal:      c[x] == At{Epilogue}(b, j);
    //@ assert equal:      b[j] == At{Epilogue}(b, j);
    //@ assert equal:      c[x] == b[j];
    //@ assert equal:      Equal{Epilogue,Here}(b+j, n-j, c+x);
    //@ assert index:      n-j+x == m+n;
    //@ assert unchanged:  Unchanged{Epilogue,Here}(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c+x, 0, n-j);
    //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    //@ assert unchanged:  Unchanged{Epilogue,Here}(a, 0, i);
    //@ assert unchanged:  Unchanged{Epilogue,Here}(b, 0, j);
    //@ assert unchanged:  Unchanged{Epilogue,Here}(c, 0, i+j);
    //@ assert multiset:   MultisetUnion(a, i, b, j, c);
    //@ assert multiset:   MultisetUnion(b, j, a, i, c);
    //@ assert equal:      Equal{Epilogue,Here}(b, j, n, c, j+i);
    //@ assert equal:      Equal{Here,Here}(b, j, n, c, j+i);
    //@ assert multiset:   MultisetUnion(b, n, a, i, c);
    //@ assert multiset:   MultisetUnion(a, i, b, n, c);
}

    //@ assert unchanged:  Unchanged{Epilogue,Here}(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    //@ assert increasing:  0 < x ==> c[x-1] <= c[x];
    //@ assert increasing:  0 < x ==> WeaklyIncreasing(c, x-1, m+n);
    //@ assert increasing:  WeaklyIncreasing(c, 0, m+n);
}
}

```

Listing 10.31: Implementation of merge (4): the epilogue, second branch

10.10. The `inplace_merge` algorithm

The algorithm `inplace_merge` is the second of the two merging algorithms of the C++ standard library [15, p. 28.7.5]. Like `merge`, it combines two ordered sequences into a single ordered sequence, and like `merge` it is not a sorting algorithm in its own right: it does not order an arbitrary array, but presupposes that its input already consists of two ordered pieces.

What distinguishes the two algorithms is where the result is placed. `merge` writes into a third array that has to be disjoint from both of its inputs. `inplace_merge` has no such array at its disposal and rearranges the elements where they already lie. Its two input sequences are therefore not two arrays but one array split at an index m , and its signature reflects this.

```
void inplace_merge(value_type* a, size_type m, size_type n);
```

The array $a[0 \dots n)$ is assumed to consist of the two ordered ranges $a[0 \dots m)$ and $a[m \dots n)$. Upon termination, $a[0 \dots n)$ is ordered as a whole and holds exactly the elements it held on entry. The case $n = m + 1$, in which the second range holds a single element, is the step `insertion_sort` performs in each of its iterations (§10.7).

10.10.1. Formal specification of `inplace_merge`

Listing 10.32 shows the formal specification of `inplace_merge`.

```
/*@
  requires  valid:      \valid(a + (0..n-1));
  requires  split:      0 <= m <= n;
  requires  increasing: Increasing(a, m);
  requires  increasing: Increasing(a, m, n);

  terminates      \true;
  exits          \false;
  assigns         a[0..n-1];

  ensures      reorder:  MultisetReorder{Old,Here}(a, n);
  ensures      increasing: Increasing(a, n);
*/
void inplace_merge(value_type* a, size_type m, size_type n);
```

Listing 10.32: Formal specification of `inplace_merge`

Since `inplace_merge` delivers an ordered array holding the elements it received, its postconditions are those of the generic sorting contract of Listing 10.1, taken over unchanged. The postcondition `increasing` states that $a[0 \dots n)$ is ordered, and the postcondition `reorder` states that the algorithm has done no more than rearrange its elements. They are, word for word, the postconditions of `insertion_sort`, `heap_sort` and their neighbours in this chapter. `inplace_merge` differs from those algorithms in what it demands, not in what it promises.

What it demands is stated by three preconditions beyond `valid`. The precondition `split` places the dividing index inside the array, and the two preconditions `increasing` require the two ranges it creates to be ordered. These are not conveniences that could be dropped to obtain a more general algorithm: the implementation shown below relies on both ranges being ordered, and without that assumption its postcondition is plainly false.

It is worth noting what does *not* appear in this contract. The postcondition of merge had to be expressed with MultisetUnion [10.24], because merge relates three arrays in a single program state, which MultisetReorder [7.59] cannot express. inplace_merge relates one array in two program states, and that is precisely the situation MultisetReorder was introduced for; the ordinary sorting contract therefore suffices. Nor do the postconditions mention *m* at all. Once the merge has been carried out, the index at which the input happened to be divided is of no further interest to the caller.

10.10.2. Implementation of inplace_merge

The implementation is shown in the following listing.

Two indices traverse the array. The index *i* marks the end of the prefix $a[0 \dots i]$ that has been merged already, and the index *j* marks the beginning of the part $a[j \dots n)$ of the second range that has not yet been touched. Between them lies $a[i \dots j)$, the part of the first range that is still waiting. Initially *i* is 0 and *j* is *m*.

```

void inplace_merge(value_type* a, size_type m, size_type n)
{
    size_type i = 0u;
    size_type j = m;

    /*@
    loop invariant bound:      0 <= i <= j <= n;
    loop invariant bound:      m <= j;
    loop invariant increasing: WeaklyIncreasing(a, j);
    loop invariant increasing: WeaklyIncreasing(a, j, n);
    loop invariant partition:  Partition(a, 0, i, n);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, j, n);
    loop invariant reorder:    MultisetReorder{Pre,Here}(a, n);
    loop assigns    i, j, a[0..n-1];
    loop variant    n - i;
    */
    while (i < j && j < n) {
        if (a[i] <= a[j]) {
            i++;
        }
        else {
            /*@ ghost Before: ;
            //@ assert lower:      StrictLowerBound(a, i, j, a[j]);
            //@ assert lower:      LowerBound(a, j, n, a[j]);
            rotate(a + i, j - i, j + 1u - i);
            //@ assert equal:      a[i] == \at(a[j],Before);
            //@ assert unchanged:  Unchanged{Before,Here}(a, 0, i);
            //@ assert unchanged:  Unchanged{Before,Here}(a, j+1, n);
            //@ assert lower:      StrictLowerBound(a, i+1, j+1, a[i]);
            //@ assert increasing: WeaklyIncreasing(a, i+1, j+1);
            i++;
            j++;
        }
    }
}

```

Listing 10.33: Implementation of inplace_merge

Each iteration compares the two candidates for position *i*, namely $a[i]$ and $a[j]$. If $a[i]$ is the smaller of the two, then it already stands where it belongs and *i* advances. Otherwise $a[j]$ has to be brought to position *i*, and the elements $a[i \dots j)$ have to make room by moving one position to the right. Taken

together, that is a rotation of the slice $a[i \dots j + 1)$ by one position, and it is carried out by a single call to `rotate` [7.32].

This call has the same shape as the one in `insertion_sort` (§10.7), which inserts an element into the ordered prefix before it in exactly the same way. The three lemmas about `Rotate` [7.28] presented there apply here unchanged, and `inplace_merge` needs no lemma of its own. Each of them carries one step of the proof. `Rotate_Shift` restates the rotation, which `rotate` establishes in the coordinates of its own argument $a + i$, in the coordinates of a ; every step that follows rests on it. `RotateOne_StrictLowerBound` then places the moved element below the elements it has displaced, which is the assertion labelled `lower` after the call, and `RotateOne_MultisetReorder` states that the rotation is a reordering of the slice, which is what re-establishes the invariant `reorder`.

The remaining six assertions appeal to no lemma. The two before the call record the bounds on $a[j]$ that the branch condition and the order of the two ranges supply. Of the four after it, three name the facts about the rotation that the provers need before the lemmas can be applied — which element ended up at position i , and which parts of the array the call left alone — and the fourth states that the displaced elements $a[i + 1 \dots j + 1)$, being the former $a[i \dots j)$ moved one position to the right, are ordered still.

10.10.2.1. The loop invariants

Two invariants labelled `increasing` describe the order that has been achieved so far. The first one, `WeaklyIncreasing(a, j)`, states that $a[0 \dots j)$ is ordered. It covers the merged prefix and the remainder of the first range in a single statement: what has been placed is in order, and what is still waiting is in order behind it. The second one states that $a[j \dots n)$ is ordered as well, which it is, being a suffix of the original second range.

The invariant `partition` employs the predicate `Partition` [10.7] of §10.4 to state that no element of the merged prefix $a[0 \dots i)$ exceeds any element of the remainder $a[i \dots n)$. This is what makes the prefix final: nothing that is still to come can belong in front of it.

The remaining two invariants concern the elements themselves rather than their order. The invariant `unchanged` records that $a[j \dots n)$ has not been touched since entry, and the invariant `reorder` that the array as a whole is at all times a reordering of the array received. The loop variant is $n - i$, since i advances in either branch.

The loop stops for one of two reasons, and the postcondition follows in a slightly different way in each case. If j has reached n , the second range is exhausted and the invariant `WeaklyIncreasing(a, j)` is already the ordering of the whole array. If instead i has reached j , the first range is exhausted, and the array consists of the ordered prefix $a[0 \dots i)$ followed by the ordered remainder $a[i \dots n)$. The invariant `partition` then supplies the one comparison $a[i - 1] \leq a[i]$ that `WeaklyIncreasing_Join` needs in order to join the two. In both cases `WeaklyIncreasing_Increasing` takes the final step from `WeaklyIncreasing` to `Increasing`, as the convention of §6.2 prescribes.

As with `merge`, ties are resolved in favour of the first range, and a rotation preserves the relative order of the elements it moves, so `inplace_merge` is stable. This property does not appear among the postconditions, for the reason given after Open Issue 10.9.1.

10.10.3. The price of doing without an auxiliary array

`inplace_merge` is what `merge_sort`, the subject of the next section, would need in order to dispense with the auxiliary array that `merge` forces upon it, and it is worth being explicit about what that would cost. §10.11 returns to the question and says why `merge_sort` does not take that route; the present section supplies the figures it argues from.

The number of comparisons is not the problem. The index `i` advances in every iteration and never exceeds `n`, so the loop performs at most `n` comparisons — the same linear bound that `merge` achieves, and the bound the C++ standard requires of `std::inplace_merge` when additional memory is available.

The moves are another matter. `merge` moves each element exactly once, and pays for this with a second array as large as its result. `inplace_merge` does without that array, but every element of the second range that overtakes `k` elements of the first is moved past all `k` of them, one rotation at a time. In the worst case — the second range preceding the first one entirely — the number of moves is thus quadratic in `n`. The space is saved, but it is paid for in time.

Implementations of `std::inplace_merge` avoid this by rotating larger blocks: they split both ranges at a median, rotate the middle once, and repeat the process on the two halves, which brings the number of moves down to $\mathcal{O}(n \cdot \log(n))$ at the price of a considerably more intricate algorithm. We have kept the simpler formulation, which is the one that shares its rotation step — and hence its lemmas — with `insertion_sort`.

10.11. The merge_sort algorithm

The `merge_sort` algorithm²² is the divide-and-conquer member of this chapter. It splits the input array into two halves, sorts each half, and combines the two ordered halves with a single call to `merge` [10.25], the algorithm of the preceding section. Where `bubble_sort`, `selection_sort` and `insertion_sort` grow an ordered prefix one element at a time, `merge_sort` produces two ordered ranges of roughly equal length and joins them in one sweep. Its asymptotic complexity is $\mathcal{O}(n \cdot \log(n))$, matching that of `heap_sort`.

10.11.1. The auxiliary array

`merge` writes its result into an array that must be disjoint from both of its inputs, so the two halves cannot be combined in place. `merge_sort` therefore needs working space of the size of the input, and it receives it as a third parameter `b` rather than obtaining it itself. This is a deliberate departure from the corresponding algorithm of the C++ Standard Library, `std::stable_sort` [15, §28.7.1.2], which takes no such parameter and allocates a temporary buffer internally. The standard acknowledges the cost of doing without one: `std::stable_sort` is required to perform $\mathcal{O}(n \cdot \log(n))$ comparisons only if sufficient extra memory is available, and is permitted $\mathcal{O}(n \cdot \log^2(n))$ otherwise. None of the algorithms in this document allocates memory, and so every array they work on is supplied by the caller. Making the auxiliary array an explicit parameter keeps that convention, and it has the further merit of exposing in the signature a resource that `std::stable_sort` merely hides.

§10.10 presents a `merge` that needs no such array, which raises the question why `merge_sort` does not simply use it. The gain would be real. `inplace_merge` would remove the parameter `b` and with it the preconditions `valid` and `sep` that accompany it; the two halves of `a[0 .. n)` lie next to each other, which is exactly the situation `inplace_merge` is written for, and its postcondition is exactly what `merge_sort`

²²See https://en.wikipedia.org/wiki/Merge_sort.

has to establish. The implementation would lose its call of `copy` [7.12], and the proof would lose the step from `MultisetUnion` to `MultisetReorder` discussed below.

What it would cost is the complexity that gives the algorithm its name. A rotation moves as many elements as the range it turns, so merging the two halves in place moves in the order of $m \cdot (n - m)$ elements, which for halves of equal length is quadratic in n . The recursion would then satisfy $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n^2)$ and so be quadratic itself, where the implementation shown below moves $\mathcal{O}(n \cdot \log(n))$ elements.

The number of *comparisons* would not change. `inplace_merge` performs at most one comparison for each position it fills, so an in-place `merge_sort` would still compare $\mathcal{O}(n \cdot \log(n))$ times. It would thus meet the stronger of the two bounds quoted above — the one the C++ standard grants only to an implementation that has extra memory — while using none, and be quadratic all the same. The auxiliary array does not buy comparisons. It buys moves, and the standard’s clause, which counts only comparisons, does not record that.

Our `merge_sort` inherits the stability of `merge`: on a tie the element of the left half is taken first, and the relative order of equal elements is preserved. As for `merge` itself, this property does not appear among the postconditions, for the reason set out after Open Issue 10.9.1. The name `merge_sort` rather than `stable_sort` is chosen accordingly: it names the strategy the implementation follows, not a property the contract leaves unstated.

10.11.2. Formal specification of `merge_sort`

The following listing presents the formal specification of `merge_sort`. Apart from the auxiliary array, it is the generic sorting contract of Listing 10.1: the array $a[0 \dots n)$ is ordered upon termination, and its elements are a reordering of those it held on entry.

```

/*@
requires   valid:      \valid(a + (0..n-1));
requires   valid:      \valid(b + (0..n-1));
requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

terminates           \true;
decreases            n;
exits                \false;
assigns              a[0..n-1], b[0..n-1];

ensures   reorder:    MultisetReorder{Old,Here}(a, n);
ensures   increasing: Increasing(a, n);
*/
void merge_sort(value_type* a, size_type n, value_type* b);

```

Listing 10.34: Formal specification of `merge_sort`

The auxiliary array contributes three clauses. The precondition `valid` requires $b[0 \dots n)$ to be writable, and the precondition `sep` requires it to be disjoint from $a[0 \dots n)$, which is what allows the two halves of a to be merged into b . The `assigns` clause names b alongside a , since `merge_sort` leaves the working space in an unspecified state. Nothing is promised about the contents of b on return; a caller may reuse or discard it.

The contract also carries a `decreases` clause, which no other contract in this document does. `merge_sort` is our first *recursive* function, and for a recursive function the `terminates` clause cannot be established from the loop variants of the body, there being no loop. The `decreases` clause supplies the measure instead: every recursive call is made with a strictly smaller, non-negative value of n , so the recursion

cannot continue indefinitely. §3.7 discusses why recursive procedures need such a measure and cannot instead be reduced to the basic Hoare rules by inlining.

10.11.3. From MultisetUnion to MultisetReorder

The postcondition of merge is expressed with MultisetUnion [10.24], which relates three arrays in one program state, whereas the postcondition of a sorting algorithm is expressed with MultisetReorder [7.59], which relates one array in two program states. The step from the former to the latter is precisely what the body of merge_sort performs when it merges the two halves into b and copies b back over a. It is stated once, as a lemma.

```

/*@
  lemma MultisetUnion_Reorder{K,L}:
    \forall value_type *a, *b, integer m, n;
      0 <= m <= n ==>
      MultisetUnion{K}(a, m, a+m, n-m, b) ==>
      Equal{K,L}(b, n, a) ==>
      MultisetReorder{K,L}(a, 0, n);
*/

```

Listing 10.35: MultisetUnionToReorder (ACSL)

Read at the two states in question, the lemma says the following. At the earlier state, b holds the elements of the two halves $a[0 \dots m)$ and $a[m \dots n)$ taken together. Between the two states, b is copied onto a, which is what the hypothesis Equal records. Hence a ends up holding exactly the elements it held before, and that is MultisetReorder.

The proof is a short chain of counting steps, each of which is available as an existing lemma: Count_Between carries the count from b to a across the copy, Count_Shift moves the second half from shifted to absolute index coordinates, and Count_Union adds the two halves together. Automatic provers do not find this chain, and the lemma is discharged interactively.

10.11.4. Implementation of merge_sort

The implementation is shown in the next listing.

An array of fewer than two elements is already ordered, and this is the base case of the recursion. Otherwise the midpoint m splits $a[0 \dots n)$ into the halves $a[0 \dots m)$ and $a[m \dots n)$, each of which is sorted by a recursive call. The first call is given the front of the auxiliary array as its working space and the second call the back, so that the two calls do not interfere. The two ordered halves are then merged into $b[0 \dots n)$, and copy [7.12] returns the result to a.

The assertions record the two properties of the contract as they are assembled. The ordering is the simpler of the two. The second recursive call establishes Increasing for $a[m \dots n)$ in the shifted coordinates of its own array parameter, and Increasing_Unshift restates it in the coordinates of a; the assigns clause of that call guarantees that it did not disturb the half sorted by the first call. That is exactly what merge requires of its two inputs, and merge's own postcondition then gives the ordering of $b[0 \dots n)$, which the copy transfers to a.

The multiset property is assembled in the same manner, one program state at a time. Each recursive call reorders its own half, MultisetReorder_Shift restates the second of these in absolute coordinates, and MultisetReorder_DisjointUnion joins the two halves into a statement about the whole array. merge leaves a untouched, so Unchanged_MultisetReorder carries the property across that call unchanged, and

MultisetUnion_Reorder supplies the final step across the copy. The individual statements are chained by MultisetReorder_Transitive.

```

void merge_sort(value_type* a, size_type n, value_type* b)
{
    if (n < 2u) {
        return;
    }

    const size_type m = n / 2u;
    //@ assert bound:      0 < m < n;

    merge_sort(a, m, b);
    //@ ghost Half: ;
    //@ assert increasing: Increasing(a, 0, m);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, 0, m);
    //@ assert unchanged:  Unchanged{Pre,Here}(a, m, n);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, m, n);

    merge_sort(a + m, n - m, b + m);
    //@ ghost Sorted: ;
    //@ assert increasing: Increasing(a + m, 0, n-m);
    //@ assert increasing: Increasing(a, m, n);
    //@ assert unchanged:  Unchanged{Half,Here}(a, 0, m);
    //@ assert increasing: Increasing(a, 0, m);
    //@ assert reorder:    MultisetReorder{Half,Here}(a + m, 0, n-m);
    //@ assert reorder:    MultisetReorder{Half,Here}(a, m, n);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, m, n);
    //@ assert reorder:    MultisetReorder{Half,Here}(a, 0, m);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, 0, m);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, 0, n);

    merge(a, m, a + m, n - m, b);
    //@ ghost Merged: ;
    //@ assert increasing: Increasing(b, 0, n);
    //@ assert multiset:    MultisetUnion(a, m, a + m, n-m, b);
    //@ assert unchanged:  Unchanged{Sorted,Here}(a, 0, m);
    //@ assert unchanged:  Unchanged{Sorted,Here}(a, m, n);
    //@ assert unchanged:  Unchanged{Sorted,Here}(a, 0, n);
    //@ assert reorder:    MultisetReorder{Sorted,Here}(a, 0, n);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, 0, n);

    copy(b, n, a);
    //@ assert equal:      Equal{Merged,Here}(b, n, a);
    //@ assert increasing: Increasing(a, 0, n);
    //@ assert reorder:    MultisetReorder{Merged,Here}(a, 0, n);
    //@ assert reorder:    MultisetReorder{Pre,Merged}(a, 0, n);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, 0, n);
}

```

Listing 10.36: Implementation of merge_sort

Part V.

Verification of data structures

Introduction

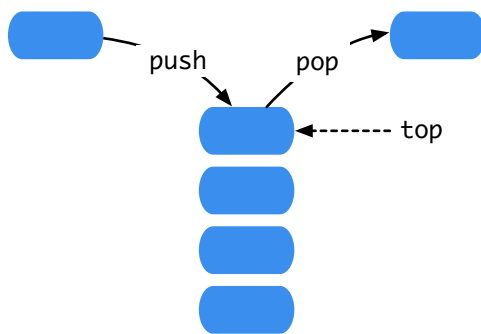
This part initiates the study of data structure verification. As a first representative example, we focus on a stack abstraction, which allows us to introduce the relevant specification patterns and proof techniques in a controlled setting. Further data structures will be incorporated in subsequent extensions of this document.

11. The Stack data type

So far, we have used the ACSL specification language primarily to specify and verify individual C functions in isolation. In practical software development, however, one is often concerned with the implementation of a whole family of functions centered around a non-trivial data structure. Such functions are not independent. Rather, they are required to interact in a disciplined way and to satisfy consistency properties that relate their behaviors.

In this setting, it is no longer sufficient to reason about each function in isolation. Instead, we must also formulate and verify properties that describe how several functions operate together on shared data and how their effects are meant to compose.

As a first example of such a data structure, we consider the *stack* abstraction. A stack is a deliberately simple abstraction, yet it already exhibits non-trivial interaction between its operations. Its defining characteristic is a strict *last-in, first-out* discipline, which constrains the order in which stored elements may be accessed and removed. Despite its simplicity, the stack therefore provides a suitable setting for studying consistency properties that relate multiple operations.



A *stack* is a data type for storing objects with the property that, if an object *a* is *pushed* onto the stack *before* an object *b*, then *a* can only be removed (*popped*) after *b*. In other words, a stack is a *first-in, last-out* data type. The *top* operation returns the element that was most recently pushed onto the stack and has not yet been removed.

Figure 11.1.: Basic operations of a stack

The development presented here builds on the results of a previous diploma thesis by one of the authors [23], which established the overall approach to the formal specification and verification of the Stack data type. The methodology followed in this chapter is the one developed in [24], where it is presented in a general form and illustrated by the same case study.

The C++ Standard Library provides the generic container adaptor `std::stack` [15, §26.6.6], which models a stack whose maximal size is not fixed a priori and may grow to an arbitrary number of elements, subject only to the availability of memory. Its interface and observable behavior serve as a reference that we follow as closely as the constraints of our C implementation allow.

In contrast, we consider only stacks with a finite *capacity*. Such a stack can store at most a fixed number *c* of elements, which is determined at the moment the stack is initialized. This bound remains constant throughout the lifetime of the stack and allows us to define the data type without relying on dynamic memory allocation.

When a stack is created or initialized, it contains no elements, that is, its *size* is 0. The operations *push* and *pop* increase and decrease the size of the stack by at most one, respectively.

11.1. Methodology overview

In this section we present a methodology for this task using the data type of stacks as a running example. The stack abstraction serves as a representative case study through which the general principles of our approach can be made precise.

Three abstraction levels

Our methodology distinguishes three systematically related levels, illustrated in Figure 11.2.

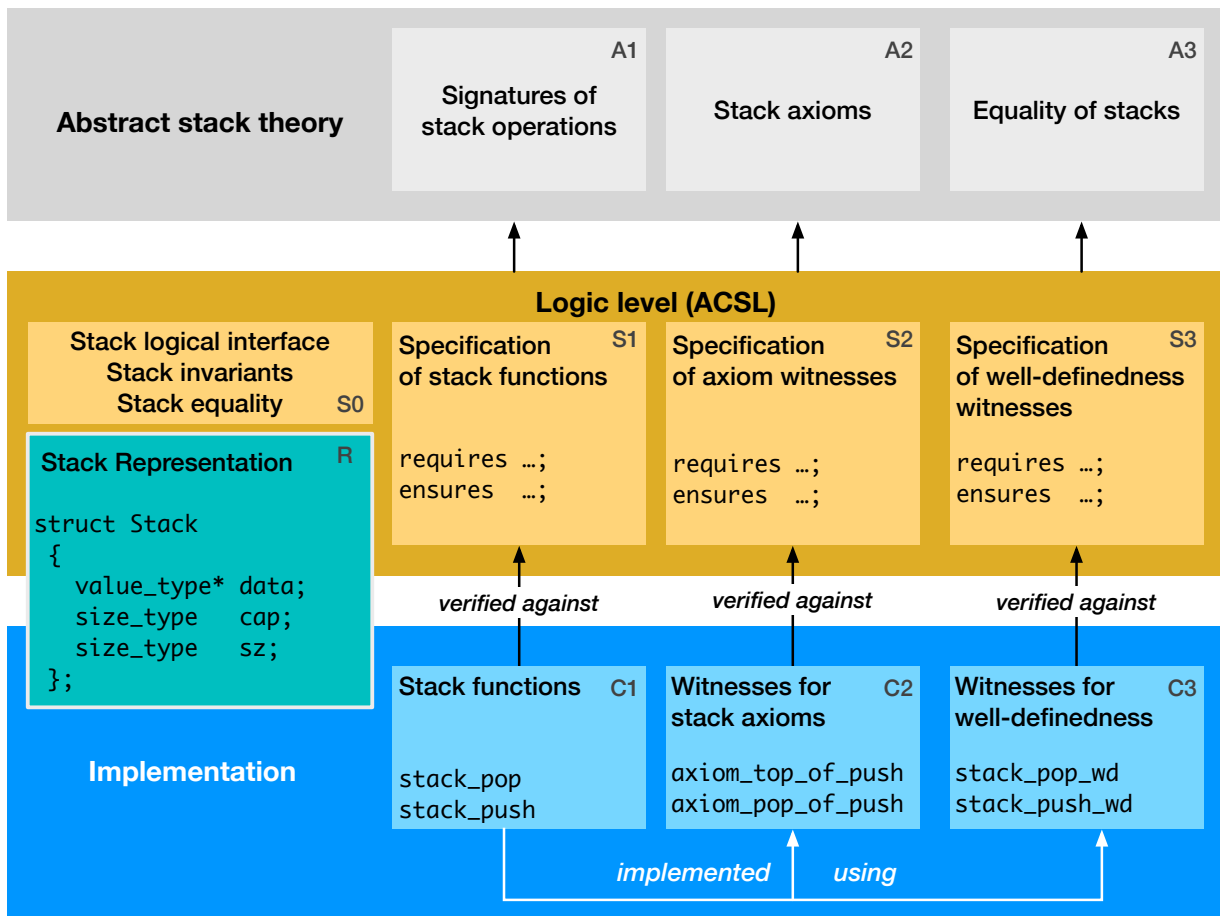


Figure 11.2.: The methodology illustrated on the Stack data type

1. At the *abstract level*, the stack is a purely mathematical model, independent of any concrete representation. It is given by the signatures of its operations (A1) and by the axioms (A2) that govern how these operations interact. The axioms in turn induce a canonical notion of equality (A3).
2. At the *logic specification level*, we follow a white-box approach in which reusable ACSL definitions capture fundamental properties of the C representation type Stack (R). Weak type invariants characterize the valid instances of that type and thus, together with a notion of equality, delimit the domain on which algebraic reasoning takes place (S0). These definitions form the basis for the contracts of the stack functions (S1) and for the specification of *witnesses* that encode the stack axioms (S2) and the well-definedness of the operations (S3).

3. At the *implementation level*, C functions implement the abstract operations (C1). Alongside them, witness functions structurally link several execution paths in order to satisfy the contracts for the stack axioms (C2) and for well-definedness (C3).

The following sections elaborate these three levels in turn, starting with the abstract axiomatization (§11.2), followed by a discussion of the concrete C representation of stacks (§11.3), then the logic specifications, comprising invariants and observational equality (§11.4 and §11.5), and culminating in the verified implementation (§11.6) together with its witnesses (§11.7 and §11.8).

Witnessing

Verifying the contract of an individual stack function ensures that this function behaves according to its specification. The axioms of the abstract stack theory, however, relate *several* operations and thus describe their interaction rather than the behavior of a single function. Contracts alone cannot express such properties, as they speak about one execution path at a time.

To bridge this gap we use *witnessing*. A witness consists of an auxiliary C function together with an auxiliary contract. The function orchestrates several execution paths by composing other functions within a single function body, while its contract states the desired relationship between these paths. Witnessing thereby enables relational reasoning in an otherwise function-centric verification framework.

A witness is verified only against the previously established contracts of the stack functions and the invariants of the data type, never against their implementations. The contracts therefore act as an abstraction barrier between the abstract specification and the concrete implementation. We employ witnesses for two purposes: to establish the stack axioms (§11.7) and to establish well-definedness (§11.8).

Observational equality and well-definedness

The stack axioms implicitly assume a notion of equality between stacks. At the level of C implementations this notion must be made explicit. For our finite-capacity stacks, equality is defined *observationally*. Two stacks are considered equal if they exhibit the same observable behavior, even if their internal capacities differ (§11.5).

Once equality is defined in this way, it is no longer immediate that all stack operations respect this equivalence. In particular, an operation whose behavior depends on the internal capacity of a stack might map equal stacks to stacks that are no longer equal. Whether the operations are *well-defined* in this sense is settled by the well-definedness witnesses, whose contracts express exactly the required preservation of observational equality (§11.8).

11.2. Axioms and invariants of finite-capacity stacks

To specify the interaction of the stack operations, we introduce a small collection of axioms. These axioms characterize stacks implicitly through the properties of their operations, rather than through a concrete representation.

Let V be a value type in the usual programming sense, that is, a type whose objects can be copied and compared for equality. We denote by S the type of stacks whose elements are of type V . The stack operations considered in this chapter have the following signatures:

$$\text{Constructor:} \quad \text{init} : \mathbb{N} \rightarrow S \quad (11.1)$$

$$\text{Observers:} \quad \text{capacity} : S \rightarrow \mathbb{N} \quad \text{size} : S \rightarrow \mathbb{N} \quad \text{top} : S \rightarrow V \quad (11.2)$$

$$\text{Modifiers:} \quad \text{push} : S \times V \rightarrow S \quad \text{pop} : S \rightarrow S \quad (11.3)$$

Every stack $s \in S$ satisfies the following invariant, capturing the idea of a stack with finite capacity:

$$0 \leq \text{size}(s) \leq \text{capacity}(s). \quad (11.4)$$

Closely related to these invariants are the two auxiliary predicates $\text{empty} : S \rightarrow \mathbb{B}$ and $\text{full} : S \rightarrow \mathbb{B}$. They are defined in terms of the primitive observer functions size and capacity and therefore impose no additional constraints on stacks.

$$\text{empty}(s) \iff \text{size}(s) = 0, \quad (11.5)$$

$$\text{full}(s) \iff \text{size}(s) = \text{capacity}(s). \quad (11.6)$$

To qualify as a stack, these operations are required to satisfy a number of basic consistency properties. We collectively refer to these properties as the *stack axioms*.

Initialization of a stack:

$$\text{size}(\text{init}(c)) = 0, \quad (11.7)$$

$$\text{capacity}(\text{init}(c)) = c. \quad (11.8)$$

Adding an element to a stack:

$$\text{size}(\text{push}(s, v)) = \text{size}(s) + 1, \quad \text{if } \neg \text{full}(s), \quad (11.9)$$

$$\text{capacity}(\text{push}(s, v)) = \text{capacity}(s), \quad \text{if } \neg \text{full}(s), \quad (11.10)$$

$$\text{top}(\text{push}(s, v)) = v, \quad \text{if } \neg \text{full}(s). \quad (11.11)$$

Removing an element from a stack:

$$\text{size}(\text{pop}(s)) = \text{size}(s) - 1, \quad \text{if } \neg \text{empty}(s), \quad (11.12)$$

$$\text{capacity}(\text{pop}(s)) = \text{capacity}(s), \quad \text{if } \neg \text{empty}(s), \quad (11.13)$$

$$\text{pop}(\text{push}(s, v)) = s, \quad \text{if } \neg \text{full}(s), \quad (11.14)$$

$$\text{push}(\text{pop}(s), \text{top}(s)) = s, \quad \text{if } \neg \text{empty}(s). \quad (11.15)$$

A note on exception handling We deliberately refrain from specifying the behavior of $\text{push}(s, v)$ when s is full, as well as the behavior of $\text{pop}(s)$ and $\text{top}(s)$ when s is empty. Specifying stack operations

in such *exceptional* situations is a substantial problem in its own right, and a wide range of approaches can be found in the literature.

Rather than pursuing this topic further, we restrict ourselves to a single illustrative example that demonstrates how seemingly innocuous exception specifications may lead to unintended and even inconsistent consequences.

Suppose that the element type V is extended by an additional error value `err`, and that the `top` operation is required to satisfy

$$\text{top}(s) = \text{err}$$

whenever s is empty. This immediately raises the question of how the operation `push( $s$ , err)` should behave. At first glance, it might appear reasonable to simply ignore such a value and to postulate

$$\text{push}(s, \text{err}) = s. \tag{11.16}$$

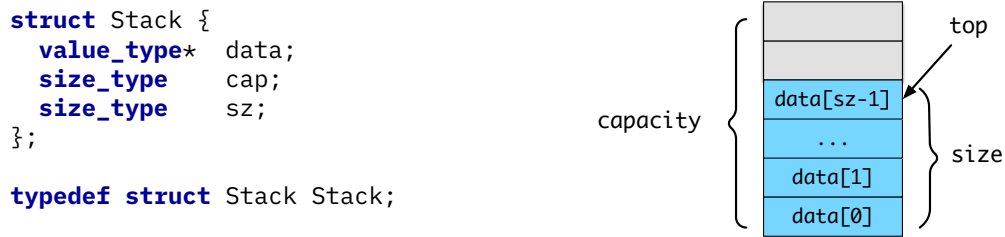
However, this assumption leads to a contradiction. Indeed, for any stack s that is neither full nor empty, we obtain

$$\begin{aligned} \text{size}(s) &= \text{size}(\text{pop}(\text{push}(s, \text{err}))) && \text{by (11.14),} \\ &= \text{size}(\text{pop}(s)) && \text{by (11.16),} \\ &= \text{size}(s) - 1 && \text{by (11.12).} \end{aligned}$$

This equality is impossible. Consequently, no such stack s can exist, and the axioms collapse into an inconsistent theory. This example illustrates that exception handling cannot be added naïvely at the axiomatic level without a careful reconsideration of the overall specification.

11.3. The structure Stack and its associated functions

We now introduce a concrete C implementation that realizes the stack axioms presented in §11.2. Listing 11.3 defines the concrete representation of type Stack. The schematic view on the right merely illustrates the source code.



Listing 11.3: Definition of type Stack and schematic interpretation

Starting at address data, Stack contains an array of fixed length cap, called the *capacity*, which represents the maximum number of elements that an instance of Stack can hold. The field sz records the *size* of a Stack object, that is, the number of elements currently stored in the stack.

Based on the abstract operations introduced in §11.2, we now declare the concrete interface of the Stack data type. Table 11.4 relates these function declarations to the corresponding methods of the C++ interface `std::stack`, where such a correspondence exists.

C function	<code>std::stack</code> counterpart	Remark
<code>stack_init(s, a, n)</code>	<code>std::stack<T,Cont> st(c);</code>	Construction from container.
<code>stack_equal(s, t)</code>	operator ==	Same purpose.
<code>stack_size(s)</code>	<code>size()</code>	Same purpose.
<code>stack_capacity(s)</code>	—	No fixed capacity concept.
<code>stack_empty(s)</code>	<code>empty()</code>	Same purpose.
<code>stack_full(s)</code>	—	No fixed capacity concept.
<code>stack_top(s)</code>	<code>top()</code>	Same purpose (undefined if empty).
<code>stack_push(s, v)</code>	<code>push(v)</code>	Defined only for non-full stacks.
<code>stack_pop(s)</code>	<code>pop()</code>	Same purpose (undefined if empty).

Table 11.4.: C Stack interface versus `std::stack`.

Most of these functions closely correspond to the operations provided by the C++ `std::stack` template class [15, §26.6.6.1]. In particular, `stack_equal` corresponds to the equality operator `==`. The function `stack_init` is used to bring a stack into a well-defined initial state. By contrast, the function `stack_full` has no direct counterpart in `std::stack`. Similarly, `stack_push` is only defined for Stack objects that are not full. This difference reflects a deliberate design choice: our implementation operates on a fixed-capacity data structure and therefore avoids dynamic memory allocation, whereas `std::stack` relies on it.

11.4. Logical interface and invariants of Stack

In this section, we introduce a collection of logic functions and predicates that will be used throughout this chapter. Together, these definitions constitute a *logical interface* to stacks and establish an abstraction barrier between the concrete representation and its formal specification. They allow us to reason about stacks at an abstract level, without reference to implementation details. In addition, we introduce the predicate `StackInvariant` [11.7], which characterizes exactly those instances of `Stack` that represent valid stacks.

11.4.1. Logical interface: functions and predicates

We begin by introducing in Listing 11.5 three auxiliary logic functions: `StackCapacity`, `StackSize`, and `StackData`. These functions provide logical access to the fields `cap`, `sz`, and `data` of a `Stack` instance, respectively. They can be used conveniently in specifications without exposing the underlying concrete representation.

```
/*@  
  logic value_type* StackData{S}(Stack* s)    = s->data;  
  
  logic integer     StackCapacity{S}(Stack* s) = s->cap;  
  
  logic integer     StackSize{S}(Stack* s)    = s->sz;  
*/
```

Listing 11.5: Stack (ACSL)

Listing 11.6 defines the logic function `StackTop` and the predicates `StackEmpty` and `StackFull`, which characterize empty and full stacks (see Equations (11.5) and (11.6)).

```
/*@  
  logic value_type StackTop{S}(Stack* s) = StackData(s)[StackSize(s)-1];  
  
  predicate StackEmpty{S}(Stack* s)      = StackSize(s) == 0;  
  
  predicate StackFull{S}(Stack* s)       = StackSize(s) == StackCapacity(s);  
*/
```

Listing 11.6: StackObservers (ACSL)

Finally, we note that the numeric results of all logic functions introduced here are using the ACSL type `integer`. This choice is intentional. Using bounded logic types in combination with arithmetic operations in axioms can easily lead to inconsistencies, whereas unbounded integers provide a sound foundation for reasoning about sizes, indices, and cardinalities.

11.4.2. Invariants of Stack

Not every instance of type `Stack` represents a valid stack. For example, with the definition of `Stack` given in Listing 11.3, the initializer `Stack s = { {0,0,0,0}, 4, 5 }` is invalid, since it violates the size–capacity inequalities stated in (11.4): a stack of capacity 4 cannot contain 5 elements.

In ACSL, structural consistency conditions are naturally expressed as *type invariants*. More precisely, ACSL distinguishes between *strong* and *weak* type invariants [25, § 2.12]. A strong invariant must hold at every program point during execution. A weak invariant, by contrast, is required to hold only at function

boundaries — upon entry and upon exit — and may be temporarily violated within a function body. At the time of writing, however, support for type invariants is not yet implemented in Frama-C. For this reason, and in order to distinguish formally between well-formed and ill-formed instances of `Stack`, we introduce in the following listing the predicate `StackInvariant`.

```

/*@
  predicate StackInvariant{L}(Stack* s) =
    0 < StackCapacity(s)                &&
    0 <= StackSize(s) <= StackCapacity(s) &&
    \valid(StackData(s) + (0..StackCapacity(s)-1)) &&
    \separated(s, StackData(s) + (0..StackCapacity(s)-1));

  predicate StackValid{L}(Stack* s) = \valid(s) && StackInvariant(s);
*/

```

Listing 11.7: `StackInvariant` (ACSL)

It is worth emphasizing that `StackInvariant` does not merely restate the mathematical properties of the abstract data type *stack*. In particular, additional conditions on memory validity and separation are required to guarantee that stack operations are well-defined at the level of concrete program execution. More precisely, the invariant ensures that:

- The capacity is strictly positive.
- The size is nonnegative and does not exceed the capacity, as expressed by (11.4).
- The underlying data array is valid over the entire memory region determined by the capacity.
- The stack object itself is disjoint from the memory region used to store its elements.

Listing 11.7 also defines the predicate `StackValid`, which combines two complementary aspects of correctness. First, the condition `\valid(s)` ensures that the pointer `s` itself denotes a valid memory location, so that accessing the structure fields is well-defined in the sense of the C memory model. Second, the predicate `StackInvariant` encapsulates the type invariants of the stack object.

11.5. Equality of stacks

Defining equality for instances of non-trivial data types is a subtle problem, particularly in the presence of mutable state and object identity. This issue is well known in object-oriented programming: for example, the book *Programming in Scala* [26] devotes an entire chapter to the definition and correct use of equality. In a formal verification setting the problem is even more delicate, since an imprecise notion of equality can invalidate otherwise correct reasoning.

Conceptually, one distinguishes between *structural* equality and *observational* equality. Structural equality compares the complete internal representation of objects. Observational equality, by contrast, identifies objects that exhibit the same externally visible behavior, even if their internal representations differ.

For stacks we adopt an observational notion of equality. Two stacks are considered equal if they have the same size and represent the same sequence of elements, irrespective of their capacity. Equality is therefore defined in terms of observable contents, not in terms of member-wise comparison of all fields. Thus the two stacks shown in Figure 11.8 are considered equal even though their internal arrays differ outside the range of stored elements. In particular, equal stacks may have different capacities, and one may be full while the other is not. This mirrors the behavior of the C++ standard container `std::vector`, whose equality is likewise determined by its stored elements rather than its reserved capacity.

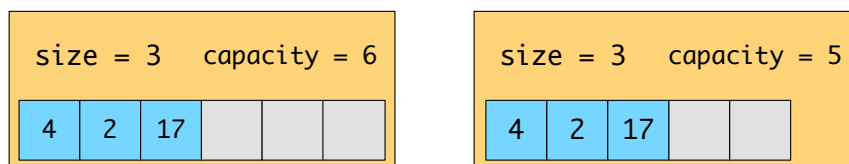


Figure 11.8.: Example of two equal stacks

The predicate `StackEqual` in Listing 11.9 formalizes equality of stacks in ACSL. Its definition is factored through the auxiliary predicate `StackPrefix`. This predicate uses `Equal` [4.31] to express that the contents of a stack `s` coincide with a prefix of the contents of `t`.

Based on this notion, `StackEqual` requires that both stacks have the same size and that `s` is a prefix of `t`. Equal sizes exclude proper prefixes and therefore ensure that both stacks store exactly the same sequence of elements. The capacity of a stack is intentionally disregarded, since equality is defined solely in terms of observable contents.

```

/*@
predicate StackPrefix{S,T}(Stack* s, Stack* t) =
    StackSize{S}(s) <= StackSize{T}(t) &&
    Equal{S,T}(StackData{S}(s), StackData{T}(t));

predicate StackEqual{S,T}(Stack* s, Stack* t) =
    StackSize{S}(s) == StackSize{T}(t) &&
    StackPrefix{S,T}(s, t);

lemma StackEqual_Reflexive{S}:
    \forallall Stack* s; StackEqual{S,S}(s, s);

lemma StackEqual_Symmetric{S,T}:
    \forallall Stack *s, *t;
        StackEqual{S,T}(s, t) ==> StackEqual{T,S}(t, s);

lemma StackEqual_Transitive{S,T,U}:
    \forallall Stack *s, *t, *u;
        StackEqual{S,T}(s, t) ==> StackEqual{T,U}(t, u) ==> StackEqual{S,U}(s, u);
*/

```

Listing 11.9: StackEqual (ACSL)

StackEqual as an equivalence relation Calling a relation “equality” is not merely a matter of terminology. At the very least, it must satisfy the defining properties of an equivalence relation. Accordingly, the listing establishes the following properties:

$$\text{Reflexivity:} \quad \forall s \in S : \quad s = s, \quad (11.17a)$$

$$\text{Symmetry:} \quad \forall s, t \in S : \quad s = t \implies t = s, \quad (11.17b)$$

$$\text{Transitivity:} \quad \forall s, t, u \in S : \quad s = t \implies t = u \implies s = u. \quad (11.17c)$$

As already mention in Equations (5.1), we adopt the usual convention that the implication symbol $\implies$ associates to the right.

In Listing 11.9, these properties are formulated explicitly as lemmas about `StackEqual`. They are therefore not postulated but must be proved within the verification framework. In fact, Frama-C/WP is able to discharge the corresponding verification conditions automatically with the help of SMT provers. This provides formal confirmation that `StackEqual` indeed behaves as a well-defined notion of equality.

Well-definedness with respect to stack equality Any relation satisfying the properties (11.17) is called an *equivalence relation*. The set of all instances that are equal to a given instance s is called the equivalence class of s .

The two stacks shown in Figure 11.8 illustrate that an equivalence class of `StackEqual` may contain more than one element. This is a familiar situation in mathematics: for example, the rational number $\frac{1}{2}$ represents the same equivalence class as $\frac{2}{4}$ or $\frac{7}{14}$. Accordingly, the stacks s and t in the figure can be regarded as two *representatives* of the same equivalence class.

In such a setting, an important question arises: can functions defined on stacks be specified in a way that is independent of the chosen representative? In other words, are these functions *well-defined* with respect to the equivalence relation `StackEqual`? We will answer this question in Section 11.8.

11.6. Specification and implementation of stack functions

In previous chapters, the verification of individual functions was usually carried out directly at the beginning of the chapter. For abstract data types, however, this approach is no longer sufficient. Before individual operations can be verified in isolation, a number of conceptual issues must be addressed, including data type invariants, user-defined equality, and the notion of well-definedness with respect to equivalence classes.

Having settled these foundations, we now turn to the verification of the individual stack operations. In §11.7, the verification results are lifted to a more abstract level. We show there that the combined behavior of the verified operations satisfies the stack axioms introduced in §11.2.

11.6.1. Verification of `stack_equal`

The function `stack_equal` decides at execution time whether two stacks are equal in the sense of `StackEqual` [11.9]. The specification of `stack_equal` in Listing 11.10 requires that both arguments satisfy the type invariant `StackInvariant` [11.7].

```
/*@
  requires   valid:      StackValid(s) && StackValid(t);
  terminates \true;
  exits     \false;
  assigns    \nothing;
  ensures    equal:      \result == true  <==> StackEqual{Here,Here}(s, t);
  ensures    not_equal:  \result == false <==> !StackEqual{Here,Here}(s, t);
*/
bool stack_equal(const Stack* s, const Stack* t);
```

Listing 11.10: Formal specification of `stack_equal`

The two postconditions `equal` and `not_equal` are not merely stylistic. In our setting `bool` is a type alias for `int`. The return value of `stack_equal` is therefore not restricted by the type system to the values `true` (1) and `false` (0). The first equivalence ensures that the value `true` corresponds exactly to the logical predicate `StackEqual`. However, it does not exclude other integer values in the non-equal case. The second equivalence closes this gap by enforcing that logical false is represented precisely by `false`. Together, the two clauses establish a bijective correspondence between the two-valued predicate `StackEqual` and its concrete integer representation.

The implementation of `stack_equal`, shown in Listing 11.11, first compares the sizes of the two stacks. If the sizes coincide, it invokes the function `equal` [4.32] to compare the first `sz` elements of the underlying data arrays. Thus, equality is determined solely by the number and sequence of stored elements. The stack capacity does not enter the comparison and therefore has no influence on the result.

```
bool stack_equal(const Stack* s, const Stack* t)
{
    return (s->sz == t->sz) && equal(s->data, s->sz, t->data);
}
```

Listing 11.11: Implementation of `stack_equal`

11.6.2. Verification of `stack_init`

The contract of `stack_init` is shown in Listing 11.12. Its preconditions require a valid stack point, a strictly positive capacity, and a contiguous storage area of sufficient size that is separated from the stack descriptor itself.

The assigns clauses make the data dependencies explicit. The data pointer and the capacity field of the `Stack` object are set from the respective function arguments, while the size field is set to 0, independent of any input state. The postconditions state that the stack is properly initialized and the stack invariants are satisfied.

```
/*@
  requires  valid:      \valid(s);
  requires  capacity:   0 < cap;
  requires  storage:    \valid(data + (0..cap-1));
  requires  sep:        \separated(s, data + (0..cap-1));

  terminates          \true;
  exits               \false;
  assigns             s->data \from data;
  assigns             s->cap  \from cap;
  assigns             s->sz   \from \nothing;

  ensures  valid:      StackValid(s);
  ensures  storage:    StackData(s) == data;
  ensures  capacity:   StackCapacity(s) == cap;
  ensures  empty:      StackSize(s) == 0;
*/
void stack_init(Stack* s, value_type* data, size_type cap);
```

Listing 11.12: Formal specification of `stack_init`

The implementation of `stack_init` is shown in the next listing. It performs exactly the initialization prescribed by the contract.

```
void stack_init(Stack* s, value_type* data, size_type cap)
{
    s->data = data;
    s->cap  = cap;
    s->sz   = 0u;
}
```

Listing 11.13: Implementation of `stack_init`

11.6.3. Verification of `stack_size`

The function `stack_size` returns the current number of elements stored in a stack. Its specification, shown in Listing 11.14, requires that the argument is a valid stack. The contract further states that the function has no side effects (`assigns \nothing`) and always terminates without exceptional control flow.

The central postcondition expresses the functional behaviour of the operation. It states that the returned value is exactly the logical size of the stack as given by the logic function `StackSize`.

```

/*@
  requires    valid: StackValid(s);
  terminates   \true;
  exits         \false;
  assigns      \nothing;
  ensures      size: \result == StackSize(s);
*/
size_type stack_size(const Stack* s);

```

Listing 11.14: Formal specification of `stack_size`

The implementation of `stack_size`, shown in the next listing, directly returns the value stored in the structure field `sz`. Since `StackSize` is defined to correspond to this field for valid stacks, the implementation can easily be verified.

```

size_type stack_size(const Stack* s)
{
  return s->sz;
}

```

Listing 11.15: Implementation of `stack_size`

11.6.4. Verification of `stack_capacity`

The function `stack_capacity` returns the maximum number of elements that can be stored in a stack. Its specification, shown in Listing 11.16, requires that the argument is a valid stack. The contract further states that the function has no side effects (`assigns \nothing`) and always terminates without exceptional control flow.

The functional behaviour is expressed by the postcondition stating that the return value is exactly the logical capacity of the stack as given by the logic function `StackCapacity`.

```

/*@
  requires    valid:    StackValid(s);
  terminates   \true;
  exits         \false;
  assigns      \nothing;
  ensures      capacity: \result == StackCapacity(s);
*/
size_type stack_capacity(const Stack* s);

```

Listing 11.16: Formal specification of `stack_capacity`

The implementation of `stack_capacity`, shown in the following listing, simply returns the value stored in the structure field `cap`.

```

size_type stack_capacity(const Stack* s)
{
  return s->cap;
}

```

Listing 11.17: Implementation of `stack_capacity`

11.6.5. Verification of `stack_empty`

The function `stack_empty` provides a runtime check for the logical predicate `StackEmpty` [11.6]. Its specification is shown in Listing 11.18. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is specified by two equivalent characterizations. The first postcondition states that the function returns **true** exactly when the logical predicate `StackEmpty` holds for the given stack. The second postcondition states the complementary condition: the function returns **false** exactly when the stack is not empty. Together, these clauses establish a precise correspondence between the runtime result and the logical notion of stack emptiness.

```
/*@
  requires  valid:      StackValid(s);
  terminates
  exits    \false;
  assigns   \nothing;
  ensures   empty:      \result == true  <==> StackEmpty(s);
  ensures   not_empty: \result == false <==> !StackEmpty(s);
*/
bool stack_empty(const Stack* s);
```

Listing 11.18: Formal specification of `stack_empty`

The implementation of `stack_empty`, shown in the following listing, checks whether the size of the stack is zero.

```
bool stack_empty(const Stack* s)
{
  return stack_size(s) == 0u;
}
```

Listing 11.19: Implementation of `stack_empty`

11.6.6. Verification of `stack_full`

The function `stack_full` provides a runtime check for the logical predicate `StackFull` [11.6]. Its specification is shown in Listing 11.20. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is expressed by postconditions that relate the return value of the function to the logical predicate `StackFull`. In particular, the result is **true** exactly when the stack is full, and **false** otherwise.

```
/*@
  requires  valid:      StackValid(s);
  terminates
  exits    \false;
  assigns   \nothing;
  ensures   full:       \result == true  <==> StackFull(s);
  ensures   not_full:  \result == false <==> !StackFull(s);
*/
bool stack_full(const Stack* s);
```

Listing 11.20: Formal specification of `stack_full`

The implementation of `stack_full`, shown in the following listing, checks whether the current size of the stack is equal to its capacity.

```

bool stack_full(const Stack* s)
{
    return stack_size(s) == s->cap;
}

```

Listing 11.21: Implementation of `stack_full`

11.6.7. Verification of `stack_top`

The function `stack_top` returns the element currently stored at the top of a stack. Its specification is given in Listing 11.22. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is specified conditionally. If the stack is not empty, the returned value must coincide with the logical value given by `StackTop`. No requirement is imposed on the result when the stack is empty.

```

/*@
    requires    valid:  StackValid(s);
    terminates      \true;
    exits          \false;
    assigns        \nothing;
    ensures    top:    !StackEmpty(s) ==> \result == StackTop(s);
*/
value_type stack_top(const Stack* s);

```

Listing 11.22: Formal specification of `stack_top`

The implementation of `stack_top` is shown in Listing 11.23. It returns the element stored at index `sz-1` of the internal array, which corresponds to the top element of the stack. If the stack is empty, the expression evaluates to the first array element. Since the specification does not constrain the result in this case, this behaviour is admissible.

```

value_type stack_top(const Stack* s)
{
    if (!stack_empty(s)) {
        return s->data[s->sz - 1u];
    }
    else {
        return s->data[0u];
    }
}

```

Listing 11.23: Implementation of `stack_top`

11.6.8. Verification of `stack_push`

The predicate `StackPush( $s, v, t$ )` in Listing 11.24 characterizes the situation in which the stack t results from pushing the element v onto the stack s . It states that t has the same capacity as s (Axiom (11.10)), that its size is larger by one (Axiom (11.9)), that v is the new top element of t (Axiom (11.11)), and that the remaining elements of t form the same sequence as the elements of s .

```

/*@
predicate StackPush{S,T}(Stack* s, value_type v, Stack* t) =
    StackTop{T}(t)      == v      &&
    StackCapacity{S}(s) == StackCapacity{T}(t) &&
    StackSize{S}(s) + 1 == StackSize{T}(t)      &&
    StackPrefix{S,T}(s, t);
*/

```

Listing 11.24: StackPush (ACSL)

The specification of the function `stack_push`, shown in Listing 11.25, uses the predicate `StackPush` [11.24] to describe the effect of this operation when the stack is not full. In this case, the predicate captures that the resulting stack is obtained from the original one by adding the element to the top. The contract furthermore guarantees that the resulting stack is not empty. If the stack is already full, then `stack_push` leaves the stack unchanged.

```

/*@
requires  valid:      StackValid(s);
terminates          \true;
exits           \false;
assigns          s->sz, s->data[s->sz];
ensures   valid:      StackValid(s);

behavior  full:
    assumes          StackFull(s);
    assigns          \nothing;
    ensures  unchanged: StackEqual{Old,Here}(s,s);

behavior  not_full:
    assumes          !StackFull(s);
    assigns          s->sz, s->data[s->sz];
    ensures  push:      StackPush{Old,Here}(s, v, s);
    ensures  not_empty: !StackEmpty(s);

complete behaviors;
disjoint behaviors;
*/
void stack_push(Stack* s, value_type v);

```

Listing 11.25: Formal specification of `stack_push`

The implementation of `stack_push` is given in Listing 11.26. It first checks whether the stack is not full. If this condition holds, the value to be pushed is written to the array position `data[sz]`, and only afterwards is the field `sz` incremented.

```

void stack_push(Stack* s, value_type v)
{
    if (!stack_full(s)) {
        s->data[s->sz++] = v;
    }
}

```

Listing 11.26: Implementation of `stack_push`

11.6.9. Verification of stack_pop

The predicate StackPop in Listing 11.27 characterizes the situation in which the stack t results from removing the top element of the stack s . It states that both stacks have the same capacity (Axiom (11.13)), that the size of t is smaller by one (Axiom (11.12)), and that the sequence of elements stored in t coincides with the prefix of s obtained by removing its top element.

```
/*@
  predicate StackPop{S,T}(Stack* s, Stack* t) =
    StackCapacity{S}(s) == StackCapacity{T}(t)  &&
    StackSize{S}(s) - 1 == StackSize{T}(t)      &&
    StackPrefix{T,S}(t, s);
*/
```

Listing 11.27: StackPop (ACSL)

The specification of the function stack_pop is shown in Listing 11.28. The contract uses the predicate StackPop [11.27] to describe the effect of this operation when the stack is not empty.

```
/*@
  requires  valid:      StackValid(s);
  terminates
  exits    \false;
  assigns  s->sz;
  ensures  valid:      StackValid(s);

  behavior empty:
    assumes StackEmpty(s);
    assigns \nothing;
    ensures empty:      StackEmpty(s);
    ensures unchanged: StackEqual{Old,Here}(s, s);

  behavior not_empty:
    assumes !StackEmpty(s);
    assigns s->sz;
    ensures pop:        StackPop{Old,Here}(s, s);
    ensures not_full:   !StackFull(s);

  complete behaviors;
  disjoint behaviors;
*/
void stack_pop(Stack* s);
```

Listing 11.28: Formal specification of stack_pop

The implementation of stack_pop is given in the next listing. It checks whether the stack is non-empty and, if so, decreases the field size by one. No memory is modified when the stack is empty, in accordance with the specification.

```
void stack_pop(Stack* s)
{
  if (!stack_empty(s)) {
    --s->sz;
  }
}
```

Listing 11.29: Implementation of stack_pop

11.7. Witnesses for the stack axioms

In the preceding section, we verified each stack operation individually against its specification. These verification results ensure that every function behaves correctly with respect to its contract when considered in isolation. They do not, however, guarantee that the operations interact in the intended way when combined.

The axioms of the abstract stack data type formulated in §11.2 describe algebraic relationships between several stack operations. Establishing that the implementation satisfies these axioms therefore requires reasoning about combinations of operations rather than about individual functions alone.

To this end, we introduce small auxiliary C functions, called *axiom witnesses*. Each witness combines stack operations in the manner prescribed by a given axiom and is equipped with an ACSL contract that captures the corresponding property.

Proving the contract of such a witness establishes that the concrete implementation of the involved operations satisfies the corresponding algebraic property. In this way, we show that the verified implementations introduced in §11.6 jointly realize the axiomatic specification of stacks.

Each witness function follows the same general scheme. It executes the operations occurring in the axiom and then returns either an observed value or the modified stack state. Its contract has three parts. The *requires* clauses state the side condition of the axiom, such as `!StackFull(s)`, together with the assumptions that the stack functions themselves make: `StackValid(s)` throughout and, for the two witnesses that initialize a stack, the validity and separation of the memory the stack is built on. The property under discussion is expressed by a distinguished postcondition, typically labeled `size`, `capacity`, `top`, or `equal`. A second postcondition, labeled `valid`, records that the witness leaves behind a valid stack, so that the type invariant of §11.4 survives the combination of operations. Whenever the axiom refers to the initial state of the stack, this is captured by the pre-state label `Old`, as in `StackSize{Old}(s)`. The operator `\old` used elsewhere in this document (§7.4) is the corresponding form for a C lvalue; for a logic function such as `StackSize` the label is written directly.

Omission of *assigns* and *terminates* clauses For better readability, these are omitted from the witness specifications. The functions are not reused in other proofs and are not part of a modular verification chain. Their sole purpose is to establish the logical properties expressed by the stack axioms, rather than to serve as callable, reusable components.

11.7.1. Witnesses for the initialization of stacks

The operation `stack_init` prepares a stack of a given capacity: it takes a `Stack` object and an array supplied by the caller and turns them into an empty stack over that array. In the abstract stack theory, this corresponds to the constructor `init` (11.1), which produces an initial stack state. The associated axioms characterize fundamental properties of such a stack, in particular its size and capacity.

In this subsection, we verify these properties using axiom witnesses. Each witness initializes a stack and then observes the relevant property via the corresponding stack operation. The contract expresses the equality stated by the respective axiom.

11.7.1.1. Size of an initialized stack, Axiom (11.7)

Axiom (11.7) states that initializing a stack yields a stack whose size is zero.

```

/*@
  requires   valid:   \valid(s);
  requires   valid:   \valid(a + (0..capacity-1));
  requires   sep:     \separated(s, a + (0..capacity-1));
  requires   pos:     0 < capacity;

  ensures    size:    \result == 0;
  ensures    valid:   StackValid(s);
*/
size_type axiom_size_of_init(Stack* s, value_type* a, size_type capacity)
{
  stack_init(s, a, capacity);
  return stack_size(s);
}

```

Listing 11.30: Implementation of axiom_size_of_init

The witness of Listing 11.30 initializes a stack and returns its size; the clause size requires that value to be zero.

11.7.1.2. Capacity of an initialized stack, Axiom (11.8)

Axiom (11.8) states that initializing a stack yields a stack whose capacity equals the specified bound.

```

/*@
  requires   valid:    \valid(s);
  requires   valid:    \valid(a + (0..capacity-1));
  requires   sep:      \separated(s, a + (0..capacity-1));
  requires   pos:      0 < capacity;

  ensures    capacity: \result == capacity;
  ensures    valid:    StackValid(s);
*/
size_type axiom_capacity_of_init(Stack* s, value_type* a, size_type capacity)
{
  stack_init(s, a, capacity);
  return stack_capacity(s);
}

```

Listing 11.31: Implementation of axiom_capacity_of_init

The witness of Listing 11.31 hands that bound to stack_init and returns the capacity of the result; the clause capacity requires the two to agree.

11.7.2. Witnesses for adding an element to a stack

We now consider properties introduced by adding elements.

Elements are added to a stack by calling the function stack_push. In the abstract stack theory, this operation corresponds to the constructor push (11.3). The associated axioms describe how fundamental stack properties, such as size and top element, change when a new element is pushed onto a stack.

In this subsection, we present axiom witnesses for these properties. Each witness combines stack_push with other stack operations as prescribed by the corresponding axiom, while the contract expresses the required relationship between the resulting states.

11.7.2.1. Size after a push, Axiom (11.9)

Axiom (11.9) specifies how the size of a stack changes when an element is pushed onto it. If the stack is not full, pushing a new element increases its size by one.

```
/*@
  requires  valid:    StackValid(s);
  requires  not_full: !StackFull(s);

  ensures   size:     \result == StackSize{Old}(s) + 1;
  ensures   valid:    StackValid(s);
*/
size_type axiom_size_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_size(s);
}
```

Listing 11.32: Implementation of axiom_size_of_push

The witness of Listing 11.32 pushes an element and returns the resulting size; the clause size compares that value with the size the stack had on entry.

11.7.2.2. Capacity after a push, Axiom (11.10)

Axiom (11.10) states that pushing an element onto a stack does not change its capacity.

```
/*@
  requires  valid:    StackValid(s);
  requires  not_full: !StackFull(s);

  ensures   capacity: \result == StackCapacity{Old}(s);
  ensures   valid:    StackValid(s);
*/
size_type axiom_capacity_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_capacity(s);
}
```

Listing 11.33: Implementation of axiom_capacity_of_push

The witness of Listing 11.33 differs from the preceding one only in the observer it applies; the clause capacity requires the capacity to be the one the stack had on entry.

11.7.2.3. Top element after a push, Axiom (11.11)

Axiom (11.11) states that, for a stack that is not full, the element pushed onto the stack becomes the top element of the resulting stack.

```

/*@
  requires    valid:    StackValid(s);
  requires    not_full: !StackFull(s);

  ensures     top:      \result == v;
  ensures     valid:    StackValid(s);
*/
value_type axiom_top_of_push(Stack* s, value_type v)
{
    stack_push(s, v);
    return stack_top(s);
}

```

Listing 11.34: Implementation of axiom_top_of_push

The witness of Listing 11.34 pushes the value v and returns the top element; the clause `top` requires the two to coincide.

11.7.3. Witnesses for removing an element from a stack

We now turn to the axioms describing the behavior of a stack when an element is removed. Elements are removed by calling the function `stack_pop`, which corresponds to the modifier `pop` (11.3) of the abstract stack theory.

These axioms characterize both the direct effect of removing the top element and the interaction between `stack_push` and `stack_pop`. In particular, they describe how the size and top element change after a pop operation and how pushing and subsequently popping an element restores the previous stack state.

In this subsection, we verify these properties using axiom witnesses, combining `stack_pop` with other stack operations as prescribed by the corresponding axioms.

11.7.3.1. Size after a pop, Axiom (11.12)

Axiom (11.12) specifies how the size of a stack changes when an element is removed. For a non-empty stack, applying `stack_pop` decreases its size by one.

```

/*@
  requires    valid:    StackValid(s);
  requires    empty:    !StackEmpty(s);

  ensures     size:     \result == StackSize{Old}(s) - 1;
  ensures     valid:    StackValid(s);
*/
size_type axiom_size_of_pop(Stack* s)
{
    stack_pop(s);
    return stack_size(s);
}

```

Listing 11.35: Implementation of axiom_size_of_pop

The witness of Listing 11.35 pops an element and returns the resulting size; the clause `size` compares that value with the size the stack had on entry.

11.7.3.2. Capacity after a pop, Axiom (11.13)

Axiom (11.13) states that removing an element from a stack does not change its capacity.

```
/*@
  requires  valid:    StackValid(s);
  requires  empty:    !StackEmpty(s);

  ensures   capacity: \result == StackCapacity{Old}(s);
  ensures   valid:    StackValid(s);
*/
size_type axiom_capacity_of_pop(Stack* s)
{
  stack_pop(s);
  return stack_capacity(s);
}
```

Listing 11.36: Implementation of axiom_capacity_of_pop

The witness of Listing 11.36 again differs from its predecessor only in the observer it applies; the clause capacity requires the capacity to be the one the stack had on entry.

11.7.3.3. Popping what was just pushed, Axiom (11.14)

Axiom (11.14) describes the interaction between the operations stack_push and stack_pop. It states that if an element is pushed onto a non-full stack and then immediately removed again, the resulting stack is equal to the original stack. Thus, on non-full stacks, stack_pop acts as a left inverse of stack_push.

```
/*@
  requires  valid:    StackValid(s);
  requires  not_full: !StackFull(s);

  ensures   equal:    StackEqual{Old,Here}(s, s);
  ensures   valid:    StackValid(s);
*/
void axiom_pop_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  stack_pop(s);
}
```

Listing 11.37: Implementation of axiom_pop_of_push

The witness of Listing 11.37 pushes the value *v* and pops it again; the clause equal relates the two states of the same stack. Note that the equality asserted here is StackEqual, the observational equality of §11.5, and not identity of the underlying representation: the array beyond the stored elements may well have been written to in the meantime. This is the point at which the axioms depend on how equality of stacks was defined.

11.7.3.4. Pushing back what was just popped, Axiom (11.15)

Axiom (11.15) describes the complementary interaction between stack_pop and stack_push. It states that if an element is removed from a non-empty stack and the former top element is subsequently pushed back, the original stack is recovered. In this sense, stack_push acts as a right inverse of stack_pop, provided that the value being pushed is the top element of the original stack.

```
/*@
requires    valid:      StackValid(s);
requires    not_empty:  !StackEmpty(s);

ensures     equal:      StackEqual{Old,Here}(s, s);
ensures     valid:      StackValid(s);
*/
void axiom_push_of_pop_top(Stack* s)
{
    const value_type v = stack_top(s);
    stack_pop(s);
    stack_push(s, v);
}
```

Listing 11.38: Implementation of axiom_push_of_pop_top

The witness of Listing 11.38 first records the top element, then pops and pushes it back; as before, the clause equal relates the two states of the same stack in the sense of StackEqual.

Summary Taken together, the witnesses presented in this section establish that the implementations of the stack operations introduced in §11.6 satisfy the algebraic properties formulated in the stack axioms of §11.2.

11.8. Witnesses for well-definedness

As discussed in §11.5, the predicate `StackEqual` induces an equivalence relation on stacks. Since different stack instances may represent the same abstract stack, it is important that stack operations behave independently of the chosen representative. Functions with this property are said to be *well-defined* with respect to `StackEqual`.

Not every stack function satisfies this requirement. Some operations may depend on aspects of the concrete representation that are intentionally ignored by `StackEqual`. In the following, we therefore settle, for each of the relevant stack operations, whether it is well-defined with respect to this equivalence relation. For most of them the answer is yes; for two it is no, and the reason is instructive.

Functions not subject to well-definedness The notion of well-definedness is relevant only for operations whose result should depend solely on the abstract stack represented by a concrete instance. Two functions fall outside this consideration.

- The function `stack_equal` implements the equality test corresponding to the predicate `StackEqual`, whose relationship has already been discussed in §11.6.1. Asking whether it respects `StackEqual` is asking whether equal stacks are found equal to the same stacks, which is precisely the symmetry and transitivity of `StackEqual`, already proved as lemmas in §11.5.
- The function `stack_init` (§11.6.2) constructs a new stack and does not operate on an existing abstract stack at all.

Consequently, the choice of representative does not arise for these two functions, and the notion of well-definedness is therefore not applicable to them.

The shape of a well-definedness witness In analogy to the witnesses introduced for the stack axioms in §11.7, the well-definedness of a function is established by exhibiting a corresponding witness. Such a witness applies the function to two equal stacks and asserts that the results agree: equal values, for the observers, and equal stacks, in the sense of `StackEqual`, for the modifiers. As in the case of axiom witnesses, these functions are not reused in other proofs and are not part of a modular verification chain. For readability, we therefore omit `assigns` and `terminates` clauses in their specifications.

11.8.1. Well-definedness of `stack_size`

Since `stack_size` does not modify its `Stack` argument, well-definedness reduces to the requirement that equal stacks yield equal results. Listing 11.39 shows how this property is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid: StackValid(s) && StackValid(t);
  requires   equal: StackEqual{Here,Here}(s, t);

  ensures    equal: \result == true;
*/
bool stack_size_wd(const Stack* s, const Stack* t)
{
  return stack_size(s) == stack_size(t);
}
```

Listing 11.39: Implementation of `stack_size_wd`

The witness applies `stack_size` to two equal stacks and returns the comparison of the two results. The clause labeled `equal` requires that comparison to be **true**, that is, the two observations agree.

11.8.2. Well-definedness of `stack_empty`

The same argument applies to `stack_empty`, which likewise leaves its argument unchanged. Listing 11.40 shows the corresponding witness.

```
/*@
  requires    valid:  StackValid(s) && StackValid(t);
  requires    equal:  StackEqual{Here,Here}(s, t);

  ensures     equal:  \result == true;
*/
bool stack_empty_wd(const Stack* s, const Stack* t)
{
  return stack_empty(s) == stack_empty(t);
}
```

Listing 11.40: Implementation of `stack_empty_wd`

Again the two observations are compared, and the clause labeled `equal` requires them to agree. This is no surprise: `StackEmpty` is defined from the size alone, which we have just shown to be well-defined.

11.8.3. `stack_capacity` is not well-defined

The first operation to fail the requirement is `stack_capacity`. Equality of stacks was deliberately defined without reference to capacity (§11.5), and the two stacks of Figure 11.8 are equal although their capacities differ. Applying `stack_capacity` to them therefore yields different results, so the operation is not well-defined with respect to `StackEqual`.

No witness is required to see this: the contract of `stack_capacity` (§11.6.4) returns exactly the quantity that `StackEqual` disregards, and the figure already exhibits two representatives that separate the results.

This is not a defect. The capacity is a property of a concrete stack instance, not of the abstract stack it represents, and a program that asks for it is asking about the representation. What matters is that the operations meant to speak about the abstract stack do not depend on it.

11.8.4. `stack_full` is not well-defined

Since `StackFull` is defined from size and capacity, the failure of `stack_capacity` propagates to `stack_full` as well. Two equal stacks may differ in capacity, and then one of them can be full while the other is not. Unlike the previous case, this one is worth recording as a proved statement, and Listing 11.41 does so.

```

/*@
  requires   valid:   StackValid(s) && StackValid(t);
  requires   equal:   StackEqual{Here,Here}(s, t);
  requires   size:    StackSize(s) == StackCapacity(s);
  requires   cap:     StackCapacity(s) < StackCapacity(t);

  ensures    equal:   \result == false;
*/
bool stack_full_not_wd(const Stack* s, const Stack* t)
{
  //@ assert StackFull(s);
  //@ assert !StackFull(t);
  return stack_full(s) == stack_full(t);
}

```

Listing 11.41: Implementation of `stack_full_not_wd`

The contract pins down such a pair of stacks. They are equal and therefore of the same size; the clause `size` makes that common size the capacity of `s`, so `s` is full, while the clause `cap` gives `t` a strictly larger capacity, so `t` is not. The two assertions in the body state these consequences explicitly. The witness returns the comparison of the two results, and the clause labeled `equal` requires it to be **false**: here the two observations *disagree*, which is exactly the failure of well-definedness.

11.8.5. Well-definedness of `stack_top`

With `stack_top` we return to the operations that see only the abstract stack. Listing 11.42 shows the corresponding witness.

```

/*@
  requires   valid:    StackValid(s) && StackValid(t);
  requires   not_empty: !StackEmpty(s) && !StackEmpty(t);
  requires   equal:    StackEqual{Here,Here}(s, t);

  ensures    equal:    \result == true;
*/
bool stack_top_wd(const Stack* s, const Stack* t)
{
  return stack_top(s) == stack_top(t);
}

```

Listing 11.42: Implementation of `stack_top_wd`

It applies `stack_top` to two equal, non-empty stacks and compares the results; the clause labeled `equal` requires them to agree. The additional precondition `not_empty` is inherited from the contract of `stack_top`, which says nothing about an empty stack.

11.8.6. Well-definedness of `stack_push`

Since `stack_push` does not return a value but modifies its stack argument, well-definedness must be expressed relationally. In particular, the operation must map equal stacks to equal stacks.

To make this reasoning sound, aliasing between the two stacks must be excluded. If both arguments referred to overlapping memory regions, modifying one stack could implicitly affect the other, invalidating the relational argument. To capture this requirement, we introduce the predicate `StackSeparated` [11.43], which asserts that the memory regions of two stacks are disjoint.

```

/*@
  predicate StackSeparated(Stack* s, Stack* t) =
    \separated(s, s->data + (0..s->cap-1), t, t->data + (0..t->cap-1));
*/

```

Listing 11.43: StackSeparated (ACSL)

Listing 11.44 shows how the well-definedness of `stack_push` is formalized and verified using a *well-definedness witness*.

```

/*@
  requires   valid:   StackValid(s) && StackValid(t);
  requires   equal:   StackEqual{Here,Here}(s, t);
  requires   not_full: !StackFull(s) && !StackFull(t);
  requires   sep:     StackSeparated(s, t);

  ensures   valid:   StackValid(s) && StackValid(t);
  ensures   equal:   StackEqual{Here,Here}(s, t);
*/
void stack_push_wd(Stack* s, Stack* t, value_type v)
{
  stack_push(s, v);
  stack_push(t, v);
}

```

Listing 11.44: Implementation of `stack_push_wd`

The witness pushes *the same* value `v` onto two equal and separated stacks, and the clause labeled `equal` requires the resulting stacks to be equal again. The contract also demands that *both* stacks be non-full. That requirement does not follow from their equality: as §11.8.4 has just shown, one of two equal stacks may be full while the other is not. Well-definedness of `stack_push` therefore holds on the non-full stacks, which is exactly the domain on which Axiom (11.9) states what pushing does.

11.8.7. Well-definedness of `stack_pop`

Since `stack_pop` modifies its stack argument, well-definedness is expressed relationally as well: equal stacks must be mapped to equal stacks. As in the case of `stack_push_wd` [11.44], aliasing between the two stacks is excluded using the predicate `StackSeparated`.

```

/*@
  requires   valid:   StackValid(s) && StackValid(t);
  requires   equal:   StackEqual{Here,Here}(s, t);
  requires   sep:     StackSeparated(s, t);

  ensures   valid:   StackValid(s) && StackValid(t);
  ensures   equal:   StackEqual{Here,Here}(s, t);
*/
void stack_pop_wd(Stack* s, Stack* t)
{
  stack_pop(s);
  stack_pop(t);
}

```

Listing 11.45: Implementation of `stack_pop_wd`

The witness of Listing 11.45 pops from two equal and separated stacks, and the clause labeled `equal` requires the results to be equal again. No emptiness assumption is needed here, because `stack_pop` is total: its contract (§11.6.9) covers the empty case by leaving the stack untouched.

Summary The operations that observe only the abstract stack, namely `stack_size`, `stack_empty` and `stack_top`, are well-defined with respect to `StackEqual`. So are the two modifiers, provided the stacks passed to them are separated and, in the case of a push, not full. The two operations that fail, `stack_capacity` and `stack_full`, fail for the same reason: they report on the concrete representation rather than on the abstract stack it stands for. Together with the axiom witnesses of §11.7, this completes the passage from the abstract stack theory of §11.2 to its verified C implementation.

Part VI.

Appendices

Introduction

This part collects supplementary material that supports the main exposition but is not required for its linear reading. It provides additional reference information and background details that may be consulted independently, depending on the reader's interests and the intended use of the document. In particular, the appendices document the tools and settings of the verification setup, together with the results they produce (Appendix [A](#)).

A. Results of Formal Verification with Frama-C

This chapter introduces the formal verification tools used throughout this tutorial. We then present the extent to which the examples from Chapters 4–11 can be deductively verified.

Within Frama-C, the Frama-C/WP plug-in [1] enables deductive verification of C programs annotated using the ANSI/ISO-C Specification Language (ACSL) [2]. The Frama-C/WP plug-in relies on weakest-precondition calculations to generate proof obligations. These obligations can subsequently be discharged by external automatic theorem provers or by interactive proof assistants.

For the exact configuration of Frama-C/WP used in this release, we refer to Section 1.3.

A.1. Verification settings

This section summarizes the most important Frama-C options that were used for almost all functions.²³

```
-pp-annot
-no-unicode
-wp
-wp-rte
-warn-unsigned-overflow
-warn-unsigned-downcast
-no-warn-unaligned-pointer
-rte-no-pointer-call
-wp-model Typed
-wp-session ../..
-wp-interactive=batch
-wp-split
-wp-timeout 2
```

The option `-wp-split` lets Frama-C/WP decompose a proof obligation into several simpler ones. The numbers of verification conditions reported in §A.3 are the ones that result from it.

The options `-no-warn-unaligned-pointer` and `-rte-no-pointer-call` concern runtime-error guards that Frama-C/WP cannot express at all. Its Typed memory model has no counterpart for `\aligned` or for `\valid_function`, so the guards for unaligned pointers and for calls through function pointers are skipped whether they are requested or not. No example in this document casts a pointer or calls a function through one, so nothing that the remaining options would have covered is left unverified; the two options merely keep Frama-C/WP from being asked for guards that it would only drop again.

We deliberately use a relatively small timeout value for the provers. A timeout is of course only meaningful together with the machine it refers to: two seconds suffice for all examples in this document on

²³For the function `random_number`, the option `-warn-unsigned-overflow` is disabled, as explained in §7.19.

a MacBook Pro with an Apple M4 processor. Depending on the employed hardware and tool versions, individual examples may require a larger timeout to obtain successful proofs. For the precise versions of the employed provers we refer to Table 1.1 on Page 11.

A.2. Consistency of the specifications

A specification that contradicts itself proves everything. If the preconditions of a function cannot be satisfied together, or if a loop invariant holds in no iteration whatsoever, then every proof obligation that rests on them is valid, and a function can be verified completely without its contract stating anything about it at all. The tables of the preceding section therefore report only one half of the picture: they count discharged proof obligations, but they say nothing about the strength of the annotations from which those obligations arose.

To search for such contradictions, Frama-C/WP provides *smoke tests*. Instead of asking whether an annotation follows from what precedes it, a smoke test asks whether `\false` follows from it. The searches cover unsatisfiable preconditions and behaviour assumptions, loop invariants that no iteration can maintain, and program points that no execution can reach. Their verdicts are inverted with respect to ordinary proof obligations: a smoke goal that is *proved* is the unwelcome outcome, as it demonstrates that the assumptions in question imply `\false`.

The smoke tests are performed in a separate run of Frama-C/WP, which adds the following options to those of §A.1.

```
-wp-smoke-tests  
-wp-smoke-timeout 10  
-wp-cache none
```

The proof cache is disabled so that no stored result can take the place of an actual search. The timeout is larger than the one used for the ordinary proof obligations because a smoke goal is expected to exhaust it. Only the automatic provers participate: a goal that is expected not to be proved admits no meaningful interactive proof script, so CoQ is not applied.

All examples in this document have been examined in this manner, together with the two contracts of `rewrite_array_nothing` and `rewrite_array_unchanged` from §7.1, which are verified without being examples in their own right. No contradiction was found.

This outcome must not be read as a proof of consistency. Each search is bounded by the timeout and by what the automatic provers can derive within it, and the vast majority of these searches end in a timeout rather than in a prover's conclusion. A smoke test can thus refute the consistency of a specification, but it can never establish it. What it does rule out is that the results of the preceding section rest on a contradiction that a prover would have uncovered quickly.

A.3. Verification results

In our verification scenario, each proof obligation is processed by a fixed sequence of automatic and interactive theorem provers arranged as a *pipeline*.²⁴ Each prover attempts to discharge the verification

²⁴Sequential processing is enforced by passing the option `-wp-par 1` to Frama-C/WP.

conditions it receives and forwards only the remaining obligations to the next prover. The resulting *verification pipeline* is illustrated in Figure A.1.

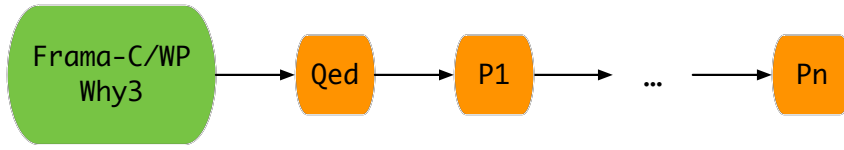


Figure A.1.: Verification pipeline of automatic and interactive theorem provers

For each algorithm, the following tables list the total number of generated verification conditions, the percentage of verified conditions, and the number of verification conditions discharged by each prover. Table cells marked with a $\cdot$ indicate a value of zero. In all cases, all generated verification conditions could be verified.

The distribution of verified conditions across provers should not be interpreted as a measure of their relative strength or quality. Due to the sequential pipeline structure, each verification condition is passed through the prover sequence until it is discharged (see Figure A.1).

Legend of verification result tables In the verification result tables, the first two columns report outcomes classified internally by the Frama-C/WP plug-in:

- WP^T : verification conditions classified as *terminating* by WP, corresponding to termination-related conditions that WP proves syntactically, for instance trivial termination inferred from the program structure, such as the presence of annotated loops and the absence of function calls;
- WP^U : verification conditions classified as *unreachable* by WP, corresponding to conditions that WP considers valid because the corresponding program points are syntactically unreachable, so that associated annotations (assertions, requires, ensures, etc.) cannot be triggered.

In other words, these two categories reflect WP-internal classifications of verification conditions prior to prover dispatch. All remaining columns give the number of verification conditions discharged by the respective external provers (e.g. QED, ALT-ERGO, Z3, CVC5, Coq).

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
find	§4.1	32 / 32	(100%)	1	1	23	7	·	·	·
find_last	§4.4	36 / 36	(100%)	1	1	20	14	·	·	·
find2	§4.2	33 / 33	(100%)	1	1	21	10	·	·	·
find3	§4.3	33 / 33	(100%)	1	1	12	16	·	·	3
find4	§4.3.4	33 / 33	(100%)	·	·	13	17	·	·	3
find5	§4.3.4	22 / 22	(100%)	·	·	7	12	·	·	3
find_if_not	§4.5	39 / 39	(100%)	1	1	12	20	·	·	5
find_first_of	§4.6	47 / 47	(100%)	·	·	38	9	·	·	·
adjacent_find	§4.7	43 / 43	(100%)	1	1	29	12	·	·	·
mismatch	§4.8	33 / 33	(100%)	1	1	23	8	·	·	·
equal	§4.8	9 / 9	(100%)	·	·	8	1	·	·	·
all_of	§4.9	28 / 28	(100%)	·	·	7	16	·	·	5
any_of	§4.9	9 / 9	(100%)	·	·	7	2	·	·	·
none_of	§4.9	9 / 9	(100%)	·	·	7	2	·	·	·
search	§4.10	60 / 60	(100%)	·	·	48	12	·	·	·
search_n	§4.11	80 / 80	(100%)	1	1	55	23	·	·	·
find_end	§4.12	54 / 54	(100%)	·	·	33	21	·	·	·
count	§4.13	42 / 42	(100%)	1	1	15	18	·	·	7

Table A.2.: Results for Nonmutating

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
clamp	§5.4	27 / 27	(100%)	1	1	21	4	·	·	·
make_pair	§5.3	5 / 5	(100%)	1	1	3	·	·	·	·
max_element	§5.5	44 / 44	(100%)	1	1	32	10	·	·	·
max_element2	§5.6	50 / 50	(100%)	1	1	37	11	·	·	·
max_seq	§5.7	16 / 16	(100%)	·	·	12	4	·	·	·
min_element	§5.8	50 / 50	(100%)	1	1	37	11	·	·	·
minmax_element	§5.9	107 / 107	(100%)	·	·	79	28	·	·	·

Table A.3.: Results for MinMax

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
lower_bound	§6.3	33 / 33	(100%)	1	1	14	17	·	·	·
upper_bound	§6.4	33 / 33	(100%)	1	1	16	15	·	·	·
equal_range	§6.5	26 / 26	(100%)	·	·	20	6	·	·	·
equal_range2	§6.5	125 / 125	(100%)	·	·	74	44	·	·	7
binary_search	§6.6	22 / 22	(100%)	·	·	15	7	·	·	·

Table A.4.: Results for BinarySearch

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
fill	§7.3	16 / 16	(100%)	1	1	9	5	.	.	.
swap	§7.4	10 / 10	(100%)	1	1	5	3	.	.	.
swap_ranges	§7.5	25 / 25	(100%)	.	.	14	11	.	.	.
copy	§7.6	18 / 18	(100%)	1	1	9	7	.	.	.
copy_backward	§7.7	19 / 19	(100%)	1	1	8	9	.	.	.
reverse_copy	§7.8	20 / 20	(100%)	1	1	9	9	.	.	.
reverse	§7.9	26 / 26	(100%)	.	.	9	17	.	.	.
rotate_copy	§7.10	18 / 18	(100%)	.	.	7	11	.	.	.
rotate	§7.11	30 / 30	(100%)	.	.	19	10	1	.	.
replace_copy	§7.12	27 / 27	(100%)	1	1	15	10	.	.	.
replace	§7.13	25 / 25	(100%)	1	1	14	9	.	.	.
remove_copy	§7.14	32 / 32	(100%)	1	1	18	12	.	.	.
remove_copy2	§7.15	82 / 82	(100%)	1	1	19	44	.	1	16
remove_copy3	§7.16	122 / 122	(100%)	1	1	30	69	.	1	20
remove	§7.17	114 / 114	(100%)	1	1	22	69	.	1	20
shuffle	§7.18	70 / 70	(100%)	.	.	28	32	.	.	10
random_number	§7.19	40 / 40	(100%)	2	2	21	15	.	.	.

Table A.5.: Results for Mutating

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
iota	§8.1	19 / 19	(100%)	1	1	10	7	.	.	.
accumulate	§8.2	26 / 26	(100%)	1	1	9	13	.	.	2
inner_product	§8.3	29 / 29	(100%)	1	1	9	16	.	.	2
partial_sum	§8.4	62 / 62	(100%)	1	1	19	39	.	.	2
adjacent_difference	§8.5	40 / 40	(100%)	1	1	13	25	.	.	.
partial_sum_inv	§8.6	44 / 44	(100%)	.	.	10	31	.	.	3
adjacent_difference_inv	§8.6	43 / 43	(100%)	.	.	10	30	.	.	3

Table A.6.: Results for Numeric

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
heap_parent	§9.3	14 / 14	(100%)	1	1	3	9	.	.	.
heap_child	§9.3	77 / 77	(100%)	1	1	34	40	.	.	1
is_heap_until	§9.4	49 / 49	(100%)	1	1	23	23	.	.	1
is_heap	§9.5	27 / 27	(100%)	.	.	12	14	.	.	1
push_heap	§9.7	161 / 161	(100%)	.	.	50	98	.	1	12
heap_sift_down	§9.8	178 / 178	(100%)	.	.	65	92	.	6	15
pop_heap	§9.9	104 / 104	(100%)	.	.	16	69	.	.	19
make_heap	§9.10	86 / 86	(100%)	.	.	33	40	.	.	13
sort_heap	§9.11	79 / 79	(100%)	.	.	22	44	.	.	13

Table A.7.: Results for Heap

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
is_sorted_until	§10.2	34 / 34	(100%)	1	1	18	11	·	·	3
is_sorted	§10.3	9 / 9	(100%)	·	·	7	2	·	·	·
partial_sort	§10.4	159 / 159	(100%)	·	·	51	89	·	·	19
bubble_sort	§10.5	110 / 110	(100%)	·	·	41	55	·	1	13
selection_sort	§10.6	91 / 91	(100%)	·	·	33	41	·	·	17
insertion_sort	§10.7	86 / 86	(100%)	·	·	27	44	1	1	13
heap_sort	§10.8	55 / 55	(100%)	·	·	15	30	·	·	10
merge	§10.9	533 / 533	(100%)	·	·	347	162	1	10	13
inplace_merge	§10.10	102 / 102	(100%)	·	·	31	54	2	1	14
merge_sort	§10.11	110 / 110	(100%)	·	·	25	69	1	·	15

Table A.8.: Results for Sorting

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
stack_equal	§11.6.1	24 / 24	(100%)	·	·	9	15	·	·	·
stack_init	§11.6.2	15 / 15	(100%)	1	1	3	10	·	·	·
stack_size	§11.6.3	8 / 8	(100%)	1	1	1	5	·	·	·
stack_capacity	§11.6.4	8 / 8	(100%)	1	1	1	5	·	·	·
stack_empty	§11.6.5	12 / 12	(100%)	·	·	7	5	·	·	·
stack_full	§11.6.6	13 / 13	(100%)	·	·	7	6	·	·	·
stack_top	§11.6.7	19 / 19	(100%)	·	·	9	10	·	·	·
stack_push	§11.6.8	40 / 40	(100%)	·	·	28	12	·	·	·
stack_pop	§11.6.9	33 / 33	(100%)	·	·	23	10	·	·	·

Table A.9.: Results for Stack

Algorithm		Verification Conditions		Individual Provers						
				WP ^T	WP ^U	QD	AE	Z3	C5	CQ
axiom_size_of_init	§11.7.1.1	12 / 12	(100%)	·	·	9	3	·	·	·
axiom_capacity_of_init	§11.7.1.2	12 / 12	(100%)	·	·	9	3	·	·	·
axiom_size_of_push	§11.7.2.1	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_capacity_of_push	§11.7.2.2	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_top_of_push	§11.7.2.3	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_size_of_pop	§11.7.3.1	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_capacity_of_pop	§11.7.3.2	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_pop_of_push	§11.7.3.3	9 / 9	(100%)	·	·	5	4	·	·	·
axiom_push_of_pop_top	§11.7.3.4	10 / 10	(100%)	·	·	6	4	·	·	·

Table A.10.: Results for Stack / StackAxioms

Algorithm		Verification Conditions	Individual Provers						
			WP ^T	WP ^U	QD	AE	Z3	C5	CQ
stack_size_wd	§11.8.1	8 / 8 (100%)	.	.	4	4	.	.	.
stack_empty_wd	§11.8.2	8 / 8 (100%)	.	.	4	4	.	.	.
stack_full_not_wd	§11.8.4	10 / 10 (100%)	.	.	5	5	.	.	.
stack_top_wd	§11.8.5	8 / 8 (100%)	.	.	4	4	.	.	.
stack_push_wd	§11.8.6	9 / 9 (100%)	.	.	3	6	.	.	.
stack_pop_wd	§11.8.7	9 / 9 (100%)	.	.	3	6	.	.	.

Table A.11.: Results for Stack / StackWellDefinedness

A.3.1. Lemmas verified with Coq

The CoQ column of the preceding tables counts the proof obligations that no automatic prover could discharge. Those counts say how many such obligations each example has, but not what they are. Table A.12 names them, and gives for each the listing that shows it and the number of examples that carry it.

That every one of them belongs to an ACSL lemma is not an accident of where the difficulties happen to fall; it is how they are dealt with. An obligation that the automatic provers cannot discharge is not proved where it arose, but restated as an ACSL lemma and proved there. What this buys is control over the context, and stability.

- A proof obligation inside a function body carries whatever the surrounding code and annotations put into scope; that context is neither chosen nor easily changed, and it can shift with every edit to the function.
- A lemma carries none of it—though it is not thereby closed, since it still quantifies over the program states its arrays live in, and still has the definitions and the neighboring lemmas of its listing at hand. What it lacks is the program, and that is what makes it durable: an interactive proof invested in a lemma is not invalidated by edits to the algorithms that use it. It can still be spent in vain—code that comes to relate three program states, where the lemma relates two, calls for a differently shaped statement—but what changes then is which lemma is wanted, not whether the proof of the old one still holds.

A lemma and its proof address a particular mathematical difficulty alone, and the C level receives the result as a single named fact.

What stays hard after the lifting from the level of code to the logic level is almost always induction. An automatic prover instantiates the axioms it is given; it does not invent an induction scheme. A lemma whose statement follows from unfolding a recursive definition one step is within reach, and a lemma that relates two ranges of an array is not. The Count family shows both halves of this. Count_Hit, Count_Miss and Count_One describe a single position and are discharged automatically, while Count_Union, which splits a range in two, and Count_Bounds, which bounds a count by the length of its range, are not. The same holds for the inductive predicates: HeapAncestor_Bounds and HeapAncestor_Root are discussed in §9.2, where the argument is spelled out.

The division is moreover a property of the lemma and not of the example that uses it: each lemma listed below is proved by CoQ in every example that includes it, and no other lemma of this document is ever proved that way.

The last column counts the examples that include the lemma, which is also the number of times it is proved. A lemma is not verified once for the whole document: it is re-proved as part of every example whose specification reaches it. A lemma of Count.acsl therefore weighs eighteen times as much as one that

only a single example includes, which is why that one file accounts for more of the Coq column than any other, and why the cost of an interactive proof is paid in proportion to how widely the lemma is shared.

Table A.12.: The ACSL lemmas that are verified with Coq

Lemma	Listing	Examples
Accumulate_Unchanged	8.5	4
AdjacentDifference_Inverse	8.20	2
Count_Between	10.27	2
Count_Bounds	4.55	18
Count_Equal	4.54	18
Count_Increasing	4.55	18
Count_Shift	4.55	18
Count_SomeEqual	4.56	2
Count_Unchanged	4.54	18
Count_Union	4.54	18
CountNotEqual_AllEqual	7.49	2
CountNotEqual_Count	7.45	3
CountNotEqual_Increasing	7.45	3
CountNotEqual_Lower	7.44	3
CountNotEqual_Unchanged	7.45	3
CountNotEqual_Upper	7.44	3
Find_Increasing	4.8	5
Find_Limit	4.8	5
Find_NoneEqual	4.8	5
FindNotEqual_AllEqual	4.20	5
FindNotEqual_Increasing	4.20	5
FindNotEqual_Limit	4.20	5
FindNotEqual_Unchanged	4.20	5
Heap_Maximum	9.7	10
HeapAncestor_Bounds	9.9	3
HeapAncestor_Root	9.9	3
HeapForest_Maximum	9.8	3
Increasing_Shift	6.3	9
Increasing_Unshift	6.3	9
IndexOfNotEqual_Core	7.51	2
IndexOfNotEqual_Lower	7.50	2
IndexOfNotEqual_Unchanged	7.51	2
InnerProduct_Unchanged	8.9	1
LowerBound_Shift	6.14	2
MultisetAdd_Distinct	9.18	2
MultisetReorder_DisjointUnion	7.59	13
MultisetReorder_LowerBound	10.8	1
MultisetReorder_PartitionLowerBound	10.8	1
MultisetReorder_Shift	7.59	13
MultisetReorder_SomeEqual	9.22	2
MultisetReorder_UpperBound	10.8	1
MultisetRetain_Distinct	9.18	2
MultisetSwap_Middle	7.63	5
MultisetUnion_Reorder	10.35	1

continued on the next page

Table A.12 – continued from the previous page

Lemma	Listing	Examples
MultisetUnion_TailLeft	10.27	2
RotateOne_MultisetReorder	10.21	2
StrictLowerBound_Shift	6.14	2
StrictUpperBound_Shift	6.14	2
Unchanged_Shift	7.2	25
UpperBound_Shift	6.14	2
WeaklyIncreasing_Increasing	6.3	9
WeaklyIncreasing_Shift	10.26	2

Bibliography

- [1] WP Plug-in. <http://frama-c.com/wp.html>.
- [2] ANSI/ISO C Specification Language. <http://frama-c.com/acsl.html>. 2025.
- [3] CEA List, INRIA, and Université Paris-Saclay. *Frama-C. A Framework for the Analysis of C Programs*. 2026. URL: <https://frama-c.com/> (visited on 02/06/2026).
- [4] CEA LIST, Laboratory of Applied Research on Software-Intensive Technologies. http://www-list.cea.fr/gb/index_gb.htm.
- [5] INRIA-Saclay, French National Institute for Research in Computer Science and Control. <http://www.inria.fr/saclay/>.
- [6] LRI, Laboratory for Computer Science at Université Paris-Sud. <http://www.lri.fr/>.
- [7] Fraunhofer-Institut für Offene Kommunikationssysteme (FOKUS). <http://www.fokus.fraunhofer.de>.
- [8] Virgile Prevosto. *ACSL Mini-Tutorial*. <http://frama-c.com/download/acsl-tutorial.pdf>.
- [9] Patrick Baudin et al. *ACSL 1.13 Implementation in Argon 18.0*. <https://frama-c.com/download/acsl-implementation-18.0-Argon.pdf>. 2018.
- [10] Why – Where Programs Meet Provers. <https://www.why3.org>. 2025.
- [11] Sylvain Conchon, Evelyne Contejean, and Johannes Kanig. *The Alt-Ergo SMT Solver*. <http://alt-ergo.lri.fr>. 2018.
- [12] Clark Barrett and Cesare Tinelli. *Homepage of CVC5*. <https://cvc5.github.io>. 2025.
- [13] Microsoft Research. *The Z3 Theorem Prover*. <https://github.com/Z3Prover/z3>. 2025.
- [14] The ROCQ Consortium. <https://rocq-prover.org>. <https://rocq-prover.org>. 2025.
- [15] Richard Smith. *Working Draft, Standard for Programming Language C++*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf>. publicly available draft of C++ 17 standard. 2017.
- [16] *Programming languages – C, Committee Draft*. <http://www.open-std.org/JTC1/SC22/WG14/www/docs/n1362.pdf>. 2009.
- [17] C.A.R. Hoare. “An Axiomatic Basis for Computer Programming”. In: *Communications of the ACM* 12 (1969), pp. 576–583.
- [18] Robert W. Floyd. “Assigning Meanings to Programs”. In: *Proc. Symposium on Applied Mathematics*. Ed. by J. T. Schwartz. Vol. 19. Mathematical Aspects of Computer Science. Providence, RI: American Mathematical Society, 1967, pp. 19–32.
- [19] Donald E. Knuth, James H. Morris, and Vaughan R. Pratt. “Fast Pattern Matching in Strings”. In: *SIAM J Comput* 6.2 (June 1977), pp. 323–350.
- [20] Timon Lapawczyk. “Formale Verifikation von Heap-Algorithmen mit Frama-C”. bachelor thesis. Humboldt-Universität zu Berlin, July 2016.
- [21] D.E. Knuth. *Sorting and Searching*. Vol. 3. The Art of Computer Programming. Addison-Wesley, 1973.

- [22] Sellbitze. *How to Implement Classic Sorting Algorithms in Modern C++*. <https://stackoverflow.com/questions/24650626/how-to-implement-classic-sorting-algorithms-in-modern-c>. Blog post. Aug. 2014.
- [23] Kim Völlinger. “Einsatz des Beweisassistenten Coq zur deduktiven Programmverifikation”. https://www2.informatik.hu-berlin.de/top/_media/www/mitarbeiter/diplomarbeit-kim-voellinger.pdf. Diplomarbeit. Humboldt-Universität zu Berlin, Aug. 2013.
- [24] Kim Völlinger and Jens Gerlach. “From Algebraic Data Types to Formally Verified Data Structures: The Stack Data Type”. Submitted to SEFM 2026. 2026.
- [25] *ANSI/ISO C Specification Language (ACSL)*. Reference manual for the specification language used by Frama-C to describe functional properties of C programs. CEA List, INRIA, and Université Paris-Saclay. 2026. URL: <https://frama-c.com/html/acsl.html> (visited on 02/06/2026).
- [26] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in Scala*. Artima, 2008. URL: http://www.artima.com/shop/programming_in_scala.

Index of ACSL definitions

Caveat: This index has been automatically generated from the ACSL sources. For the time being it mentions only the page where an ACSL definition is first included in the text. Moreover, if a listing had to be split, then the page number refers to the first part of the listing even if the specific ACSL definition appears in the second part.

A

Accumulate	172, 173
Accumulate_Init	172
Accumulate_Next	172
Accumulate_Unchanged	172
Accumulate_Unchanged_Shrink	172
AccumulateBounds	172, 174
AccumulateBounds_Unchanged	172
AccumulateDefault	173, 179
AccumulateDefault_Next	173
AccumulateDefault_One	173
AccumulateDefault_Unchanged	173
AccumulateDefault_Zero	173
AccumulateDefaultBounds	173
AccumulateDefaultBounds_Shrink	173
AdjacentDifference	182–184
AdjacentDifference_Inverse	185, 187
AdjacentDifference_InverseBounds	185, 187
AdjacentDifference_Section	183
AdjacentDifference_Step	183, 184
AdjacentDifferenceBounds	183
AllEqual	52, 66, 71, 106, 122, 123
AllSomeNot	52
ArrayBounds	83
ArrayBoundsShift	109, 243
ArrayExtrema	83
ArraySwap	166, 223
ArrayUpdate	121, 198, 209, 214, 215, 219, 220
ArrayUpdate_MultisetUpdate	209
ArrayUpdate_Shrink	121
ArrayUpdate_UpperBound	121
At	121, 159, 161, 207

C

C_Bit	168
Count	77, 80, 150, 162, 249
Count_Between	252
Count_Bounds	77

Count_Cut	77
Count_Empty	77, 78
Count_Equal	77, 252
Count_Hit	77, 78
Count_Increasing	77
Count_Miss	77
Count_One	77, 78
Count_Shift	77
Count_Single	77
Count_Single_Bounds	77
Count_Single_Shift	77
Count_SomeEqual	79
Count_Tail	252
Count_Unchanged	77
Count_Union	77
CountFind	79
CountFindNotEqual	154
CountNotEqual	150, 151, 154
CountNotEqual_AllEqual	154
CountNotEqual_Count	150
CountNotEqual_Decrement	154
CountNotEqual_Empty	150
CountNotEqual_FindNotEqual	154
CountNotEqual_Hit	150
CountNotEqual_Increasing	150
CountNotEqual_Lower	150
CountNotEqual_Miss	150
CountNotEqual_SomeNotEqual	154
CountNotEqual_Unchanged	150
CountNotEqual_Union	150
CountNotEqual_Upper	150
CountNotEqual_WeaklyIncreasing	150
CountNotEqual_Zero	154
D	
DefaultBounds	185
Difference	182
Difference_Next	182
Difference_Unchanged	182
Difference_UnchangedRange	182
Difference_Zero	182
E	
Equal	62–65, 68, 125, 126, 128, 130, 136, 252, 277
Equal_Shift	252
F	
Find	44, 46, 53, 55
Find_Empty	44
Find_Hit	44
Find_Increasing	44
Find_Limit	44

Find_Lower	44
Find_MissHit	44
Find_MissMiss	44
Find_NoneEqual	44
Find_ResultEqual	44
Find_ResultNoneEqual	44
Find_SomeEqual	44
Find_Upper	44
Find_WeaklyIncreasing	44
FindNotEqual	52, 53, 55, 67, 154
FindNotEqual_AllEqual	53
FindNotEqual_Empty	53
FindNotEqual_Hit	53
FindNotEqual_Increasing	53
FindNotEqual_Limit	53
FindNotEqual_Lower	53
FindNotEqual_MissHit	53
FindNotEqual_MissMiss	53
FindNotEqual_ResultAllEqual	53
FindNotEqual_ResultNotEqual	53
FindNotEqual_SomeNotEqual	53
FindNotEqual_Unchanged	53
FindNotEqual_Upper	53
FindNotEqual_WeaklyIncreasing	53

G

Greater_Less	82
GreaterOrEqual_Less	82

H

HasConstantSubRange	71, 72
HasConstantSubRange_Sizes	71
HasEqualNeighbors	59–61
HasSubRange	68, 69, 71, 75
HasSubRange_Sizes	68
HasValueOf	56–58
Heap	198, 199, 203, 205, 217
Heap_ChildBounds	197
Heap_Childs	197
Heap_Maximum	198, 199, 223
Heap_ParentBounds	197
Heap_ParentChild	197
Heap_ParentLeft	197
Heap_ParentRight	197
Heap_Shrink	198
Heap_Unchanged	198
HeapAncestor	199, 219
HeapAncestor_Bounds	199, 200
HeapAncestor_Root	199, 200
HeapCompatible	198, 199, 216
HeapCompatible_Update	198, 199

HeapForest	199, 217
HeapForest_Heap	199, 217, 223, 225
HeapForest_Maximum	199, 219
HeapForestCompatible	199, 220
HeapForestCompatible_Update	199, 220
HeapLeft	197, 201
HeapNodes	197
HeapParent	197, 199, 200
HeapParent_Zero	197
HeapRight	197

I

Increasing	97, 98, 100, 227, 231, 233, 235
Increasing_Equal	99, 246
Increasing_Shift	99
Increasing_Shrink	97
Increasing_Unshift	99
Increasing_WeaklyIncreasing	99
IncreasingLemmas	99
IndexOfNotEqual	155
IndexOfNotEqual_Core	155
IndexOfNotEqual_Count	155
IndexOfNotEqual_Empty	155
IndexOfNotEqual_Left	155
IndexOfNotEqual_Lower	155
IndexOfNotEqual_Next	155
IndexOfNotEqual_NotEqual	155
IndexOfNotEqual_Right	155
IndexOfNotEqual_Segment	155
IndexOfNotEqual_StrictlyWeakIncreasing	155
IndexOfNotEqual_Unchanged	155
IndexOfNotEqual_Upper	155
InnerProduct	176, 177
InnerProduct_Init	176
InnerProduct_Next	176
InnerProduct_Unchanged	176
InnerProductBounds	176
IotaGenerate	170

L

Less_Antisymmetry	82
Less_Irreflexivity	82
Less_Transitivity	82
LessOrEqual_Less	82
LessThanComparable	82
LowerBound	83, 92, 93, 101
LowerBound_Shift	109

M

MaxElement	83, 89, 95, 198
MinElement	83, 92, 95
Multiset_AddMinusRetain	207

Multiset_AddMinusRetainReorder	207
Multiset_MinusAddRetain	207
MultisetAdd	206, 208
MultisetAdd_Distinct	206
MultisetMinus	206
MultisetMinus_Distinct	206
MultisetOperations	206
MultisetParity	208, 209, 214, 219
MultisetParity_Combined	208, 215
MultisetParity_MultisetReorder	208, 216
MultisetParity_UnchangedFirst	208
MultisetParity_UnchangedSecond	208
MultisetReorder	162–164, 166, 207, 211, 224, 235, 239, 243, 249, 259, 263
MultisetReorder_DisjointUnion	163
MultisetReorder_LowerBound	235
MultisetReorder_PartitionLowerBound	235
MultisetReorder_Shift	163
MultisetReorder_SomeEqual	209, 210, 223, 235
MultisetReorder_Symmetric	163
MultisetReorder_Transitive	163
MultisetReorder_UpperBound	235
MultisetReorderLemmas	209
MultisetRetain	206, 207
MultisetRetain_Distinct	206
MultisetRetainRest	207
MultisetRetainRest_UnchangedFirst	207
MultisetRetainRest_UnchangedSecond	207
MultisetSwap	166
MultisetSwap_FrontMiddle	166, 223
MultisetSwap_Middle	166, 243
MultisetUnion	249, 259, 263
MultisetUnion_Empty	252
MultisetUnion_Left	252
MultisetUnion_Reorder	263
MultisetUnion_Right	252
MultisetUnion_Symmetric	252
MultisetUnion_TailLeft	252
MultisetUnion_Unchanged	252
MultisetUnionLemmas	252
MultisetUnionToReorder	263
MultisetUpdate	209, 215, 219
N	
NoneEqual	41, 42, 45, 50, 66, 148
NoneEqual_SomeEqual	41
NumericInverse	185
P	
PartialSum	179
PartialSum_Inverse	185
PartialSum_One	179

PartialSum_Section	179
PartialSum_Step	179
PartialSum_Unchanged	179
Partition	230, 234, 235, 260
PartitionLemmas	235
ProductBounds	176
R	
RandomNumberModulo	168
Remove	157, 159, 160, 251
Replace	143–146
Reverse	131–135
Reverse_Shift	132, 142
ReverseLemmas	132
Rotate	136–140, 142, 245, 260
Rotate_Shift	246
RotateLemmas	246
RotateOne_MultisetReorder	246
RotateOne_StrictLowerBound	246
S	
SomeEqual	41, 42, 50, 52, 56, 57, 66, 79, 111, 210
SomeNone	41
SomeNotEqual	52
SomeNotEqual_AllEqual	52
Stack	275
StackCapacity	275
StackData	275
StackEmpty	275, 282
StackEqual	277, 279
StackEqual_Reflexive	277
StackEqual_Symmetric	277
StackEqual_Transitive	277
StackFull	275, 282
StackInvariant	275, 276, 279
StackObservers	275
StackPop	285
StackPrefix	277
StackPush	283, 284
StackSeparated	294
StackSize	275
StackTop	275
StackValid	276
StrictLowerBound	83, 92, 93, 95, 103
StrictLowerBound_Shift	109
StrictUpperBound	83, 89, 95, 101, 106
StrictUpperBound_Shift	109
U	
Unchanged	54, 118, 120, 121, 135, 239
Unchanged_Extend	120
Unchanged_MultisetReorder	163

Unchanged_Shift	120
Unchanged_Shrink	120
Unchanged_Symmetric	120
Unchanged_Transitive	120, 187
UnchangedLemmas	120
UpperBound	83, 89, 103
UpperBound_Shift	109

W

WeaklyIncreasing	98, 100, 227, 232, 251
WeaklyIncreasing_AddElement	251
WeaklyIncreasing_Equal	251
WeaklyIncreasing_Increasing	99, 100, 227, 232, 255
WeaklyIncreasing_Join	251
WeaklyIncreasing_Shift	251
WeaklyIncreasing_Shrink	251
WeaklyIncreasing_Unchanged	251
WeaklyIncreasingLemmas	251

Index of examples

Caveat. This index has been generated automatically from the ACSL sources and currently records only the page on which a given ACSL definition is first introduced. Moreover, when a code listing is split across multiple pages, the page number shown refers to the first part of the listing, even if the ACSL definition itself appears later.

A

accumulate	175, 176
adjacent_difference	183, 184
adjacent_difference_inv	187
adjacent_find	60, 61
all_of	66, 67
any_of	66, 67
axiom_capacity_of_init	287
axiom_capacity_of_pop	290
axiom_capacity_of_push	288
axiom_pop_of_push	290
axiom_push_of_pop_top	290
axiom_size_of_init	286
axiom_size_of_pop	289
axiom_size_of_push	288
axiom_top_of_push	288

B

binary_search	110, 111
bubble_sort	240

C

clamp	85, 86
copy	127–129, 262, 263
copy_backward	129, 130
count	80, 172

E

equal	63, 65, 279
equal_range	105–107, 109
equal_range2	105, 107

F

fill	122, 123
find	38, 40–42, 67, 69
find2	42, 45–48, 50, 57, 58, 64, 72, 75, 111
find3	46–48, 55
find4	47
find5	47, 48

find_end	75, 76
find_first_of	57, 58
find_if_not	55, 67
find_last	50, 51
H	
heap_child	201, 202, 219
heap_parent	200, 201
heap_sift_down	198, 199, 201, 208, 217, 222, 223, 225
heap_sort	248
I	
inner_product	177, 178
inplace_merge	258, 259
insertion_sort	139, 244, 245
iota	170, 171
is_heap	205
is_heap_until	203–205, 231
is_sorted	233
is_sorted_until	98, 231, 233
L	
lower_bound	101–103, 105, 107, 111
M	
make_heap	224, 225, 236, 248
make_pair	84, 95
max_element	87, 88, 90, 93
max_element2	89–91, 94
max_seq	91
merge	250, 254, 255, 261
merge_sort	262, 264
min_element	92–94, 243
minmax_element	81, 94, 95
mismatch	64, 65
N	
none_of	66, 67
P	
partial_sort	210, 235, 236, 239
partial_sum	173, 180, 181, 183
partial_sum_inv	186
pop_heap	210, 221, 222, 227, 236
push_heap	198, 208, 211, 212, 217, 219, 220, 225, 236
R	
random_number	165, 167, 168
remove	158, 160, 161
remove_copy	148–151, 153, 158
remove_copy2	148, 151–153, 158
remove_copy3	52, 148, 152, 157–161
replace	143, 145

replace_copy	144, 145
reverse	134, 135, 141, 142
reverse_copy	132, 133
rotate	140, 141, 245, 246, 260
rotate_copy	137–139

S

search	69, 70, 72, 76
search_n	72
selection_sort	242, 243
shuffle	164, 165
size_pair_type	84
sort_heap	98, 226, 227, 248
stack_capacity	281
stack_empty	282
stack_empty_wd	293
stack_equal	279
stack_full	282, 283
stack_full_not_wd	293
stack_init	280
stack_pop	285
stack_pop_wd	295
stack_push	284
stack_push_wd	295
stack_size	280, 281
stack_size_wd	292
stack_top	283
stack_top_wd	294
swap	124, 126, 135, 165, 166, 243
swap_ranges	125, 126

U

upper_bound	103–105, 107, 229, 245
-------------------	------------------------